



Data Readers

Exensio Release 4.1 and Higher



DOCUMENTS



CONTENTS

PDF Solutions, Inc. Confidential Information

85-079701-T11

NOTICE

This document has been developed and created by PDF Solutions[®], Inc.

The software programs described in this document are copyrighted and are confidential information and proprietary products of PDF Solutions, Inc. This Manual is furnished to authorized users of PDF Solutions products, and is intended solely to facilitate the use of PDF Solutions tools and products, as specified in written agreements. The recipient of this document, by its retention and use, agrees to hold this document and its content in strict confidence and to protect the same from loss, theft or unauthorized use. No part of this publication may be reproduced, stored in a retrieval system, translated, transcribed, or transmitted, in any form, or by any means without prior explicit written permission from PDF Solutions, Inc. This Notice is an integral part of this document and shall not be removed or altered.

While every precaution has been taken in the preparation of this document PDF Solutions assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document or from the use of programs and source code that may accompany it, if/as applicable. In no event shall PDF Solutions be liable for any loss of profit or any other commercial damage caused or alleged to have been caused directly or indirectly by this document.

PDF Solutions, Inc. reserves the right to make changes in specifications and other information contained in this publication without prior notice. The reader should in all cases consult PDF Solutions to determine whether any such changes have been made.

Download Disclaimer

Through this documentation you can link to other websites which are not under the control of PDF Solutions. We have no control over the nature, content and availability of those sites. In no event shall we, PDF Solutions, be liable for any loss or damage including without limitation, indirect or consequential loss or damage, or any loss or damage whatsoever arising from loss of data or profits arising out of or in connection with the use of any external website sites.

Trademarks

Characterization Vehicle[®], Circuit Surfer[®], CV[®], dataPOWER[®] and dataPOWER[®], Exensio[®], maestria[®], ModelWare[®], pdFasTest[®], PDF Solutions[®], Yield Ramp Simulator[®], and YRS[®] are registered trademarks of PDF Solutions, Inc. or its subsidiaries, and Design-to-Silicon-Yield[™], dataPOWER[®] VSF[™], dP-bitMAP[™], dP-Defect[™], dP-Mining[™], dP-SSA[™], dP-Variability Analysis[™], dP-WorkFlow[™], Exensio-ALPS[™], Exensio-Control[™], Exensio-Test[™], Exensio-Yield[™], pdBRIX[™], pdCV[™], Template[™], and YA-FDC[™] are our common law trademarks.

The following logos are trademarks or registered trademarks of PDF Solutions:



Copyright© 1991-2021 PDF Solutions, Inc. All Rights Reserved.

PDF Solutions, Inc.

2858 De La Cruz Blvd, Santa Clara CA, 95050 USA

Tel: (408) 280-7900 ~ Fax: (408) 280-7915

email: info@pdf.com (general), doc@pdf.com (documentation),

dpsupport@pdf.com (support)

web: www.pdf.com

PDF Solutions, Inc. Confidential Information

CONTENTS

Notice	2
--------------	---

CHAPTER 1

Exensio Data Readers — Overview.....	9
---	----------

Reader Configuration Overview	10
Format Files	11
Automated Data Loading — DpLoad.pl.....	12
Exensio –Hosted Unified Schema — Data Loading And Insertion With DpLoad	27
DpLoadMgr.pl	29
Database Readers and the Database Model.....	30
Programs	30
Limits	31
Tester Summary Worksheet Storage	31
Bin Summaries.....	33
Server Tools — Database Storage of Tool Information	33
<i>data</i> BASE Readers.....	34
Conditions and Indexes.....	34
Program Class	35
Limits	36
Dynamic Programs.....	37
Rework.....	38
Unit Scaling	39
Bin Map Data.....	39
Site-Level Alignment.....	40
Outlier Filtering	43
Normalization and Alignment Concepts	44
Meaning of Data Normalization	44
Alignment	44
What is a Center Die?	46
How the Defect Reader Determines Center Die	49
Statistics Update Process — UpStat.....	50
Specification for Calculating Cluster Factor and Defect Density (m and d ₀).....	51

CHAPTER 2

ASCII Data Reader	55
Importing Data / Format File Configuration	55
ASCII Format File — Overview	56
Objective	56
Lexical Conventions	61
Format File Blocks	62
Limits and Scaling	65
Assignment of Variables	70
Assignment Using “=”	70
Assignment From the Input Stream File Name	70
Invalid Data	71
Oracle Rollback Table	71
Separators	72
Operators	73
Loop Control	74
Conditional Control	74
dbascii — Oracle Parameter Limit	75
Built-in Functions	76
Database	122
Designation of “Bad/Ugly” Die Locations at the Program Level	122
Tagging Lots and Wafers	123
dpEXPORT	123
Putting It All Together	124
The Exensio –Yield Interface	126
Error Message Alerts for Alarm and Events Rules Manager	139
Multibin Data Integration	140
Example Data File - Multibin	140
Example Format File - Multibin	141

CHAPTER 3

LEH Data Reader	147
Importing Data / Format File Configuration	147
LEH Format File — Overview	148
Objective	148
Lexical Conventions	151
Format File Blocks	152
Assignment of Variables	156

Assignment Using “=”	156
Assignment From the Input Stream File Name	156
Oracle Rollback Table.....	157
Separators	158
Operators	158
Loop Control	159
Conditional Control.....	159
Built-in Functions.....	160
The Exensio –Yield Interface.....	175

CHAPTER 4

WEH Data Reader179

Importing Data / Format File Configuration	179
WEH Format File — Overview	181
Objective	181
Lexical Conventions.....	184
Format File Blocks	186
Assignment of Variables	190
Assignment Using “=”	190
Assignment From the Input Stream File Name	190
Oracle Rollback Table.....	191
Separators	192
Operators	192
Loop Control	193
Conditional Control.....	193
Built-in Functions.....	194
The Exensio –Yield Interface.....	210

CHAPTER 5

FAB Data Reader215

Importing Data / Format File Configuration	215
Fab Format File — Overview	217
Objective	217
Lexical Conventions.....	219
Format File Blocks	220
Limits and Scaling	224
Assignment of Variables	228

Assignment Using “=”	228
Assignment From the Input Stream File Name	228
Oracle Rollback Table	229
Invalid Data	230
Separators	230
Operators	231
Loop Control	232
Conditional Control	232
Built-in Functions	233
The Exensio –Yield Interface	257
Error Message Alerts for Alarm and Events Rules Manager	260
Example 1 — FabSite	262
Example 2 — FabWaf	264
Example 3 — FabLot	267

CHAPTER 6

STDF3 Data Reader	271
Importing Data / Format File Configuration	271
STDF3 Worksheet Reader Interface	272
STDF3 Format File — Overview	273
Database	276

CHAPTER 7

STDF4 Worksheet Reader and ASCII Utility	279
STDF4 Worksheet Reader Interface	279
Exensio –YieldSTDF4 Format File — Overview	288
Importing STDF4 Data From a Database — stdf4ascii	296
Usage	297
Usage Examples	298

CHAPTER 8

Defect Reader	299
Importing Data	299
Lexical Conventions	300
Format File Blocks	301
Assignment Of Variables	304
Assignment Using “=”	304

Assignment From the Input Stream File Name	305
Separators	306
Operators	306
Loop Control	307
Conditional Control	307
Built-in Functions	308
Tagging Wafers	341
Running the Defect Reader	342
Tencor™ Binary to ASCII Conversion Script	346
defectMAP Format File — Overview	347
Example One	347
Example Two	361
Using Defect Image Pointers	364
Recommended Installation and Setup for Informix	364
Recommended Installation and Setup for Oracle	365
Example Format File - Defect Image Pointers	368
Example Data File - Defect Image Pointers	381

CHAPTER 9

LTX77 Data Reader	383
Importing Data / Format File Configuration	383
LTX77 Worksheet Reader Interface	384
LTX77 Format — Overview	385
Format File	386
Database	388

CHAPTER 10

BitMap Reader	389
Importing Data	389
Lexical Conventions	390
Format File Blocks	391
Assignment Of Variables	394
Assignment Using “=”	394
Assignment From the Input Stream File Name	394
Separators	396
Operators	397
Loop Control	398

Conditional Control	398
Built-in Functions	399
Running the bitMAP Reader	436
bitMAP Format File - Overview	439

CHAPTER 11

Events Reader.....467

Importing Data.....	467
Events Format File — Overview.....	468
Objective	468
Lexical Conventions.....	469
Format File Blocks	470
Assignment of Variables	474
Assignment Using “=”	474
Assignment From the Input Stream File Name	474
Oracle Rollback Table.....	475
Invalid Data	476
Separators	476
Operators	477
Loop Control	478
Conditional Control.....	478
Built-in Functions.....	479
Database	501
Tagging Lots	501
Putting It All Together.....	503
EVENTS File Format Specification (Oracle Only)	510
Running The Events Reader.....	512
Event Reader Windows Service – Installation and Configuration.....	516
Install EPT_Reader Windows Service.....	516
EPT_Reader Service Recovery Options	518
Start/Stop EPT_Reader Service	519
Uninstall Event Reader Windows Service	519

INDEX

.....	521
-------	-----

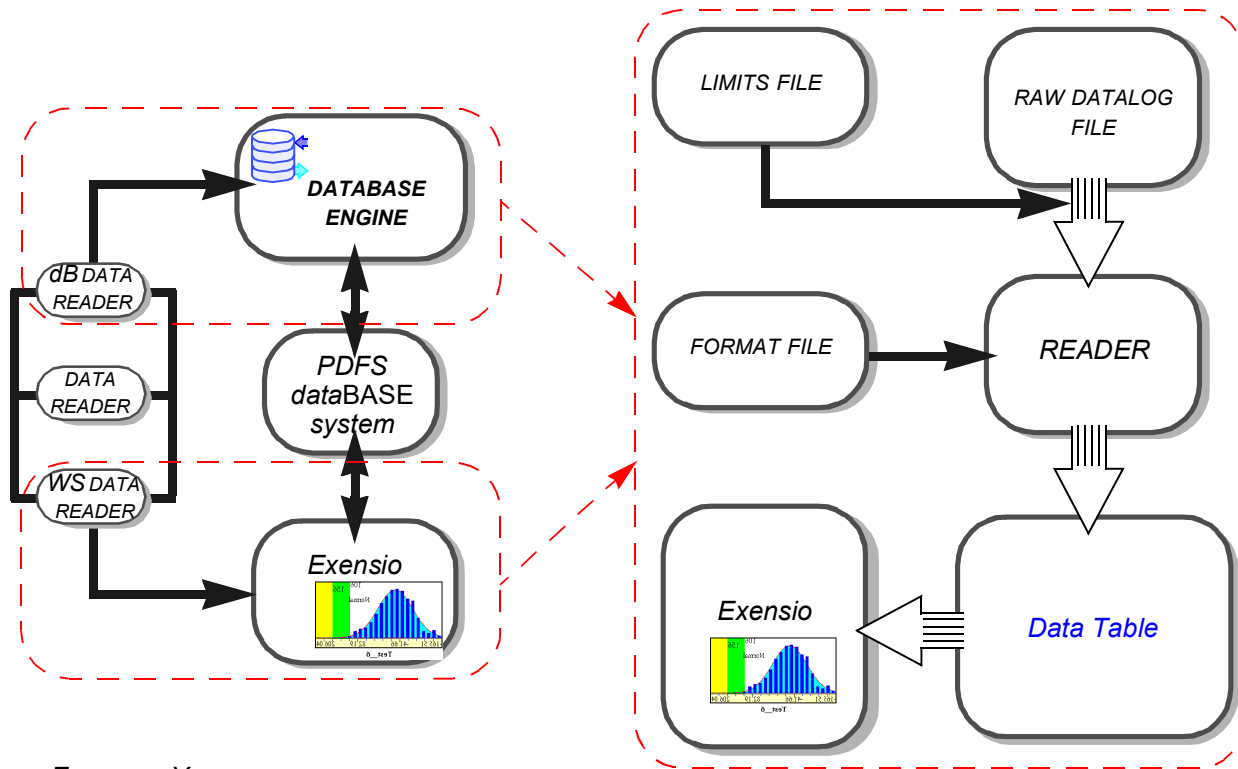
CHAPTER 1

EXENSIO DATA READERS — OVERVIEW

All Exensio readers function as the device for importing raw datalog files into the analysis environment. The reader organizes and formats data into a standard form that can be handled by Exensio, regardless of how the data is formatted in the original file. Data readers can access data directly from files or through a database. Whether the data is imported through the database or directly from a data file, the objective is the same: to access your data and format it properly for comprehensive analysis with Exensio — wafer mapping tools, data mining, yield analysis. etc.

To accommodate these two primary means of importing data, there are two types of data readers. Database readers, the primary means of importing data into Exensio, will import the formatted data into the Informix, Oracle or Cassandra database for access through the Exensio data retrieval system. Worksheet readers import the formatted data directly into the Exensio –Yield environment. Database readers operate in the background, as part of the overall process of configuring importing data from the database. Worksheet reader sessions are configured and implemented from a reader-specific interface that is called by the user when it is time to load a new raw data file.

READER CONFIGURATION OVERVIEW



EXENSIO –YIELD READER CONFIGURATION

The end result of the datalog import process using the Exensio data reader protocol is a raw data worksheet that contains all of the information you will need to perform all of the analysis functions available with your configuration of Exensio –Yield, *wafer*MAP, etc.

Worksheet Readers

To initiate the importing of data from a datalog file with your system’s worksheet readers, choose **Tools > Import >** The appropriate reader import dialog will appear. This section provides a general overview that should familiarize you with the basic functionality of the Exensio data reader. The functionality specific to each type of reader and their import dialogs is described in the documentation provided for that reader (STDF3, ASCII, etc.).

FORMAT FILES

The format file is a short ASCII file that is used by the reader, while it processes the datalog file, that determines how the imported data will be formatted in the final raw data worksheet. It specifies the format in which the data will be expected from an incoming source.

Format files may differ between readers, however they will all be composed in a similar way. The main purpose of a format file is to specify the pieces of information from your data that will be key conditions, key indexes, conditions, and indexes.

Format files typically can be created with any text editor. The name of the format file should be representative of the information contained in it, and, in the case of WS readers, stored in the Exensio –Yield **formats** directory. Format files typically use the **.fmt** file extension. For certain types of format files such as STDF, there is a graphical user interface for the creation of the files, accessed via **Data > Create Format File >** Where applicable, these are described in the documentation for the individual readers.



In the case of dB readers, several other items that are required by the database system need to be specified, e.g. program name, equipment, etc. This information is described in detail in the documentation for each reader.

AUTOMATED DATA LOADING — DpLoad.pl

The DpLoad.pl Perl script searches given directories for data files, and runs the associated data reader on these files, dumping the results to a database. The location of the data files and the reader, along with other options, are specified in a configuration file that is passed to the script as the first argument. DpLoad.pl also works with other tools, such as scripts and pre-processors.

Usage Statement

The following is the usage statement of DpLoad.pl as of release 4.0.1:

Usage:

usage:

```
DpLoad.pl [-v] [-V] [-u]
DpLoad.pl configFile [-once] [-run command]
          [-notify script_name] [-log [logfile]] [-log_db db]
          [-reverse] [-nosort] [-files num] [-sleep val]
          [-cleanup val] [-modtime val] [-rm_spaces] [-warn]
          [-mtime] [-timeout val]
```

Where:

```
configFile :The name of the configuration file
-once      :when used, the DpLoad.pl will process the data
            files once and exit.
-run       :when used, the provided command will be executed
            on the Processed files.
            Example: DpLoad.pl config.cfg -run compress
-notify    :when used, the provided script_name will be
            executed and passed the NotProcessed file
            name.
            Example: DpLoad.pl config.cfg -notify email.pl
-log        :Enable logging of events and error files. When
            provided, the specified logfile will be used
            instead of the default log file.
-log_db    :Enable database logging of events and errors.
            When provided, the specified database will be
            used.
-reverse    :Load the data files in the order of newest
            first. By default, the oldest files are loaded
            first.
-nosort     :Load the data files in the order of filenames.
            By default, the oldest files are loaded first.
-files      : Specify number of files in the staging area to
            be processed each visit. It overrides
            ${IDS_FILES_PER_LOOP} environment variable.
            Default if neither is specified is 20 files.
-sleep      :Specify sleep time after completion of config
            file execution. It overrides
            ${IDS_SLEEP_SECONDS} environment variable.
            Default if neither is specified is 10 seconds.
-cleanup    :Specify age of files in the Processed directory
            to be removed. It overrides
            ${IDS_CLEANUP_DAYS} environment variable.
            Default if neither is specified is 0 days
            (never delete).
```

```

-unused      :Specify age of files in the UnusedFiles
               directory to be removed. It overrides
               ${IDS_RM_UNUSED_DAYS} environment variable.
               Default if neither is specified is 0 days
               (never delete).
-modtime     :Specify age of files in the staging area to be
               processed. It overrides ${IDS_MOD_TIME}
               environment variable. Default if neither is
               specified is 30 seconds.
-rm_spaces   :Replace the spaces in the file names with
               underscores.
-warn        :Moves the datafile to the Warnings directory
               along with the .warn file. If this option is
               not used, the datafile is moved to the
               Processed directory while the .warn file is
               moved to the Warnings directory.
-mtime       :Load the data files sorted by modification time,
               access time is the default if this option is
               not used.
-timeout     :When used, DpLoad.pl will check running time
               elapsed so far and will stop gracefully if
               that time exceeds timeout value. Check will be
               done after completion of config file execution
               loop. Timeout value is in seconds, with a
               maximum allowed value of 200 million.

```

The **email.pl** script is provided as an example of how the user could use the **-notify** option to E-mail the specified users when a file went into the **NotProcessed** directory.

Configuration File

The configuration file can have any number of lines specifying the location of the files to be processed, a path to the reader and several other options. A typical file with three entries uses the following template:

```

Search Dir1:FullPath of Reader:Extension1:Reader Options:
Search Dir2:FullPath of Reader:Extension2:Reader Options:
Search Dir3:FullPath of Reader:Extension3:Reader Options:

```



For UpStat and other tools that do not require data files as input, only the second and fourth fields are significant.

NA:/somedir/UpStat:NA:-from 0 -to 2 dbname1 dbname2:

Parallelization

Parallelization allows you to run multiple instances of certain readers and tools in parallel. Tools that can use the parallelization functionality are:

- Scheduler
- Descrambler
- Membrain
- bitmap reader
- Fab reader

- Any script or executable that does not need database access. For example, the `stdf4ascii` reader.

To enable this feature, you need to add an extra field to the `DpLoad.pl` configuration line. Below, an example illustrates this. The first line is a typical `DpLoad.pl` configuration line, the second and third are lines with parallelization enabled.

- `/u01/data:/u01/bin/bitmap:%-db dbname -class 17 -db_accept -fmt /u01/bit.fmt:`
- `/u01/data`
- `:/u01/bin/bitmap:%-db dbname -class 17 -db_accept -fmt /u01/bit.fmt:-parallel 4:`
- `/u01/data`
- `:/u01/bin/bitmap:%-db dbname -class 17 -db_accept -fmt /u01/bit.fmt:-parallel 8 -freeze 60`

The parallelization options are **-parallel** and **-freeze**.

- The **-parallel** option is followed by a number; this number must be a valid integer between 2 and 16. This option tells `DpLoad.pl` how many instances of the reader or tool to run in parallel.
- The **-freeze** option is followed by a valid integer between 1 and 1800. This option tells `DpLoad.pl` to stop feeding more instances, if one or more of the current running instances have been running for more than the number of seconds set by the **-freeze** option. Once no instances have been running for more than this number of seconds, `DpLoad.pl` will resume feeding instances to be parallelized.
 - **-freeze** is optional. If it not used, `DpLoad.pl` will make sure to keep a constant number of processes running in parallel, as defined by the **-parallel** option.

Looping

The added looping functionality allows you to specify the number of times the same configuration line will be executed before moving to the next line in the configuration file. This feature is primarily added for the tools that do not operate on files — i.e. the extension field in the config line is set to “NA”.

The syntax of this feature is similar to the parallelization option and freeze options. Example: **-parallel 5 -loops 10**. This feature is a standalone feature, meaning it can be used without using the parallel option.

Parallel/Looping Configuration Files

If using parallel or looping options, then the configuration files will look like:

```
Search Dir1:FullPath of Reader:Extension1:Reader Options:
parallel and/or looping options
```

```
Search Dir2:FullPath of Reader:Extension2:Reader Options:
parallel and/or looping options
```

```
Search Dir3:FullPath of Reader:Extension3:Reader Options:
parallel and/or looping options
```

Configuration

All the empty lines and the lines that start with the character “#” in the configuration file are considered comment lines and will be skipped. All options must be included in one line (even if the line wraps). Environment variables can be used in the configuration file.

Search Dir: Location of files to be processed.

FullPath of Reader: The reader to be run by the script.

Extension: Specifies the extensions of the files that should be processed by the script. % if all files should be processed.



When the extension is set to “NA”, fields one and three will be ignored. DpLoad.pl executes the tool in the second field, and passes the arguments in the fourth field.

Options: Command-line arguments passed to the reader.

- ASCII reader — “[Command-Line Arguments](#),” pg. 127.
- LEH reader — “[Command-Line Arguments](#),” pg. 176.
- LTX77 reader — “[Command-line Arguments](#),” pg. 385.
- STDF3 reader — “[Command-line Arguments](#),” pg. 276.
- defectMAP reader — “[Command-line Arguments](#),” pg. 342.

The following six directories should be present under the Search Dir: (If they are not present, they get created by DpLoad.pl)

Wd: Working directory.

Processed: Processed files are moved to this directory.

NotProcessed: Files that were not processed properly are moved to this directory.

UnUsedFiles: Files rejected by the readers.

ReworkFiles: Files that have reworked data and where Rework Action is “load no data.”

Warnings: Warning files.

Operation

The DpLoad.pl script searches the **Search Directory** for any files with the desired **Extension**. If any files are found, the script runs the specified reader, passing it **Options** as arguments. If the file is properly processed without errors it is moved to the **Processed** directory. Otherwise, it is moved to the **NotProcessed** directory and an error file is also placed in the **NotProcessed** directory.

Warning files are moved to the **Warnings** directory.

Files that have reworked data and where Rework Action is “do not load” are moved to the **ReworkFiles** directory.

Each file associated with a new program (Job Name) needs to be processed twice before it is actually moved to the **Processed** directory, this is necessary to give the user the ability to specify several options on the database side before the file is completely processed. The reader recognizes that these options have been set when the **Accept_Data** flag is set on the database side or the reader is passed the **-db_accept** option. The file, **NOT_ACCEPTED**, in the **Working** directory lists all the files that do not have there **Accept_Data** flag set.

All the files that remain in the **Processed** directory for more than the value of **IDS_CLEANUP_DAYS** (as defined in the *Environment Variable* section, below) will be deleted.

If DpLoad.pl sees **UpStat** as the program name it ignores the search directory entry and the extension and it uses the program and option fields to invoke the UpStat program.

The command line argument **-once** is an optional argument. When specified, all the data directories listed in the configuration file will be visited once and all the applicable files in those directories will be processed. Applicable files are those with the specified file extension in the config file and have a time stamp older than the current time by the value of **IDS_MOD_TIME** environment variable. Since each directory is visited only once when the option is used, the environment variable **IDS_FILES_PER_LOOP** is ignored.

How to Run DpLoad.pl

- Run and keep a log file
./DpLoad.pl cfgfile.cfg > MyLog &
- Run and keep no log file (no output to screen)
./DpLoad.pl cfgfile.cfg > /dev/null &

Environment Variables

The following variables can be set on the c-shell command line using the **setenv** command (e.g. **setenv IDS_MOD_TIME 60**), or can be included in the **.cshrc** file.

IDS_FILES_PER_LOOP	Indicates how many files should be processed per iteration per config entry. Defaults to 5.
IDS_SLEEP_SECONDS	Sleep time after processing all configuration file entries. Defaults to 10 seconds
IDS_CLEANUP_DAYS	The age of the files at which DpLoad.pl removes them from the Processed directory. The age is determined by comparing the time-stamp on the file against the current time. Defaults to 30 days. When this variable is set to 0, the files will not be deleted.
IDS_RM_UNUSED_DAYS	The age of the files at which DpLoad.pl removes them from the Unused Files and Rework Files directories. The age is determined by comparing the time-stamp on the file against the current time. Defaults to 10 days. When this variable is set to 0, the files will not be deleted.
IDS_MOD_TIME	Time, in seconds, before a file is processed after being copied to the data directory. Defaults to 10 seconds.

Algorithm

The DpLoad.pl script's main functionality follows this algorithm:

```

for ever
    for every entry in the config file
        process x files per entry
        (where x is $IDS_FILES_PER_LOOP)
        (do not process a file unless it
        has not been modified within
        $IDS_MOD_TIME) (Guard Time)
        remove files (per entry) processed
        before T days
        (T is $IDS_CLEANUP_DAYS)
    end for
    sleep y seconds (where y is $IDS_SLEEP_SECONDS)
end for

```

Automated Loading of Exported Data Files

To automate the loading of exported data files, the DpLoad.pl utility accepts "DPIMPORT" as a special keyword in the list of reader's arguments. When DPIMPORT is found, DpLoad.pl executes the `dpimport_fmt.pl` utility to generate a format file for the current data file and replaces DPIMPORT with the path to the generated format file.

For example, consider the following line in the DpLoad.pl configuration file:

```
/my/data:/my/bin/dbascii:res: -db mydb -fmt DPIMPORT -db_accept:
```

In this case, every file that has the .res extension in the /my/data directory is passed to the dpimport_fmt.pl utility:

```
% /my/bin/dpimport_fmt.pl test.res
```

It generates the format file (e.g. test.fmt) in the /my/data/wd directory. then the reader is executed as follows:

```
% /my/bin/dbascii /my/data/test.res -db mydb -fmt /my/data/wd/test.fmt -db_accept
```

The automatically generated format file is removed after processing the file.

Changing Case

In DpLoad.pl configuration file, you can specify the case change specific fields (database columns) using the special keywords **-DPIMPORT_LOWERCASE** or **-DPIMPORT_UPPERCASE** on the same line where DPIMPORT is specified. The fields affected are:

Lot, SrcLot, LotClass, WaferID, Process, Equip1 ...Equip6, ProcStep, Stage, Cust_name, Recipe, Operator, TestMode, Technology

This is sometime useful in maintaining consistency when importing data from different sites, where case conventions are not always the same.

Ignoring Bin Colors

DpLoad.pl's configuration file support the **-nobincolor** option in dpimport_fmt.pl. To use this option, add **-DPIMPORT_NOBINCOLOR** after **"-fmt DPIMPORT"** in the configuration file — as in the following example in the configuration file:

```
/my/data: /my/Path/to/dbascii:%: -db myDB -db_accept
-fmt DPIMPORT -DPIMPORT_NOBINCOLOR
```

(The **-nobincolor** option in dpimport_fmt.pl is documented in the *"Default Bin Colors,"* in the *"Importing Data"* section of the Exensio Administrator Tools manual.)

Limits-Only Format File

DpLoad.pl's configuration file supports the **-limitsonly** option in dpimport_fmt.pl.

To use this option, add **-DPIMPORT_LIMITSONLY** after **"-fmt DPIMPORT"** in the configuration file, as in the following example in the configuration file:

```
/my/data: /my/Path/to/dbascii:%: -db myDB -db_accept
-fmt DPIMPORT -DPIMPORT_LIMITSONLY
```

(The **-limitsonly** option in dpimport_fmt.pl is documented in the *"Limits-Only Format File"* sub-section in the *"dpEXPORT and dfEXPORT"* chapter of the Exensio Administrator Tools manual.)

DpLoad.pl File Tracking

DpLoad.pl File Tracking enables users to track data files, error messages, reader versions and other useful information. This feature uses the standard Exensio – Yield database schema to store all the above information.

File Tracking is triggered by using the DpLoad.pl **-log_db** command-line option, followed by the tracking database name. This tracking database is created in the same way that a standard database is created, using DpCreateDb.pl.

Installation — To install and use DpLoad.pl File Tracking, the following is required:

- Perl 5.8.8 and its library — supplied by PDF Solutions
- DpLoad.pl version 9.x
- DpCreateDb.pl, version 9.x

Usage — Assuming you have a correctly configured DpLoad.pl configuration file, and Exensio –Yield supplied perl_db and libraries are correctly installed. Below are some general steps:

1. Test that the environment is ready by running the following two commands. The first command will make sure that perl_db and its libraries are installed, also it will check that you have the correct version of DpLoad.pl.

• DpLoad.pl -v

2. Now you need to create Exensio –Yield schema specifically used as file your tracking database. Issue the below command to create the new database. Enter:

DpCreateDb.pl <Enter>

3. Once all the above is complete with no problems or errors, you can now run DpLoad.pl and by adding one more option to it, enable the new feature. Enter:

DpLoad.pl myconfig.cfg -log -files 100 -log_db dbname

Optional Storage of Meta Information — The new file tracking feature provides optional meta information tracking. The optional meta storage is enabled if the reader, script or pre-processor creates a meta file that contains some key meta information to be stored in the file tracking database.

The meta file follows some basic standards:

- DpLoad.pl will be looking for a **meta.jnk** file in the same location as the **err.jnk** file.
- The new file consists of multiple lines. The first three lines and the last line are informative; the remaining lines hold the meta information.
- The “|” symbol is used as separator.
- First line:
 - VERSION|1.0
- Second line:
 - INFILE|filename

- Third line:
 - BOM
- Remaining lines
 - PROGRAM|prog_1_v2
 - LOT|AD2000.01
 - LOT|AD2000.03
 - WAFER|
 - ...
- Last line:
 - EOM

Miscellaneous — There are some codes stored in the file tracking database that are very useful for querying the database and generating statistics about loading progress and performance.

- **pgc_key** in DP_LOG table:
 - If **-class** option is used and the readers are dbascii, bitmap, events, fab, leh, weh, defect or cv, then the **-class** value is stored.
 - If **-class** option is not used and the readers are dbascii, bitmap, events, fab, leh, weh, defect or cv, then:

Reader	Code
dbascii	-100
bitmap	-101
events	-102
fab	-103
leh	13
defect	14
weh	19
cv	26

- For all other types of programs or tools, whether or not the **-class** option is used, **-199** is stored.
- **error_code** in DP_LOG table — When inserting a new entry in the DP_LOG table, the system populates the **error_code** field with **-1**, indicating that the loading process is underway. Upon completion of the loading process, this field is updated to:

Code	Definition
0	File was loaded successfully
90	File was not processed and it has been moved to NotProcessed directory
4	File is a rework and has been moved to ReworkedFiles directory. <i>Seldom used.</i>
9	File is rejected and has been moved to UnusedFiles directory. <i>Seldom used.</i>
10	File is not accepted and is being kept in the staging directory. <i>Seldom used.</i>
11	This code is specific to Informix database. When a dead lock happens, the file is kept in the staging directory to be picked up for loading again. <i>Seldom used.</i>

Code	Definition
80	File was loaded but generated warnings. <i>Seldom used.</i>
91	File was not processed, because the error file format doesn't follow error file standards, and has been moved to NotProcessed directory. <i>Seldom used.</i>

Configuration File Locking

This feature will prevent running multiple DpLoad.pl instances on the same configuration file entries.

Problem

Assume you have a configuration file, **pcm.cfg**, and this file includes the following configuration line:

```
STAGING_DIR2:/bin/dbascii:csv:-class x -fmt file.fmt -db dbname
```

Also, you have another configuration file, **pcm_prod.cfg**, that includes a line from the above configuration file.

```
STAGING_DIR1:/bin/dbascii:csv:-class x -fmt file.fmt -db dbname
```

```
STAGING_DIR2:/bin/dbascii:csv:-class x -fmt file.fmt -db dbname
```

```
NA:/bin/UpStat:NA:dbname
```

If you run DpLoad.pl against the first configuration file, then you run the DpLoad.pl against the second configuration file, you will get no error, but there will be a significant problem. The problem is that two DpLoad.pl instances are hitting on the same staging directory trying to work on the same files, potentially causing conflicts.

Solution

In order to avoid this scenario, the system can now implement a file lock that is created under the staging directory. Assume the staging directory above is **/data/pcm**. The file lock will be created under **/data/pcm/Wd/tmp_csv**.

The file lock name is always **DpLoad.pl_cfg.lock**.

The file lock contains useful information that will identify the problem. Contents are:

```
[pid           : 12345]
[username      : dpower]
[hostname      : cursa]
[date/time     : 2/25/2008 3:08]
[config file   : configuration file name]
[staging dir   : /data/pcm]
[launch dir    : launch_dir]
[DpLoad.pl     : script full path and name]
```

Assume that User_A launches DpLoad.pl against pcm.cfg above, then later User_B also launches DpLoad.pl against pcm_prod.cfg. The pcm_prod.cfg contains a line that is already running in the first pcm.cfg configuration file. This will trigger DpLoad.pl to inform User_B that there is a problem in that configuration file and it should be fixed before attempting again. DpLoad.pl will also print out the file lock to the screen, and it will also be output to the log file if **-log** is used.

Notes

DpLoad.pl will stop the execution of a configuration file if one or more lines share the same exact staging directory and same file extension.

This feature only works on lines that are reader or reader-like lines. In other words it works on readers/tools that run against data files. DpLoad.pl will ignore all the lines for UpStat or for other tools that do not work on data files.

Error Handling Standards in DpLoad.pl

How the Readers Handle Errors

All Exensio –Yield readers follow some standards in outputting error files and also the exit codes they use. The DpLoad.pl tool is designed in accordance with these standards.

DpLoad.pl also supports running “non-reader tools” such as preprocessors, which do not operate on data files. DpLoad.pl uses the extension field in its configuration file to distinguish between reader and non-reader tools. If the extension field is “NA”, then the configuration line is for a non-reader tool.

Exit Codes

Readers use certain exit codes for special file handling. These codes are reserved for Exensio –Yield readers only. It is highly recommended that you do not use these exit codes — **4, 9, 10, 11, 12** — in DpLoad.pl or other tools, other than as exit codes.

In DpLoad.pl, each of these codes will result in an unintended action. These actions are:

Exit Code	Reader Action
4	move data file to ReworkFiles directory
9	move file to UnusedFiles directory
10	keep the file in its current location
11	handle Informix deadlocks
12	exit code for non-reader tools

Do not use these exit codes when creating scripts, tools or pre-processors.

For non-reader tools, the special exit code is 12. This exit code means that the tool or preprocessor did not do anything. This is only useful in case of using database logging; this tells DpLoad.pl not to keep an entry in the DP_LOG table for this execution.

No errors

Even when script executes successfully, the reader creates an error file with the name **err.jnk** in the current working directory. This file has one line: two zeros separated by a tab — “0\t0”

If the reader finishes execution with this error file, the file is considered processed and will be moved to the **Processed** directory.

For non-reader tools, the error file is the tool name with a “.log” extension. Similar to readers, it is created in the current working directory. For example, if the tool name is **parse_pcm.pl**, the log file will be **parse_pcm.pl.log**.

DpLoad.pl checks if the first argument in the configuration file is in a valid and existing directory. If so, then the log file is created in it. If it is “NA,” the log file is created in the current directory from which DpLoad.pl was launched.

Errors & Warnings

When a reader/tool produces errors and/or warnings, these are logged into the **err.jnk** standard file. The **err.jnk** error file contents follow reader's standards.

For non-reader tools, error file is the log file.

First line

The first line has two values separated by a tab. For example: `1\t0, 2\t0, 3\t1` or `0\t2`, and so on. For a script or preprocessor it will usually be `1\t0`, meaning only one error.

Remaining lines

Starting from the second line, each line will look like:

```
1\t9001\tE\t01-29-2008\t04:33:35    unable to ...
in case of error line
```

OR

```
1\t9002\tW\t01-29-2008\t04:33:35    value truncated to...
In case of warning line.
```

The above fields are tab delimited. The first three fields are not meaningful and are always the same values, in case of errors. In case of warning, only the first field changes, indicating warning number 1, 2, 3... and so on.

The fourth and fifth fields are date and time. When using the DpLoad.pl file tracking function, only the third field of the first five fields is stored (E or W). The remaining four fields are truncated and the system only stores starting from the "unable to..." text. This is done because the date/time stamp will be an obstacle in storing unique error messages.

When developing a reader-like tool, if you don't store date/time, just put values such as "0" instead of date and "0" instead of time. For example, an error line for a preprocessor might look like the following:

```
1 9001 E 0 0    unable to open input data file ...
```

It is recommended that you follow one of the two methods below for the error lines:

```
1\t9001\tE\t01-29-2008\t04:33:35    unable to..."
1\t9001\tE\t0\t0\t                    unable to..."
```



If the error/warning line does not start with: "1\t9001\tE" or "#\t9002\tW" then the whole line will be stored as error text. In case of warnings, the "#" sign means a number 1 or greater.

No error file

Sometimes DpLoad.pl doesn't find the **err.jnk** error file. This will be handled one of two ways: (for non-reader tools, it is the log file)

- **Zero exit code** — This will be also considered a success and file is moved to **Processed**.
- **Non-zero exit code** — DpLoad.pl considers this as a core dump or abnormal error. The file is moved to **NotProcessed** and an error file is generated by DpLoad.pl that includes two lines:
 - `1\t0`
 - `1\t0\n1\t9001\tE\t0\t0\tAbnormal exit without an error file.\n"`

For non-reader tools, the behavior is the same. But, again, the system does not operate on files here.

Empty error file

This will be considered a core dump or abnormal error. The file is moved to **NotProcessed** and an error file is generated by DpLoad.pl that include two lines:

- `1\t0`
- `1\t0\n1\t9001\tE\t0\t0\tAbnormal exit with an empty error file.\n"`

For non-reader tools, the behavior is the same.

Non-standard error file

If the error/warning file produced by the reader/tool does not follow the standards, DpLoad.pl will not be able to determine how this data file need to be handled. In this case the file is moved to the **NotProcessed** directory and the **err.jnk** error file is prepended with information that will specify that the error file does not follow standards.

Warnings

In the above mentioned cases warnings came with errors. In some other cases, only warnings are generated.

Similar to the errors, if the reader/tool produces only warnings, an **err.jnk** file will be produced. The first line will look like "`0\t#`" where # will be the number of warnings. The warnings will be stored in the database in the same way as described above, except that a different code will be logged in the DP_LOG table to indicate warnings-only results. The the file will be moved to the **Processed** directory and the warning file will be moved to the **Warnings** directory. If the **-warn** DpLoad.pl option is used, the file and warning files are both moved to the **Warnings** directory.

Version The **-v** option provides DpLoad.pl and DpLoadMgr.pl version numbers. DpLoad.pl also has the **-V** option to show more version details.

General Note When Using Database Logging — All readers have a **-v** option that prints the reader option and exits. The output of this option has a line similar to this:

Version 8.0 - May 11, 2008.

Essentially, it has a “version” string. This is what DpLoad.pl looks for when using database logging, in order to log the tool version into the logging database. For non-reader tools, this standard also needs to be followed. In other words, adding a **-v** option has a result similar to readers.

EXENSIO –HOSTED UNIFIED SCHEMA — DATA LOADING AND INSERTION WITH DPLOAD

The following overview describes the path that data files follow, from initial input to parsing to database. In other words, the process of how data is loaded into the Exensio –Hosted Unified Schema environment — by auto-insertion, manual insertion, or data push/pull — to arrive at a specified data partition.

1. **Data Input** — Exensio –Hosted UI receives input files, with predefined parser and loading configuration options:
 - Pre-and post-parser configuration options are configured on-site by the customer.
 - Loading configuration command-line options including partition names/destinations.

2. **Data Parsing** — Exensio –Hosted UI launches the parser (using the same functionality as the legacy dataConductor system).

Parsers launch on a remote or local server, configured by personnel who sets up the installation. (**remote_execute_script.sh** is the template script used to build the customer-specific script that launches the parser remotely.)

3. **Parsed .res files delivered to mapped output directories** — The parser delivers the resulting .res files, using a map file (**dpLoad_map.cfg***) to determine where the parser will send the files. The parser produces multiple .res files for multiple data types (hard bin, soft bin, summary, etc.). The map file specifies the output directory where each output data type goes.

* Example dpLoad_map.cfg Map File —

Each line has the following general format:

```
ProgClassNum:DataType:OutputFileDirectory<app
server>:HostIP:OutputFileDirectory<parsing server>
```

```
1:sbin:/dcfilex/fa00100/datalogs/dbascii/sbin_1:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/sbin_1
1:hbin:/dcfilex/fa00100/datalogs/dbascii/hbin_1:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/hbin_1
2:sbin:/dcfilex/fa00100/datalogs/dbascii/sbin_2:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/sbin_2
2:hbin:/dcfilex/fa00100/datalogs/dbascii/hbin_2:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/hbin_2
3:ssum:/dcfilex/fa00100/datalogs/dbascii/ssum_3:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/ssum_3
3:hsum:/dcfilex/fa00100/datalogs/dbascii/hsum_3:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/hsum_3
5:sbin:/dcfilex/fa00100/datalogs/dbascii/sbin_5:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/sbin_5
5:hbin:/dcfilex/fa00100/datalogs/dbascii/hbin_5:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/hbin_5
12:tsum:/dcfilex/fa00100/datalogs/dbascii/tsum_12:172.31.0.14:/dcfilex/fa00100/
datalogs/dbascii/tsum_12
```

Typically, the **OutputFileDirectory** on both the app server and the parsing servers are identical, since they must be on the same mount point.

4. **Auto-insertion launches DpLoad** — The auto-insertion process creates a DpLoad config file that runs the dbascii data loader (data reader) and launches multiple instances of the **run_dpload.sh** script, one per data type. This script launches DpLoad on the remote loading server.

- To create the DpLoad config file, the auto-insert process reads in the map file (above) to learn where the .res files reside and on which host to run the DpLoad commands to dbascii.
- dbascii loading is done in parallel with the parsing, to optimize performance.

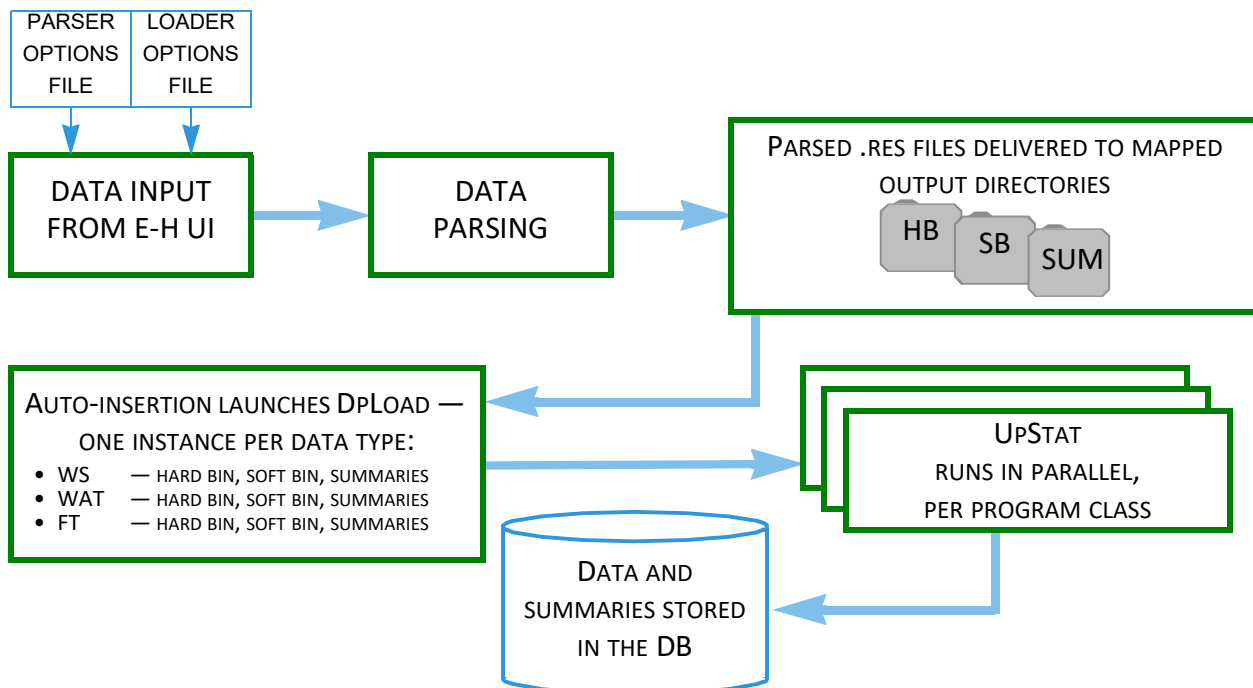
Parallel loading of data can be handled based on program class/data type:

- | | | |
|--|---|--|
| <ul style="list-style-type: none"> • Wafer Sort • hard bin • soft bin • summaries | <ul style="list-style-type: none"> • WAT • hard bin • soft bin • summaries | <ul style="list-style-type: none"> • Final Test • hard bin • soft bin • summaries |
|--|---|--|

Once the dbascii loading is complete, DpLoad is stopped and the logging is presented to the user (via the Exensio –HostedUI).

5. **UpStat instances run in parallel, per program class,** — UpStat instances will run independently, per program class, on the loading machines at the database level. This occurs regardless of how data files are loaded (manual insertion, auto-insertion or data push/pull). This means, for example, that three instances of UpStat — one per program class — could be run in parallel on three machines.

6. **Data and summaries are stored** — The data and the required summarization now resides in the database.



DpLoadMgr.pl

DpLoadMgr.pl is a shell that provides options to easily manage multiple DpLoad.pl processes. It allows the user to start, stop, kill and check the status of all DpLoad.pl processes. DpLoadMgr.pl requires its own config file (e.g. mgr.cfg).



DpLoadMgr.pl must reside in the same directory as DpLoad.pl.

Configuration File

The configuration file contains the name of the DpLoad.pl config files and DpLoad.pl command-line options, and an optional group name. For example:

```
pcm.cfg: -reverse -log -files 100
/ids/home/cfg/sort.cfg:
${IDSHOME}/dbscripts/load/defect.cfg: -log -once:group1
```

Usage Statement

The following is the usage statement of DpLoadMgr.pl:

Usage:

```
DpLoadMgr.pl -v
DpLoadMgr.pl -f <file> [-status | -start | -stop | -kill]
[-all]
```

Where:

```
-f          :followed by a config file for the DpLoadMgr.pl.
             Each line in the config file contains:
             <DpLoad.pl config file name>:<DpLoad.pl
             command-line options>
-status    :display the status of DpLoad.pl process(es).
-start     :start DpLoad.pl process(es).
-stop      :stop DpLoad.pl process(es). Wait until the
             current job (reader) is done.
-kill      :kill DpLoad.pl process(es). No waiting.
-all       :apply the -status,-start,-stop and -kill to all
             the DpLoad.pl processes. If not used, the
             user is prompted to select the desired
             DpLoad.pl process from a list.
```

Groups

Groups in the DpLoadMgr.pl file give users additional control over DpLoadMgr.pl actions. Rather than DpLoadMgr.pl actions being performed on either one entry or on all entries, with grouping, you can perform actions on a group of entries, depending on the group identifier field. The group identifier field is an optional field added at the end of each entry in the DpLoadMgr.pl file (see “[Configuration File](#),” above). When there are no groups, the behavior of DpLoadMgr.pl is unchanged. If groups are used, the user will be prompted to pick a group instead of being prompted to pick only one entry.



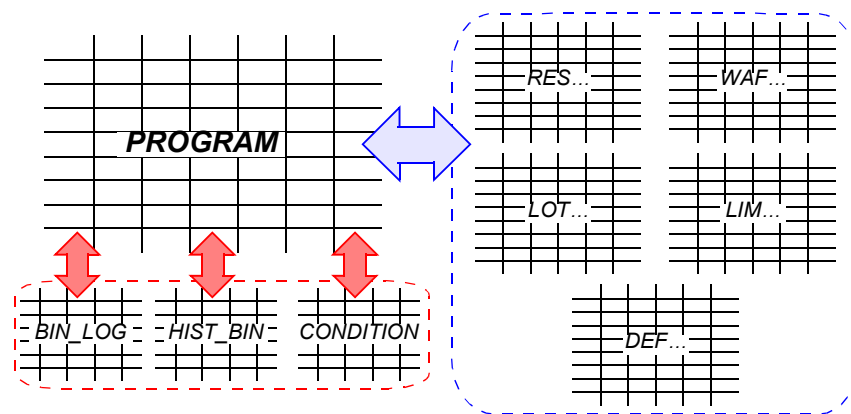
DATABASE READERS AND THE DATABASE MODEL

PROGRAMS

For the purposes of Exensio –Yield, the term “*program*” is defined as a **set of parametric tests which is stable over time and uses a constant set of test limits.**

The program is central in the design of Exensio –Yield. Each type of program generates an entry in the **PROGRAM** table in the database and generates five more tables: **RES...**, **WAF...**, **LOT...**, **LIM...** and **DEF...** (where “...” indicates that the table names are postfixed by the program’s serial key (**pg_key**)).

Additional database tables — **BIN_LOG**, **HIST_BIN** and **CONDITION** — are linked to **PROGRAM** to store binning and condition information.



Each program is associated with a fixed set of Exensio –Yield indexes and conditions which cannot be changed after program creation. This set is automatically generated by the data readers, according to a given format file.

All data files must contain a test program, and the test program must belong to a program class, as described in “*Program Class*,” pg. 35.

A new record entered into **PROGRAM** — which causes the instantiation of a set of the previously described tables — should describe a raw data source, which over time will:

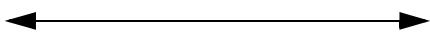
- Always produce the same set of tests ($t1 \dots tm$). (...or at least a reasonably big subset.)

A given entry in **PROGRAM** can be either of program type static, semi_dynamic or dynamic. In the static case (default), the format of previously generated tables **RES...**, etc. will never change. Thus, if for some reason, a future dlog file contains additional tests, they will be neglected. Semi_Dynamic is similar to static, but in addition the data readers automatically fill in fields in tables pointed to by **OP_LOG**, if available in the dlog files. Dynamic programs, in contrast, will automatically expand the tables to handle the additional tests. This however, is a computationally expensive operation.

Since this part of the data model is highly denormalized (for optimal performance, the bulk of the data is stored in the **RES...** tables) and is closely linked to the worksheet structure of the analysis front end, it is worthwhile to further explain the setup of the tables involved.

LIMITS

Test limits are stored in the database table, **LIM...** (depicted below) which is in essence a transformation of the Limit view in the worksheet.

<i>sbin_num</i>	<i>limit_name</i>	<i>t1</i>
<i>1</i>	<i>LSL</i>	 <i>(tot. 50 columns)</i>
<i>1</i>	<i>US</i>	
<i>1</i>	<i>L</i>	
	<i>LP</i>	

LIM... TABLE LAYOUT

Thus, for all pass bins, a complete set of limits can be stored for all tests *t1 ... tm* (m=50).

TESTER SUMMARY WORKSHEET STORAGE

Some test systems produce both datalog and summary worksheet data. Although the latter would logically belong to a run-level summary table (and in an aggregated stage to the **WAF...** and **LOT...** tables), this is not feasible for the following reasons:

- Summary worksheets typically have a different resolution than dlog files. E.g., leakage currents are measured on all pins, thus causing multiple entries (columns) in **RES...**, but result in only one entry in the tester summary worksheet.

- New releases of test programs very often have different test limits, thus must cause a new entry in **PROGRAM** and consequently instantiate a new set of tables (including **LIM...**). Only rarely do new releases contain additional tests.

As a result, tester summary worksheet information is typically stored as independent entries in **PROGRAM**. Corresponding PPID names will end with **..._SUM**. Since summary worksheet formats do not typically change between test program updates, PPID names do need to encode test program version and update level. For summaries, the **prog_type** field in the **PROGRAM** table should be set to *Dynamic*.

The resulting sum (summary) view when loaded into the Exensio –Yield analysis front end is then as follows:

Parameter	Units	mean	std.dev.	count	min	max
Lot	index					
Wafer	index					
TP_vers.	index					
Tested	pcs	1010	5	5	1000	1020
Passed	pcs	690	2	5	680	700
Tst_a	pcs	135	7	5	120	150
Tst_b	pcs	70	12	5	50	90
Tst_c	pcs	60	10	5	40	80
Tst_d	pcs	55	20	5	20	90

The shaded information is stored on a wafer, and lot basis, respectively, in **WAF...**, **LOT...** (in a transposed format).

This layout allows immediate meaningful use of box plots (yields per lot with distribution over the wafers), histograms (yield distribution), trend charts, etc.

BIN SUMMARIES

Bin summaries are maintained in the database in the **BIN_LOG** and **HIST_BIN** tables. The **BIN_LOG** table maintains binning summaries at lot and wafer, levels for all programs; while the **HIST_BIN** table maintains historical binning summaries at program level.

The bin names, bin numbers, pass/fail flags, as well as yield information is maintained in these tables.

m and d0

The **BIN_LOG** table maintains the yield model variables m , d_0 , y_1 , $\dots$, y_5 . The formula used to calculate m and d_0 is: $y = me^{-A\phi}$ where:

- m — cluster factor — multiplicative factor used to scale yield to the exponential curve fit. Based on y_1 - y_n , the expected yield if area = zero.
- d_0 — defect density — density of defects expected in an area of the wafer.
- y_1 — **(1x1)** — yield where the die group is 1x1.
- y_2 — **(2x1)** — yield where the die group is 2x1.
- y_3 — **(1x2)** — yield where the die group is 1x2.
- y_4 — **(2x2)** — yield where the die group is 2x2.
- y_5 — **(4x4)** — yield where the die group is 4x4.

As a prerequisite for these yield calculations, the total number of occurrences of the die groups must be greater than or equal to 20.

Wafer level $y_1 \dots y_5$, m , and d_0 are calculated by the readers, while the lot-level values are calculated by the UpStat process.

The information needed by the readers is an index or a test that includes the binning, and two indexes that have the die X and die Y information. For the ASCII reader, these indexes are declared using the functions **DbBinIndex**, **DbBinTest**, and **DbDieXYIndexes**.

For more information, see [“Specification for Calculating Cluster Factor and Defect Density \(\$m\$ and \$d_0\$ \)”](#), pg. 51.

SERVER TOOLS — DATABASE STORAGE OF TOOL INFORMATION

When a tool connects to the database, tool information is logged into database. Tool name and version are stored in the **DP_FUNC** table, and an execution time stamp is stored in the **DP_LOG** table. If a system is using the same tool but with different versions, then a maximum of two entries per day is logged to the **DP_LOG** table (for that tool).

dataBASE READERS

CONDITIONS AND INDEXES

Items that are declared as indexes can affect the binning, wafer, and lot database tables.

Binning

The **HIST_BIN** and **BIN_LOG** database tables are updated by a reader only if an index or a parameter exists that is identified as containing bin information. For binary readers, such as STDF and LTX, this is an automatic operation. For the ASCII reader, the database functions, **DbBinIndex** or **DbBinTest**, are necessary to identify such indexes. **The bin index should always be declared as an integer.**

The **BIN_LOG** table fields — $y_1, y_2, y_3, y_4, y_5, m, d_0$ — are filled by the data readers as long as the following criteria are true:

- an index is defined as containing the wafer ID
- an index is defined as containing the X-coordinates
- an index is defined as containing the Y coordinates

For the ASCII reader, the database function, **DbDieXYIndexes**, is necessary to identify XY information.

The ASCII, STDF3, and STDF4 readers fill the **BIN_LOG** table from summary data that includes binning information, where program class is specified as “Summary.”

For both STDF3, and STDF4 readers this is an automatic operation. For the ASCII reader, the built-in function, **DbBinSum**, is used to indicate the condition that is the bin number, and the tests which are reporting binning information. The ASCII reader can handle bin summaries that are percentages or total counts. For more information on this function, refer to the ASCII reader document.

Lots

The **LOT...** and **LOT** database tables are updated from a binary reader as long as the lot is defined in the file. For the ASCII reader, the database functions, **DbLot** or **vDbLot**, are necessary to identify the Lot ID. Lot ID should not be typically declared as an index.

The one exception, where Lot ID is declared as an index is in the LEH reader, primarily because that reader does not use the **OP_LOG** database table.

Wafers

The **WAF...** and **WAFER** database tables are updated by a reader only if an index or a parameter is identified as containing wafer information. For binary readers, such as STDF and LTX, this is an automatic operation.

For the ASCII reader, the database function, **DbWaferIndex**, is necessary to identify wafer information. **The wafer index should always be declared as a string.**

PROGRAM CLASS

A program can be related to a specific class. This only affects a new program. Existing programs determine the program class from the database. If a program is not related to a program class, no data for that program can be inserted in the database. This is equivalent to the **accept_data** flag (in the PROGRAM table) being OFF.

Some predefined classes also affect **ppid** in the PROGRAM table, summaries for example have **ppid** ending in **_sum**, and BinMap ending in **_ink**.

Thirty-two predefined classes are reserved by Exensio –Yield and may not be altered, except for renaming of a program class. New program classes may be added at any time. The following items may be configured through the use of program classes:

Data Tables

BIN_LOG HIST_BIN RES...LOT...WAF...LIM...

Any of these data tables can be included in the data insertion process or may be ignored by the readers for a particular program class.

Statistics

cnt	number of observations, screened data
avg	average, from screened data
stdev	standard deviation, from screened data
q1	1st quartile, from screened data
q2	median, from screened data
q3	3rd quartile, from screened data
p1	1st percentile point, from screened data
p5	5% point, from screened data
p10	10% point, from screened data
p90	90% point, from screened data
p95	95% point, from screened data
p99	99% point, from screened data
nlo	number of outliers left from median; non-screened data based on outliers (boxplot or database)

nuo	number of outliers right from median; non-screened data based on outliers (boxplot or database)
nls	number of data points below specification limits — Exensio –Hosted only
nus	number of data points above specification limits — Exensio –Hosted only
lol	lower outlier limit, used for outlier filtering
uol	upper outlier limit, used for outlier filtering
min	minimum value from screened data within outliers
max	maximum value from screened data within outliers
sum	sum from screened data
ss	sum of squares from screened data

Any of these statistics may be included/excluded for a particular program class. This default configuration may be redefined at the program level through the use of the **STATS_CFG** table flags.

LIMITS

There are eight types of limits:

LSLlow spec. limits
HSLhigh spec. limits
LPLlow production limits
HPLhigh production limits
LOLlow outlier limits
HOLhigh outlier limits
LWLlow what if limits
HWLhigh what if limits

Limits may exist as part of the data files or as separate files. Limits that are part of the data files are inserted in the database only when a new program is created. Limits, in these cases, are not updated for subsequent processing of data files belonging to the same program.

In the case of the ASCII and Fab readers, limits may also exist as separate files and these files may be used to update limits for existing programs using the **-limitonly** command line argument. For more information on this functionality, refer to the ASCII and Fab reader chapters.

One program in the database can have associated with it, more than one set of limits. The distinction between each set is a date/time stamp stored in the **limit_date** column in the **LIM...** database table can only be maintained by the ASCII reader and the Fab reader using the **-limitonly** option and optionally, the built-in function **DbHistLimits**.

If the **-limitsonly** option is used without the **DbHistLimits** built-in function, the date stamp for the current limits is updated to the current date (today) and the new limits are inserted as the current limits (date stamp 01/01/2500). The current limits are those used as the default limits for retrieval into a worksheet.

The **DbHistLimits** built-in function may be used with the **-limitsonly** option to insert new limits in one of the following two ways:

- Updating current limits only (update, no insertion). This option is achieved by supplying a date to the **DbHistLimits** function that exceeds the current date (today).
- Inserting new limits with a specific date stamp. This option is achieved by supplying a date that is less than the current date (today). (In this case, the current limits in the database are set to the limits with the greatest date.)

The **DbUpdateLimits** built-in function in the ASCII reader allows for the update of a partial set of limits. In the previous behavior, limit sets were updated by a delete-then-insert operation. This built-in function replaces the delete/insert operation with an actual update, where tests with no limits in the current data file maintain whatever limits these tests have in the database. In other words, only those tests that have valid limits in the data file are updated.

For more detailed information on the ASCII reader, refer to [“ASCII Historical Limits,”](#) pg. 66. For more information on the Fab reader, refer to [“FAB Data Reader,”](#) pg. 215.

DYNAMIC PROGRAMS

All readers accept the command line argument “**-dynamic**,” indicating a dynamic program type. For dynamic programs, the Results, Summary, and Limits tables may have to be dynamically altered to handle new parameters that did not exist at the time the program was created.

Altering tables with database transaction logging requires log-space that is at least twice as large as the sum of the sizes of all tables being altered.

When a data reader detects that not enough log-space is available for altering tables, the readers run their own operation, similar to a table alter, that requires a relatively small amount of log-space.

This operation consists of copying the old **RES...**, **LOT...**, and **WAF...** tables to new tables with the extra parameters and dropping the old tables.

The reader does this only when necessary. The **LIM...** table is still altered and the assumption is that there is always have enough log-space for this table to be altered in the normal way.

When this operation is performed, the reader does not insert any data from the file. It simply alters the tables and exits with code 10, which tells the DpLoad.pl script ([“Automated Data Loading — DpLoad.pl,”](#) pg. 12) to keep the file in place for another pass, at which time all the tables will be of the correct size.

REWORK

Definitions

Rework — an entire lot (e.g., ESort and Final Test) or several, or all wafers out of a lot (e.g. ESort) are retested again.

rework_action — database field in **PROGRAM** table defining what activity is to be performed in case of rework for a given program.

Rework action may be set for all readers from the command line using the option **-rework_action**. For the ASCII reader, the function **DbReworkAction** also may be used to set **rework_action**. The command line argument and the function **DbReworkAction** only affect new programs. For an existing program, **rework_action** is determined from the database.

rework_flag — database field in **OP_LOG** (lot level) or **WF_LOG** (wafer level) indicating if rework has occurred.

According to rework action for a particular program, the data readers will either dynamically try to detect rework or rework may be forced from the data file. With the ASCII reader, this may be accomplished using the built-in function, **DbRework**.

Rework is detected if:

- wafer-oriented data — **Lot ID**, and **Wafer ID** already exist in the database for the same program
- final test data — Lot ID already exists in the database for the same program

rework_flag in **OP_LOG** / **WF_LOG** is set in the following way:

- **0** — no rework
- **1** — latest reworked data
- **n** — oldest rework data

The following are the possible values for **rework_action**:

- **1** — no action
- **2** — append and mark
- **3** — overwrite
- **4** — load no data

When rework action is “do not load” the data readers exit with code “4” and the DpLoad.pl script will move reworked files to the **ReworkFiles** directory. (See [“Automated Data Loading — DpLoad.pl,”](#) pg. 12.)

Start-time/ End-time

For the ASCII reader, detection of rework is by default independent of both start-time and end-time, as supplied from the data file. Two command-line arguments, **-start_time** and **-end_time**, allow the user to make rework detection dependent on time.

As an example using start-time, two cases exist:

- **Start-time in the data file equal to start-time in the database (in addition to wafer, lot, program)** — In this case, the reader considers the new data file as a reload of the same data. In which case, the old data is deleted and is replaced by the new data file with no change in the value of the **rework_flag**.
- **Start-time in the data file is different from start-time in the database** — In this case, the **rework_flag** is calculated based on start-time, so that the latest **rework_flag** value, (1) - as an example, corresponds with the latest date.

UNIT SCALING

All readers will scale data according to the scale factors stored in the database. Scale factors are inserted in the database when a new program is detected.

For the ASCII reader, scale factors may be set using the built-in function, **ScaleFactor**.

BIN MAP DATA

Bin map data is stored in the database in a unique format. Typically, for most other data types, columns in the **RES...** table are the parameters, while the rows of the **RES...** table are the devices. In the case of bin map data there is essentially only one result, which is the bin number.

For efficient storage and retrieval of this type of data, X and Y coordinates are the key conditions that define the **RES...** table columns, while the Wafer ID defines the rows. All format files for bin map data should use this configuration.

The “binmap” (4) program class includes the **BIN_LOG** table for storing bin summaries and yield information. For the **BIN_LOG** table to be successfully updated, Wafer ID as well as X and Y coordinates must exist and be identified. This is an automatic operation for binary readers. For the ASCII reader it is necessary to use the built-in functions, **DbWaferIndex** and **DbDieXYIndexes**.

Die Sampling

Die sampling is automatically implemented for wafers containing 32K - 512K die, depending on the database platform. In order to allow for loading bin map data in excess of 40,000 die per wafer, die sampling is used to maintain a die count lower than 10,000 in the raw data. The bin statistics are still calculated using all the data.

Informix If the **-nosampling** options is not used, sampling starts at 10,000 die.

Then, sampling rate is determined by the following rules:

- If # die is less than 40,000 Then the sample rate is 2.
- If # die is less than 90,000 Then the sample rate is 3.
- If # die is less than 160,000 Then the sample rate is 4.
- If # die is less than 250,000 Then the sample rate is 5.

Oracle If the **-nosampling** options is not used, sampling starts at 32,000 die

The sampling rate is determined by the following rules:

- If # die is less than 128,000 Then the sample rate is 2.
- If # die is less than 288,000 Then the sample rate is 3.
- If # die is less than 512,000 Then the sample rate is 4.

No Sampling It is possible to load *binMAP* data with no sampling. When the dbascii reader command line option, **-nosampling**, is used, the reader will not perform any sampling, and more than 10,000 tests can be loaded for Informix databases or more than 32,000 for Oracle databases. See “**-nosampling,**” pg. 132.

SITE-LEVEL ALIGNMENT

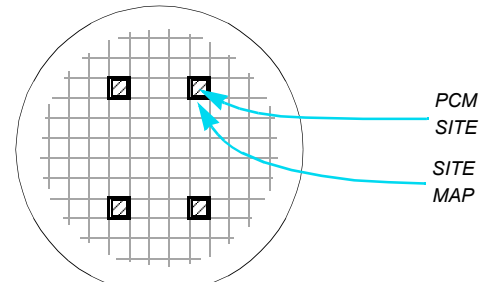
The ASCII reader has the ability to create an association between a site number (location on a wafer) and an area on the wafer that encompasses a predefined number of surrounding die. The purpose of this association is to allow for the automatic alignment of data at site level with data at dieX/Y level. The prime example being alignment of PCM data with wafer sort data.

This may be accomplished using the ASCII reader during the loading of PCM data. The requirements are the existence in the data set of a wafer configuration, a site index (defined using the DbSiteIndex built-in function), and DieX and DieY indexes (defined using the DbDieXYIndexes built-in function). The supplied DieX and DieY indexes map the site to a particular device on the wafer. This mapping is, in general, only an approximation used to define a larger area (set of surrounding die) to be aligned to the site.

The built-in function, DbDieMap, tells the ASCII reader to perform this automatic alignment using one of a set of predefined areas, as in the following list:

- **single** — site map 1.

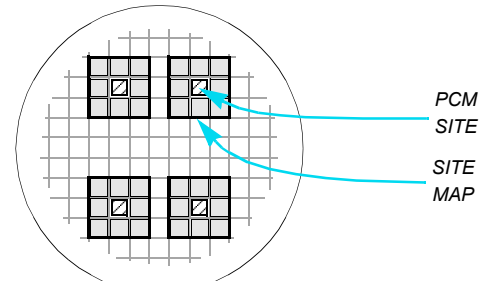
In this case the surrounding area is the die itself.



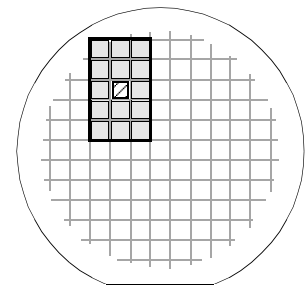
- **rectangle 3X3** (average) — site map 2.

In this case the surrounding area is a 3x3 rectangle around the site, with average used as the statistic for collapsing the results of the nine devices in this area to one representative number in

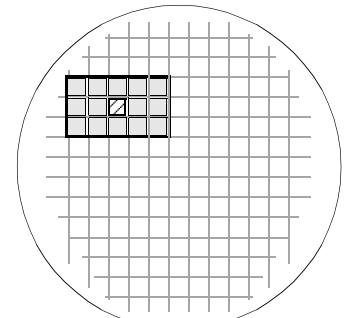
the case where automatic alignment of die-level data to site-level data is performed. In this case, the number 2 is passed as the argument to the built-in function, DbDieMap.



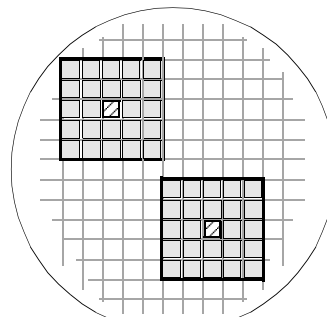
- **rectangle 3X5** (average) — site map 3.



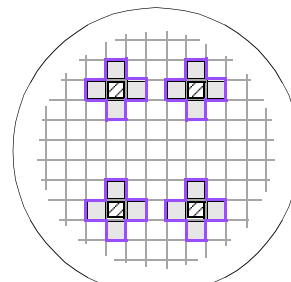
- **rectangle 5X3** (average) — site map 4.



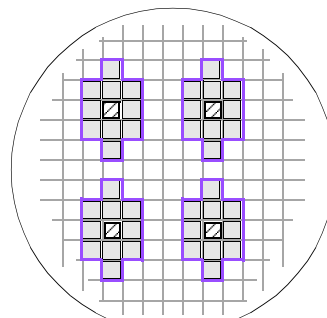
- **rectangle 5X5** (average) — site map 5.
- **rectangle 5X5** (median) — site map 6.



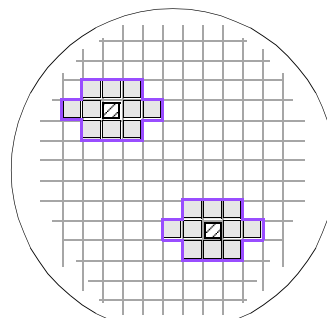
- **ellipse 3X3** (average) — site map 7



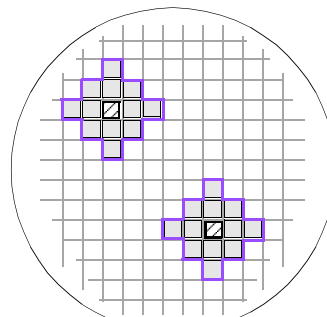
- **ellipse 3X5** (average) — site map 8



- **ellipse — 5X3** (average) — site map 9



- **diamond 5X5** (average) — site map 10
- **diamond 5X5** (median) — site map 11



OUTLIER FILTERING

Outliers may be excluded from the calculation of the summaries (lot and wafer level) based on database outlier limits or on BoxPlot criteria. The database outlier limit filtering option is invoked from the command line (for all readers) using the argument “**-dboutliers**”.

The BoxPlot filtering is invoked using the argument “**-outliers ni**” — where the lower and upper limits are:

- $\text{lowerlimit} = q1 - ni * (q3 - q1)$
- $\text{upperlimit} = q3 + ni * (q3 - q1)$

These limits are used in case the data file has at least 24 data points for any particular parameter. If the number of data points is less than 24, then the spec limits are used to calculate the outlier limits, based on the following formulas:

- $\text{lowerlimit} = \text{LSL} - ni * (\text{USL} - \text{LSL})$
- $\text{upperlimit} = \text{USL} + ni * (\text{USL} - \text{LSL})$

In case the number of data points is less than 24 and there are no Spec limits available in the database, outlier filtering is not possible.



All data points between -1e-38 and 1e-38 are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.

Normalization and Alignment Concepts

MEANING OF DATA NORMALIZATION

Whenever data is “normalized,” this implies the following:

- center die has index (0,0)
- positive X is right
- positive Y is up
- notch orientation matches the global database orientation chosen by user at the time the database was created

These criteria mean that when a reader is performing normalization on a data set, the reader might be forced to shift, re-index and/or rotate data.

- Defect data normalization is forced by the defect reader.
- Bitmap data normalization is forced by the bitmap reader.
- Binmap data normalization (and any data type with diex, diey indexes) is optional and is invoked by the dbascii **-normalize** command-line option or the **DbNormalize** built-in function.

The direct benefit of having normalized data in a database is the ability to quickly and easily overlay different data types and deduce statistical relationships based on the raw data.

ALIGNMENT

Binmap data alignment with defect data is optional and is invoked by the dbascii **-dfalign** command-line option or the **DbDefect** built-in function in dbascii. If a Binmap program is aligned, then the dbascii reader automatically forces it to be normalized.

Aligning a Binmap program to a defect program allows UpStat to precompute kill ratio statistics for that pair. Because for the same product, one defect program is typically created while multiple Binmap programs might exist, the user would have the choice to normalize all of the Binmap programs, but can align only one.

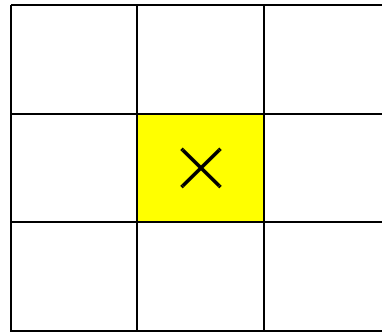
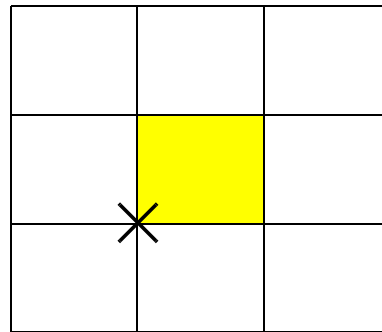
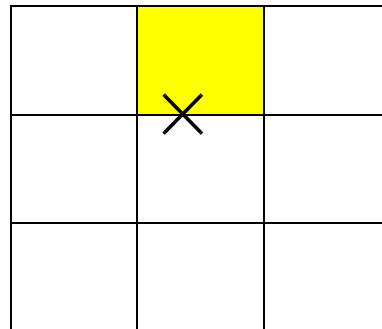
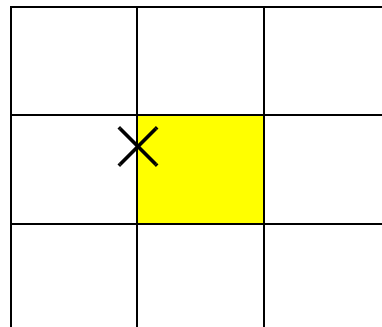
If UpStat finds multiple binmap programs aligned to one defect program, it returns an error.

Similarly, the bmupstat utility makes use of the fact that defect and bitmap data are normalized, and it precomputes various statistics that relate these two closely related data types.

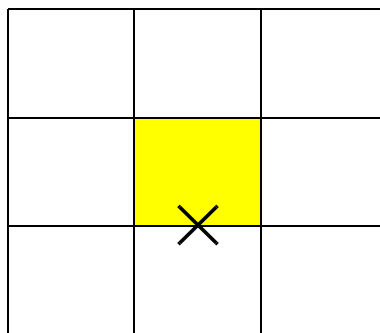
WHAT IS A CENTER DIE?

The center die is the die that contains the physical wafer center. If the wafer center lies on a die edge, then (after normalization) the lower and left edges are considered part of the die, while the upper and right edges are considered part of the neighboring die.

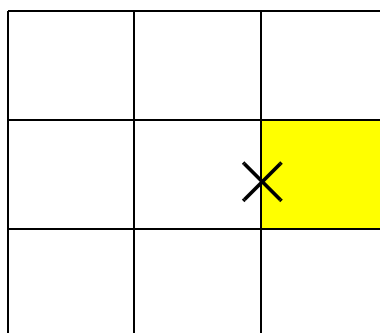
Here are some examples, with **X** marking the physical wafer center and the colored die indicating that the die is the center die (0,0):

Example A**Example B****Example C****Example D**

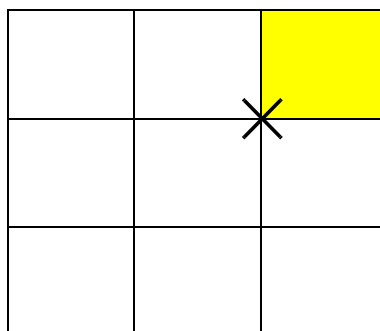
Example E



Example F



Example G



HOW THE DEFECT READER DETERMINES CENTER DIE

Using the 8th and 9th arguments in defect reader built-in function **vDbWmapCfg()** (*“vDbWmapCfg (...)”*, pg. 336), the format file passes to the reader the vector from the wafer center to lower left corner of the die (0,0) as it is defined in the file. Obviously, if the dimensions of this vector are less than the die dimensions, then the wafer center does physically lie in die (0,0) and thus no shifting is done by the reader. If not, the reader will shift the die indexes so that the vector from the wafer center to lower left corner of the die is less than the die dimensions and negative in value. Due to measurement tool inaccuracies, and in the case where the wafer center is within 200 microns of the die edge, the reader will force the wafer center to the nearest edge.

Taking KLARF data as an example (*“defectMAP Format File — Overview,”* pg. 347), the following fields are the key to determining the parameters of the wafer configuration:

SampleSize — Determines wafer diameter

SampleOrientationMarkType — Determines the flat type

OrientationMarkLocation — Determines testing orientation

DiePitch — Determines die dimensions

SampleCenterLocation — Determines distance from lower left corner of die to wafer center

STATISTICS UPDATE PROCESS — UPSTAT

UpStat is a program that updates lot level database statistics (**LOT...** database tables) for Exensio –Yield. The need for such a process arises because certain lot level statistics (q1, q2 and q3) can only be calculated from all the raw data belonging to a certain lot. This data is not available to the data readers when lots are split over multiple files.

UpStat also calculates the lot level yield per die group parameters ($m, d_0, y_1, y_2, y_3, y_4$ and y_5) in the **BIN_LOG** table, based on the wafer level parameters calculated by the data readers.

The **UPDATE_STATS** database table indicates which lots require updating, one row per lot. Rows are added to this table:

1. ...by the various data readers when more than one data file is encountered for the same lot.
2. ...when a program mask and/or lot mask are provided to UpStat on the command line.
3. ...when the lot read by the data readers has a source lot (parent lot). A row per source lot will be added to the table.

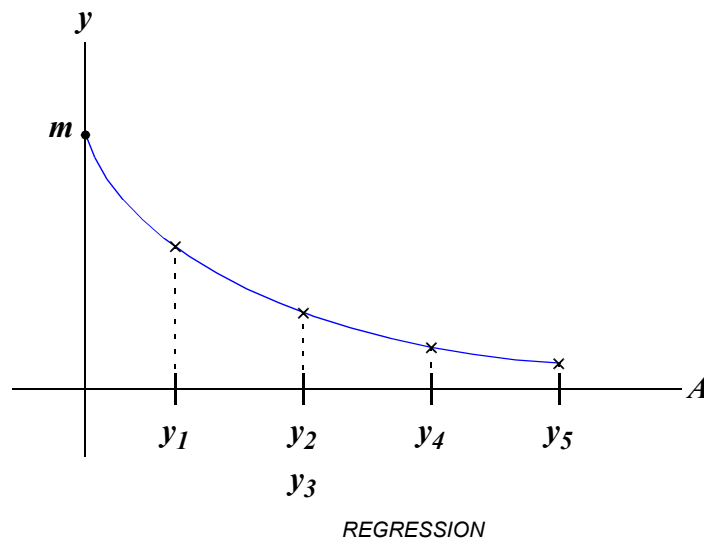
For more detailed information on UpStat, refer to the “*Statistics Update Process — UpStat*” chapter of the Exensio –Yield Administrator Tools manual.

SPECIFICATION FOR CALCULATING CLUSTER FACTOR AND DEFECT DENSITY (m AND d_0)

The equation for calculating m and d_0 as used within Exensio –Yield is:

$$y = m e^{-A d_0}$$

Yield (y) is the percentage of good die out of the theoretical maximum on a wafer; A is the area of the die; d_0 is the defect density; m is usually known as the area of usage or the *cluster factor*, the part of the yield loss that is not accounted for by random defects.



Regression Calculation

To calculate m and d_0 , a non-linear regression is performed on bin map data (X and Y coordinates for each die along with a pass/fail indicator). Since multiple measurements are required for any regression, the wafer is subdivided into groups of die to obtain multiple measurements on yield (y) for different die areas (A).

The algorithm begins by subdividing the wafer into groups of four die in a 2 x 2 arrangement. Since there are several ways in which this may be done; the algorithm uses the tiling that produces the largest number of 2 x 2 groups. Then yield is calculated: a group of four die “fails” if any of the die in the group fail. The factor A for this yield is 4. The 2 x 2 tiling will not use all the die making up the wafer. To avoid biased estimates, further subdivisions are performed only on the die within 2 x 2 tilings, not on all the die within the original wafer.

Next, 2 x 1 and 1 x 2 subdivisions are performed on the die within the 2 x 2 tiling. Yields are calculated with an area factor of 2. Finally, the same function is performed for the 1 x 1 subdivision (only one tiling is possible) with an area factor of 1.

If at least 20 groupings of 4 x 4 die can be found, then the algorithm will begin with a 4 x 4 tiling of groups of 16 die, using the same fail criteria. Then the algorithm performs the 2 x 2 grouping, as explained. If a 4 x 4 tiling is possible, the regression calculation will be performed with 5 points, rather than 4.

Now there are four or five (y_i, A_i) pairs for the non-linear regression. ($i = 1, 2, \dots, 5$.) The Gauss-Newton method with derivative is used to minimize the sum squared residuals for the yield equation with respect to m and d_0 .

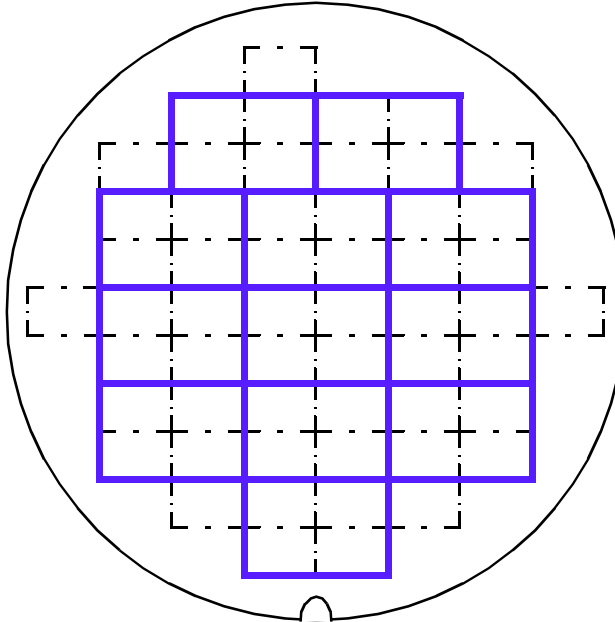


FIGURE 1

The algorithm performs the 2 x 2 tiling, as illustrated above. There are several ways this may be done (starting at the upper left corner, one die to the left, one die down). The algorithm selects the tiling (as highlighted in the illustration) that results in the largest number of 2 x 2 squares. Here the yield is calculated for these 12 squares. A square fails if any of the 4 die within it fail. This results in an (x, y) pair for the non-linear regression with $x = 4$ and $y = \text{yield } (\%)$.

The 2 x 2 tiling defines a new grid of die which is then further subdivided into 2 x 1 and 1 x 2 squares:

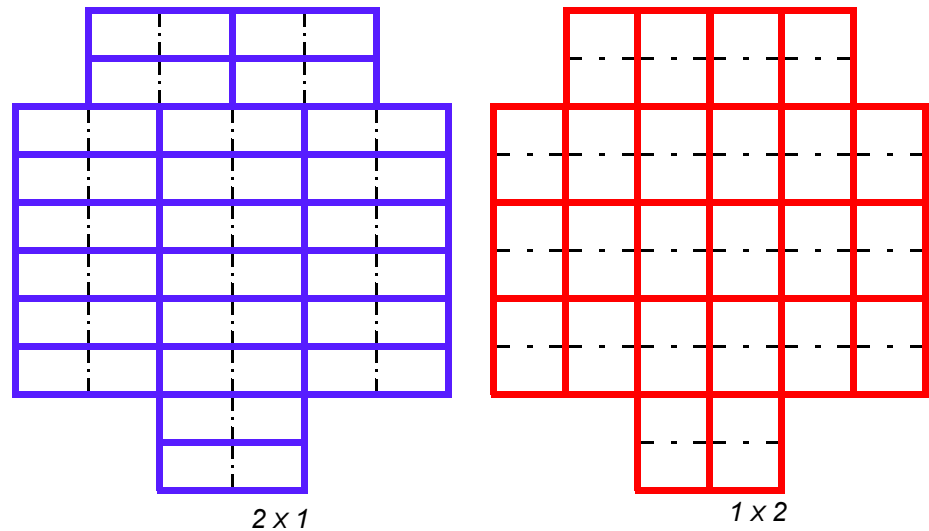


FIGURE 2

These form ($x = 2, y = \text{yield}(\%)$) pairs for the non-linear regression, with a 2 x 1 or 1 x 2 grouping counted as a “fail” if either die within it fails. The subdivisions may be performed in several ways, just as in the 2 x 2 algorithm.

Finally, a yield % is computed (as with the 48 die in Figure 2). This provides an ($x = 1, y = \text{yield}(\%)$) pair for the non-linear regression.

The non-linear regression equation:

$$y = me^{-Ad_0}$$

$$(x = A, y = \text{yield})$$

is then solved for m and d_0 using the four or five (x, y) pairs obtained.

CHAPTER 2

ASCII DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file. These readers can access data directly from files or through a database using the Exensio interface. Whether the data is imported through the database or directly from a data file, the objective is the same: to access your data and format it properly for comprehensive analysis.

To accommodate these two primary means of importing data, there are two types of data readers. Database readers, the primary means of importing data into Exensio, will import the formatted data into the Informix, Oracle or Cassandra database for access through the Exensio data retrieval system. Worksheet readers import the formatted data directly into the Exensio –Yield environment. Database readers operate in the background, as part of the overall process of configuring importing data from the database. Worksheet reader sessions are configured and implemented from a reader-specific interface that is called by the user when it is time to load a new raw data file.

For a general overview of how Exensio readers function, refer to “[Exensio Data Readers — Overview](#),” pg. 9.



The dbascii executable accepts lot IDs and wafer IDs that start with "NA."



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

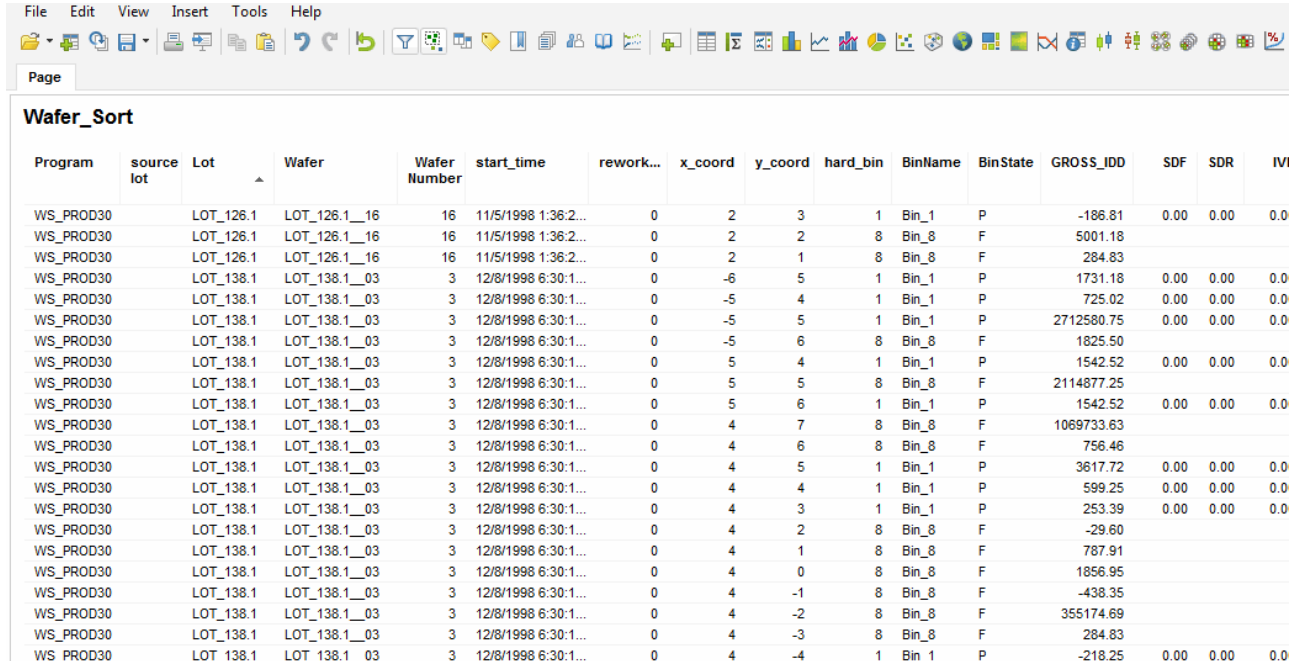
ASCII FORMAT FILE — OVERVIEW

The Exensio –Yield format file is a way to specify the ASCII format in which the data will be expected from an incoming source (usually a disk file). The format file is written in an easy to use script language specifically designed to navigate through an ASCII file and extract all relevant information, storing it in a pre-defined database. The language has a set of keywords (*“Keywords,”* pg. 61) and built-in functions (*“Built-in Functions,”* pg. 76) and it allows for user-defined variables, constants and separators. Blanks, horizontal and vertical tabs, new-lines, form-feeds and comments as described below (collectively referred to as “white space”) serve only to separate tokens.

OBJECTIVE

The objective of the ASCII reader is to import data into a Exensio –Yield database so it can then be retrieved into a data table. It is organized it into a standard form, regardless of how the data is organized in the original ASCII file.

Following is an example of a raw data table in the Exensio environment.



The screenshot shows the Exensio software interface with a menu bar (File, Edit, View, Insert, Tools, Help) and a toolbar. Below the toolbar is a 'Page' tab. The main window displays a table titled 'Wafer_Sort'. The table has 16 columns: Program, source lot, Lot, Wafer, Wafer Number, start_time, rework..., x_coord, y_coord, hard_bin, BinName, BinState, GROSS_IDD, SDF, SDR, and IVI. The data rows show various wafer information, including program names like WS_PROD30, lot numbers, wafer numbers, start times, coordinates, bin names, and various numerical values.

Program	source lot	Lot	Wafer	Wafer Number	start_time	rework...	x_coord	y_coord	hard_bin	BinName	BinState	GROSS_IDD	SDF	SDR	IVI
WS_PROD30		LOT_126.1	LOT_126.1__16	16	11/5/1998 1:36:2...	0	2	3	1	Bin_1	P	-186.81	0.00	0.00	0.0
WS_PROD30		LOT_126.1	LOT_126.1__16	16	11/5/1998 1:36:2...	0	2	2	8	Bin_8	F	5001.18			
WS_PROD30		LOT_126.1	LOT_126.1__16	16	11/5/1998 1:36:2...	0	2	1	8	Bin_8	F	284.83			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	-6	5	1	Bin_1	P	1731.18	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	-5	4	1	Bin_1	P	725.02	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	-5	5	1	Bin_1	P	2712580.75	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	-5	6	8	Bin_8	F	1825.50			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	5	4	1	Bin_1	P	1542.52	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	5	5	8	Bin_8	F	2114877.25			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	5	6	1	Bin_1	P	1542.52	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	7	8	Bin_8	F	1069733.63			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	6	8	Bin_8	F	756.46			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	5	1	Bin_1	P	3617.72	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	4	1	Bin_1	P	599.25	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	3	1	Bin_1	P	253.39	0.00	0.00	0.0
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	2	8	Bin_8	F	-29.60			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	1	8	Bin_8	F	787.91			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	0	8	Bin_8	F	1856.95			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	-1	8	Bin_8	F	-438.35			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	-2	8	Bin_8	F	355174.69			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	-3	8	Bin_8	F	284.83			
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	4	-4	1	Bin_1	P	-218.25	0.00	0.00	0.0

EXENSIO RAW DATA TABLE

Add Additional Results to Existing Programs

The dbascii reader supports the addition and removal of new result types from a format file even after the program has been created.

Similar to creating a new program for the first time, if an existing program with one result type exists in a database, and you introduce a new result type into the format file, the first time you load the file with the new results, the data itself does not get loaded. Only the program structure is modified. The file itself needs to be loaded again. This is automatically taken care of in DpLoad.pl ([“Automated Data Loading — DpLoad.pl,”](#) pg. 12).

- The dbascii reader will give an error if any change to the conditions section in the format file (string/real or key/nonkey) is introduced after the program has been created.

Maximum Number of Parameters

The dbascii reader can accommodate the following number of parameters:

- **Cassandra:** The maximum number of parameters is that can be handled is 10 million.
- **Oracle:** The maximum number of parallel tables that can be handled is 225, which translates to exactly 202,500 parameters.
- **Informix:** The maximum number of parallel tables is 14, which translates to approximately 113,000 parameters – depending on the parameter data types and the number and data types of indexes in the program.

For Informix, the usage statement of dbascii uses the following argument to accommodate large numbers of tables/parameters:

-extend_par: Allow up to 14 parallel tables for a single program.

Maximum Number of Indexes

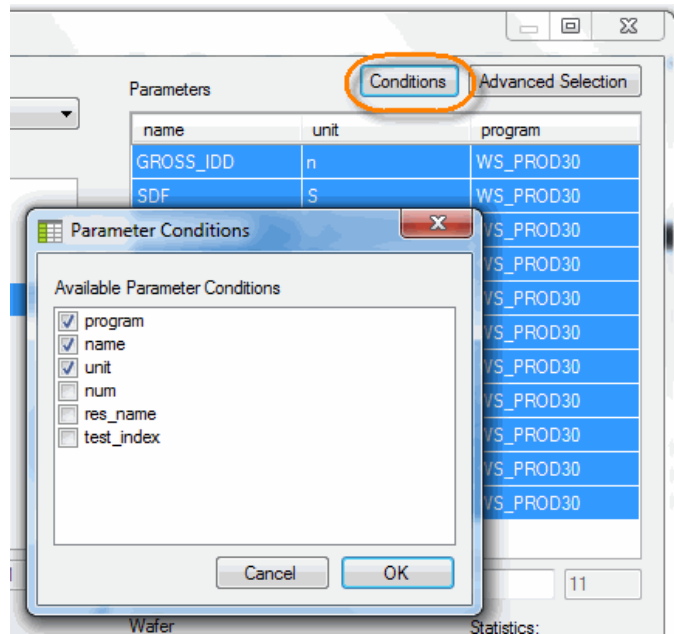
The dbascii reader can accommodate the up to 200 indexes.

Conditions and Indexes — Definitions

conditions

In a Exensio/Exensio –Yield data table — the column headings in the raw data table (typically program, test/parameter name, etc.). These conditions are the identifiers for parameters that uniquely identify each column.

In an Exensio data table — conditions are used in database retrieval to define the retrieved data set, but they are not displayed in the data table by default. To view a program's conditions from the GUI retrieval window, the user clicks on the **Conditions** button above the **Parameters** list box and from the Parameter Conditions dialog selects the conditions to be included in the data set.



key conditions In an Exensio –Yield data table — the group of conditions that uniquely identify the data column. The combination of all key conditions for any particular data column will be unique to that column.

In an Exensio data table — the user can click on the **Advanced Selection** button to open the Advanced Parameter Selection dialog and then select the **Include key condition values in parameter names** option to include conditions in parameter column headings.

indexes In an Exensio –Yield data table — the blue columns in the raw data table. Each row in the Data Table represents a device or portion of a device that uses a “tag” or index. The indexes of a data row contain information about the device, such as lot, wafer, part number or X/Y coordinates for wafer mapping.

In an Exensio data table — the indexes are the “identifier” columns to the left of the parameter columns: Program, Lot, Wafer, etc.

key indexes In Exensio –Yield and Exensio data tables — the group of indexes that uniquely identify the data row. The combination of all key indexes for any particular data row will be unique to that row. Each unique set of index values (i.e. program WSPROD30, LOT_24.1, wafer LOT_24.1__05, X/Y coordinates, etc...) make up the unique index combination that identifies a unique data row.

Limits — associated with the data table is a limits table of the types shown below. When used, limits are applied per parameter, usually of types **Production**, **Specification**, **Outlier** and **What-If**. User-defined limits can also be created, per parameter.

Page

Wafer_Sort

Program	source lot	Lot	Wafer	Wafer Number	start_time	rework...	x_coord	y_coord	hard_bin	BinName	BinState	GROSS_IDD	SDF
WS_PROD30		LOT_126.1	LOT_126.1__16	16	11/5/1998 1:36:2...	0	2	3	1	Bin_1	P	-186.81	0.00
WS_PROD30		LOT_126.1	LOT_126.1__16	16	11/5/1998 1:36:2...	0	2	2	8	Bin_8	F	5001.18	
WS_												284.83	
WS_												1731.18	0.00
WS_												725.02	0.00
WS_												2712580.75	0.00
WS_												1825.50	
WS_												1542.52	0.00
WS_												2114877.25	
WS_												1542.52	0.00
WS_												1069733.63	
WS_												756.46	
WS_												3617.72	0.00
WS_												599.25	0.00
WS_												253.39	0.00
WS_												-29.60	
WS_												787.91	
WS_												1856.95	
WS_												-438.35	
WS_												355174.69	
WS_												284.83	
WS_												-218.25	0.00
WS_												1699.73	
WS_												260124.45	
WS_												2321713.25	
WS_												6891647.00	
WS_												1448.19	0.00
WS_												1.85	0.00
WS_												1008.00	0.00
WS_												1102.33	0.00
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	3	-1	1	Bin_1	P	442.04	0.00
WS_PROD30		LOT_138.1	LOT_138.1__03	3	12/8/1998 6:30:1...	0	3	0	8	Bin_8	F	1102.33	

Manage Limits and Outliers

Available Limit Sets

Name	Type	Limit Count
Production	DataBase	11
Specification	DataBase	11
Outlier	DataBase	11
What-If	DataBase	11

Columns and Limit Values:

Type to search

Column	Lower	Target	Upper
GROSS_IDD	-101211.09375	NaN	204314.25
SDF	3.82495066730826E-08	NaN	4.12231493385207E-08
SDR	8.86242688125094E-08	NaN	9.58459480671081E-08
IVF	6.87499968066163E-09	NaN	7.57812568252803E-09
ORF	3.84465259628541E-08	NaN	4.14627479017327E-08
IVR	1.1640625352527E-08	NaN	1.25781252080515E-08
ORR	8.87891360434878E-08	NaN	9.64089608146423E-08

By Columns ☒ By Expression ☐

OK Cancel Reset

EXENSIO LIMITS TABLE

limit conditions — limit conditions are the group of conditions that uniquely identify a row in the limits table. The combination of all limit conditions for any particular row in the limits table will be unique to that row.

Every column (parameter) in the data table could possibly have limits associated with it, and therefore a corresponding row in the limits sheet. Limit conditions determine which parameters are to appear as new rows in the limits sheet.

The preceding two tables were created from the following ASCII file, where Pin and Chan were chosen as key conditions, Test Number as a limit condition and Unit and Test Name as conditions. Lot, Wafer and Device were all chosen as key indexes.

Program : RAM4a
LOT# : u08607

WAFER# : 1
Device# : 1

Test#:1000 opens Shorts
PIN NAME CHAN VMIN VMAX UNIT ACTUAL

XIN 98 -1.6 -0.2 v -0.5368
MCK 50 -1.6 -0.2 v -0.5712

```

RESETB 16 -1.6      -0.2    v -0.4329
UCSB   230 -1.6     -0.2    v -0.6191

```

```

WAFER#: 1
Device#: 2

```

```

Test#:1000 opens_Short
PIN NAME CHANVMIN  VMAX  UNITACTUAL
-----
XIN      98 -1.6      -0.2    v -0.4464
MCK      50 -1.6      -0.2    v -0.3744
RESETB   16 -1.6      -0.2    v -0.5294
UCSB    230 -1.6     -0.2    v -0.2917

```

```

WAFER#: 1
Device#: 3

```

```

Test#:1000 opens_Short
PIN NAME CHANVMIN  VMAX  UNITACTUAL
-----
XIN      98 -1.6      -0.2    v -0.5618
MCK      50 -1.6      -0.2    v -0.6282
RESETB   16 -1.6      -0.2    v -0.5731
UCSB    230 -1.6     -0.2    v -0.6112

```

```

WAFER#: 2
Device#: 1

```

```

Test#:1000 opens_Short
PIN NAME CHANVMIN  VMAX  UNITACTUAL
-----
XIN      98 -1.6      -0.2    v -0.6145
MCK      50 -1.6      -0.2    v -0.5467
RESETB   16 -1.6      -0.2    v -0.5880
UCSB    230 -1.6     -0.2    v -0.4543

```

```

WAFER#: 2
Device#: 2

```

```

Test#:1000 opens_Short
PIN NAME CHANVMIN  VMAX  UNITACTUAL
-----
XIN      98 -1.6      -0.2    v -0.5211
MCK      50 -1.6      -0.2    v -0.4046
RESETB   16 -1.6      -0.2    v -0.5282
UCSB    230 -1.6     -0.2    v -0.7116

```

```

WAFER#: 2
Device#: 3

```

```

Test#:1000 opens_Short
PIN NAME CHANVMIN  VMAX  UNITACTUAL
-----
XIN      98 -1.6      -0.2    v -0.5241
MCK      50 -1.6      -0.2    v -0.7736
RESETB   16 -1.6      -0.2    v -0.5443
UCSB    230 -1.6     -0.2    v -0.4897

```

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	Char	If	Target
KeyCond	False	AND	Integer	Else	LOL
LimCond	EQ	Const	Real	Exit	HOL
Index	NE	Var	String	LSL	LWL
KeyIndex	NOT	Begin	Boolean	HSL	HWL
Result	LE	End	While	LPL	FailBin
FileName	GE	Step	For	HPL	mod
Script	LT	GT	To	Tune	

All built-in functions described in “*Built-in Functions*,” pg. 76, also constitute part of the list of key words.

Comments The characters /* introduce a comment that terminates with the characters */, such comments do not nest and they can not occur within strings. The characters // introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
job_nam      =      "MyJob"      //String Constant
Product      =      "MyProduct" //String Constant
NumOfBins    =      32           //Integer Constant
```

Variables

The following types of variables are supported:

- integer (four bytes)
- real (four bytes)
- char (one byte)
- string (maximum length of 254 characters)
- boolean (can only take on the values true or false)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

All variables declared in the format file are initialized to invalid values. Therefore, all declared variables have to be given an initial value by the user. The default values are:

- NA for string variables
- invalid value for integer and real variables

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                     condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                     condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key
                                     index
```

A minimum of one key condition and one key index must be included in the VAR block. The allowed types for conditions and indexes are Real, Integer and String. Also, note that the order that these conditions appear in is important and is different for dB and WS readers. The first three conditions should always be:

<i>dB Reader</i> —		
Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)
<i>WS Reader</i> —		
TestNum	Integer	(For Test Number)
Tname	String	(For Test Name)
Unit	String	(For Units)



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*



A maximum of 50 conditions can be defined in the format file.

Results

The dbascii data reader supports choosing more than one result, each of a possibly different type. Results are declared in the VAR block and the following syntax is used:

```
Variable1,Variable2,...TypeResult// To declare a result
```

Every format file must have at least one result declared.

The following types of results are supported:

- integer (four bytes)
- short (two bytes)
- real (four bytes)
- char (one byte)
- fixed length char (a fixed length string, which can be between 2 and 63 bytes (inclusive) — e.g. char2, char12, char46, char63)
- string (maximum length of 254 characters)

The Main Block

The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

LIMITS AND SCALING



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.

Limits

There are nine types of limits stored in the Limits Set:

LSL	Low Specification Limits
HSL	High Specification Limits
LPL	Low Production Limits
HPL	High Production Limits
LOL	Low Outlier Limits
HOL	High Outlier Limits
LWL	Low What If Limits
HWL	High What If Limits
TGT	Target Limits

And a test Fail Bin: **FailBin**

These are pre-defined keywords of type Real, except for FailBin which is of type integer. Limits are logged to the **LIM...** table; FailBin is logged to the **DEF...** table) when the **LogLimits** built-in function is called. **LogLimits** uses the current values of the limit conditions to determine where to place the limits.

Information about each limit set is stored in the LIM_LOG database table. The actual limits are stored in the LIM... table, which is linked with LIM_LOG by the LIM_LOG primary key, lim_key.

If the limits are not included in the data file and need to be read from a separate file, the **LimFile** function can be used to close the data file and open a new file containing the limits. Typically this is done after all the data has been read. Any non-key conditions that were not defined in the data file, but exist in the limit file can also be updated when the **LogLimits** function is called.

If the **-limitonly** argument is passed to the reader, the **limits_type**, **target_value** and **fail_bin** in **DEF...** are the only fields updated other than the limit table. None of the conditions can be updated in this way and the Program must already exist in the database. With this option limits can be updated for any existing Program. To update non-key conditions the **-conditions** option may be used with the **-limitonly** option. This allows for the updating of any non-key conditions from the limits file.

ASCII Historical Limits

One program in the database can have more than one set of limits associated with it.

The distinction between each limits set is determined by the **lim_type** column stored in the **LIM_LOG** table. **lim_type** is a three character string that specifies the type of the limit set as follows:

- D--** — date/time stamp only.
- DM-** — Date/time stamp and meta type.
- D-R** — Date/time and program revision.
- DMR** — Date/time stamp, meta type and program revision.
- M-** — Meta type only.
- MR** — Meta type and program revision.
- R** — Program revision only.

Meta Type — Meta type is a three character string that specifies which meta type is associated with the limit set. A program can have only one meta type, it is set once and cannot be changed (if used). Valid values for meta type are:

- LOT** — refers to a certain lot. To set the LOT ID value, users must use **vDbLimLot**, or **vDbLot** and **DbLot** if limits are being loaded with data.
- SRC** — refers to a certain source lot. To set the source lot id value, users must use **vDbLimSrcLot**, or **vDbSrcLot** and **DbSrcLot** if limits are being loaded with data.
- PKG** — refers to a certain package. To set the Package value, users must use **vDbLimPackage**, or **vDbPackage** and **DbPackage** if limits are being loaded with data.
- PRD** — refers to a certain product. To set the product value, users must use **vDbLimProduct**, or **vDbProduct** and **DbProduct** if limits are being loaded with data.
- PRC** — refers to a certain process. To set the Process value, users must use **vDbLimProcess**, or **vDbProcess** and **DbProcess** if limits are being loaded with data.
- FML** — refers to a certain Family. To set the family value, users must use **vDbLimFamily**, or **vDbFamily** and **DbFamily** if limits are being loaded with data.
- TCH** — refers to a certain technology. To set the technology value, users must use **vDbLimTechnology**, or **vDbTechnology** and **DbTechnology** if limits are being loaded with data.
- EQ1** — refers to a certain Equipment1. To set the Equipment1 value, users must use **vDbLimEquipment**, or **vDbEquipment** and **DbEquipment** if limits are being loaded with data.
- EQ2** — refers to a certain Equipment2. To set the Equipment2 value, users must use **vDbLimEquipment**, or **vDbEquipment** and **DbEquipment** if limits are being loaded with data.

Historical limits can only be maintained by the ASCII reader using the **-limitsonly** option and optionally, the built-in function **DbHistLimits** and **DbLimitsSet**. **DbHistLimits** and **DbLimitsSet** built-in functions are used to insert new limits and set the **lim_type** for the new set of limits.

The ASCII reader still supports the old method of loading limits with date/time stamp as the only distinction between different limit sets, or a combination of date/time, meta type, and program revision.

Users can load limits using date/time stamp only by using the **DbHistLimits** built-in function alone, or use **DbLimitsSet** with only the date/time argument valid and the rest NA, or use neither function.

Date Stamp Only

If the **-limitsonly** option is used without the **DbHistLimits** or **DbLimitsSet** built-in function, the date stamp for the current limits is updated to the current date (today) and the new limits are inserted as the current limits (date stamp 01/01/2500). The current limits are those used as the default limits for retrieval into a worksheet.

The **DbHistLimits** built-in function or **DbLimitsSet** with only the date argument provided, may be used with the **-limitsonly** option to insert new limits in one of the following two ways:

1. Updating current limits only (update, no insertion). This option is achieved by supplying a date to the **DbHistLimits** or **DbLimitsSet** function that exceeds the current date (today).
2. Inserting new limits with a specific date stamp. This option is achieved by supplying a date that is less than the current date (today). (In this case, the current limits in the database are set to the limits with the greatest date.)

Combination of Date Stamp, Meta Type, and Program Revision

The **DbLimitsSet** built-in function, with Meta Type or Program Revision provided, may be used with the **-limitsonly** option to insert new limits with a specific **lim_type**, which can be date stamp, a certain meta type, or a certain program revision.

Updating Limits

The built-in function **DbUpdateLimits** allows for the update of a partial set of limits. In the previous behavior, limit sets were updated by a delete-then-insert operation. This built-in function replaces the delete/insert operation with an actual update, where tests with no limits in the current data file maintain whatever limits these tests have in the database. In other words, only those tests that have valid limits in the data file are updated.

Default Limits For Multiple Limits Sets

The default limits set is the one with the column **def_flag = Y** in the LIM_LOG table. There can only be one default limits set. As long as all the limits sets for a particular program in the database are not of the **lim_type = D--**, the most recently entered set will be assigned as default.

As soon as a single limits set of type **D--** is introduced into the program, the logic for assigning default status changes. All future sets that are not of **D--** type will be loaded as non-defaults. All future sets that are of **D--** type will be compared against existing **D--** type sets.

If the new set is the first ever **D--** set for the program, then it becomes the default. If it is not, there are three possible scenarios:

1. The new limits set has no date stamp: the new set becomes the new default set. The old default set is maintained but made non-default (i.e. **def_flag = N**)
2. The new limits set has a future date stamp (greater than today's date): the new set replaces (overwrites) the existing default set.

3. The new limits set has a past date stamp (less than today's date): the new set is loaded as non-default and the existing default set remains the same.

Scaling

The built-in function **ScaleFactor** may be used to set the scaling factor for any parameter. In the case of database, scale factors in the database are updated by the reader only when a new program is created or when new parameters are added to a dynamic program. For existing programs, the scaling factors used are those stored in the database.

When logging results using the functions **LogResult** or **vLogResult**, the scaling factor for the current parameter is set to the integer value set using the function **ScaleFactor**. If this function has not been called, the scale factor defaults to 0.

The legal scale factors are limited to the following set of values:

-15, -12, -9, -6, -3, -2, 0, +2, +3, +6, +9, +12, +15

Maximum Wafers Per Lot

The maximum number of wafers that can be imported with the dbascii reader, per lot, is 1,000.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the ASCII data reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index lot_id the value of “FileName” one would

write:

lot_id = FileName//lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

INVALID DATA

Results, conditions and indexes can sometimes assume values in the data file that are invalid, and should not be logged to the data table. To achieve this a list of invalid data values is maintained that the user can add to or delete from using the built-in functions described in detail in the built-in functions section.

As an example if the result 999.99 is used to designate invalid data in the data file it can be added to the list of invalid data by calling the function **AddInvReal(999.99)**.

ORACLE ROLLBACK TABLE

With an Oracle database, when a reader error is detected and the reader exits unexpectedly, for reasons such as:

- Connection to the Oracle engine abruptly lost.
- The user disrupting the reader by pressing **Ctrl+C** multiple times.

...the reader may exit before the buffer has fully executed.

When this happens, a rollback sequence takes place to prevent the Oracle database from becoming corrupted. The rollback statements are stored in a dynamic temporary table, DP_ROLLBACK_STMTS_..., per program.

If the reader exits without fully executing the rollback statements, the table will continue to hold the remaining rollback statements, for that pg_key. This allows the reader to recover the rollback statements and execute them, should the above exit cases occur.

When a premature exit occurs, this rollback recovery operation is triggered automatically; the administrator does not have to perform any function to make this operation occur.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

```
AddSep(':')
```


OPERATORS

<i>Arithmetic:</i>	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Assignment	=
	Modulus	mod

Logical: Used only with variables of type Boolean.

Logical Or	OR
Logical And	AND
Negation	NOT

<i>Relational:</i>	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression  
    statements  
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression  
    statements  
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression  
    if-statements  
Else  
    else-statements  
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

dbascii — ORACLE PARAMETER LIMIT

The Oracle database dbascii reader allows storage of parametric data with up to 108,000 parameters.

BUILT-IN FUNCTIONS

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of ASCII Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 77
 - **Go To** — pg. 77
 - **Separators** — pg. 78
 - **Skip Forward/Backward** — pg. 78
 - **Miscellaneous** — pg. 79
- **Data Retrieval** — pg. 79
 - **Data** — pg. 79
 - **Limits** — pg. 80
 - **Strings** — pg. 81
 - **Sub-Strings** — pg. 81
 - **Integers** — pg. 83
 - **Real** — pg. 83
- **String Manipulation** — pg. 83
- **Die -Level Traceability (DLT)** — pg. 85
- **Database** — pg. 86
 - **Data Partitioning** — pg. 86
 - **Fab** — pg. 86
 - **Technology** — pg. 87
 - **Family** — pg. 87
 - **Process** — pg. 88
 - **Product** — pg. 88
 - **Program** — pg. 89
 - **Lot** — pg. 92
 - **Step** — pg. 92
 - **Stage** — pg. 93

- [Recipe](#) — pg. 94
- [Equipment](#) — pg. 94
- [Package](#) — pg. 98
- [Operator](#) — pg. 98
- [Customer](#) — pg. 99
- [Wafer Configuration](#) — pg. 99
- [Indexes](#) — pg. 102
- [Date-Time](#) — pg. 103
- [Rework](#) — pg. 105
- [Limits](#) — pg. 107
- [Tagging](#) — pg. 108
- [Coordinate Normalization](#) — pg. 109
- [Binning](#) — pg. 111
- [Miscellaneous](#) — pg. 114
- [Mathematical](#) — pg. 117
- [Debugging](#) — pg. 117
- [System](#) — pg. 118
- [Zones](#) — pg. 118
- [Triggers](#) — pg. 120
- [Miscellaneous](#) — pg. 121

File Navigation

Go To	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the error code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.

Separators	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.
Skip Forward/ Backward	SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
	SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.
	SkipChars (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.
Open File	OpenFile (string) —	Accepts one argument of type string and has no return. Replaces the currently open file with the file name that is passed in the argument. The passed argument may be the full path name or may be used with the -file_path command line argument (<i>"-file_path,"</i> pg. 130).
		If the function is called with no arguments (e.g. OpenFile()), the original data file that is passed to the reader is reopened without the need to specify its name. If the new file does not exist, the error code is set to 1 and the original file remains open.

OpenDirFile (string, string) —

Accepts two argument of type string and has no return. Replaces the currently open file with the file name that is passed in the second argument.

The first passed argument is the full path name, with “/” at the end of the directory in which the file resides. If the second argument is the empty string “”, then the original data file that is passed to the reader is reopened without the need to specify its name, assuming the first argument is the correct path to data file.

If the new file does not exist, the error code is set to 1 and the original file remains open.

Miscellaneous **NotEndOfFile —**

Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.

GetLineLen —

Accepts no arguments and returns an integer. Returns the number of characters from the FilePointer to the end of line.

Data Retrieval*Data* **LogResult (result) —**

Accepts one argument of type result and has no return. Reads in the current word as a result and logs it to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values.

vLogResult (result, real) — Accepts two argument of type result and real and has no return. Logs the second argument as the result to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values. The first argument passed to this function specifies the name and type of the result, while the second argument is the result itself.

AddInvString (string) —

Accepts one argument of type string and has no return. Adds the passed argument to the list of invalid data.

AddInvReal (real) —

Accepts one argument of type real and has no return. Adds the passed argument to the list of invalid data.

AddInvInteger (integer) — Accepts one argument of type integer and has no return. Adds the passed argument to the list of invalid data.

AddInvChar (char) — Accepts one argument of type char and has no return. Adds the passed argument to the list of invalid data.

DelInvString (string) — Accepts one argument of type string and has no return. Deletes the passed argument from the list of invalid data.

DelInvReal (real) — Accepts one argument of type real and has no return. Deletes the passed argument from the list of invalid data.

DelInvInteger (integer) — Accepts one argument of type integer and has no return. Deletes the passed argument from the list of invalid data.

DelInvChar (char) — Accepts one argument of type char and has no return. Deletes the passed argument from the list of invalid data.

OpenFile (string) — Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the **-file_path** command line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

Limits **LogLimits** — Accepts no arguments. Logs the current values of LPL, HPL, LSL, HSL, LOL, HOL, LWL and HWL to the limits table. The limits are placed in the row that matches the current values of all LimCond conditions.

LimFile (string) — Accepts one argument of type string and has no return. The passed argument is the file to be opened after the data file is closed. The File Pointer is placed at the beginning of the new file.

ClearLimits — Accepts no arguments. Resets all limits (LSL, LPL, ...) to invalid.

<i>Strings</i>	GetWord —	Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.
	GetPrevWord —	Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.
	GetQuotedWord (char) —	Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error code is set if no quoted word is found.)
	GetLine —	Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.
<i>Sub-Strings</i>	GetChars (integer) —	Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.
	GetCharsTrim (integer, char) —	Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)
	ExtractString —	Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.
	GetLeftChars (integer) —	Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetRightChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetMidChars (integer, integer) — Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordAfter (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.

If there are multiple occurrences of the passed character, the function returns all characters after the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

<i>Integers</i>	GetInt —	Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.
	GetPrevInt —	Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.
	<i>Real</i>	
	GetReal —	Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.
	GetPrevReal —	Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.
	String Manipulation	
	ToLower (string) —	Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.
	ToUpper (string) —	Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.
	IsNumber (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.
	IsString (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.
	Right (string, integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetRightChars(), but operates on the passed string instead of the current word.
	Left (string, integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetLeftChars(), but operates on the passed string instead of the current word.
	Mid (string, integer, integer) —	Accepts three arguments of type string, integer and integer and returns a string. Same as GetMidChars(), but operates on the passed string instead of the current word.

After (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordAfter()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

Before (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `'StrToInt ("16A3")'`, the output would be the integer 16; for `'StrToReal ("16.1A3")'`, the output would be 16.1.

IntToStr (integer) — Accepts one argument of type integer and returns a string. Translates the passed integer to a string.

StrTrim (string, char) — Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.

StrLen (string) — Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.

Die -Level Traceability (DLT)

Argument —

Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command line argument **-arg**.

DbDieId —

Accepts no arguments and has no returns. Calls GetWord, returning the current word. If the current word does not exist as a Die ID in the database (DIE Table), the Die ID is added. If it does exist, the function establishes the appropriate relations with other tables. Necessary to activate DLT in Fab stage. If "NA" is the current word, no DLT is passed.

vDbDieId (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. The function is the same as **DbDieId**, but it uses the passed argument instead of reading from the file.

DbDieIdIndex (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the Die ID index. Necessary to activate DLT in both final-test and module stages. When combined with **DbDieId/vDbDieId**, the reader will error out.

DbModuleIndex (string) —

Accepts three arguments of type string and has no return. The passed arguments must be the names (in double quotes) of three declared indexes. Those indexes become the module ID, module die-x, and module die-y, respectively. Necessary for loading Module-stage data. **DbModuleIndex** functions like a combination of **DbWaferIndex (string)** and **DbDieXYIndexes (string, string)** for module data.

vDbModuleCfg —

Accepts six arguments and has no return. The first four arguments of type string, while the last two of type integer. Sets the configuration of the Module, similar to vDbWmapCfg. The arguments are as follows:

- Module ID
- Module type: maps module to module type in the WMAP_CONFIG database table.
- Module parent: Maps module to source module in the MDL2MDL database table.
- Product: Maps product to module.
- Die_X cnt: Module width.
- Die_Y cnt: Module height

Database

<i>Data Partitioning</i>	DbPartitionAction (string) —	Accepts one argument of type string and has no return. Sets partition_action in the PROGRAM table. This function overwrites the command line argument -partition_action. Valid values for DbPartitionAction are: •sw = weekly by start time •sm = monthly by start time •sq = quarterly by start time •ew = weekly by end time •em = monthly by end time •eq = quarterly by end time •iw = weekly by insert time •im = monthly by insert time •iq = quarterly by insert time See the “<i>Partitioning Data</i>” section in the <i>dpEXPORT</i> section of the Administrator Tools manual.
<i>Fab</i>	DbFab --- default --- —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.
	DbFab --- with -semi_dynamic option --- —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbFab (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFab, but uses the passed argument instead of reading from the file.
	vDbLimFab (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbFab, but it is used in association with a limit set. The function populates the FAB database table and the LOT database table’s fab column.

Technology **DbTechnology --- default ---**

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbTechnology --- with -semi_dynamic option ---

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbTechnology (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.

vDbLimTechnology (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbTechnology, but it is used in association with a limit set.

Family **DbFamily —**

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the program type is fixed and the current word does not exist as a family in the database (FAMILY Table), the function does nothing. If the program type is not fixed (semi-dynamic, dynamic) and the current word does not exist as a family in the database (FAMILY Table), the family is added to the database and the function establishes the appropriate relations with other tables.

vDbFamily (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.

vDbLimFamily (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbFamily, but it is used in association with a limit set.

Process DbProcess --- default --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbProcess --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but uses the passed argument instead of reading from the file.

vDbLimProcess (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but it is used in association with a limit set.

Product DbProduct --- default --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbProduct --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbProduct (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file.

vDbLimProduct (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but it is used in association with a limit set.

DbDieCnt (int) — Accepts one argument of type integer and has no return. The function populates the PRODUCT database table's **die_count** column.

DbLimDieCnt (int) — Accepts one argument of type integer and has no return. The function populates the PRODUCT database table's die_count column. It is used with function DbLimProduct in association with a limit set.

DbProdWmapCfg — Accepts no arguments and has no return. When called, the function associates the PRODUCT wafer map configuration to the PROGRAM — if the PROGRAM is not associated with a wafer map configuration.

The reader would create a similar wafer map configuration to the one associated with the PRODUCT, giving it the standard naming convention, and linking it to the PROGRAM.

Program **DbProgram** —

Accepts no arguments and returns a string. Calls GetWord, returning the current word. Sets the database test program name to the current word. This function should always be called when dumping to database.



*If the **-class** reader option is not used, this function must be called after calling the **DbProgClass** function.*



Data readers and database retrieval supports program names up to a limit of 255 characters.

vDbProgram (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the database test program name to the passed argument. This function should always be called when dumping to database.



*If the **-class** reader option is not used, this function must be called after calling the **DbProgClass** function.*



Data readers and database retrieval supports program names up to a limit of 255 characters.

DbProgRel (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. Sets the database test program release to the current word. The passed argument specifies the format of the date string being read.

vDbProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as DbProgRel but uses the first passed argument instead of GetWord as the date string. The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.
ddd — Day of the week as a 3-letter abbreviation.
mm — Month as a 2-digit number.
mmm — Month as a 3-letter abbreviation.
yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
yyyy — Year as a 4-digit number.



*With the Oracle database, the **ddd** option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in the date string.*

vDbLimProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as vDbProgRel but uses the first passed argument instead of GetWord as the date string. It is used in association with a limit set.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.
ddd — Day of the week as a 3-letter abbreviation.
mm — Month as a 2-digit number.
mmm — Month as a 3-letter abbreviation.
yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
yyyy — Year as a 4-digit number.



*With the Oracle database, the **ddd** option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in the date string.*

- DbProgRev** — Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.
- vDbProgRev (string)** — Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.
- DbProgClass (int)** — Accepts one argument of type integer and has no return. Sets pgc_key in the PROGRAM table. This function overwrites the command line argument **-class**.
- vDbProgGroup (string)** — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.
- The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

- vDbProgProduct (string)** — Accepts one argument of type string and returns a string. Returns the passed argument. The function associates a program with a specific product. The value passed is filled in the PRODUCT table and the corresponding key is filled in the **program.identifier_key** field.

Lot DbLot —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Lot in the database (LOT Table), The Lot is added. If it does exist the function establishes the appropriate relations with other tables. Necessary if Lot summaries are to be calculated.

vDbLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbLot, but uses the passed argument instead of reading from the file

vDbLimLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbLot, but uses the passed argument instead of reading from the file. This function is used in association with a limit set.

DbSrcLot —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Source Lot in the database (LOT Table), the Source Lot is added. If it does exist, the function establishes the appropriate relations with other tables. Works in conjunction with DbLot, sets the src_lot column in the LOT table and OP_LOG table.

vDbSrcLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbSrcLot, but uses the passed argument instead of reading from the file.

vDbLimSrcLot (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbSrcLot, but it is used in association with a limit set.

vDbLotClass (string, string, string, string) —

Accepts four arguments of type string: (**Lot_Name**, **Method_Name**, **Method_Type**, **Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

vDbLimLotClass (string) — Accepts one argument of type string and has no return. Works in conjunction with **vDbLimLot**. Sets the lot class from the passed argument for the lot specified using **vDbLimLot**. This function is used in association with a limit set.

DbUpdateLot (int) — Accepts one argument of type integer, and has no return. The first argument is a fab flag that allows for updating the Fab relationship if argument >0.

The function allows for the update of the LOT table's fab relationships.

This is useful when a particular lot already exists in the database and there is a need to update the fab relationships to that lot.

The fab is updated to the fab identified by either of the functions, **DbFab** or **vDbFab**.

Step DbStep --- default --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStep --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStep (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStep**, but uses the passed argument instead of reading from the file.

Stage DbStage --- default --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStage --- with -semi_dynamic option ---

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStage (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbStage, but uses the passed argument instead of reading from the file.

Recipe **DbRecipe —**

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the program type is fixed and the current word does not exist as a recipe in the database (RECIPE Table), the function does nothing. If the program type is not fixed (semi-dynamic, dynamic) and the current word does not exist as a recipe in the database (RECIPE Table), the recipe is added to the database and the function establishes the appropriate relations with other tables.

vDbRecipe (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbRecipe, but uses the passed argument instead of reading from the file.

Equipment **vDbEquipment (int, string, string, string) —**

Accepts four arguments and has no return.

The 1st argument is an integer between 1 and 6, which indicates the equipment number — as in the OP_LOG database table (**eqkey1, eqkey2, ...**).

The 2nd argument is the equipment I.D. (string).

The 3rd argument is the equipment type (string).

vDbConsEquipment (int, string, string, string) —

Provides the ability to set the equipment wafer consolidation entry in the OP_LOG table (e.g. eqkey1...eqkey6) from the format file. Accepts four arguments and has no return.

The 1st argument is an integer between 1 and 6, which indicates the equipment number — as in the OP_LOG database table (**eqkey1**, **eqkey2**, ...).

The 2nd argument is the equipment I.D. (string).

The 3rd argument is the equipment type (string).

The 4th argument is the equipment class (string).

Functionality: If **vDbConsEquipment** is used for a specified equipment number (e.g. 1-6), the provided metadata is used under the following conditions:

- The provided metadata is used when the reader performs consolidation (when **-cons** (“*-cons*,” pg. 128) or **DbWaferAppend** is used).
- If the reader fills a new OP_LOG row using the current file’s metadata (default), the equipment metadata is overwritten by the data provided by the **vDbConsEquipment** function.
- If the reader fills a new OP_LOG row using the latest meta data (using **DbUpdateOplogCons**), the equipment meta data is overwritten by the data provided by the **vDbConsEquipment** function.

The 4th argument is the equipment class (string).

vDbLimEquipment (int, string, string, string) —

Accepts four arguments and has no return.

The 1st argument is an integer between 1 or 2, which indicates the equipment number.

The 2nd argument is the equipment I.D. (string).

The 3rd argument is the equipment type (string).

The 4th argument is the equipment class (string).

Same as **vDbEquipment**, but it is used in association with a limit set.

DbTester --- default --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a tester in the database (EQUIPMENT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbTester --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a tester in the database (EQUIPMENT Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbTester (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTester, but uses the passed argument instead of reading from the file.

DbHandler --- default --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a handler in the database (EQUIPMENT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbHandler--- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a handler in the database (EQUIPMENT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbHandler (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbHandler, but uses the passed argument instead of reading from the file

DbEquip3 — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey3 in the OP_LOG table.)

vDbEquip3 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip3 , but uses the passed argument instead of reading from the file.
DbEquip4 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey4 in the OP_LOG table.)
vDbEquip4 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip4 , but uses the passed argument instead of reading from the file.
DbEquip5 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey5 in the OP_LOG table.)
vDbEquip5 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip5 , but uses the passed argument instead of reading from the file.
DbEquip6 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey6 in the OP_LOG table.)
vDbEquip6 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip6 , but uses the passed argument instead of reading from the file.

<i>Package</i>	DbPackage —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Package in the database (PKG Table), the Package is added. If it does exist, the function establishes the appropriate relations with other tables.
	vDbPackage (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbPackage, but uses the passed argument instead of reading from the file.
	DbPackageType —	Accepts no arguments and returns a string. Calls GetWord returning the current word. Works in conjunction with DbPackage. Sets the package type from the passed argument for the package specified using DbPackage.
	vDbPackageType (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbPackageType, but uses the passed argument instead of reading from the file. Works in conjunction with DbPackage. Sets the package type from the passed argument for the package specified using DbPackage.
	vDbLimPackage (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbLimPackage, but it is used in association with a limit set.
	vDbLimPackageType (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbLimPackageType, but it is used in association with a limit set.
<i>Operator</i>	DbOperator --- default --- —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a name in the PEOPLE Table, The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbOperator --- with -semi_dynamic option ---

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a name in the PEOPLE Table, it is added with the “role” field set to Operator. If it does exist the function only establishes the appropriate relations with other tables.

vDbOperator (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbOperator, but uses the passed argument instead of reading from the file.

Customer **vDbCustomer(string, ...)** — Accepts 12 arguments of type string, and has no return. Populates the CUSTOMER database table and establishes a relationship between the customer entry and the current lot. This function may be called several times in the same format file, in this way relating one lot to many customers. The arguments passed to the function match exactly those of the CUSTOMER table. These fields (in their respective order) include: **customer name, address 1, address 2, address 3, postal code, city, state, country, contact, email, fax, phone.**

Wafer Configuration **vDbWmapCfg (...)** —

Accepts fifteen arguments and has no return. The passed arguments fill the WMAP_CONFIG database table and are of the following types:

wmap_name - String
wf_size - Real
wf_units - String
flat - Char
flat_type Char
die_wd - Real
die_ht - Real
center_x - Real
center_y - Real
pos_x - Char
pos_y - Char
fld_rows Integer
fld_cols Integer
row_offset integer
col_offset integer



The first argument for the function, **wmap_name**, is ignored by **dbascii**. Internally, **dbascii** sets its value to the program name. This guarantees a one-to-one relationship between program and wafer configuration. The **wmap_name** argument exists for historical purposes only.

Acceptable values for:

- **Flat** —
 - **R** - right
 - **L** - left
 - **T** - top
 - **B** - bottom
- **flat_type** —
 - **F**- flat
 - **N** - notch
- **pos_x** —
 - **R** - right
 - **L** - left
- **pos_y** —
 - **U** - up
 - **D** - down

vDbZWmapCfg (int, int, int, int, int, int, int, int) —

Accepts eight arguments of type integer and has no return. Populates the **Z_WMAP_CONFIG** database table. The eight passed arguments are: **quad_offset**, **radial_offset**, **radials**, **circles**, **rows**, **cols**, **area_flag**, **accept_percent**.

DbZonalBins (int) —

Accepts one argument of type integer and has no return. The argument is the bin number for which zonal summaries will be enabled. Related to **zstats_flag** and **zcnt_flag** from the **PROG_CLASS** table. By default, if either of the above two flags are set, yield zonal summaries will be enabled. This function is used to add bin zonal summaries to the yield summaries.

vDbWaferClass (string, string, string, string) — Accepts four arguments of type string and has no returns. The passed arguments are of the following types:

wafer_name - string

method_name - string
method_type - string
class_name - string

Only those wafers that are used in the data file will have their classes and methods logged. Wafers that are only passed to **vDbWaferClass()** but not used elsewhere in the data file will be ignored.

For those wafers logged, the following tables will be populated: CLS_METHOD, WAFER_CLASS, and WFCLS2WF. CLS_METHOD is populated with the **method_name** and **method_type**. WAFER_CLASS is populated with the **class_name**. WFCLS2WF is populated with the corresponding **wafer_names** and **class_names**.

DbWaferAppend —

Accepts no arguments and has no return.

Turns on wafer consolidation from the format file. Consolidates partial wafer/lot parametric data. Wafer Consolidation is a way to consolidate a wafer that has been retested and has data in different data files. Use **DbWaferAppend** and the ASCII reader will load the file, creating a new entry in the database with a new lg_key and consolidating the data of the split wafer data files into one entry with the latest information. The new entry would have **test_mode = C** in WF_LOG table.

When there are multiple readings for individual die locations, the latest data is used.

This function is identical to calling the dbascii **-cons** command-line option ("*-cons*," pg. 128).

DbWfTestMode (string, string) —

Accepts two arguments of type string and has no return. Defines which wafers should be included in the consolidation process, based on test mode. The first argument should be the wafer ID. The second argument should be the test mode the user wants associated with that wafer ID. (This function allows populating the **test_mode** column in the WAF_LOG table.)



*Note 1: Only the first character of the **test_mode** string passed to this function is stored in the DB.*

*Note2: If this function is not called, the default value for **test_mode** is “S”.*

*Note 3: In the case of consolidation, the **test_mode** value “C” is not accepted*

DbWfDesc (string, string) —

Accepts two arguments of type string and has no return. The first argument is the wf_id; the second argument is the wafer description (maximum 64 characters). This function allows a wafer description to be added to the WAFER table (in the **wf_desc** column), but only when a new wafer is encountered.

Bricking **vDbBrick (int, int) —**

Accepts two arguments of types integer and integer and has no return. The passed arguments fill the RET2DIE database table. The first passed argument must be the die_x coordinate, and the second passed argument must be the die_y coordinate.

Indexes **DbBinIndex (string) —**

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the bin index used for updating the BIN_LOG and HIST_BIN database tables.

DbBinTest (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the Tests. That Test becomes the bin test used for updating the BIN_LOG and HIST_BIN database tables.

DbBinTypeIndex (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the bin type index used for updating the BIN_LOG and HIST_BIN database tables.

This function is only meaningful when program class is `bin_map` (4). It is used to allow for loading of soft bin and hard bin into the same program. The bin type index is used to indicate the rows that will be used to populate the `BIN_LOG` table. Those rows, with bin type different from “hbin” are ignored when calculating bin summaries.

DbWaferIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the wafer index used for updating the `WAFER` and `WAF...` database tables.

DbDieXYIndexes (string, string) — Accepts two arguments of type string and has no return. The passed arguments must be the names (in double quotes) of the two declared indexes. Those indexes become **die_x** and **die_y**, respectively. This is necessary for `BIN_LOG m` and **d0** calculations.

DbSiteIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of the one declared index. That index becomes the **site** index. This is necessary, for example, to identify the site index for PCM data, allowing for alignment of site-level data with **die_x**, **die_y**-level data.

All `RES...` tables created by the `dbascii` reader having a site index (identified by **DbSiteIndex**) will automatically have database indexes created on the combination of the `lg_key` column and the site index column. This is specifically done for performance reasons when using the PSA analysis tool.

Date-Time **DbStartTime (string)** — Accepts one argument of type string and returns a string. Calls `GetWord` returning the current word. The passed string is the format (SQL `DATETIME`) that describes the date as it appears in the file, the date is used as the `Start_Time` of the lot.

vDbStartTime (string, string) —

Accepts two arguments of type string and returns a string. Returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the Start_Time of the lot.

The following describes the formatting of the DATETIME string:

%b — Abbreviated month name.
%B — Full month name.
%d — Day of the month as a decimal [01,,31].
%H — 24 hour clock.
%I — 12 hour clock
%M — Minute as a decimal [00,,59].
%m — Month as a decimal [01,,12].
%p — a.m. or p.m.
%S — Second as a decimal [00,,59].
%y — Year as a decimal [00,,99].
%Y — Year as a 4-digit decimal.
%% — Allows for percent in the string.

As an example the format for the following string:

“jul 1 96 05:10:46”

would be:

“%b %d %y %H:%M:%S”

vDbLimStartTime (string, string) —

Accepts two arguments of type string and returns a string. Returns the first argument.

The 1st argument is the date-time as a string and the 2nd is the format (SQL DATETIME for Informix) that describes the date as it appears in the file. The date is used as the Start_Time of the lot.

The following describes the formatting of the Informix DATETIME string:

%b — Abbreviated month name.
%B — Full month name.
%d — Day of the month as a decimal [01,,31].
%H — 24 hour clock.
%I — 12 hour clock

%M — Minute as a decimal [00,.,59].
%m — Month as a decimal [01,.,12].
%p — a.m. or p.m.
%S — Second as a decimal [00,.,59].
%y — Year as a decimal [00,.,99].
%Y — Year as a 4-digit decimal.
%% — Allows for percent in the string.

This function is similar to `vDbStartTime`, but is used in association with a limit set, to populate the LOT database tables `start_time` column.

DbEndTime (string) — Accepts one argument of type string and returns a string. Calls `GetWord` returning the current word. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the `End_Time` of the lot (updates the `TEST_LOG` table only)

vDbEndTime (string, string) — Accepts two arguments of type string and returns a string. Returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the `End_Time` of the lot (updates the `TEST_LOG` table only).

Rework **DbReworkAction (int) —** Accepts one argument of type integer and has no return. Sets `rework_action` in the `PROGRAM` table. This function overwrites the command line argument **-rework_action**.

DbRework — Accepts no arguments and has no return. Overrides the **rework_flag** field in the **OP_LOG** table. This function is used to bypass detection of rework by the reader. The data is considered rework regardless of the contents of the database.

DbTimeResRework (string, string) — Accepts two arguments of type string and returns nothing. The first argument is the name of the key format file string index that holds the date-time string that will be used to detect rework in the `RES...` table. The second argument is the format (SQL DATETIME) that describes the date-time string as it appears in the file.

The following describes the formatting of the DATETIME string:

%b - Abbreviated month name.

%B - Full month name.

%d - Day of the month as a decimal [01,,31].

%H - 24 hour clock.

%I - 12 hour clock

%M - Minute as a decimal [00,,59].

%m - Month as a decimal [01,,12].

%p - a.m. or p.m.

%S - Second as a decimal [00,,59].

%y - Year as a decimal [00,,99].

%Y - Year as a 4-digit decimal.

%% - Allows for percent in the string.

As an example the format for the following string:

"jul 1 96 05:10:46"

would be:

"%b %d %y %H:%M:%S"

Note: This format file function only works with append and mark rework action (**-rework_action = 2**).

DbNumberResRework (string) —

Accepts one argument of type string and returns nothing. The argument is the name of the key format file integer index that holds the sequential number that will be used to detect rework in the RES... table.

Note: **DbNumberResRework()** is supported for Cassandra, but must be used with the **-cons** and **-first_pass** command-line arguments.

Note: This format file function only works with append and mark rework action (**-rework_action = 2**).

Limits DbHistLimits (string) — Accepts one argument of type string and has no returns. The passed argument is the date stamp for the limits being inserted. This function has no effect except when using the ASCII reader with the **-limitonly** option.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.
ddd — Day of the week as a 3-letter abbreviation.
mm — Month as a 2-digit number.
mmm — Month as a 3-letter abbreviation.
yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
yyyy — Year as a 4-digit number.

DbUpdateLimits — Accepts no arguments and has no return. Allows for the update of a partial set of limits. Tests with no limits in the current data file maintain whatever limits these tests have in the database. In other words, only those tests that have valid limits in the data file are updated.

For related functionality, refer to [“DbHistLimits \(string\),”](#) pg. 107.

DbLimitsSet (string, string, string) — Accepts three arguments of type string and has no return. The passed arguments set the Limit Set Type.

The arguments are:

Limits Date — date stamp for the insertion of the limits being. NA if no date is available.

Meta Type — A three character string that specifies the meta type for the limit set.

Valid values are:

LOT — lot

SRC — source lot

PKG — package

PRD — product

PRC — process

FML — family

EQ1 — equipment1

EQ2 — equipment2

TCH — technology

It can be set to NA if no meta type is available. If used, the appropriate limit function setting the meta type value should be used. For example, if meta type is LOT, the value of the lot id can be specified using the vDbLimLot function.

Revision: Sets the database test program revision. NA if no revision is available.



Loading limits based on Auto, Latest or Oldest is not functional if date is not specified in DbLimitsSet. If not specified, the system will always return the default limits set.



WARNING:

*Once the program has been created, the DbLimitsSet function will only work with the **-limitonly** option ("**-limitonly**," pg. 131).*

Tagging DbTagAction (int, real, real) —

Accepts three arguments of types integer, real and real and has no return. Sets tag action and automatic tagging criteria for bad yield and scrap yield. The first argument is the tag action (1 — tagging is disabled; 2 — tagging is enabled). The second argument is bad yield (all lots/wafers with yield lower than the bad yield value are tagged as bad). The third argument is scrap yield (all lots/wafers with yield lower than the scrap yield value are tagged as scrap).

If automatic tagging is not needed, bad yield and scrap yield may be set to **-1**.

This function should only be called once from the format file.

DbLotTag (int) —

Accepts one argument of type integer and has no return. Used for manual lot tagging. The passed argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

DbWfTag (string, int) — Accepts two arguments of type string and integer and has no return. Used for manual wafer tagging. The first argument should be the wafer ID. The second argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (int) — Accepts one argument of type integer and has no return. Used for manual source lot tagging. The passed argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

Coordinate Normalization

DbNormalizeMap — Accepts no arguments and has no return. Should only be used for normalization indicating that the program **is not** to be aligned with defect data in the database. Normalization is applicable to any data type where die_x and die_y indexes have been identified using the DbDieXYIndexes function.

See “[Coordinate Normalization](#),” pg. 138, for more information.

DbOrgDieXYIndexes (string, string) — Accepts two arguments of type string and has no return. The passed arguments must be the names (in double quotes) of the two declared indexes that hold the original diex and diey indexes, respectively. These indexes must be different than the indexes that are passed to the **DbDieXYIndexes** format file function and must be of type integer. Typically used in conjunction with the map normalize functions (“[Coordinate Normalization](#),” pg. 138).

When this function is applied, wafer map end-users will have the option of changing the die X/Y coordinates to the original, pre-normalized coordinates. They will also have the option of displaying the original coordinates in labels and tool-tips.

In case of the Binmap program class, the above description is true for conditions instead of indexes.

DbAlignDefect —

Accepts no arguments and has no return. Should only be used for normalization indicating that the program **is** to be aligned with defect data in the database. This function is only applicable to data belonging to the bin map program class. It has no effect on any other data types.

See [“Coordinate Normalization,”](#) pg. 138.

DbAlignDefectOnly —

Accepts no arguments and has no return. Tells the reader to align defect data with binmap data.

Reader will not perform normalization. Using this function assumes that the user has provided data in a normalized state. The function flags the reader that binmap data should be aligned with defect data.

The following restriction applies: the data’s positive X is “right” and positive Y is “top.”

Not to be used with the **DbAlignDefect** or **DbNormalize** format file functions.

Do not use with the **-dfalign** or **-normalize** command-line options.

This function must be called after **DbProgClass**, if it is used in the format file.

Fill **vDbWmapCfg** in the standard way.

See [“Coordinate Normalization,”](#) pg. 138.

DbDieMap (int) —

Accepts one argument of type integer and has no return. Refer to [“Site-Level Alignment,”](#) pg. 40 for a detailed explanation of DbDieMap usage.

Binning DbBinName (integer, string, char) —

Accepts three arguments of types integer, string and char and has no return.

The first passed argument must be the bin number (2-bytes only*), the second is the bin name and the third is a 'P' for a Pass Bin or an 'F' for a Fail Bin.

RESTRICTIONS:

- The passed **bin_name** value cannot exceed 254 characters.

DbBinNameOnly (string, char) —

Accepts one argument of type string and one argument of type character, and has no return.

The first argument is the bin name (**bin_name**).

The second argument is the bin pass-fail type (**PassFail** — **P** or **F**).

RESTRICTIONS:

- The passed **bin_name** value cannot exceed 254 characters.
- This function can only be used with program class 12 (Final Summary).
- This function can NOT be combined with the regular binning built-in functions:
 - **DbBinName**
 - **DbBinNameOrder**
 - **DbBinNameColor**
 - **DbBinNameColorOrder**
- The existing function, **DbBinSum**, MUST be used to declare the **bin_num** and **test_type** conditions.
- The bin number condition MUST be declared as non-key. No value should be assigned to this condition. Rather, it will be used internally by the reader.
- The same bin name can not apply to more than one unique test in the data.

DbBinNameColor (integer, string, char, integer, integer, integer) —

Accepts 6 arguments of types integer, string, char, integer, integer, integer and has no return.

- The first passed argument must be the bin number (2-bytes only*).
- The second passed argument is the bin name.
- The third is a 'P' for a Pass Bin or an 'F' for a Fail Bin.
- The next three arguments are of type integer and they specify the RGB value for each bin. The range for each argument (color) is 0-255.

If **DbBinName** is used, colors will be set by the reader based on the default stored in the database.

RESTRICTIONS:

- The passed **bin_name** value cannot exceed 254 characters.

DbBinNameOrder (integer, string, char, integer) —

Accepts four arguments of types integer, string, char and integer and has no return.

- The first passed argument must be the bin number.
- The second argument must be the bin name.
- The third argument must be a **P** for a Pass Bin or an **F** for a Fail Bin.
- The fourth argument must be the bin order.

RESTRICTIONS:

- The passed **bin_name** value cannot exceed 254 characters.

DbBinNameColorOrder (integer, string, char, integer, integer, integer, integer) —

Accepts seven arguments of types integer, string, char, integer, integer, integer, integer and has no return.

- The first passed argument must be the bin number (2-bytes only*).
- The second passed argument must be the bin name.
- The third passed argument must be a **P** for a Pass Bin or an **F** for a Fail Bin.
- The next three arguments are of type integer and they specify the RGB value for each bin. The range for each argument (color) is 0-255.
- The last argument is bin order.

If **DbBinName** or **DbBinNameOrder** are used, colors will be set by the reader based on the default stored in the database.

RESTRICTIONS:

- The passed **bin_name** value cannot exceed 254 characters.

DbUpdateBinName —

Accepts no arguments and has no returns. Calling this function for any data type that fills in the HIST_BIN table will force an update of bin names with every file loaded.

DbUpdateBinColor —

Accepts no arguments and has no returns. Calling this function for any data type that fills in the HIST_BIN table will force an update of bin colors with every file loaded.

DbBinSum (string, string, string, char, string) —

Accepts five arguments of types string, string, string, char, string and has no return. This function is only used with program class “Summary (3)” to fill the **BIN_LOG** table.

- Argument 1 — condition name that has the bin numbers
- Argument 2 — condition name that is used to identify parameters that have binning information
- Argument 3 — the value of the condition specified in argument 2 that identifies a column with binning information

- The 4th argument has only two valid values, 'C' and 'P'. 'C' is used to indicate that the bin summaries are counts; while 'P' indicates that the bin summaries are in percentages.
- Argument 5 — this argument is only used when the 4th argument has value 'P' to indicate the index name that has the total counts

Also, dbascii forces the arguments that are passed to the **DbBinSum()** function to be stored in the database as lower-case. The arguments affected are **arg1**, **arg2**, and **arg5**.

DbRetestBin (int) —

Accepts one argument of type integer and has no return. The passed argument is the bin number. Used with the **DbBinSum** function, this function is used to indicate which fail bins in the data files are being retested. This function can be called only with bin summary data, where not all of the failed bins are retested. This function can be called multiple times in the format file for every failed bin that is being retested. This function only affects lot fail bin summaries.

Restrictions/Requirements

- This function is used with bin summaries data, only where data for a lot comes in multiple files listing bin counts, some files are called first-pass files and the rest of the files are called retest files in which failed devices are retested. This means that the **DbBinSum** function MUST be called in the format file in order to use this **DbRetestBin** function.
- **DbTestMode** or **vDbTestMode** MUST be called in the format file and this function can be only called if the passed test mode is "O" — which means this file is a retest.
- This function only populates the BIN_LOG_RETEST table and in order to see the effect of this function, UpStat must run with SSC option.

Miscellaneous **DbRowCnt (int) —**

Accepts one argument of type integer and has no return. The passed argument sets the row_cnt column in the OP_LOG table. (This field is typically filled by the actual number of rows generated from the data file. The function DbRowCnt is used to over-ride that value.)

- DbTestMode** — Accepts no arguments and has no return. Calls `GetWord` and assigns the first character of the current word to **test_mode** in the database **OP_LOG** table.
- vDbTestMode (Char)** — Accepts one argument of type `Char` and has no return. Assigns the passed character to **test_mode** in the database **OP_LOG** table.
- DbWfNum (string, int)** — Accepts two arguments of type `string` and `integer` and has no return. The first argument should be the wafer ID, while the second argument should be the wafer number associated with that wafer ID. (This function allows populating the **wf_num** column in the **WAFER** table. It should be called for every wafer/wafer number combination in the data file.)
- vDbOutliers (real)** — Accepts one argument of type `real` and has no return. Sets the “ni” (outliers factor) in the **OP_LOG** table. This function over-rides the command-line arguments **-outliers** and **-dboutliers**. A passed value of -1.0 is equivalent to **-dboutliers**, while any positive real is equivalent to **-outliers <ni>**.
- DbArchLog (int)** — Accepts one argument of type `integer` and has no return. The passed argument is the value of the **lg_key** column from the **ARCHIVE_LOG** table.
- This function is primarily used by the *dpEXPORT/ import* functionality to indicate to the reader which data set is being restored from archived files.
- DbUglyDie (int)** — Accepts one argument of type `integer` and has no return. It is used to indicate whether a die is normal (0), bad (1), or ugly (2). This function is typically called prior to calling the **LogResult** function and only affects bin map data.
- DbParGrp (string, int, string)** — Accepts three arguments of type `string`, `integer`, `string`, and has no return. Allows for the creation of parameter groups from within the format file. This function may be called several times, thus creating multiple parameter groups related to the current program. The arguments passed to the function include the following fields (in their respective order): **parameter group name**, **condition number**, and **wild-card string**.

The function creates the parameter groups based on the value of one of the conditions (argument 2) and based on the value of that condition for the different parameters (argument 3, which could be an exact match or a wild-card match).

DbSumLevel (string) —

Accepts one arguments of type string, and has no return. The argument can be either “W” for Wafer level summaries, “L” for Lot level summaries or “S” for source lot level summaries. The function allows for loading wafer/lot/source lot summaries generated by the *dpEXPORT* utility or by dbascii using the **-export** or **-exportonly** options. By using this function, dbascii fills only the WAF... or LOT... tables.

DbSumLevel cannot be used with the built-in function, **DbNoSumCalc**. If these two functions are used together, dbascii will exit with an error.



*When loading data into the database, if **DbSumLevel** is used dbascii will append “_W” (wafer level) “_L” (lot level) or “_S” (source lot level) to the program name.*

DbNoSumCalc — Accepts no arguments and has no returns. Used when raw data is imported and no summary calculations are wanted. Therefore, if used when loading raw data, dbascii will not calculate wafer, lot, or source lot level summaries.

You cannot use this with the **DbSumLevel** built-in function. If these two functions are used together, dbascii will exit with an error.

vDbGuid (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Sets the **guid** column in the OP_LOG table.

DbMetaData (string, string) — Accepts two arguments of type string and has no return. The first argument is the MetaName. It must already exist in the META_DEF table. This function is used to create meta-indexes to be used with advanced data retrieval.

If it does not exist, the reader will error out. This is to prevent data integration issues.

The second argument is the MetaValue. A max of 100 meta values per file are allowed.

This format function will only work for Big Data installations (Cassandra).

Mathematical **ScaleFactor (int)** — Accepts one argument of type integer and has no return. Sets the value of the current scaling factor.

Abs (real) — Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.

Sqr (real) — Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.

POW (real, int) — Accepts two arguments — first of type real, second of type integer; and returns a real. The function returns a real, which is the result of the 1st argument (type real), raised to the power of the 2nd argument (type integer).

Debugging **Print (...)** — Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.

PrintToFile (...) — Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.

ErrorCode — Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the error code are **GoTo**, **GoBackTo**, and **OpenFile**.

ExitScript (string) — Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

exitcode (int) — Accepts one argument of type integer and has no return. This function sets the exit code with which the ExitScript function would exit. If not used, the **exitscript** function exits with code = 1.

System **System (string)** — Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function. Accepts up to 254 characters.

Zones The dbascii reader automatically define zones for each program. The reader uses dieX and dieY coordinates to create default zones for certain programs, when the following built-in functions are called:

- **DbCreateDefaultzones ()**
- **DbCircularZones (string, int)**
- **DbRadiusZones (int)**

DbCreateDefaultzones () — The **DbCreateDefaultzones** built-in function accepts between one and seven arguments (following) of type string that represent the zones for the system to create. The function has no returns. Possible arguments:

- **RW** — Row
- **CL** — Column
- **CR** — Circle
- **RD** — Radial
- **QD** — Quadrants
- **SP** — Step
- **Z9** — 9 Zone

DbCreateDefaultzones works with the following zone types:

By Row: Zonal analysis by row displays bin or parametric results based on die row zones.

By Column: Zonal analysis by column displays bin or parametric results based on die column zones.

Circular (where n is the circular area or the circular radius): Circular zonal analysis displays bin or parametric results based on circular zones.

Radials (where n is the number of radials): Radial zonal analysis displays bin or parametric results based on radial (pie slice) zones.

Quadrant: Quadrant zonal analysis displays bin or parametric results based on quadrant zones.

Stepper Field: Stepper Field zonal analysis displays bin or parametric results by stepper field zones.

9 Zone: 9 Zone analysis displays three circles of equal radius, with the outer two circles divided into four quadrants.

DbCircularZones (string, int) —

Accepts two arguments of type string and integer. The system does not return any code or message for this built-in function.

The first argument is type string and it represents the type of circular zone: equal area or equal distance (between zone lines). Acceptable arguments are:

- area** — equal area
- distance** — equal distance between zones

The second argument is type integer and represents the number of circles.



If this built-in function is not used, the reader creates a default 10 circle chart with equal distance between the zones.

DbRadiusZones (int) — Accepts one argument of type integer and has no returns. The argument represents the number of radials.



If this built-in function is not used, the reader creates a default 12 radial chart.

Triggers

The two trigger functions, **vDbTrigger()** and **vDbTriggerInput()** are used together to populate the database tables, DP_TRIGGER and DP_TRIGGER_RUNS with trigger information used by *dpMonitor*. This includes the name and type of trigger and the combination of input name-input type, and all values associated with each input name-input type combination.

These two dbascii functions only insert this trigger data into the two trigger database tables. No other dbascii data processing is affected by them.

Additionally, two command-line arguments — **-trigger_db** (pg. 136) and **-triggeronly** (pg. 135) — can be used to specify the database schema into which the trigger data will be inserted and to specify that only trigger information will be inserted.

vDbTrigger (string, string, string, char) —

Accepts three arguments of type string and one character. Passes trigger information to the database.

Argument 1 passes the trigger name. Argument 2 passes the trigger type. Argument 3 passes the trigger description (optional). Argument 4 is the trigger method; it must be set to **C** (for custom trigger).

This function must be used with **vDbTriggerInput()**.

vDbTriggerInput (string, char, string, string) —

Accepts three arguments of type string and one character. Passes trigger information to the database.

Argument 1 passes the trigger name. The combination input_type (argument 2 (character)) and input_name (argument 3) is connected to a set of input_values (argument 4).

So you can call this function several times for the same trigger name, but with different input name-input type combinations — each combination with its own set of input_values.

This function must be used with **vDbTrigger()**.

Format File Example

```
// void DbTrigger(char *inTrigName, char *inTrigType, char
*inTrigDesc, char inTrigMeth)
vDbTrigger("NewLot", "Lotloaded", "LOOOTOOT", 'C')
//DbTriggerInput(char *inTrigName, char inInpType, char
*inInpName, char * inInpVal)
vDbTriggerInput("NewLot", 'c', "LotName", "LOTABCD")
```

Miscellaneous ExportExt (string, integer) —

Accepts two arguments, of type string and integer, respectively. The string specifies the extension name. The integer is a flag. When set, the reader appends the program class number to the specified extension.

The extension of limit files cannot be changed. It remains .lim.

**DbUpdateOplogCons —**

Accepts no arguments and has no return. Should be used with **-cons** command-line argument, to assign latest OP_LOG indexes for the consolidated entry, according to start_time/end_time time. When not used, the consolidated entry will use OP_LOG indexes from the file being loaded, regardless of start_time/end_time.

DATABASE

In addition to the database functions described in the previous section, variables that are declared as indexes can affect the binning, wafer and lot database tables.

The HIST_BIN and BIN_LOG database tables are updated from the ASCII reader only if an index or a Parameter exists that is declared as containing bin information using the database function “DbBinIndex” or “DbBinTest”. **The bin index should always be declared as an integer.** Bin names and pass-fail flags can be updated using the database function “DbBinName”.

In order to calculate the fields **m, d0, y1, y2, y3, y4, y5** in the BIN_LOG table, the **die_X** and **die_Y** indexes have to be declared using the function **DbDieXYIndexes**.

The LOT... and LOT database tables are updated from the ASCII reader only if the DbLot or vDbLot functions are used to read in the Lot ID.

The WAF... and WAFER database tables are updated from the ASCII reader only if an index exists that is declared as containing wafer information using the database function “DbWaferIndex”. **The wafer index should always be declared as a string.**

All files must contain a Test Program, and the Test Program must belong to a Program Class. This is done using the **-class** option described below.

DESIGNATION OF “BAD/UGLY” DIE LOCATIONS AT THE PROGRAM LEVEL

Any X/Y coordinates can be designated as “bad” (non-die, such as PCM) or “ugly” (partially built) so they can be excluded from zonal and ANOVA analysis in *binMAP* and from bin statistics in the database. All die marked either bad or ugly will be ignored for all the bin mapping calculations (unless re-designated as “good” from the *waferMAP* interface by a user with DBA privileges).

The function, **DbUglyDie (int)**, has been added to the ASCII reader. It accepts one argument of type integer, and is used to indicate whether a die is normal (0), bad (1), or ugly (2). This function is typically called prior to calling the **LogResult** function and only affects bin map data.

TAGGING LOTS AND WAFERS

Lots and wafers can be tagged manually or automatically. Automatic tagging is based on yield and allows for tagging lots and wafers as bad or scrap. Manual tagging is possible from the ASCII reader. This tagging state can then be used as data retrieval criteria.

The predefined tag states are:

- Normal (0)
- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)

To set the tagging flag from the ASCII reader, four functions are used:

- **DbTagAction (int, real, real)** — sets tag action and automatic tagging criteria
- **DbLotTag (int)** — used for manual lot tagging
- **DbWfTag (string, int)** — used for manual wafer tagging
- **DbSrcLotTag (int)** — used for manual source lot tagging

dpEXPORT

The database ASCII reader has the capability to generate data files that follow the *dpEXPORT* file format.

The following three command-line arguments are used for this functionality:

- | | |
|-------------|--|
| -export | Generates a <i>dpEXPORT</i> file and writes to the database. If DbSumLevel is used, dbascii will generate an export file for summary data. |
| -exportonly | Generates a <i>dpEXPORT</i> file without writing to the database. If DbSumLevel is used, dbascii will generate an export file for summary data. |

When using this export capability, the ASCII reader should be run using exactly the same command-line arguments and format files used to write to the database. (With the **-exportonly** option, the **-db** argument that is used to specify the database name is not needed.)

dpEXPORT is fully described in the “*dpEXPORT* and *dfEXPORT*” chapter of the Exensio –Yield Administrator Tools manual.

PUTTING IT ALL TOGETHER

The combination of keywords, commands and processing blocks make up the format file. A thorough concept of the input stream and a few lines in a format file will allow the user to process any defined ASCII file. In writing the format file, it is important to know the inherent nesting order of the input stream. Once this is known, it is only a matter of matching the keywords with the values being expected. Every format file can be started from the following skeleton:

```
Script      default
Const
Var
           // At least the default conditions (Test name, test number and units)
           // At least one key index
           // At least one result

Begin
           // Add separators and invalid data
           // For Database at least DbProgram has to be called
           // Set condition and index values
           // Log results
           // Log Limits

End
```

The Format file that generated Table 1 follows:



Script Example

```
VAR
//Conditions
TestNam      string      Cond
Unit         string      Cond
TestNum      integer     LimCond
PinName      string      KeyCond
Chan         integer     KeyCond
//Indexes
Wafer        integer     KeyIndex
Device       integer     KeyIndex

//Results
res          real        Result

Str          string
Lot          string

BEGIN
AddSep(' ')  // Makes space a separator of words in the data file
AddSep(':')  // Makes ":" a separator of words in the data file

DbWaferIndex("Wafer") //Specifies Index that is the Wafer ID

SkipWords(1) // Skips one Word
Str = DbProgram //Next Word becomes the database test program name
SkipWords(1)  // Skips one Word
Lot = DbLot   // Assigns the next word to the string index "Lot" and
              // adds lot to database
Str = GetWord // Assigns the next word to the string variable "Str"
```

```
While (NotEndOfFile)// Loops until end of file

    if(Str = "WAFER#")
        Wafer = GetInt // Assigns the next integer to the index "Wafer"
    Else
        If(Str = "Device#")
            Device = GetInt // Assigns the next integer to the index "Device"
        Else
            If(Str = "Test#")
                TestNum = GetInt
                TestNam = GetWord
            Else
                If(Str = "PIN")
                    SkipLines(2)
                Else
                    PinName = Str
                    Chan = GetInt
                    LSL = GetReal// Assigns the next real to LSL
                    HSL = GetReal// Assigns the next real to HSL
                    Unit = GetWord
                    LogResult(res)// Reads in the next word as
                                // real logging it as the result
                    LogLimits // Logs LSL and HSL for the
                                // current conditions
                End If
            End If
            Str = GetWord
        End While
    END
```

THE EXENSIO –YIELD INTERFACE

All the tools necessary for building and running the ASCII reader with a specific format file are available within the Exensio –Yield environment. Any text editor could be used provided that no invisible control characters are written to the file (some text editors use these control characters for formatting purposes). However, it is recommended that you use Exensio –Yield's interface for this function. It is also recommended that you store all the format files that are created in the “ASCII” sub-directory of the Exensio –Yield Formats Directory.

Type the commands and keywords for your format file in the newly created document. Once you are finished choose the **File > Save As...** menu option. In the pop-up window type the name of the format file in the file name window and select the “text” format. Ensure that the path to your saved file is correct.

Once your format file is created and saved the ASCII Reader can be run using the newly created format. If there are any errors in your format file the reader will generate an error file (err.jnk) describing the nature of the error.

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the ASCII reader, which is the format file preceded by the -fmt switch.

Example:

```
dbascii -fmt [format file]
```

Executing the ASCII reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Command-Line Arguments

If the ASCII reader is being run manually with the database option, knowing the different command-line arguments becomes necessary. The ASCII reader accepts the following options: (The first argument should always be the data file to be processed.)

Command-Line Arguments

Option	Definition	Default
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).	If not used, this action is not taken.
-bigstring	Extend string index and parameter values to 254 characters instead of the default 63.	If not used, default of 63 characters is supported.
-bincons	The -bincons argument affects the loading of binmap data only. It should only be used with -rework_action 3. The objective is to allow for the loading of partial (wafer) binmap results where the reader consolidates these partial wafer results into one wafer (as if the data was loaded from one file, as usual). For example, data for one wafer can be split between several files, with each file loaded separately with this option. The resulting database data is the same as if all the data was loaded as one file. This is accomplished by updating the wafer at the die level from one file to the next, overwriting any new die that exist in the current file.	If not used, this action is not taken.
-bingal	Only affects binMAP data. Stores bin map images in the database, for use in reports that benefit from easy access to bin map galleries. The colors used for the different bins in these images follow the standard method of handling bin colors as stored in the database BIN_COLORS table at a program level.	No binmap images.
-cassandra <max_rows>	Stores raw data, limit data, wafer level-summaries and lot-level summaries in Cassandra. The max_rows value determines the maximum number of raw data points to be stored in each blob, per parameter, per wafer. The allowed range for max_rows is [0-16000]. A value of 0 allows the reader to calculate the optimal blob size on its own.	No default. -cassandra is required with Cassandra databases.
-cassandra_level <level>	Only used with the -cassandra command-line option. The only allowed values for level are (3, -3). A value of 3 means raw data and limit data are stored in Cassandra. The value of -3 should only be applied to use Exensio –Hosted and Guided Analytics at the same time.	If not used, this action is not taken and the default is level 1.
-class	[class] Sets Program Class to class.	If not used, program class must be set within format file.

Command-Line Arguments

Option	Definition	Default
-conditions	In combination with -limitonly option, indicates that non-key conditions should be updated.	If not used, non-key conditions are not updated.
-complete_stats	This option is related to dbascii's capability to directly load statistics at wafer level and lot level into the database. In default mode, the system assumes that the data file that contained these statistics is complete. ("Complete," in this context means that for the defined statistics set for every program class, (e.g. min., max., avg.), the data file was assumed to hold all the statistics for a specified program class.) An error is generated when any statistic was missing. You have the option to override this error and insert nulls when a statistic is missing. Add this command-line option when loading statistics files.	If not used, no action is taken.
-cons	Consolidates partial wafer/lot parametric data. Wafer Consolidation is a way to consolidate a wafer that has been retested and has data in different data files. Use -cons and the ASCII reader will load the file, creating a new entry in the database with a new lg_key, and consolidating the data of the split wafer data files into one entry with the latest information. The new entry would have test_mode = C in WF_LOG table. When there are multiple readings for individual die locations, the latest data is used. Restrictions: -cons has to be used with rework_action 2. -cons has to be used with either -start_time or -end_time command-line arguments. When latest parameter data is null, it would not override older valid data values.	If not used, this action is not taken.
-cons_latest	When used, consolidation will take place per die and across all test parameters of that latest die. The consolidated information will then contain the test results for all the latest dies. Restrictions: Must be used with the -cons option. So, to trigger this operation, the two options must be used together: i.e. -cons -cons_latest When the latest parameter data is null, older valid data values would not be overridden. In other words, when -cons_latest is used: if a parameter test result is missing from the latest retest, then the consolidated result will remain empty; as opposed to using only the -cons option, where the consolidated result will be filled using the result from the previous retest (as long as at least one parameter result is available).	If not used, this action is not taken.



Command-Line Arguments

Option	Definition	Default
-datadbs	<dbspace> Places reader created tables in specified dbspace.	Default dbspace for the (database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details. If the -datadbs option is not used and the -partition option is also not used, then the default space for the schema is used to create the RES... table.
-db	[database] (Where database is the name of the database to write to)	This option is required with a valid value.
-db_accept	(If used, the processing of the files does not wait for any user input, this is useful if none of the filtering or sampling options is needed.)	NA
-dboutliers	Filters Outliers from the calculation of Wafer and Lot summaries using database limits.	If not used, no data is excluded.
-defext	[extent] Sets the extent for the database table DEF... to extent. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 20,000 Kb.	If not used, defaults to 256 Kbytes.
-dfalign	Same as Format File Function DbAlignDefect (<i>"DbAlignDefect,"</i> pg. 110). Wafer normalization takes into account the row offset (row_offset) and column offset (col_offset) arguments. See <i>"Data and Wafer Map Normalization,"</i> pg. 136.	If not used, this action is not taken, this action is not taken unless done within format file.
-dtxit	Causes the reader to error out if supplied date format(s) is wrong.	Wrong date format is ignored.
-dynamic	<max added tests> Same as semi_dynamic, but with a possibly growing number of parameters. The max added tests is optional, and sets the maximum number of tests to be added per run [percent].	NOTE: If not used, no new parameters or tests can be added.
-end_time	Uses end time as part of the detection of rework.	end time is not used
-export	Generates a dpEXPORT file and writes to the database.	If not used, this action is not taken.

Command-Line Arguments

Option	Definition	Default
-exportonly	Generates a dpEXPORT file without writing to the database. Note: The dbascii reader does not check for licensing when using the -exportonly argument.	If not used, this action is not taken.
-extend_par	Informix only: allows up to 14 parallel tables for a single program. See “Maximum Number of Parameters,” pg. 57.	If not used, then the ascii reader reverts to the default behavior for Informix databases, which is no parallel tables. This means that depending on the data types of indexes and parameters, the maximum allowed parameters could be up to 8100.
-fcenterxy	Forces calculation of center_x and center_y, only applicable when coordinate normalization is used. This only affects new programs; existing programs will set center_x and center_y to the database values when this option is used.	If not used, this action is not taken.
-file_path	[path] Sets the path to data files that may be opened using the built-in function OpenFile.	The current working directory.
-filter	[limit] Excludes data greater than limit or less than -limit (default limit is set to 1.0e+19).	Default limit is set to 1.0e+19.
-first_pass	Consolidates partial wafer/lot parametric first pass data. This option can be used with Cassandra only and the -cons option must also be used.	If not used, this action is not taken.
-fix_cond_values	In all string condition values, replace the '[' with '(' and ']' with ')’.	If not used, this action is not taken.
-fmt	[format file] (Where format file is the format to be used)	This option is required with a valid value.
-hosted_retest	Works with Exensio –Hosted data only. Specify retest and assign retest number.	If not used, will perform the default behavior: read retest from file.
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid dbspace in the database, then [dbspace] is where the indexes will be created.	Default dbspace for the (database/schema). Note: Default dbspace is the tablespace in which the database/ schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.

Command-Line Arguments

Option	Definition	Default
-in_place_alter	(Informix only) Always perform in-place alter on dynamic tables when adding new parameters. When used, this option will force the reader to use in-place alter to alter dynamic tables, regardless of log space size.	If not used, this action is not taken. Instead, the reader uses a formula to compare dynamic table sizes to available log space and if there is not enough log space, the reader will not use in-place alter and will use another method.
-keyIndexes	When used, the consolidation will happen using all key Indexes. This argument is only used only with the -cons option.	If not used, this action is not taken.
-lim_path	[path] (Where path is the path to the directory where the limit file exists if any)	The current working directory.
-limext	[extent] Sets the extent for the database table LIM... to extent. If not used, defaults to 64 Kbytes. Acceptable range is 16 Kb to 4,000 Kb.	If not used, defaults to 64 Kbytes.
-limitonly	If used, the LIMITS database table is modified by the reader, as well as the target_value and fail_bin columns in DEF....	If not used, this action is not taken; both data and limits are loaded.
-lotext	[extent] Sets the extent for the database table LOT... to extent. If not used, defaults to 64 Kbytes. Acceptable range is 16 Kb to 4000 Kb.	If not used, defaults to 64 Kbytes.
-lowercase	Forces Lot ID and Wafer ID to Lower Case	If not used, this action is not taken.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.
-md0	An alternative algorithm for calculating m and d0 at wafer level. The option takes into account the units used for the die area in the wafer configuration, for the calculation of d0. All units will be used to scale the area to a standard square-centimeter. Four units are allowed (case-insensitive): uM — micrometer mm — millimeter cm — centimeter Inch — inch Any other units are ignored and the area is unchanged.	If not used, this action is not taken.
-n	Works with Exensio –Hosted data only. Overwrite limits.	If not used, this action is not taken.

Command-Line Arguments

Option	Definition	Default
-nodata	If the reader exits with the error message: "Zero data points generated by data file/format file combination", this behavior is modified by the -nodata switch, where no error is produced in this case and the file is considered a success by the DpLoad.pl script.	If not used, the reader will be allowed to exit with the error: "Zero data points generated by data file/format file combination".
-nolimprogram	This option can only be applied with the -limitonly option ("-limitonly," pg. 131). When a program is detected as new (it does NOT already exist in the database), then the reader will exit with 0 (successful), and move the file to the Processed directory. The file will pass through with no processing.	If not used, when the reader detects a new program it will exit with an error — program does not exist in the database.
-normalize	Same as Format File Function DbNormalizeMap ("DbNormalizeMap," pg. 109). Wafer normalization takes into account the row offset (row_offset) and column offset (col_offset) arguments. See "Data and Wafer Map Normalization," pg. 136.	If not used, this action is not taken unless done within format file.
-nosampling	<i>binMAP</i> data is loaded without sampling when -nosampling is used.	Die sampling is the default behavior for wafers with more than a certain number of die or tests. Also, default die sampling numbers differ by wafer die count and by database platform. So, when -nosampling is not used, See "Die Sampling," pg. 40 for details on default behavior.
-noxycheck	Disables validation of center_x and center_y for normalization of coordinates.	If not used, center_x and center_y will be checked; if inconsistent, reader will reject the data file.
-otca	Works with Exensio –Hosted data only. Update test_category column only, for the tests in the DC_TEST_INFO table.	If not used, this action is not taken.
-otgr	Works with Exensio –Hosted data only. Update test_group column only, for the tests in the DC_TEST_INFO table.	If not used, this action is not taken.
-otin	Works with Exensio –Hosted data only. Updates the tests in the DC_TEST_INFO table.	If not used, this action is not taken.
-otsc	Works with Exensio –Hosted data only. Update test_scale column only, for the tests in the DC_TEST_INFO table.	If not used, this action is not taken.
-otu<X>	Works with Exensio –Hosted data only. Updates user-defined <X> value in DC_TEST_INFO table, where X is an integer between 1 and 23.	If not used, this action is not taken.

Command-Line Arguments

Option	Definition	Default
-otun	Works with Exensio –Hosted data only. Update Units column for the tests in DC_TEST_INFO table.	If not used, this action is not taken.
-outliers	[ni] Filters Outliers from the calculation of Wafer and Lot summaries using Box-Plot criteria.	If not used, no data is excluded.
-parallel_stats	Enables parallel calculation of the following summaries: • parametric • binning • serial • AMP Allowed values are (2-32). Oracle or Cassandra DBs only (not enabled for Informix).	If not used, this action is not taken.
-partition	<dbspace> Partitions the RES... table in the comma-separated list of dbspaces.	If the -partition option is not used, then the data space specified using the -datadbs option is used for the RES... table. If neither of these two options is specified, then the default space for the schema is used to create the RES... table.
-partition_action	[action] Directs the ascii reader to partition the data in the RES... and WAF... tables, according to the action passed. Valid values for partition_action are: • sw = weekly by start time • sm = monthly by start time • sq = quarterly by start time • ew = weekly by end time • em = monthly by end time • eq = quarterly by end time • iw = weekly by insert time • im = monthly by insert time • iq = quarterly by insert time See the “<i>Partitioning Data</i>” section in the <i>dpEXPORT</i> section of the Administrator Tools manual.	If the -partition_action option is not used, then no partitioning takes place in the RES... AND WAF... tables.
-pct_increase	Controls the table extent increase percentage for dynamic tables created by the reader. Oracle only.	0

Command-Line Arguments

Option	Definition	Default
-profile	The -profile command line argument in dbascii and UpStat allows for SQL trace capability for both Oracle and Informix. This argument is hidden, and will not show in the usage statement. The objective is to add the option of creating trace files that can be used for performance debugging. The Oracle DBA needs to re-run the script ora_setup.sql, which is provided in the dbscripts/config directory to enable the trace capability. Refer to the Exensio/Exensio –Yield installation and configuration manual for more information regarding the ora_setup.sql script.	If not used, this action is not taken.
-replace_pipe <char>	Used only with -cassandra <max_rows> option. Replaces the pipe/vertical bar character (“ ”) in string results or indexes with the passed character. Because Cassandra does not accept the “ ” character as a value in string tests and indexes, using the character with dbascii will generate an error message similar to the following: “String Test value <string value> invalid for Cassandra. (pipe) char is not allowed in strings.”	If not used, this action is not taken and errors will be generated with “ ” character in Cassandra.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results are never aged out.
-resext	[extent] Sets the extent for the database table RES... to extent. If not used, defaults to 5012 Kbytes. Acceptable range is 64 Kb-128,000 Kb.	If not used, defaults to 5012 Kbytes.
-retest	Consolidates retests. Affects all parametric data and bin data. This is related to the client-side Raw Retest Consolidation capability (“<i>Raw Retest Consolidation Tool</i>,” in the Exensio –Yield End-User Manual). The algorithm for consolidation involves the replacement of all failing devices from previous runs and can be done at wafer or lot/sub lot level. The results of the consolidation are correct wafer and lot summaries and all the passes at raw level, including a consolidated set.	If not used, this action is not taken.
-rework_action	[action] Sets the rework_action field in the PROGRAM table to action. Valid values for rework action are 1, 2, 3, 4.	1 WARNING: rework data will be ignored.
-semi_dynamic	A semi dynamic “Program” assumes a fixed number of parameters, like the default, but in addition fills up all tables with keys in the OP_LOG table such as Product and Equipment.	NOTE: If not used, and -dynamic is not used, readers will not fill in fields in tables pointed to by OP_LOG.

Command-Line Arguments

Option	Definition	Default
-special_char	When using -export or -exportonly, exported file names that contain special characters may cause problems. This feature replaces the special characters in the file names (only). This feature fixes program, lot and wafer name special characters in the exported file, but does NOT change anything inside the file itself (i.e. the actual data) - Special characters that are fixed: <i>/ converted to ##</i> <i>; converted to ##</i> <i>' converted to !!</i> <i>` converted to @@</i> <i>(converted to !#</i> <i>) converted to #!</i> Note: dbascii also handles special characters for the program name part of the exported file. By default, it replaces both "/" and ";" with "##". This default action happens independent of the use of the -special_char argument.	If not used, this action is not taken.
-ssc	This option affects rework in the dbascii reader. If you use this option and the following are true: Program Class is 3 or 12 test_mode is anything other than s/S (Stop on fail), r/R (Complete Lot Rework), or o/O (Reject Retest (recycle) or Retest opens/shorts) ...then the rework action is assumed to be in the append mode (rework_flag is not calculated).  Works in conjunction with the UpStat utility, as defined in "Summary Consolidation" section, "Statistics Update Process — Upstat" chapter of the Exensio Administrator Tools manual.	If not used, this action is not taken.
-start_time	Uses start time as part of the detection of rework.	start time is not used
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.	If not used, summaries are never aged out.
-tag_action	[Tag Action] Sets Test Program Tag Action (integer).	2 WARNING: tagging is enabled.
-triggeronly	Used to insert only trigger data into the database tables.	If not used, this action is not taken.

Command-Line Arguments

Option	Definition	Default
-trigger_db	Defines the name of the database or central schema database that the reader will insert the trigger data for. Note: The -trigger_db option can be used with the -triggeronly option to insert only trigger information into the specific database.	If not used, trigger data will be inserted to the DB name passed using -db option.
-u	usage	NA
-updatebinname	Using this option for any data type that fills in the HIST_BIN table will force an update of bin names with every file loaded.	If not used, this action is not taken. This action is not taken unless done within format file.
-updatelot	[flag] Same as the built-in function DbUpdateLot (<i>“DbUpdateLot (int),”</i> pg. 93). Used when DbUpdate lot is required, but is not included in the format file. The flag following -updatelot should have only one character, where the character is the fab flag. For example, -updatelot 1 would indicate that the fab relationship should be updated.	If not used, this action is not taken.
-uppercase	Forces Lot ID and Wafer ID to Upper Case	If not used, this action is not taken.
-use_db_wmap	By default, <i>binMAP</i> images are generated using the wafer configuration in the data file. The -use_db_wmap command line option allows the user to use the wafer configuration in the database instead. This command line option is only valid when the -bingal option is used.	If not used, <i>binMAP</i> images are generated using the wafer configuration in the data file and this action is not taken.
-v	version	NA
-validate_wmcfg	When used, it will check to see if the wafermap configuration from the data file is consistent with the current configuration in the database. If it is not, then loading will stop.	If not used, no wafermap configuration validation will be performed.
-wafext	[extent] Sets the extent for the database table WAF... to extent. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 20000 Kb.	If not used, defaults to 256 Kbytes.

Data and Wafer Map Normalization

Any data that includes X-Y coordinates, such as bin map data and defect data may be inspected in different wafer orientations. Different die indexing is also typically used. Thus, in order to align these two types of data, the data readers read and store the data in a “normalized” fashion that guarantees that they are aligned in the database.

Normalization takes the wafer configuration for each data type and shifts, rotates and/or mirrors raw data and the wafer configuration such that in the normalized state:

1. All inspection data is oriented according to a database global orientation



FRONT



CONTENTS



INDEX

exensio

specified when the schema was created using DpCreateDb.pl.

2. The die indexes (0,0) include the physical center of the wafer.
3. Positive x-direction is to the right. Positive y-direction is up.

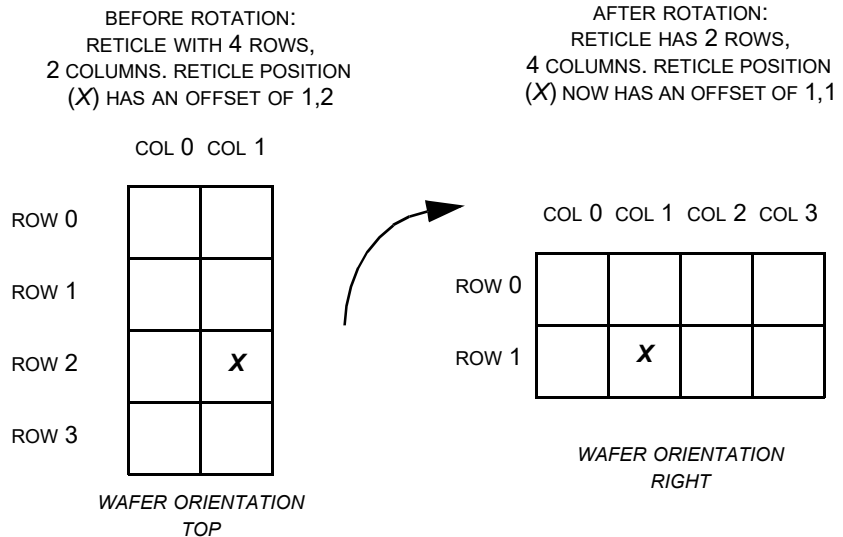
Once the data is normalized in a consistent manner, both back-end and front-end tools can make use of this fact to create summaries (Kill Ratio, Yield Impact, etc.) and map quickly for visualization.

The ascii reader has a command-line option to normalize data (*"-normalize,"* pg. 132). The wafer configuration that comes with Binmap data must be correct in order to achieve proper alignment. Often, administrators have problems identifying which die is the center die.

Additionally, the ascii reader has a command-line option to align a binmap program to a defect program (*"-dfalign,"* pg. 129). This means that pre-computed summaries will be computed between the defect program and the aligned binmap program. Typically, there will be many normalized binmap programs but only one aligned to the defect program.

Wafer configuration normalization takes into account the row offset (**row_offset**) and column offset (**col_offset**) arguments. Normalization is activated by the **-normalize** and **-dfalign** functions in the ASCII reader.

Reticle row and column offset normalization example:



REGARDLESS OF WAFER ORIENTATION AND NORMALIZATION,
COLUMN/ROW NUMBERING SYSTEM IS CONSTANT, WITH
COLUMN NUMBERS INCREASING TO THE RIGHT AND ROW
NUMBERS INCREASES GOING DOWN

Coordinate Normalization

The dbascii reader has the ability to normalize the XY coordinates for such data types as wafer sort (program class 1), bin map (program class 4), PCM (program class 5) to a standard configuration. The standard configuration has center X and center Y as (0,0) with the +X direction set to right ‘R’ and the +Y direction to up ‘U’. The flat location is set by the administrator when a new database is created. The default is bottom ‘B’.

This feature is necessary for the purposes of automatically aligning different data sources at die X/Y level.

dbascii Reader Functions; DbNormalizeMap, DbAlignDefect and DbAlignDefectOnly —

- **DbNormalizeMap** (pg. 109) — accepts no arguments and has no return. Should only be used for normalization indicating that the program **is not** to be aligned with defect data in the database. Normalization is applicable to any data type where die_x and die_y indexes have been identified using the DbDieXYIndexes function.
- **DbAlignDefect** (pg. 110) — accepts no arguments and has no return. Should only be used for normalization indicating that the program **is** to be aligned with defect data in the database. This function is only applicable to data belonging to the bin map program class. It has no effect on any other data types.
- **DbAlignDefectOnly** (pg. 110) — Reader will not perform normalization. Using this function assumes that the user has provided data in a normalized state. The function flags the reader that binmap data should be aligned with defect data.

The distinction between the usage of **DbNormalizeMap** and **DbAlignDefect** is to allow for multiple binmap programs for the same set of lots (i.e. hard bin/soft bin), in which case one could use the **DbNormalizeMap** function to allow for aligning with defect data through the *defectMAP* tool, but not for the purpose of calculating summaries, i.e. kill ratios, automatically in the database. **DbAlignDefect**, alternatively, identifies the program as the one to be automatically aligned with defect data for the purpose of calculating database summaries. In both cases, the *defectMAP* tool will allow the users to do the graphical analysis for both types of programs.

Original Configuration — The dbascii Reader maintains a copy of the original wafer map configuration in the ORG_WMAP database table. The WMAP_CONFIG table will be populated by the standardized configuration.

Normalization of XY coordinates requires that an accurate wafer map configuration is available with the data files. Without this configuration, normalization will not be possible.

Additionally, the **DbOrgDieXYIndexes** function (pg. 109) makes the original die X/Y coordinates available to the user after normalization.

ERROR MESSAGE ALERTS FOR ALARM AND EVENTS RULES MANAGER

If the reader, for any reason, outputs an error message, then the reader can send that error as a message to an AEM (Alarm and Event Rules Manager) web server, running a web service called AEM Supervision that understands the specific format of the SOAP XML message.

This action only takes place if the reader is started with the options **-wsreport AemReport -endpoint <URL>**. The URL is used to define the address of the AEM server that will receive the message.

EXAMPLE DATA FILE - MULTIBIN

[illegible]

EXAMPLE FORMAT FILE - MULTIBIN

```

SCRIPT multibin

CONST
    ProgClass = 7 // MultiBin
VAR
    //Conditions
    TestName      string      LimCond
    TestUnit      string      Cond
    TestNumber    string      Cond

    //Indexes
    Wafer         string      KeyIndex
    die_X         integer     KeyIndex
    die_Y         integer     KeyIndex
    Dut           integer     Index
    hbin          integer     Index

    //Results
    res           real        Result

    //Variables
    Str           string
    Str2          string
    Str3[10]      string
    tagAct        integer
    program       string
    process       string
    slot          integer
    badY          real
    scrY          real
    waferID       string
    waferNumber   integer
    wfUnit        string
    wfSize        real
    flatType      string
    flat          string
    dieWid        real
    dieHt         real
    centerX       real
    centerY       real
    posX          string
    posY          string
    retRows       integer
    retCols       integer
    retRowOff     integer
    retColOff     integer
    binNumber     integer
    binName       string
    binFlag       string

    tmp_real      real
    i             integer

```

```

j                integer
sleng            integer

TestNameA[33]    string
TestUnitA[33]    string
TestNumberA[33]  string

```

```
BEGIN
```

```
AddSep(' , ')
```

```
TestNumber = "NA"
```

```
TestUnit = "NA"
```

```
TestName = "NA"
```

```
DbNormalizeMap
```

```
DbWaferIndex("Wafer")
```

```
DbDieXYIndexes("die_X", "die_Y")
```

```
DbBinIndex("hbin")
```

```
DbProgClass(ProgClass)
```

```
GoTo("PROGRAM")
```

```
Str = DbProgram
```

```
GoTo("FAB")
```

```
Str = DbFab
```

```
GoTo("TECHNOLOGY")
```

```
Str = DbTechnology
```

```
GoTo("PROCESS")
```

```
process = DbProcess
```

```
GoTo("PRODUCT")
```

```
Str = DbProduct
```

```
GoTo("LOT")
```

```
Str = DbLot
```

```
GoTo("SRC_LOT")
```

```
Str = DbSrcLot
```

```
GoTo("START_TIME")
```

```
Str = GetWord
```

```
Str = vDbStartTime(Str, "%Y/%m/%d %H:%M:%S")
```

```
GoTo("END_TIME")
```

```
Str = GetWord
```

```
Str = vDbEndTime(Str, "%Y/%m/%d %H:%M:%S")
```



```

//-----
//  WAFERINFO
//-----
Goto("WAFER_SIZE")
wfSize = GetReal
wfUnit = "mm"

Goto("XDieSize")
dieWid  = GetReal

Goto("YDieSize")
dieHt   = GetReal

Goto("XCenterDie")
CenterX = GetReal

Goto("YCenterDie")
centerY = GetReal

Goto("Notch")
flat    = GetWord

Goto("flat_type")
flatType = GetWord

GoTo("POSITIVE_X")
posX = GetWord

GoTo("POSITIVE_Y")
posY = GetWord

Goto("RETROWS")
retRows = GetInt

Goto("RETCOLS")
retCols = GetInt

Goto("RETROWOFF")
retRowOff = GetInt

Goto("RETCOLOFF")
retColOff = GetInt

vDbWmapCfg(program, wfSize, wfUnit, flat[1], flatType[1],
dieWid, dieHt, centerX, centerY, posX[1], posY[1], retRows,
retCols, retRowOff, retColOff)

```

```
//-----
// <BIN> ... </BIN>
//-----
Goto("<BIN>")
Str = GetWord
While (Str <> "</BIN>")
    BinNumber = StrToInt(Str)
    BinName = GetWord
    BinFlag = GetWord
    DbBinName(BinNumber,BinName,BinFlag[1])
    Str = GetWord
End While

//-----
// <PAR> ... </PAR>
//-----
Goto("<PAR>")
Str = GetWord
i = 1
While (Str <> "</PAR>")
    TestNumberA[i]= Str
    TestNameA[i]= GetWord
    TestUnitA[i]= GetWord
    Str = GetWord
    i = i + 1
End While

//-----
// <WAFER> ... </WAFER><DATA> ... </DATA>
//-----
GoTo("WAFER")
    Wafer = GetWord
    DbWfNum(Wafer, StrToInt(After(Wafer,'_')))

//-----
// <DATA> ... </DATA>
//-----
Goto("<DATA>")
Str = GetWord
While (Str <> "</DATA>")

    die_X = GetInt
    Goto("DIE_Y")
    die_Y = GetInt

    Goto("SITE")
    Str = getWord

    Goto("HARD_BIN")
    hbin = GetInt

    Goto("DUT")
    Dut = GetInt
```



```
//-----  
// BIN number  
//-----  
For j=1 to i-1  
    TestNumber= TestNumberA[j]  
    TestName= TestNameA[j]  
    TestUnit= TestUnitA[j]  
    LogResult(res)  
  
    LPL = -0.5  
    LSL = -0.5  
    HPL = 0.5  
    HSL = 0.5  
    LogLimits  
    ClearLimits  
    j = j + 1  
End For  
  
Str = GetWord  
  
End While  
END
```


CHAPTER 3

LEH DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio environment, regardless of how the data is formatted in the original file. The Exensio LEH reader imports LEH (Lot Equipment History) data directly into the Exensio database system. The objective is to access your data and format it properly for comprehensive analysis with Exensio and other tools in the Exensio system.

The major difference between the LEH reader and the general purpose ASCII reader (*“ASCII Data Reader,”* pg. 55) is the ability of the LEH reader to handle data files that include multiple lots (one program); and the ability to handle program type **“Update - Rows,”** where data belonging to a particular row of the results table is split between many data files.

For a general overview of how Exensio readers function, refer to *“Exensio Data Readers — Overview,”* pg. 9.



The Exensio LEH Reader is a database reader and has no worksheet reader functionality. See [“Exensio Data Readers — Overview,”](#) pg. 9.



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

LEH FORMAT FILE — OVERVIEW

The Exensio format file is a way to specify the ASCII format in which the data will be expected from an incoming source. The format file is written in an easy to use script language specifically designed to navigate through an ASCII file and extract all relevant information, storing it in a pre-defined database. The language has a set of keywords (*“Keywords,”* pg. 151) and built-in functions (*“Built-in Functions,”* pg. 160) and it allows for user-defined variables, constants and separators. Blanks, horizontal and vertical tabs, new-lines, form-feeds and comments as described below (collectively referred to as “white space”) serve only to separate tokens.

OBJECTIVE

The objective of the LEH reader is to import data into a Exensio table, organizing it into a standard form regardless of how the data is organized in the original ASCII file.

Following is an example of a raw data table in the Exensio environment.

LEH Worksheet

				CNE_P	CNE_P	CNE_P	CNE_P	CNE_P
				53	53	53	53	53
Index	Index	Index	Index	TI	TO	PE	RP	LO
It.lot_id	product	trackin	trackout	CNE_P	CNE_P	CNE_P	CNE_P	CNE_P
LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVA	VF005	AV
LotB	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVA	VF005	AV
LotC	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVP	VF005	AV
LotD	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVP	VF005	AV
LotE	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotF	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotG	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotH	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV

RAW DATA TABLE FOR TYPICAL LEH WORKSHEET

LEH Design

- Program type is hard-coded to **Update - Rows** (4).
- Program class is hard-coded to **LEH** (13).
- Typically, “program” is the process flow or technology; “parameters” are the process steps; unique rows are lot and row type; and the results are items like equipment, track-in date, recipe,
- Mandatory indexes include lot and row type.
- “Nice-to-have” indexes include track-in and track-out.

- Results are strings. Maximum length:
 - 20 characters — Informix databases
 - 40 characters — Oracle databases
- Different result types are stored as new rows, with the row type index indicating the data type.

Data types must be one of the following: **TI, TO, RI, RO, TL, PE, OP, RP, LT, WN, RF, LO, PR, WC, QT, TC, RV, RT, PI, or PO**

- **TI** — Track-In
- **TO** — Track-Out
- **RI** — Process-in time.
- **RO** — Process-out time.
- **TL** — Tooling
- **PE** — Processing Equipment
- **OP** — Operator
- **RP** — Recipe
- **LT** — Current Lotid
- **WN** — Wafer Numbers
- **RF** — Rework
- **LO** — Location
- **PR** — Process
- **WC** — Wafer Count
- **QT** — Queue Time
- **TC** — Tech Code
- **RV** — Revision
- **RT** — Reticle
- **PI** — Track-In Operator
- **PO** — Track-Out Operator

The hard-coded data types are:

- PE** — must be equipment
- TI** — must be track-in
- TO** — must be track-out
- RP** — must be recipe
- OP** — must be operator

All other data types are configurable from the format file. The descriptions of the data types (names) are stored in the database table, **PROG_DATATYPE** and are set using the built-in function [DbDataDesc](#).

- As new data is inserted into the database, the track-in index is updated by the reader to the minimum value of the track-in row (row type = TI) while the track-out index is updated by the reader to the maximum value of the track-out row (row type = TO).

Even though row types are automatically translated at retrieval time to columns, the storage in the database is such that row type is a key index and each row maintains one type of data, i.e. equipment, track-in date, The following table is how the worksheet, “*LEH Worksheet*,” pg. 148, would look in the database RES... table. (Only a portion of that table is shown here.)

Index row type	Index lt.lot_id	Index product	Index trackin	Index trackout	CNE_P 53
					CNE_P
LO	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	AV
PE	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	CNEVA
RP	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	VF005
TI	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00
TO	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00

TH

Source Lot The LEH reader supports storage of data at source lot level in addition to lot level. The options are to either ignore source lot and store lot-level only, or to store both. The decision to use source lot should be made in consideration of how source lot is handled with other data sources.

Normalized Database Table The LEH_LOG table (see the “*LEH_LOG*” table description in the *Exensio Data Dictionary manual*) is automatically filled by the LEH reader and supports advanced searches through LEH data at the step level. In essence, the reader is maintaining two views to the same data; one designed for fast retrieval, while the other is designed for fast navigation through the data.

Rework The LEH reader does not support rework in the RES... table, as it does with other readers. Instead, rows are updated when the same program-lot-data type is encountered. However, LEH_LOG can support rework by invoking the command-line option -rework. By default, LEH_LOG will have no rework (i.e. new entries will overwrite old entries for the same program-lot-test).

Conditions and Indexes — Definitions

conditions — the column headings in the raw data table (**parameter name**, **unit**, etc. in the previous illustration). These conditions are the identifiers for parameters that uniquely identify each column.

key conditions — the group of conditions that uniquely identify the data column. The combination of all key conditions for any particular data column will be unique to that column.



FRONT



CONTENTS



INDEX

- indexes** — the row headings in the raw data table. Each row in the Data Table typically represents a lot that uses a “tag” or index. The indexes of a data row contains information about the lot.
- key indexes** — the group of indexes that uniquely identify the data row. The combination of all key indexes for any particular data row will be unique to that row.

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR CharIf
KeyCond	False	ANDIntegerElse
To	EQ	ConstReal Exit
Index	NE	Var String LT
KeyIndex	NOT	BeginBoolean GT
Result	LE	End WhileScript
FileName	GE	Step For mod

All built-in functions described in “*Built-in Functions*,” pg. 160, also constitute part of the list of key words.

Comments The characters /* introduce a comment that terminates with the characters */, such comments do not nest and they can not occur within strings. The characters // introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
job_nam      =      "MyJob"      //String Constant
Product      =      "MyProduct" //String Constant
NumOfBins    =      32           //Integer Constant
```


Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 254 characters)
- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

All variables declared in the format file are initialized to invalid values. Therefore, all declared variables have to be given an initial value by the user. The default values are:

- NA for string variables
- `invalid value` for integer and real variables

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The first three conditions should always be:

Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)

Note that the order that these conditions appear in is also important. The allowed types for conditions and indexes are Real, Integer and String.



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results The LEH reader supports choosing only one result of type string. Results are declared in the VAR block, and the following syntax is used:

```
Variable1,... Type      Result// To declare a result
```

Every format file must have one result declared.

The Main Block The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the LEH reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index lot_id the value of “FileName” one would

write:

lot_id = FileName//lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

ORACLE ROLLBACK TABLE

With an Oracle database, when a reader error is detected and the reader exits unexpectedly, for reasons such as:

- Connection to the Oracle engine abruptly lost.
- The user disrupting the reader by pressing **Ctrl+C** multiple times.

...the reader may exit before the buffer has fully executed.

When this happens, a rollback sequence takes place to prevent the Oracle database from becoming corrupted. The rollback statements are stored in a dynamic temporary table, `DP_ROLLBACK_STMTS_...`, per program.

If the reader exits without fully executing the rollback statements, the table will continue to hold the remaining rollback statements, for that `pg_key`. This allows the reader to recover the rollback statements and execute them, should the above exit cases occur.

When a premature exit occurs, this rollback recovery operation is triggered automatically; the administrator does not have to perform any function to make this operation occur.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

```
AddSep(':',')
```

OPERATORS

Arithmetic:	Addition	+		
	Subtraction	-		
	Multiplication	*		
	Division	/		
	Assignment	=		
	Modulus	mod		
Logical:	Used only with variables of type Boolean.			
Logical Or	OR			
Logical And	AND			
Negation	NOT			
Relational:	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression  
statements  
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression  
statements  
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression  
if-statements  
Else  
else-statements  
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

The built-in functions fall into five main categories: File navigation, data retrieval, string manipulation, database functions and mathematical functions.

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of LEH Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 161
 - **Go To** — pg. 161
 - **Separators** — pg. 161
 - **Skip Forward/Backward** — pg. 161
 - **Miscellaneous** — pg. 161
- **String Manipulation** — pg. 165
- **Database** — pg. 167
 - **Fab** — pg. 167
 - **Technology** — pg. 167
 - **Family** — pg. 168
 - **Process** — pg. 168
 - **Product** — pg. 168
 - **Program** — pg. 169
 - **Lot** — pg. 171
 - **Tagging** — pg. 171
 - **Indexes/Conditions** — pg. 172
 - **Date-Time** — pg. 172
 - **Miscellaneous** — pg. 173
- **Mathematical** — pg. 173
- **Debugging** — pg. 173
- **System** — pg. 174

File Navigation

	Go To	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the Error Code is set to 1.
		GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
		GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
		GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>		AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
		DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
		ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.
<i>Skip Forward/ Backward</i>		SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
		SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.

SkipChars (integer) — Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.

Miscellaneous **NotEndOfFile —** Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.

GetLineLen — Accepts no arguments and returns an integer. Returns the number of characters from the FilePointer to the end of line.

Data Retrieval

Data **LogResult (result) —** Accepts one argument of type result and has no return. Reads in the current word as a result and logs it to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values.

vLogResult (result, string) — Accepts two arguments of type result and string and has no return. Logs the second argument as the result to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values. The first argument passed to this function specifies the name of the result, while the second argument is the result itself.

DbDataDesc (string, string) — Accepts two arguments of type string and has no return. The first argument indicates one of the unique values of the row type index, while the second argument is a description of that row type.

For example, calling:
 DbDataDesc("RP", "Recipe")
 indicates that a row type "RP" is used in the data and Recipe is what is being stored in that row as a result.

For a list of valid row types, refer to "[LEH Design](#)," pg. 148.

OpenFile (string) — Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the **-file_path** command-line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

Strings **GetWord —** Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.

GetPrevWord — Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

GetQuotedWord (char) — Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error Code is set if no quoted word is found.)

GetLine — Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.

Sub-Strings **GetChars (integer) —** Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.

GetCharsTrim (integer, char) — Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)

- ExtractString** — Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.
- GetLeftChars (integer)** — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.
- GetRightChars (integer)** — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.
- GetMidChars (integer, integer)** — Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.
- GetWordAfter (char)** — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.
- If there are multiple occurrences of the passed character, the function returns all characters after the first instance.
- If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

Integers **GetInt —**

Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.

GetPrevInt —

Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.

Real **GetReal —**

Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.

GetPrevReal —

Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.

String Manipulation

ToLower (string) —

Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.

ToUpper (string) —

Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.

IsNumber (string) —

Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.

IsString (string) —

Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.

Right (string, integer) — Accepts two arguments of type string and integer and returns a string. Same as `GetRightChars()`, but operates on the passed string instead of the current word.

Left (string, integer) — Accepts two arguments of type string and integer and returns a string. Same as `GetRightChars()`, but operates on the passed string instead of the current word.

Mid (string, integer, integer) — Accepts three arguments of type string, integer and integer and returns a string. Same as `GetMidChars()`, but operates on the passed string instead of the current word.

After (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordAfter()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

Before (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `StrToInt("16A3")`, the output would be the integer 16; for `StrToReal("16.1A3")`, the output would be 16.1.

IntToStr (string) —	Accepts one argument of type integer and returns a string. Translates the passed integer to a string.
StrTrim (string, char) —	Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.
StrLen (string) —	Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.
Argument —	Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument -arg .

Database

<i>Fab</i>	DbFab —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbFab (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFab, but uses the passed argument instead of reading from the file.
<i>Technology</i>	DbTechnology —	Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the technology name to the current word. The relationship to the PROCESS table is established only if the function DbProcess (or vDbProcess) is used to set the current process. This function should be called any time a new technology is encountered in the data file.
	vDbTechnology (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Sets the technology name to the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.

Family **DbFamily** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the family name to the current word. The relationship to the PRODUCT table is established only if the function DbProduct (or vDbProduct) is used to set the current product. This function should be called any time a new family is encountered in the data file.

vDbFamily (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the family name to the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.

Process **DbProcess** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. This function should be called any time a new process is encountered in the data file.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but uses the passed argument instead of reading from the file. This function should be called any time a new process is encountered in the data file.

Product **DbProduct** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables. This function should be called any time a new product is encountered in the data file.

vDbProduct (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file. This function should be called any time a new product is encountered in the data file.

Program **DbProgram** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program name to the current word. This function should always be called when dumping to database and should always be the first database function called.



Data readers and database retrieval supports program names up to a limit of 255 characters.

vDbProgram (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the database test program name to the passed argument. This function should always be called when dumping to database and should always be the first database function called.



Data readers and database retrieval supports program names up to a limit of 255 characters.

DbProgRel (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. Sets the database test program release to the current word. The passed argument specifies the format of the date string being read.

vDbProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as DbProgRel but uses the first passed argument instead of GetWord as the date string.

The following describes the formatting of the DATE string:

- dd** — Day of the month as a 2-digit number.
- ddd** — Day of the week as a 3-letter abbreviation.
- mm** — Month as a 2-digit number.
- mmm** — Month as a 3-letter abbreviation.
- yy** — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
- yyyy** — Year as a 4-digit number.



*With the Oracle database, the **ddd** option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in date string.*

DbProgRev —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.

vDbProgRev (string) — Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.

vDbProgGroup (string) — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,  
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)  
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',  
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');  
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,  
INSERT_TIME, PG_GRP_DESC)  
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,  
"A new group");  
Commit;
```

vDbProgProcess (string) — Accepts one argument of type string and returns a string. Returns the passed argument. The function associates a program with a specific process. The value passed is filled in the PROCESS table and the corresponding key is filled in the **program.identifier_key** field.

DbNoRes —

Accepts no arguments and has no return value. When creating a new program, this function will set the **prog_type** column in the PROGRAM table to **40**. The default value for the **prog_type** column is **4** when this function is not used.

Using this function will disable creating and loading to the RES... tables. Disabling RES... loading can help with back-end performance, solving redundancy issues during loading of LEH data, where performance may be reduced due to loading the same data to LOG tables and RES... tables.

The command-line argument **-nores** can be used to perform the same function. (*"-nores,"* pg. 177)



*To disable creating RES... tables at the schema level, the **res_flag** column in PROG_CLASS table should be set to 0 for LEH program class.*



You cannot create custom indexes when disabling the creation and loading of RES... tables with DbNoRes. Only the indexes used by the DbIndexes format function will be allowed.

Lot vDbLotClass (string, string, string, string) —

Accepts four arguments of type string: (**Lot_Name**, **Method_Name**, **Method_Type**, **Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

Tagging DbLotTag (string, int) —

Accepts two arguments of type string and integer and has no return.

The first argument is the lot name.

The second argument is used for manual lot tagging. The passed argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (string, int) — Accepts two arguments of type string and integer and has no return.

The first argument is the source lot name.

The second argument is used for manual source lot tagging. The passed argument should be one of the following values:

No Action	(1)
Bad	(2)
Scrap	(3)
Experiment	(4)

The above list is dependent on the contents of the **TAGS** table in the database.

Indexes/Conditions

DbIndexes (string, string, string, string) —

Accepts four arguments of type string and has no return. The passed arguments must be the name (in double quotes) of one of the declared indexes. The order of the arguments is lot index, track-in index, track-out index, and the row type index. (Also, if the track-in or track-out indexes are missing they should be set to “NA”. The lot index and row type index are mandatory.)

DbSrcLotIndex (string) — Accepts one argument of type string, which should be the name of the **source lot** index; and has no return value.

DbStepCond (string) — Accepts one argument of type string, which should be the name of the **step** condition; and has no return value.

DbStageCond (string) — Accepts one argument of type string, which should be the name of the **stage** condition; and has no return value.

Date-Time

DbDtFormat (string) — Accepts one argument of type string and has no return. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file. This affects the track-in and track-out indexes.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
%B Full month name.
%d Day of the month as a decimal [01,...,31].
%H 24 hour clock.
%I 12 hour clock
%M Minute as a decimal [00,...,59].
%m Month as a decimal [01,...,12].
%p a.m. or p.m.
%S Second as a decimal [00,...,59].
%y Year as a decimal [00,...,99].
%Y Year as a 4-digit decimal.
%% Allows for percent in the string.

As an example the format for the following string:

“jul 1 1996 05:10:46”

would be:

“%b %d %Y%H:%M:%S”

Miscellaneous DbCalcQueueTime —

Accepts no arguments and has no return value. This function will calculate lot queue time between steps. Queue time is the difference between track out of step and track in of next step. The reader will use latest lot data and will internally create a row type for this (**TT**) that is called in the RES... table. This data will be also stored in LEH_LOG table's **queue_time** column.

Mathematical Abs (real) —

Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.

Sqr (real) —

Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.

Debugging Print (...) —

Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.

PrintToFile (...) —

Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.

ErrorCode —

Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the Error Code are **GoTo**, **GoBackTo**, and **OpenFile**.

ExitScript (string) —

Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

System **System (string)** —

Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function. Accepts up to 254 characters.

THE EXENSIO –YIELD INTERFACE

All the tools necessary for building and running the LEH reader with a specific format file are available within the Exensio environment. To build a new format file, use the **File > New > Script** menu command. This command will open Exensio's text editor. Any text editor could be used provided that no invisible control characters are written to the file (some text editors use these control characters for formatting purposes). However, it is recommended that you use Exensio's interface for this function.

Type the commands and keywords for your format file in the newly created document. Once you are finished choose the **File > Save As...** menu option. In the pop-up window type the name of the format file in the file name window and select the "text" format. Ensure that the path to your saved file is correct.

Once your format file is created and saved the LEH reader can be run using the newly created format.

Running the LEH Reader

The LEH reader accepts the following options: (The first argument should always be the data file to be processed, unless the objective is to only parse the format file.)

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the LEH reader, which is the format file preceded by the -fmt switch.

Example:

```
leh -fmt [format file]
```

Executing the LEH reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Command-Line Arguments

Option	Definition	Default
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).	If not used, this action is not taken.
-cassandra	Stores LEH_LOG table in the Cassandra database. Note: When using this option, rework action is determined when creating the program and can't be changed. E.g. if the options -rework and -start_time are used when creating the program, then these options must be used every time when loading to that program and can't change.	If not used, this action is not taken.
-cass_threads	[# of threads] Number of threads to create when storing the data to Cassandra (Allowed range [1-50]; default=5).	If not used, defaults to 5.
-datadbs	<dbspace> Places reader created tables in specified dbspace.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-db	[database]	This option is required with a valid value.
-db_accept	Automatically sets the accept_data flag in the database	NA
-defext	[Kbytes] Sets def... Table extent. If not used, defaults to 128 Kbytes. Acceptable range is 32 Kb to 5012 Kb.	If not used, defaults to 128 Kbytes.
-end_time	Includes end_time as part of the detection of update, so that at data loading older data will not overwrite newer, existing data in the database.	If not used, the latest files loaded will overwrite existing database data, regardless of data time stamp.
-file_path	[Path to data File]	The current working directory.
-fix_cond_values	In all string condition values, replace the '[' with '(' and ']' with ')'.	If not used, this action is not taken.
-fmt	[format file]	This option is required with a valid value.

Option	Definition	Default
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-lowercase	Forces lot_id and wf_id to lowercase	If not used, this action is not taken.
-maxtests	[max tests] Sets the maximum number of tests to be added per run [percent].	If not used, unlimited number of tests could be added.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.
-nores	Disables creating and loading to RES... Tables. This performs the same functionality as the DbNoRes built-in function. See “DbNoRes,” pg. 171, for more information. Note: You cannot create custom indexes when disabling the creation and loading of RES... tables with -nores. Only the indexes used by the <i>DbIndexes</i> format function will be allowed.	If not used, this action is not taken.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results never age.
-resex	[Kbytes] Sets res... Table extent. If not used, defaults to 50,012 Kbytes. Acceptable range is 5000 Kb to 200024 Kb.	If not used, defaults to 50,012 Kbytes.
-rework	Enable rework in LEH_LOG.	WARNING: If not used, new entries in LEH_LOG overwrite old entries.
-start_time	Includes start_time as part of the detection of update, so that at data loading older data will not overwrite newer, existing data in the database.	If not used, the latest files loaded will overwrite existing database data, regardless of data time stamp.
-u	usage	NA

Option	Definition	Default
-uppercase	Forces lot_id and wf_id to uppercase	If not used, this action is not taken.
-v	version	NA



CHAPTER 4

WEH DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio environment, regardless of how the data is formatted in the original file. The Exensio WEH reader imports WEH (Wafer Equipment History) data directly into the Exensio database system. The objective is to access your data and format it properly for comprehensive analysis with Exensio and other tools in the Exensio system.

The major difference between the WEH reader and the general purpose ASCII reader (*"ASCII Data Reader,"* pg. 55) is the ability of the WEH reader to handle data files that include multiple lots (one program); and the ability to handle program type **"Update - Rows,"** where data belonging to a particular row of the results table is split between many data files.

For a general overview of how Exensio readers function, refer to *"Exensio Data Readers — Overview,"* pg. 9.



The Exensio WEH Reader is a database reader and has no worksheet reader functionality. See ["Exensio Data Readers — Overview,"](#) pg. 9.



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

WEH Data Retrieval

The WEH level of the Fab Retrieval window is used to retrieve Wafer Equipment History (WEH) data directly. Like the other retrieval windows, the WEH retrieval window is able to handle multiple parameters (programs) and multiple lots.

The functionality of the WEH Retrieval window is fully described in “*LEH/WEH Data Retrieval*” in the Exensio User Manual.

From the retrieval window, select the one of the statistics retrieval modes from the **Retrieval Type** drop-down list and select the **WEH** program class. (Refer to the “*Data Retrieval*” chapter in the Exensio End-User Manual.

WEH FORMAT FILE — OVERVIEW

The Exensio format file is a way to specify the ASCII format in which the data will be expected from an incoming source. The format file is written in an easy to use script language specifically designed to navigate through an ASCII file and extract all relevant information, storing it in a pre-defined database. The language has a set of keywords (*“Keywords,”* pg. 184) and built-in functions (*“Built-in Functions,”* pg. 194) and it allows for user-defined variables, constants and separators. Blanks, horizontal and vertical tabs, new-lines, form-feeds and comments as described below (collectively referred to as “white space”) serve only to separate tokens.

OBJECTIVE

The objective of the WEH reader is to import data into a Exensio table, organizing it into a standard form regardless of how the data is organized in the original ASCII file.

Following is an example of a raw data table in the Exensio environment.

WEH Worksheet

				CNE_P	CNE_P	CNE_P	CNE_P	CNE_P
				53	53	53	53	53
Index	Index	Index	Index	TI	TO	PE	RP	LO
It.lot_id	product	trackin	trackout	CNE_P	CNE_P	CNE_P	CNE_P	CNE_P
LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVA	VF005	AV
LotB	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVA	VF005	AV
LotC	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVP	VF005	AV
LotD	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00	17-Sep-98 00:00	CNEVP	VF005	AV
LotE	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotF	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotG	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV
LotH	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	18-Sep-98 00:00	18-Sep-98 00:00	CNEVA	VF005	AV

RAW DATA TABLE FOR TYPICAL WEH WORKSHEET

WEH Design

- Program type is hard-coded to **Update - Rows** (4).
- Program class is hard-coded to **WEH** (19).
- Typically, “program” is the process flow or technology; “parameters” are the process steps; unique rows are lot, wafer, and row type; and the results are items like equipment, track-in date, recipe,
- Mandatory indexes include lot, wafer and row type.
- “Nice-to-have” indexes include track-in and track-out.

- Results are strings. Maximum length:
 - 20 characters — Informix databases
 - 40 characters — Oracle databases
- Different result types are stored as new rows, with the row type index indicating the data type.

Data types must be one of the following: **TI, TO, TL, PE, OP, RP, LT, WN, RF, LO, PR, WC, QT, TC, RV, RT, PI, or PO**

- **TI** — Track-In
- **TO** — Track-Out
- **TL** — Tooling
- **PE** — Processing Equipment
- **OP** — Operator
- **RP** — Recipe
- **LT** — Current Lotid
- **WN** — Wafer Numbers
- **RF** — Rework
- **LO** — Location
- **PR** — Process
- **WC** — Wafer Count
- **QT** — Queue Time
- **TC** — Tech Code
- **RV** — Revision
- **RT** — Reticle
- **PI** — Track-In Operator
- **PO** — Track-Out Operator

The hard-coded data types are:

- PE** — must be equipment
- TI** — must be track-in
- TO** — must be track-out
- RP** — must be recipe
- OP** — must be operator
- LT** — must be child lot

All other data types are configurable from the format file. The descriptions of the data types (names) are stored in the database table, `PROG_DATATYPE` and are set using the built-in function [DbDataDesc](#).

- As new data is inserted into the database, the track-in index is updated by the reader to the minimum value of the track-in row (row type = TI) while the track-out index is updated by the reader to the maximum value of the track-out row (row type = TO).

Even though row types are automatically translated at retrieval time to columns, the storage in the database is such that row type is a key index and each row maintains one type of data, i.e. equipment, track-in date, The following table is how the worksheet, “*WEH Worksheet*,” pg. 181, would look in the database RES... table. (Only a portion of that table is shown here.)

Index row type	Index lt.lot_id	Index product	Index trackin	Index trackout	CNE_P 53 CNE_P
LO	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	AV
PE	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	CNEVA
RP	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	VF005
TI	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00
TO	LotA	PROD-Z	18-Sep-98 13:36	18-Sep-98 14:26	17-Sep-98 00:00

TH

Source Lot The WEH reader supports storage of data at two different levels:

- source lot level
- lot level

The options are to either ignore source lot and store lot-level only, or to store source lot then optionally include the child lot data.

The decision to use source lot should be made in consideration of how source lot is handled with other data sources. If source lot data exists elsewhere, the source lot level should be used.

To activate source lot level, two steps must be performed in the format file:

1. Use the built-in function **DbUseSrcLot** (pg. 206).
2. Insert the source lot variable in the first argument of the built-in function **DbIndexes** (pg. 206).

To activate lot level, one step must be performed in the format file:

1. Insert the lot variable in the first argument of the built-in function **DbIndexes** (pg. 206).

Normalized Database Table

The WEH_LOG table (see the “*WEH_LOG*” table description in the Exensio Data Dictionary manual) is automatically filled by the WEH reader and supports advanced searches through WEH data at the step level. In essence, the reader is maintaining two views to the same data; one designed for fast retrieval, while the other is designed for fast navigation through the data.

Rework The WEH reader does not support rework in the RES... table, as it does with other readers. Instead, rows are updated when the same program-lot-data type is encountered. However, WEH_LOG can support rework by invoking the command-line option -rework. By default, WEH_LOG will have no rework (i.e. new entries will overwrite old entries for the same program-lot-test).

Conditions and Indexes — Definitions

conditions	— the column headings in the raw data table (parameter name , unit , etc. in the previous illustration). These conditions are the identifiers for parameters that uniquely identify each column.
key conditions	— the group of conditions that uniquely identify the data column. The combination of all key conditions for any particular data column will be unique to that column.
indexes	— the row headings in the raw data table. Each row in the Data Table typically represents a lot that uses a “tag” or index. The indexes of a data row contains information about the lot.
key indexes	— the group of indexes that uniquely identify the data row. The combination of all key indexes for any particular data row will be unique to that row.

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	CharIf
KeyCond	False	AND	IntegerElse
To	EQ	Const	Real Exit
Index	NE	Var	String LT
KeyIndex	NOT	Begin	Boolean GT
Result	LE	End	WhileScript
FileName	GE	Step	For mod

All built-in functions described in “*Built-in Functions*,” pg. 194, also constitute part of the list of key words.

Comments

The characters `/*` introduce a comment that terminates with the characters `*/`, such comments do not nest and they can not occur within strings. The characters `//` introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
job_nam      =      "MyJob"      //String Constant
Product      =      "MyProduct" //String Constant
NumOfBins    =      32           //Integer Constant
```

Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 254 characters)
- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

All variables declared in the format file are initialized to invalid values. Therefore, all declared variables have to be given an initial value by the user. The default values are:

- NA for string variables
- `invalid value` for integer and real variables

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The first three conditions should always be:

Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)

Note that the order that these conditions appear in is also important. The allowed types for conditions and indexes are Real, Integer and String.



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results The WEH reader supports choosing only one result of type string. Results are declared in the VAR block, and the following syntax is used:

```
Variable1,... Type      Result// To declare a result
```

Every format file must have one result declared.

The Main Block The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the WEH reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index lot_id the value of “FileName” one would

write:

lot_id = FileName//lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

ORACLE ROLLBACK TABLE

With an Oracle database, when a reader error is detected and the reader exits unexpectedly, for reasons such as:

- Connection to the Oracle engine abruptly lost.
- The user disrupting the reader by pressing **Ctrl+C** multiple times.

...the reader may exit before the buffer has fully executed.

When this happens, a rollback sequence takes place to prevent the Oracle database from becoming corrupted. The rollback statements are stored in a dynamic temporary table, `DP_ROLLBACK_STMTS_...`, per program.

If the reader exits without fully executing the rollback statements, the table will continue to hold the remaining rollback statements, for that `pg_key`. This allows the reader to recover the rollback statements and execute them, should the above exit cases occur.

When a premature exit occurs, this rollback recovery operation is triggered automatically; the administrator does not have to perform any function to make this operation occur.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

```
AddSep(':',')
```

OPERATORS

Arithmetic:	Addition	+		
	Subtraction	-		
	Multiplication	*		
	Division	/		
	Assignment	=		
	Modulus	mod		
Logical:	Used only with variables of type Boolean.			
Logical Or	OR			
Logical And	AND			
Negation	NOT			
Relational:	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression
    statements
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression
    statements
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression
    if-statements
Else
    else-statements
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

The built-in functions fall into five main categories: File navigation, data retrieval, string manipulation, database functions and mathematical functions.

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of WEH Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 195
 - **Go To** — pg. 195
 - **Separators** — pg. 195
 - **Skip Forward/Backward** — pg. 195
 - **Miscellaneous** — pg. 196
- **String Manipulation** — pg. 199
- **Database** — pg. 201
 - **Fab** — pg. 201
 - **Technology** — pg. 201
 - **Family** — pg. 202
 - **Process** — pg. 202
 - **Product** — pg. 202
 - **Program** — pg. 203
 - **Lot** — pg. 205
 - **Wafer** — pg. 205
 - **Tagging** — pg. 205
 - **Indexes/Conditions** — pg. 206
 - **Date-Time** — pg. 207
 - **Miscellaneous** — pg. 207
- **Hardware** — pg. 208
- **Mathematical** — pg. 208
- **Debugging** — pg. 209
- **System (string)** — pg. 209

File Navigation

<i>Go To</i>	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the Error Code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.
<i>Skip Forward/ Backward</i>	SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
	SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.

SkipChars (integer) — Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.

Miscellaneous **NotEndOfFile —** Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.

GetLineLen — Accepts no arguments and returns an integer. Returns the number of characters from the FilePointer to the end of line.

Data Retrieval

Data **LogResult (result) —** Accepts one argument of type result and has no return. Reads in the current word as a result and logs it to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values.

vLogResult (result, string) — Accepts two arguments of type result and string and has no return. Logs the second argument as the result to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values. The first argument passed to this function specifies the name of the result, while the second argument is the result itself.

DbDataDesc (string, string) — Accepts two arguments of type string and has no return. The first argument indicates one of the unique values of the row type index, while the second argument is a description of that row type.

For example, calling:
DbDataDesc("RP", "Recipe")
indicates that a row type "RP" is used in the data and Recipe is what is being stored in that row as a result.

For a list of valid row types, refer to *"WEH Design,"* pg. 181.

OpenFile (string) — Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the **-file_path** command-line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

Strings **GetWord —** Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.

GetPrevWord — Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

GetQuotedWord (char) — Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error Code is set if no quoted word is found.)

GetLine — Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.

Sub-Strings **GetChars (integer) —** Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.

GetCharsTrim (integer, char) — Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)

- ExtractString** — Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.
- GetLeftChars (integer)** — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.
- GetRightChars (integer)** — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.
- GetMidChars (integer, integer)** — Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.
- GetWordAfter (char)** — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.
- If there are multiple occurrences of the passed character, the function returns all characters after the first instance.
- If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

Integers **GetInt** —

Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.

GetPrevInt —

Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.

Real **GetReal** —

Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.

GetPrevReal —

Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.

String Manipulation

ToLower (string) —

Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.

ToUpper (string) —

Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.

IsNumber (string) —

Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.

IsString (string) —

Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.

Right (string, integer) — Accepts two arguments of type string and integer and returns a string. Same as `GetRightChars()`, but operates on the passed string instead of the current word.

Left (string, integer) — Accepts two arguments of type string and integer and returns a string. Same as `GetRightChars()`, but operates on the passed string instead of the current word.

Mid (string, integer, integer) — Accepts three arguments of type string, integer and integer and returns a string. Same as `GetMidChars()`, but operates on the passed string instead of the current word.

After (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordAfter()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

Before (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `'StrToInt ("16A3")'`, the output would be the integer 16; for `'StrToReal ("16.1A3")'`, the output would be 16.1.

IntToStr (string) —	Accepts one argument of type integer and returns a string. Translates the passed integer to a string.
StrTrim (string, char) —	Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.
StrLen (string) —	Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.
Argument —	Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument -arg .

Database

<i>Fab</i>	DbFab —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbFab (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFab, but uses the passed argument instead of reading from the file.
<i>Technology</i>	DbTechnology —	Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the technology name to the current word. The relationship to the PROCESS table is established only if the function DbProcess (or vDbProcess) is used to set the current process. This function should be called any time a new technology is encountered in the data file.
	vDbTechnology (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Sets the technology name to the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.

Family **DbFamily** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the family name to the current word. The relationship to the PRODUCT table is established only if the function DbProduct (or vDbProduct) is used to set the current product. This function should be called any time a new family is encountered in the data file.

vDbFamily (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the family name to the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.

Process **DbProcess** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. This function should be called any time a new process is encountered in the data file.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but uses the passed argument instead of reading from the file. This function should be called any time a new process is encountered in the data file.

Product **DbProduct** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables. This function should be called any time a new product is encountered in the data file.

vDbProduct (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file. This function should be called any time a new product is encountered in the data file.

Program DbProgram —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program name to the current word. This function should always be called when dumping to database and should always be the first database function called.



Data readers and database retrieval supports program names up to a limit of 255 characters.

vDbProgram (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the database test program name to the passed argument. This function should always be called when dumping to database and should always be the first database function called.



Data readers and database retrieval supports program names up to a limit of 255 characters.

DbProgRel (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. Sets the database test program release to the current word. The passed argument specifies the format of the date string being read.

vDbProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as DbProgRel but uses the first passed argument instead of GetWord as the date string.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.

ddd — Day of the week as a 3-letter abbreviation.

mm — Month as a 2-digit number.

mmm — Month as a 3-letter abbreviation.

yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.

yyyy — Year as a 4-digit number.



*With the Oracle database, the **ddd** option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in date string.*

DbProgRev —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.

vDbProgRev (string) — Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.

vDbProgGroup (string) — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

vDbProgProcess (string) — Accepts one argument of type string and returns a string. Returns the passed argument. The function associates a program with a specific process. The value passed is filled in the PROCESS table and the corresponding key is filled in the **program.identifier_key** field.

DbNoRes —

Accepts no arguments and has no return value. When creating a new program, this function will set the **prog_type** column in the PROGRAM table to **40**. The default value for the **prog_type** column is **4** when this function is not used.

Using this function will disable creating and loading to the RES... tables. Disabling RES... loading can help with back-end performance, solving redundancy issues during loading of WEH data, where performance may be reduced due to loading the same data to LOG tables and RES... tables.

The command-line argument **-nores** can be used to perform the same function. (*“-nores,”* pg. 212)



*To disable creating RES... tables at the schema level, the **res_flag** column in PROG_CLASS table should be set to 0 for WEH program class.*



You cannot create custom indexes when disabling the creation and loading of RES... tables with DbNoRes. Only the indexes used by the DbIndexes format function will be allowed.

Lot vDbLotClass (string, string, string, string) —

Accepts four arguments of type string: (**Lot_Name**, **Method_Name**, **Method_Type**, **Class_Name**) and has no return. The function supports multiple classes, per lot.

Wafer DbWfDesc (string, string, string) —

Accepts three arguments of type string and has no return. The first argument is the wf_id; the second argument is the wafer description (maximum 64 characters), the third argument is the lot_id. The description goes into the **wf_desc** column of the WAFER table, but only if the wafer is new.

Tagging DbLotTag (string, int) —

Accepts two arguments of type string and integer and has no return.

The first argument is the lot name.

The second argument is used for manual lot tagging. The passed argument should be one of the following values:

No Action	(1)
Bad	(2)
Scrap	(3)
Experiment	(4)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (string, int) — Accepts two arguments of type string and integer and has no return.

The first argument is the source lot name.

The second argument is used for manual source lot tagging. The passed argument should be one of the following values:

No Action	(1)
Bad	(2)
Scrap	(3)
Experiment	(4)

The above list is dependent on the contents of the **TAGS** table in the database.

Indexes/Conditions

DbIndexes (string, string, string, string, string) —

Accepts five arguments of type string and has no return. The passed arguments must be the name (in double quotes) of one of the declared indexes. The order of the arguments is lot index, wafer index, track-in index, track-out index, and the row type index. (Also, if the track-in or track-out indexes are missing they should be set to “NA”. The lot index, wafer index, and row type index are mandatory key indexes.)

DbUseSrcLot —

Accepts no arguments and has no return value. It is used to activate the source lot level. At the source lot level, the source lot name is placed in the lot index and, optionally, the child lot name is placed in the “**LT**” row type.

DbStepCond (string) —

Accepts one argument of type string, which should be the name of the **step** condition; and has no return value.

DbStageCond (string) —

Accepts one argument of type string, which should be the name of the **stage** condition; and has no return value.

Date-Time DbDtFormat (string) — Accepts one argument of type string and has no return. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file. This affects the track-in and track-out indexes.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
 %B Full month name.
 %d Day of the month as a decimal [01,...,31].
 %H 24 hour clock.
 %I 12 hour clock
 %M Minute as a decimal [00,...,59].
 %m Month as a decimal [01,...,12].
 %p a.m. or p.m.
 %S Second as a decimal [00,...,59].
 %y Year as a decimal [00,...,99].
 %Y Year as a 4-digit decimal.
 %% Allows for percent in the string.

As an example the format for the following string:

“jul 1 1996 05:10:46”

would be:

“%b %d %Y%H:%M:%S”

Miscellaneous vDbWaferClass (string, string, string, string) — Accepts four arguments of type string and has no returns. The passed arguments are of the following types:

wafer_name - string
method_name - string
method_type - string
class_name - string

Only those wafers that are used in the data file will have their classes and methods logged. Wafers that are only passed to **vDbWaferClass()** but not used elsewhere in the data file will be ignored.

For those wafers logged, the following tables will be populated: CLS_METHOD, WAFER_CLASS, and WFCLS2WF. CLS_METHOD is populated with the **method_name** and **method_type**. WAFER_CLASS is populated with the **class_name**. WFCLS2WF is populated with the corresponding **wafer_names** and **class_names**.

DbWfNum (string, string, int) —

Accepts three arguments of type string, string and integer and has no return. The first argument should be the lot ID, the second argument should be the wafer ID, the third argument should be the wafer number associated with that wafer ID. (This function allows populating the **wf_num** column in the **WAFER** table. It should be called for every wafer/wafer number combination in the data file.)

DbCalcQueueTime —

Accepts no arguments and has no return value. This function will calculate lot queue time between steps. Queue time is the difference between track out of step and track in of next step. The reader will use latest wafer data and will internally create a row type for this (**TT**) that is called in the RES... table. This data will be also stored in WEH_LOG table's **queue_time** column.

Hardware ChamberRowType (string)

—

Accepts one argument of type string, two characters long, which should be the name of the data type associated with the chamber. The function has no return value.

This function allows you to generate the list of equipments in the Exensio –Control UI so that chambers can be specified. This feature is implemented because both chambers and equipments are listed, by default, since they are both stored in the same database table.

Data type inputs must be one of the following, of those allowed by the WEH reader: **TL, LT, WN, RF, LO, PR, WC, QT, TC, RV, RT, PI, or PO**.

Data types inputs cannot be one of the hard-coded date types: **PE, RP, TI, TO, OP**.

Once the user selects one of the data types, any string associated with the data type specified by the **ChamberRowType** will be the “Chamber ID”.

Mathematical Abs (real) —

Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.

	Sqr (real) —	Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.
Debugging	Print (...) —	Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.
	PrintToFile (...) —	Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.
	ErrorCode —	Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the Error Code are GoTo , GoBackTo , and OpenFile .
	ExitScript (string) —	Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message. If the passed string is a null string "" no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the Processed directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the NotProcessed directory.
System	System (string) —	Accepts one argument of type string and returns an integer. This function allows access to the "C" System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the "C" System function. Accepts up to 254 characters.

THE EXENSIO –YIELD INTERFACE

All the tools necessary for building and running the WEH reader with a specific format file are available within the Exensio environment. To build a new format file, use the **File > New > Script** menu command. This command will open Exensio's text editor. Any text editor could be used provided that no invisible control characters are written to the file (some text editors use these control characters for formatting purposes). However, it is recommended that you use Exensio's interface for this function.

Type the commands and keywords for your format file in the newly created document. Once you are finished choose the **File > Save As...** menu option. In the pop-up window type the name of the format file in the file name window and select the "text" format. Ensure that the path to your saved file is correct.

Once your format file is created and saved the WEH reader can be run using the newly created format.

Running the WEH Reader

The WEH reader accepts the following options: (The first argument should always be the data file to be processed, unless the objective is to only parse the format file.)

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the WEH reader, which is the format file preceded by the -fmt switch.

Example:

```
weh -fmt [format file]
```

Executing the WEH reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Option	Definition	Default
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-lowercase	Forces lot_id and wf_id to lowercase	If not used, this action is not taken.
-maxtests	[max tests] Sets the maximum number of tests to be added per run [percent].	If not used, unlimited number of tests could be added.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.
-nores	Disables creating and loading to RES... Tables. This performs the same functionality as the DbNoRes built-in function. See “DbNoRes,” pg. 205, for more information. Note: You cannot create custom indexes when disabling the creation and loading of RES... tables with -nores. Only the indexes used by the <i>DbIndexes</i> format function will be allowed.	If not used, this action is not taken.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results never age.
-resex	[Kbytes] Sets res... Table extent. If not used, defaults to 50,012 Kbytes. Acceptable range is 5000 Kb to 200024 Kb.	If not used, defaults to 50,012 Kbytes.
-rework	Enable rework in WEH_LOG.	WARNING: If not used, new entries in WEH_LOG overwrite old entries.
-start_time	Includes start_time as part of the detection of update, so that at data loading older data will not overwrite newer, existing data in the database.	If not used, the latest files loaded will overwrite existing database data, regardless of data time stamp.
-u	usage	NA

Option	Definition	Default
-uppercase	Forces lot_id and wf_id to uppercase	If not used, this action is not taken.
-v	version	NA

CHAPTER 5

FAB DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file. These readers can access data directly from files or through the Exensio –Yield database. Whether the data is imported through the database or directly from a data file, the objective is the same: to access your data and format it properly for comprehensive analysis.

The major difference between the Fab reader and the general purpose ASCII reader (*“ASCII Data Reader,”* pg. 55) is the ability of the Fab reader to handle data files that include multiple programs and multiple lots.



The Fab reader has replaced the Metrology reader that was offered in previous versions of the software because, unlike the old Metrology reader, the Fab reader can handle multiple-parameter programs. It also supports dynamic-type programs and multiple levels of data — lot, wafer, and site.

For a general overview of how Exensio –Yield readers function, refer to *“Exensio Data Readers — Overview,”* pg. 9.



Number of Parameter Columns Limitation — The Fab reader cannot handle more than 3,980 data columns (indexes and parameters, combined).



Worksheet Reader — The Exensio –Yield Fab Reader is a database reader and has no worksheet reader functionality. It therefore runs only on user configurations which include the Exensio –YieldExensio –Yield. See *“Exensio Data Readers — Overview,”* pg. 9.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

Fab Data Retrieval

Data retrieval into Exensio –Yield is accomplished in the same way as Ascii data, through the General Data Retrieval window — .

For site data, use the **Raw...** Retrieval type — **23-FabSite** program class. For wafer and lot raw data and statistics, use one of the **Stats...** retrieval types — **23-FabSite**, **24-FabWaf**, and **25-FabLot** program classes.

FAB FORMAT FILE — OVERVIEW

The Exensio –Yield format file is a way to specify the ASCII format in which the data will be expected from an incoming source (usually a disk file). The format file is written in an easy to use script language specifically designed to navigate through an ASCII file and extract all relevant information, storing it in a pre-defined database. The language has a set of keywords (*“Keywords,”* pg. 219) and built-in functions (*“Built-in Functions,”* pg. 233) and it allows for user-defined variables, constants and separators. Blanks, horizontal and vertical tabs, new-lines, form-feeds and comments as described below (collectively referred to as “white space”) serve only to separate tokens.

OBJECTIVE

The objective of the Fab reader is to import data into a Exensio –Yield table, organizing it into a standard form regardless of how the data is organized in the original ASCII file.

Conditions and Indexes — Definitions

conditions	— the column headings in the raw data table (parameter name , unit , etc. in the previous illustration). These conditions are the identifiers for parameters that uniquely identify each column.
key conditions	— the group of conditions that uniquely identify the data column. The combination of all key conditions for any particular data column will be unique to that column.
indexes	— the row headings in the raw data table. Each row in the Data Table typically represents a site that uses a “tag” or index. The indexes of a data row contains information about the site, such as wafer, lot,
key indexes	— the group of indexes that uniquely identify the data row. The combination of all key indexes for any particular data row will be unique to that row.
limit conditions	— limit conditions are the group of conditions that uniquely identify a row in the limits table. The combination of all limit conditions for any particular row in the limits table will be unique to that row.

Every column (parameter) in the data table could possibly have limits associated with it, and therefore a corresponding row in the limits sheet. Limit conditions determine which parameters are to appear as new rows in the limits sheet.



LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	CharIf	HPL
KeyCond	False	AND	IntegerElse	LOL
LimCond	EQ	ConstReal	Exit	HOL
Index	NE	Var	String	LSL
KeyIndex	NOT	Begin	Boolean	HSLHWL
Result	LE	End	While	LPL
FileName	GE	Step	For	GT
Script	mod			LT

All built-in functions described in “*Built-in Functions*,” pg. 233, also constitute part of the list of key words.

Comments The characters /* introduce a comment that terminates with the characters */, such comments do not nest and they can not occur within strings. The characters // introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
  job_nam      =      “MyJob”      //String Constant
  Product      =      “MyProduct” //String Constant
  NumOfBins    =      32             //Integer Constant
```

Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 255 characters)
- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

All variables declared in the format file are initialized to invalid values. Therefore, all declared variables have to be given an initial value by the user. The default values are:

- NA for string variables
- `invalid value` for integer and real variables

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                   condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                   condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The first three conditions should always be:

Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)

Note that the order that these conditions appear in is also important. The allowed types for conditions and indexes are Real, Integer and String.



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results The Fab reader supports choosing more than one result, each of a possibly different type (only real or string results are supported). Results are declared in the VAR block and the following syntax is used:

```
Variable1,Variable2,...TypeResult// To declare a result
```

Every format file must have at least one result declared.

String Results Storing string results is accomplished by defining a results variable of type string, and using that variable's name in the **LogResult** or **vLogResult** functions as the first passed argument. Note that a single parameter must always have the same data type.

Parameters that are of type string do not generate lot or wafer summaries. Instead, the last string value read is used to fill all the statistics at the wafer and lot level.

The Main Block The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

LIMITS AND SCALING



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.

Limits There are eight types of limits:

LSL	Low Spec. Limits
HSL	High Spec. Limits
LPL	Low Production Limits
HPL	High Production Limits
LOL	Low Outlier Limits
HOL	High Outlier Limits
LWL	Low What If Limits
HWL	High What If Limits

These are pre-defined keywords of type Real. They are logged to the **LIM...** table when the **LogLimits** built-in function is called. **LogLimits** uses the current values of the limit conditions to determine where to place the limits.

If the limits are not included in the data file and need to be read from a separate file, the **LimFile** function can be used to close the data file and open a new file containing the limits. Typically this is done after all the data has been read. Any non-key conditions that were not defined in the data file, but exist in the limit file can also be updated when the **LogLimits** function is called.

Information about each limit set is stored in the LIM_LOG database table. The actual limits are stored in the LIM... table, which is linked with LIM_LOG by the LIM_LOG primary key, lim_key.

If the **-limitonly** argument is passed to the reader, the **limits_type**, **target_value** and **fail_bin** in **DEF...** are the only fields updated other than the limit table. None of the conditions can be updated in this way and the Program must already exist in the database. With this option limits can be updated for any existing Program. To update non-key conditions the **-conditions** option may be used with the **-limitonly** option. This allows for the updating of any non-key conditions from the limits file.

Fab One program in the database can have more than one set of limits associated with it.

The distinction between each limits set is determined by the **lim_type** column stored in the **LIM_LOG** table. **lim_type** is a three character string that specifies the type of the limit set as follows:

- D--** — date/time stamp only.
- DM-** — Date/time stamp and meta type.
- D-R** — Date/time and program revision.

- DMR** — Date/time stamp, meta type and program revision.
- M-** — Meta type only.
- MR** — Meta type and program revision.
- R** — Program revision only.

Meta Type — Meta type is a three character string that specifies which meta type is associated with the limit set. A program can have only one meta type, it is set once and cannot be changed (if used). Valid values for meta type are:

- LOT** — refers to a certain lot. To set the LOT ID value, users must use **vDbLimLot**, or **vDbLot** and **DbLot** if limits are being loaded with data.
- SRC** — refers to a certain source lot. To set the source lot id value, users must use **vDbLimSrcLot**, or **vDbSrcLot** and **DbSrcLot** if limits are being loaded with data.
- PKG** — refers to a certain package. To set the Package value, users must use **vDbLimPackage**, or **vDbPackage** and **DbPackage** if limits are being loaded with data.
- PRD** — refers to a certain product. To set the product value, users must use **vDbLimProduct**, or **vDbProduct** and **DbProduct** if limits are being loaded with data.
- PRC** — refers to a certain process. To set the Process value, users must use **vDbLimProcess**, or **vDbProcess** and **DbProcess** if limits are being loaded with data.
- FML** — refers to a certain Family. To set the family value, users must use **vDbLimFamily**, or **vDbFamily** and **DbFamily** if limits are being loaded with data.
- TCH** — refers to a certain technology. To set the technology value, users must use **vDbLimTechnology**, or **vDbTechnology** and **DbTechnology** if limits are being loaded with data.
- EQ1** — refers to a certain Equipment1. To set the Equipment1 value, users must use **vDbLimEquipment**, or **vDbEquipment** and **DbEquipment** if limits are being loaded with data.
- EQ2** — refers to a certain Equipment2. To set the Equipment2 value, users must use **vDbLimEquipment**, or **vDbEquipment** and **DbEquipment** if limits are being loaded with data.

Historical limits can only be maintained by the ASCII reader using the **-limitsonly** option and optionally, the built-in function **DbHistLimits** and **DbLimitsSet**. **DbHistLimits** and **DbLimitsSet** built-in functions are used to insert new limits and set the **lim_type** for the new set of limits.

The ASCII reader still supports the old method of loading limits with date/time stamp as the only distinction between different limit sets, or a combination of date/time, meta type, and program revision.

Users can load limits using date/time stamp only by using the **DbHistLimits** built-in function alone, or use **DbLimitsSet** with only the date/time argument valid and the rest NA, or use neither function.

Date Stamp Only

If the **-limitsonly** option is used without the **DbHistLimits** or **DbLimitsSet** built-in function, the date stamp for the current limits is updated to the current date (today) and the new limits are inserted as the current limits (date stamp 01/01/2500). The current limits are those used as the default limits for retrieval into a worksheet.

The **DbHistLimits** built-in function or **DbLimitsSet** with only the date argument provided, may be used with the **-limitsonly** option to insert new limits in one of the following two ways:

1. Updating current limits only (update, no insertion). This option is achieved by supplying a date to the **DbHistLimits** or **DbLimitsSet** function that exceeds the current date (today).
2. Inserting new limits with a specific date stamp. This option is achieved by supplying a date that is less than the current date (today). (In this case, the current limits in the database are set to the limits with the greatest date.)

Combination of Date Stamp, Meta Type, and Program Revision

The **DbLimitsSet** built-in function, with Meta Type or Program Revision provided, may be used with the **-limitsonly** option to insert new limits with a specific **lim_type**, which can be date stamp, a certain meta type, or a certain program revision.

Default Limits For Multiple Limits Sets

The default limits set is the one with the column **def_flag = Y** in the LIM_LOG table. There can only be one default limits set.

As long as all the limits sets for a particular program in the database are not of the **lim_type = D--**, the most recently entered set will be assigned as default.

As soon as a single limits set of type **D--** is introduced into the program, the logic for assigning default status changes. All future sets that are not of **D--** type will be loaded as non-defaults. All future sets that are of **D--** type will be compared against existing **D--** type sets.

If the new set is the first ever **D--** set for the program, then it becomes the default. If it is not, there are three possible scenarios:

1. The new limits set has no date stamp: the new set becomes the new default set. The old default set is maintained but made non-default (i.e. **def_flag = N**)
2. The new limits set has a future date stamp (greater than today's date): the new set replaces (overwrites) the existing default set.
3. The new limits set has a past date stamp (less than today's date): the new set is loaded as non-default and the existing default set remains the same.

Updating Limits

The FAB reader **-update** and **-limitonly** command-line options allow for the update of a partial set of limits. (See “*Command-Line Arguments*,” pg. 258.) These command-lines options update limits where tests with no limits in the current data file are updated to whatever limits these tests have in the database. In other words, only those tests that have valid limits in the data file are updated.



*In the dbascii reader, the same functionality is achieved using the **DbUpdateLimits** format file function.*

Scaling

The built-in function **ScaleFactor** may be used to set the scaling factor for any parameter. Scale factors in the database are updated by the reader only when a new program is created. For existing programs, the scaling factors used are those stored in the database.

When logging results using the functions **LogResult** or **vLogResult**, the scaling factor for the current parameter is set to the integer value set using the function **ScaleFactor**. If this function has not been called, the scale factor defaults to 0.

The legal scale factors are limited to the following set of values:

-15, -12, -9, -6, -3, -2, 0, +2, +3, +6, +9, +12, +15

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the Fab reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index lot_id the value of “FileName” one would

write:

lot_id = FileName//lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

ORACLE ROLLBACK TABLE

With an Oracle database, when a reader error is detected and the reader exits unexpectedly, for reasons such as:

- Connection to the Oracle engine abruptly lost.
- The user disrupting the reader by pressing **Ctrl+C** multiple times.

...the reader may exit before the buffer has fully executed.

When this happens, a rollback sequence takes place to prevent the Oracle database from becoming corrupted. The rollback statements are stored in a dynamic temporary table, `DP_ROLLBACK_STMTS_...`, per program.

If the reader exits without fully executing the rollback statements, the table will continue to hold the remaining rollback statements, for that `pg_key`. This allows the reader to recover the rollback statements and execute them, should the above exit cases occur.

When a premature exit occurs, this rollback recovery operation is triggered automatically; the administrator does not have to perform any function to make this operation occur.

INVALID DATA

Results, conditions and indexes can sometimes assume values in the data file that are invalid, and should not be logged to the data table. To achieve this a list of invalid data values is maintained that the user can add to or delete from using the built-in functions described in detail in the built-in functions section.

As an example if the result 999.99 is used to designate invalid data in the data file it can be added to the list of invalid data by calling the function **AddInvReal(999.99)**.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

AddSep(',')

OPERATORS

<i>Arithmetic:</i>	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Assignment	=
	Modulus	mod

Logical: Used only with variables of type Boolean.

Logical Or	OR
Logical And	AND
Negation	NOT

<i>Relational:</i>	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression  
    statements  
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression  
    statements  
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression  
    if-statements  
Else  
    else-statements  
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

The built-in functions fall into five main categories: File navigation, data retrieval, string manipulation, database functions and mathematical functions.

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of Fab Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 234
 - **Go To** — pg. 234
 - **Separators** — pg. 234
 - **Skip Forward/Backward** — pg. 235
 - **Miscellaneous** — pg. 235
- **Data Retrieval** — pg. 235
 - **Data** — pg. 235
 - **Limits** — pg. 237
 - **Strings** — pg. 237
 - **Sub-Strings** — pg. 237
 - **Integers** — pg. 239
 - **Real** — pg. 239
- **String Manipulation** — pg. 240
- **Database** — pg. 242
 - **Technology** — pg. 244
 - **Family** — pg. 245
 - **Process** — pg. 245
 - **Product** — pg. 246
 - **Program** — pg. 242
 - **Lot** — pg. 246
 - **Step** — pg. 247
 - **Stage** — pg. 247
 - **Recipe** — pg. 248

- [Equipment](#) — pg. 248
- [Operator](#) — pg. 250
- [Conditions](#) — pg. 251
- [Indexes](#) — pg. 251
- [Date-Time](#) — pg. 251
- [Rework](#) — pg. 253
- [Limits](#) — pg. 253
- [Tagging](#) — pg. 254
- [Miscellaneous](#) — pg. 255
- [Mathematical](#) — pg. 255
- [Debugging](#) — pg. 255
- [System](#) — pg. 256

File Navigation

<i>Go To</i>	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the error code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.

<i>Skip Forward/ Backward</i>	SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
	SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.
	SkipChars (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.
<i>Miscellaneous</i>	NotEndOfFile —	Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.
	GetLineLen —	Accepts no arguments and returns an integer. Returns the number of characters from the FilePointer to the end of line.

Data Retrieval

<i>Data</i>	LogResult (result) —	Accepts one argument of type result and has no return. Reads in the current word as a result and logs it to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values.
	vLogResult (result, ...) —	Accepts two arguments of type result and real or string, and has no return. Logs the second argument as the result to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values. The first argument passed to this function specifies the name and type of the result, while the second argument is the result itself.

- AddInvString (string) —** Accepts one argument of type string and has no return. Adds the passed argument to the list of invalid data.
- AddInvReal (real) —** Accepts one argument of type real and has no return. Adds the passed argument to the list of invalid data.
- AddInvInteger (integer) —** Accepts one argument of type integer and has no return. Adds the passed argument to the list of invalid data.
- AddInvChar (char) —** Accepts one argument of type char and has no return. Adds the passed argument to the list of invalid data.
- DelInvString (string) —** Accepts one argument of type string and has no return. Deletes the passed argument from the list of invalid data.
- DelInvInteger (integer) —** Accepts one argument of type integer and has no return. Deletes the passed argument from the list of invalid data.
- DelInvChar (char) —** Accepts one argument of type char and has no return. Deletes the passed argument from the list of invalid data.
- DelInvReal (real) —** Accepts one argument of type real and has no return. Deletes the passed argument from the list of invalid data.
- OpenFile (string) —** Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the **-file_path** command line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

Limits **LogLimits** —

Accepts no arguments. Logs the current values of LPL, HPL, LSL, HSL, LOL, HOL, LWL and HWL to the limits table. The limits are placed in the row that matches the current values of all LimCond conditions. LogLimits should be preceded by a call to **LimitsProgram** or **vLimitsProgram (string)**.



All the limits conditions (LimConds) need be set before calling the function LogLimits().

LimFile (string) —

Accepts one argument of type string and has no return. The passed argument is the file to be opened after the data file is closed. The File Pointer is placed at the beginning of the new file.

ClearLimits —

Accepts no arguments. Resets all limits (LSL,LPL, ...) to invalid.

Strings **GetWord** —

Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.

GetPrevWord —

Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

GetQuotedWord (char) —

Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error code is set if no quoted word is found.)

GetLine —

Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.

Sub-Strings **GetChars (integer)** —

Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.

GetCharsTrim (integer, char) —

Accepts two arguments of type integer and char and returns a string. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)

ExtractString —

Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.

GetLeftChars (integer) —

Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetRightChars (integer) —

Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetMidChars (integer, integer) —

Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordAfter (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.

If there are multiple occurrences of the passed character, the function returns all characters after the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

Integers **GetInt —**

Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.

GetPrevInt —

Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.

Real **GetReal —**

Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.

GetPrevReal —

Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.

String Manipulation

ToLower (string) —	Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.
ToUpper (string) —	Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.
IsNumber (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.
IsString (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.
Right (string, integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetRightChars(), but operates on the passed string instead of the current word.
Left (string,integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetRightChars(), but operates on the passed string instead of the current word.
Mid (string, integer, integer) —	Accepts three arguments of type string, integer and integer and returns a string. Same as GetMidChars(), but operates on the passed string instead of the current word.
After (string, char) —	Accepts two arguments of type string and character and returns a string. Same as GetWordAfter(), but operates on the passed string instead of the current word. If there are multiple occurrences of the passed character, the function returns all characters after the last instance.
Before (string, char) —	Accepts two arguments of type string and character and returns a string. Same as GetWordBefore(), but operates on the passed string instead of the current word. If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `StrToInt("16A3")`, the output would be the integer 16; for `StrToReal("16.1A3")`, the output would be 16.1.

IntToStr (string) — Accepts one argument of type integer and returns a string. Translates the passed integer to a string.

StrTrim (string, char) — Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.

StrLen (string) — Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.

Argument —

Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument **-arg**.

Database

Program **vDbProgLotWaf (string, string, string, string, integer) —**



vDbProgLotWaf() must be called before calling any other database function, except vDbOplogIndex(). (below)

Accepts five arguments. The first four are of type string and the last one is of type integer. The passed arguments are the program, lot id, wafer id, source lot id, and wafer number (in that order). This function must be called any time a new program, lot, or wafer is encountered in the data file. Since the fab reader accepts multiple programs and multiple lots in one data file, this function is used to set the values of the program and lot as they change in the data file in a way similar to the usage of key indexes.

Source lot id and wafer number are optional. If source lot id is not available, that is indicated by “” or “NA”. If wafer number is not available, that is indicated by a negative number.

vDbOplogIndex (string, string) —



vDbOplogIndex() must be called before vDbProgLotWaf(). (above)

Accepts two arguments of type string and has no return. The 1st argument must be one of the declared variables in the format file, holding the indexed OP_LOG column. The 2nd is the name of the indexed OP_LOG column (in double quotes), with the following valid values:

**STEP_KEY
STAGE_KEY
RCP_KEY
PD_KEY
EM_KEY
PR_KEY
EQKEY1
EQKEY2
EQKEY3
EQKEY4
EQKEY5
EQKEY6**

LimitsProgram —

Accepts no arguments and returns a string. Calls GetWord returning the current word. The current word is then used as the current program, for which the **LogLimits** function applies.



When loading limits in-file (i.e. data + limits in same file), the function LimitsProgram() or vLimitsProgram() must be called before calling any other limits function.

vLimitsProgram (string) — Accepts one argument of type string and returns a string. The passed string is used as the current program, for which the **LogLimits** function applies.



When loading limits in-file (i.e. data + limits in same file), the function LimitsProgram() or vLimitsProgram() must be called before calling any other limits function.

DbProgRev —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.

vDbProgRev (string) —

Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.

vDbProgGroup (string) — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

vDbProgStep (string) — Accepts one argument of type string and returns a string. Returns the passed argument. The function associates a program with a specific step. The value passed is filled in the PROC_STEP table and the corresponding key is filled in the **program.identifier_key** field.

Technology DbTechnology — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. This function must be called every time a new technology is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbTechnology (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.

vDbLimTechnology (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbTechnology, but it is used in association with a limit set.

Family **DbFamily —**

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a family in the database (FAMILY Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. This function must be called every time a new family is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbFamily (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.

vDbLimFamily (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbFamily, but it is used in association with a limit set.

Process **DbProcess —**

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function establishes the appropriate relations with other tables. This function must be called every time a new process is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbProcess**, but uses the passed argument instead of reading from the file.

vDbLimProcess (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but it is used in association with a limit set.

Product **DbProduct** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), it is added. If it does exist the function only establishes the appropriate relations with other tables. This function must be called every time a new product is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbProduct (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbProduct**, but uses the passed argument instead of reading from the file.

vDbLimProduct (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but it is used in association with a limit set.

Lot **vDbLotClass (string, string, string, string)** —

Accepts four arguments of type string: (**Lot_Name**, **Method_Name**, **Method_Type**, **Class_Name**) and has no return. Sets the lot class from the passed argument for the lot specified using **vDbProgLotWaf**. The function supports multiple classes, per lot.

vDbLimLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbLot, but uses the passed argument instead of reading from the file. This function is used in association with a limit set.

vDbLimSrcLot (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as vDbSrcLot, but it is used in association with a limit set.

vDbLimLotClass (string) — Accepts one argument of type string and has no return. Works in conjunction with vDbLimLot. Sets the lot class from the passed argument for the lot specified using vDbLimLot. This function is used in association with a limit set.

DbUpdateLot (int) —

Accepts one argument of type integer, and has no return. The first argument is a fab flag that allows for updating the Fab relationship if argument >0.

The function allows for the update of the LOT table's fab relationships.

This is useful when a particular lot already exists in the database and there is a need to update the fab relationships to that lot.

The fab is updated to the fab identified by either of the functions, **DbFab** or **vDbFab**.

Step DbStep —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), it is added. If it does exist the function only establishes the appropriate relations with other tables. This function must be called every time a new process step is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbStep (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStep**, but uses the passed argument instead of reading from the file.

vDbMStepPStep (string, string, string) —

Accepts three arguments of type string and has no return.

The first argument is the process; the second argument is the measurement step; the third argument is the processing step. It associates a specified measurement step with a specific processing step. It may be called multiple times per file.

Stage DbStage —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), it is added. If it does exist, the function establishes the appropriate relations with other tables. This function must be called every time a new stage is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbStage (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStage**, but uses the passed argument instead of reading from the file.

Recipe **DbRecipe —** Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a recipe in the database (RECIPE Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. This function must be called every time a new recipe is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbRecipe (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbRecipe**, but uses the passed argument instead of reading from the file.

Equipment **DbMEquip —** Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey1 in the OP_LOG table.)

vDbMEquip (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbMEquip**, but uses the passed argument instead of reading from the file. This function must be called every time a new M-equipment item is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

DbPEquip — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey2 in the OP_LOG table.) This function must be called every time a new P-equipment item is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbPEquip (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbPEquip , but uses the passed argument instead of reading from the file.
DbEquip3 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey3 in the OP_LOG table.) This function must be called every time a new Equip3 item is encountered in the file, and should always be preceded by a call to vDbProgLotWaf .
vDbEquip3 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip3 , but uses the passed argument instead of reading from the file.
DbEquip4 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey4 in the OP_LOG table.) This function must be called every time a new Equip4 item is encountered in the file, and should always be preceded by a call to vDbProgLotWaf .
vDbEquip4 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip4 , but uses the passed argument instead of reading from the file.
DbEquip5 —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey5 in the OP_LOG table.) This function must be called every time a new Equip5 item is encountered in the file, and should always be preceded by a call to vDbProgLotWaf .

vDbEquip5 (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip5**, but uses the passed argument instead of reading from the file.

DbEquip6 — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey6 in the OP_LOG table.) This function must be called every time a new Equip6 item is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbEquip6 (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip6**, but uses the passed argument instead of reading from the file.

vDbLimEquipment (int, string, string, string) —
Accepts four arguments and has no return.

The 1st argument is an integer between 1 or 2, which indicates the equipment number.

The 2nd argument is the equipment I.D. (string).

The 3rd argument is the equipment type (string).

The 4th argument is the equipment class (string).

Same as vDbEquipment, but it is used in association with a limit set.

Operator DbOperator — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a name in the PEOPLE Table, it is added with the “role” field set to Operator. If it does exist, the function only establishes the appropriate relations with other tables. This function must be called every time a new operator is encountered in the file, and should always be preceded by a call to **vDbProgLotWaf**.

vDbOperator (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbOperator**, but uses the passed argument instead of reading from the file.

Conditions DbParLevelCond (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared key conditions. That condition becomes the one used as the parameter-level condition. Parameter-level condition has only three valid case-insensitive values: 'site', 'wafer', and 'lot'.

Mandatory for all Fab program classes —
23-FabSite, 24-FabWaf, and 25-FabLot.

Indexes DbDieXYIndexes (string, string) —

Accepts two arguments of type string and has no return. The passed arguments must be the names (in double quotes) of the two declared indexes. Those indexes become the **die_x** and **die_y**, respectively.

DbSiteIndex (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of the one declared index. That index becomes the **site** index.

DbWaferIndex (string) —

Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the wafer index used for updating the WAFER and WAF... database tables.

DbWaferIndex is only mandatory when handling site data.

Date-Time vDbLimStartTime (string, string) —

Accepts two arguments of type string and returns a string. Returns the first argument.

The 1st argument is the date-time as a string and the 2nd is the format (SQL DATETIME for Informix) that describes the date as it appears in the file. The date is used as the Start_Time of the lot.

The following describes the formatting of the Informix DATETIME string:

%b — Abbreviated month name.

%B — Full month name.

%d — Day of the month as a decimal [01,,31].
%H — 24 hour clock.
%I — 12 hour clock
%M — Minute as a decimal [00,,59].
%m — Month as a decimal [01,,12].
%p — a.m. or p.m.
%S — Second as a decimal [00,,59].
%y — Year as a decimal [00,,99].
%Y — Year as a 4-digit decimal.
%% — Allows for percent in the string.

This function is used in association with a limit set, to populate the LOT database tables start_time column.

DbMDate (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the measurement date-time in the OP_LOG table. This is equivalent to start time.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
%B Full month name.
%d Day of the month as a decimal [01,,31].
%H 24 hour clock.
%I 12 hour clock
%M Minute as a decimal [00,,59].
%m Month as a decimal [01,,12].
%p a.m. or p.m.
%S Second as a decimal [00,,59].
%y Year as a decimal [00,,99].
%Y Year as a 4-digit decimal.
%% Allows for percent in the string.

As an example the format for the following string:

“jul 1 96 05:10:46”

would be:

“%b %d %y %H:%M:%S”

vDbMDate (string, string) —

Accepts two arguments of type string and returns a string. Returns the passed argument. Same as **DbMDate (string)**, but uses the passed arguments instead of reading from the file. This is equivalent to start time.

DbPDate (string) — Accepts one argument of type string and returns a string. Calls GetWord returning the current word. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the processing date-time in the OP_LOG table. This is equivalent to end time.

vDbPDate (string, string) — Accepts two arguments of type string and returns a string. Returns the passed argument. Same as **DbPDate (string)**, but uses the passed arguments instead of reading from the file. This is equivalent to end time.

Rework **DbReworkAction (int) —** Accepts one argument of type integer and has no return. Sets rework_action in the PROGRAM table. This function overwrites the command line argument **-rework_action**. This function applies to all programs within one data file.

Limits **DbHistLimits (string) —** Accepts one argument of type string and has no returns. The passed argument is the date stamp for the limits being inserted. This function has no effect except when using the ASCII reader with the **-limitsonly** option.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.
ddd — Day of the week as a 3-letter abbreviation.
mm — Month as a 2-digit number.
mmm — Month as a 3-letter abbreviation.
yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
yyyy — Year as a 4-digit number.

DbLimitsSet (string, string, string) — Accepts three arguments of type string and has no return. The passed arguments set the Limit Set Type.

The arguments are:

Limits Date — date stamp for the insertion of the limits being. NA if no date is available.

Meta Type — A three character string that specifies the meta type for the limit set.

Valid values are:

LOT — lot
SRC — source lot
PKG — package
PRD — product
PRC — process
FML — family
EQ1 — equipment1
EQ2 — equipment2
TCH — technology

It can be set to NA if no meta type is available. If used, the appropriate limit function setting the meta type value should be used. For example, if meta type is LOT, the value of the lot id can be specified using the vDbLimLot function.

Revision: Sets the database test program revision. NA if no revision is available.

This function is associated with the limits.



WARNING:

Once the program has been created, the DbLimitsSet function will only work with the -limitsonly option (“-limitsonly,” pg. 259).

Tagging **DbLotTag (int)** —

Accepts one argument of type integer and has no return. Used for manual lot tagging. The passed argument should be one of the following values:

No Action	(1)
Bad	(2)
Scrap	(3)
Experiment	(4)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (int) —

Accepts one argument of type integer and has no return. Used for manual source lot tagging. The passed argument should be one of the following values:

No Action	(1)
Bad	(2)
Scrap	(3)
Experiment	(4)

The above list is dependent on the contents of the **TAGS** table in the database.

Miscellaneous	vDbWaferClass (string, string, string, string) —	Accepts four arguments of type string and has no returns. The passed arguments are of the following types: wafer_name - string method_name - string method_type - string class_name - string Only those wafers that are used in the data file will have their classes and methods logged. Wafers that are only passed to vDbWaferClass() but not used elsewhere in the data file will be ignored. For those wafers logged, the following tables will be populated: CLS_METHOD, WAFER_CLASS, and WFCLS2WF. CLS_METHOD is populated with the method_name and method_type . WAFER_CLASS is populated with the class_name . WFCLS2WF is populated with the corresponding wafer_names and class_names .
	DbInline —	Accepts no arguments and has no return. Sets the monitor_flag field in the OP_LOG table to inline data. (N)
Mathematical	ScaleFactor (int) —	Accepts one argument of type int and has no return. Sets the value of the current scaling factor.
	Abs (real) —	Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.
	Sqr (real) —	Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.
Debugging	Print (...) —	Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.
	PrintToFile (...) —	Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.

ErrorCode —

Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the error code are **GoTo**, **GoBackTo**, and **OpenFile**.

ExitScript (string) —

Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

System **System (string)** —

Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function. Accepts up to 254 characters.

THE EXENSIO –YIELD INTERFACE

All the tools necessary for building and running the Fab reader with a specific format file are available within the Exensio –Yield environment. To build a new format file, use the **File > New > Script** menu command. This command will open Exensio –Yield’s text editor. Any text editor could be used provided that no invisible control characters are written to the file (some text editors use these control characters for formatting purposes).

Type the commands and keywords for your format file in the newly created document. Once you are finished choose the **File > Save As...** menu option. In the pop-up window type the name of the format file in the file name window and select the “text” format. Ensure that the path to your saved file is correct.

Once your format file is created and saved the Fab reader can be run using the newly created format. If there are any errors in your format file the reader will generate an error file (err.jnk) describing the nature of the error.

Running the Fab Reader

The Fab reader accepts the following options: (The first argument should always be the data file to be processed, unless the objective is to only parse the format file.)

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the Fab reader, which is the format file preceded by the -fmt switch.

Example:

```
fab -fmt [format file]
```

Executing the Fab reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Command-Line Arguments

Option	Definition	Default
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).	If not used, this action is not taken.
-bigcond	For Informix: Extend condition values to 254 characters instead of the default 63. For Oracle: Extend condition values to 511 characters instead of the default 63. Added in 3.1, per EX-8732	If not used, this action is not taken, and the default maximum characters is 63.
-bigstring	When used, index and result columns of type string will be created with extended length, depending on the database platform — Oracle or Informix. If this option is used on new programs, all index and result columns of type string will be created with the extended length. If this option is removed later, only newly added result columns of type string are affected; they will be created with the default length. If the option is used on existing programs, only newly added result columns of type string are created with the extended length. Once a column is created with a specified length, it will keep accepting values at that length, weather the option is used in the future or not. • Oracle — -bigstring extends string index and parameter values to 511 characters instead of the default 63. • Informix — -bigstring extends string index and parameter values to 254 characters instead of the default 63.	If not used, the default string value limit is 63 characters.
-conditions	In combination with -limitonly option, indicates that non-key conditions should be updated.	If not used, non-key conditions are not updated.
-datadbs	<dbspace> Places reader created tables in specified dbspace.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-db	[database]	This option is required with a valid value.
-dboutliers	Exclude outliers from summaries, outlier limits retrieved from database.	If not used, no data is excluded.

Option	Definition	Default
-defext	[Kbytes] Sets DEF... Table extent. If not used, defaults to 16 Kbytes. Acceptable range is 16 Kb to 256 Kb.	If not used, defaults to 16 Kbytes.
-dynamic	[max added tests] A dynamic program assumes the possibility of a growing number of parameters. The default is a semi-dynamic program which assumes a fixed number of parameters.	WARNING: The default is a semi-dynamic program which assumes a fixed number of parameters.
-file_path	[Path to data file]	The current working directory.
-filter	[limit] Excludes data greater than limit or less than -limit (default limit is set to 1.0e+19).	Default limit is set to 1.0e+19.
-fix_cond_values	In all string condition values, replace the '[' with '(' and ']' with ')'.	If not used, this action is not taken.
-fmt	[format file]	This option is required with a valid value.
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-limext	[Kbytes] Sets LIM... Table extent. If not used, defaults to 128 Kbytes. Acceptable range is 16 Kb to 5012 Kb.	If not used, defaults to 128 Kbytes.
-limitonly	Updates only the Limits database tables.	If not used, this action is not taken; both data and limits are loaded.
-lotext	[Kbytes] Sets LOT... Table extent. If not used, defaults to 128 Kbytes. Acceptable range is 16 Kb to 5012 Kb.	If not used, defaults to 128 Kbytes.
-lowercase	Forces lot_id and wf_id to lowercase.	If not used, this action is not taken.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.

Option	Definition	Default
-nolimprogram	This option can only be applied with the -limitonly option (<i>-limitonly</i>). When a program is detected as new (it does NOT already exist in the database), then the reader will skip the current program and proceed to the next one. The new program will be ignored in the data file.	If not used, when the reader detects a new program it will exit with an error — program does not exist in the database.
-outliers	[ni] Exclude outliers from summaries, outlier limits calculated from file data.	If not used, no data is excluded.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results never age.
-resex	[Kbytes] Sets RES... Table extent. If not used, defaults to 128 Kbytes. Acceptable range is 16 Kb to 10024 Kb.	If not used, defaults to 128 Kbytes.
-rework_action	[Rework Action] Sets Test Program Rework Action (integer).	1 WARNING: rework data will be ignored.
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.	If not used, summaries never age.
-u	usage	NA
-update	Updates existing rows of data, rather than inserting new rows.	WARNING: If not used, the reader only permits inserting new rows.
-update_all	Updates existing rows of data and non-key indexes, rather than inserting new rows. NOTE: There is no need to use the -update option when the -update_all option is used, because -update_all implements the same operation, with the addition of the update of non-key indexes.	If not used, this action is not taken.
-uppercase	Forces lot_id and wf_id to uppercase.	If not used, this action is not taken.
-v	version	NA
-wafext	[Kbytes] Sets WAF... Table extent. If not used, defaults to 128 Kbytes. Acceptable range is 16 Kb to 5012 Kb.	If not used, defaults to 128 Kbytes.

ERROR MESSAGE ALERTS FOR ALARM AND EVENTS RULES

MANAGER

If the reader, for any reason, outputs an error message, then the reader can send that error as a message to an AEM (Alarm and Event Rules Manager) web server, running a web service called AEM Supervision that understands the specific format of the SOAP XML message.

This action only takes place if the reader is started with the options **-wsreport AemReport -endpoint** <URL>. The URL is used to define the address of the AEM server that will receive the message.

EXAMPLE 1 — FABSITE

Data File

```
<PAR>
1,4,Fab_23_4,lot,v,s,
2,5,Fab_23_5,lot,mv,r,
3,6,Fab_23_6,site,v,s,
4,7,Fab_23_7,site,mv,r,
</PAR>

<DATA>
PROG_23_1,LOT_10,S_LOT_11,LOT_CLASS_12,WAF_13,14,15,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
4,
1,2.0,
2,,
3,,
4,25.0,

PROG_23_2,LOT_20,S_LOT_21,LOT_CLASS_22,WAF_23,24,25,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
4,
1,,
2,22.0,
3,,
4,225.0,
</DATA>
```

Format File

```
SCRIPT fab

VAR
    //Conditions
    TestName      string      LimCond
    Unit          string      Cond
    TestNum       string      Cond
    ParLevel      string      LimCond

    //Indexes
    Site          integer     KeyIndex
    Wafer         string      KeyIndex

    //Results
    str_res       string      Result
    flt_res       real        Result

    Str           string
    Flt           real
    Program       string
    Lot           string
    SrcLot        string
    LotClass      string
    WfNum         integer
    i             integer
    j             integer
    k             integer
    no_param      integer

    TestNumberA[1000] string
    TestNameA[1000]  string
    TestLevelA[1000] string
    UnitA[1000]     string
    TestTypeA[1000]  string
```

```

BEGIN
  AddSep(',')

  DbSiteIndex("Site")
  DbWaferIndex("Wafer")

  DbParLevelCond("ParLevel")

  //reading PARAMETER
  i = 1
  GoTo("<PAR>")
  If(ErrorCode <> 0)
    ExitScript("<PAR> not found.")
  End If

  Str = GetWord
  While (Str <> "</PAR>")
    TestNumberA[i]      = GetWord
    TestNameA[i]        = GetWord
    TestLevelA[i]       = GetWord
    UnitA[i]            = GetWord
    TestTypeA[i]        = GetWord

    Str = GetWord
    i = i + 1
  End While

  //reading DATA
  GoTo("<DATA>")
  If(ErrorCode <> 0)
    ExitScript("<DATA> not found.")
  End If

  Str = GetWord
  While( Str <> "</DATA>" )
    Program              = Str

    Lot                  = GetWord
    SrcLot               = GetWord
    LotClass             = GetWord

    Wafer                = GetWord
    WfNum               = GetInt
    Site                 = GetInt

    vDbProgLotWaf(Program, Lot, Wafer, SrcLot, WfNum)

    vDbLotClass(LotClass)

    Str = DbTechnology
    Str = DbFamily
    Str = DbProcess
    Str = DbProduct

    Str = DbStage
    Str = DbStep
    Str = DbRecipe

    Str = DbMequip
    Str = DbMdate("%Y%m%d")
    Str = DbPequip
    Str = DbPdate("%Y%m%d")

    Str = DbOperator

    no_param = GetInt

    For k=1 To no_param
      j = GetInt

      TestNum = TestNumberA[j]
      TestName = TestNameA[j]
      ParLevel = TestLevelA[j]
      Unit = UnitA[j]
    
```

```

                                Str = GetWord
                                If(Str <> " ")
                                    If(TestTypeA[j] = "s")
                                        vLogResult(str_res,Str)
                                    Else If(TestTypeA[j] = "r")
                                        Flt = StrToReal(Str)
                                        vLogResult(flt_res,Flt)
                                    Else
                                        ExitScript("Invalid test
                                End If
                                End If
                                End For
                                Str = GetWord
                                End While
                                End

```

EXAMPLE 2 — FABWAF

Data File

```

<PAR>
1,4,PROG_24_4,lot,v,s,
2,5,PROG_24_5,wafer,mv,r,
3,6,PROG_24_6,wafer,mv,r,
</PAR>

<DATA>
PROG_24_1,LOT_10,S LOT_11,LOT_CLASS_12,WAF_13,14,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
3,
3,2.0,
2,,
1,25.0,

PROG_24_2,LOT_20,S LOT_21,LOT_CLASS_22,WAF_23,24,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
3,
3,,
2,22.0,
1,225.0,
</DATA>

```

Format File

```

SCRIPT fab

VAR
    //Conditions
    TestName      string      LimCond
    Unit          string      Cond
    TestNum       string      Cond
    ParLevel      string      LimCond

    //Indexes
    Wafer         string      KeyIndex

    //Results

```



```

str_res      string      Result
flt_res      real        Result

Str          string
Flt          real
Program      string
Lot          string
SrcLot       string
LotClass     string
WfNum        integer
i            integer
j            integer
k            integer
no_param     integer

TestNumberA[1000] string
TestNameA[1000]  string
TestLevelA[1000] string
UnitA[1000]      string
TestTypeA[1000]  string

BEGIN
  AddSep(',')

  DbParLevelCond("ParLevel")

  //reading PARAMETER
  i = 1
  GoTo("<PAR>")
  If(ErrorCode <> 0)
    ExitScript("<PAR> not found.")
  End If

  Str = GetWord
  While (Str <> "</PAR>")
    TestNumberA[i] = GetWord
    TestNameA[i]   = GetWord
    TestLevelA[i]  = GetWord
    UnitA[i]       = GetWord
    TestTypeA[i]   = GetWord

    Str = GetWord
    i = i + 1
  End While

  //reading DATA
  GoTo("<DATA>")
  If(ErrorCode <> 0)
    ExitScript("<DATA> not found.")
  End If

  Str = GetWord
  While( Str <> "</DATA>" )
    Program = Str

    Lot = GetWord
    SrcLot = GetWord
    LotClass = GetWord

    Wafer = GetWord
    WfNum = GetInt

    vDbProgLotWaf(Program, Lot, Wafer, SrcLot, WfNum)

    vDbLotClass(LotClass)

    Str = DbTechnology
    Str = DbFamily
    Str = DbProcess
    Str = DbProduct

    Str = DbStage
    Str = DbStep
    Str = DbRecipe
  
```

```

Str = DbMequip
Str = DbMdate("%Y%m%d")
Str = DbPequip
Str = DbPdate("%Y%m%d")

Str = DbOperator

no_param = GetInt

For k=1 To no_param
    j = GetInt

    TestNum = TestNumberA[j]
    TestName = TestNameA[j]
    ParLevel = TestLevelA[j]
    Unit = UnitA[j]

    Str = GetWord

    If(Str <> " ")
        If(TestTypeA[j] = "s")
            vLogResult(str_res,Str)

        Else If(TestTypeA[j] = "r")
            Flt = StrToReal(Str)
            vLogResult(flt_res,Flt)

        Else
            ExitScript("Invalid test

type used in <PAR>.")
        End If
    End If

End For

Str = GetWord
End While
End

```

EXAMPLE 3 — FABLOT

Data File

```

<PAR>
1,4,PROG_25_4,lot,v,s,
2,5,PROG_25_5,lot,mv,r,
3,6,PROG_25_6,lot,mv,r,
</PAR>

<DATA>
PROG_25_1,LOT_10,S LOT_11,LOT_CLASS_12,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
3,
3,,
2,2.0,
1,25.0,

PROG_25_2,LOT_20,S LOT_21,LOT_CLASS_22,
TECH_1,FAM_2,PROC_3,PROD_4,STG_5,STP_6,RCP_7,MeEq_8,
19981219,PrEq_8,20001025,OPER_9,
3,
3,22.0,
2,,
1,225.0,
</DATA>

```

Format File

```

SCRIPT fab

VAR
    //Conditions
    TestName      string      LimCond
    Unit          string      Cond
    TestNum       string      Cond
    ParLevel      string      LimCond

    //Indexes
    Lot           string      KeyIndex

    //Results
    str_res       string      Result
    flt_res       real        Result

    Str           string
    Flt           real
    Program       string
    SrcLot        string
    LotClass      string
    i             integer
    j             integer
    k             integer
    no_param      integer

    TestNumberA[1000] string
    TestNameA[1000]  string
    TestLevelA[1000] string
    UnitA[1000]     string
    TestTypeA[1000]  string

BEGIN
    AddSep(',')

    DbParLevelCond("ParLevel")

    //reading PARAMETER

```

```

i = 1
GoTo("<PAR>")
If(ErrorCode <> 0)
    ExitScript("<PAR> not found.")
End If

Str = GetWord
While (Str <> "</PAR>")
    TestNumberA[i]      = GetWord
    TestNameA[i]        = GetWord
    TestLevelA[i]       = GetWord
    UnitA[i]            = GetWord
    TestTypeA[i]        = GetWord

    Str                = GetWord
    i = i + 1
End While

//reading DATA
GoTo("<DATA>")
If(ErrorCode <> 0)
    ExitScript("<DATA> not found.")
End If

Str = GetWord
While( Str <> "</DATA>" )
    Program              = Str

    Lot                  = GetWord
    SrcLot               = GetWord
    LotClass             = GetWord

    vDbProgLotWaf(Program, Lot, "NA", SrcLot, -1)

    vDbLotClass(LotClass)

    Str = DbTechnology
    Str = DbFamily
    Str = DbProcess
    Str = DbProduct

    Str = DbStage
    Str = DbStep
    Str = DbRecipe

    Str = DbMequip
    Str = DbMdate("%Y%m%d")
    Str = DbPequip
    Str = DbPdate("%Y%m%d")

    Str = DbOperator

    no_param = GetInt

    For k=1 To no_param
        j = GetInt

        TestNum = TestNumberA[j]
        TestName = TestNameA[j]
        ParLevel = TestLevelA[j]
        Unit = UnitA[j]

        Str = GetWord

        If(Str <> " ")
            If(TestTypeA[j] = "s")
                vLogResult(str_res,Str)

            Else If(TestTypeA[j] = "r")
                Flt = StrToReal(Str)
                vLogResult(flt_res,Flt)

            Else
                ExitScript("Invalid test

```

type used in <PAR>.")



FRONT



CONTENTS



INDEX

```
End If
End For
Str = GetWord
End While
End
```


CHAPTER 6

STDF3 DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file. Exensio –Yield readers can access data directly from files or through the database. Whether the data is imported through the database or directly from a data file, the objective is the same: to access your data and format it properly for comprehensive analysis with Exensio –Yield.

To accommodate these two primary means of importing data, there are two types of data readers: database (*dB*) readers, which import the formatted data into the Informix database for access through the Exensio –Yield database system; and worksheet (*WS*) readers, which import the formatted data directly into the Exensio –Yield environment. dB readers operate in the background, as part of the overall process of configuring and importing data from the database. WS reader sessions are configured and implemented from a reader-specific interface that is called by the user when it is time to load a new raw data file.

For a general overview of how Exensio –Yield readers function, refer to “[Exensio Data Readers — Overview](#),” pg. 9.

In this document, the STDF3 WS reader functionality is described in the following section.

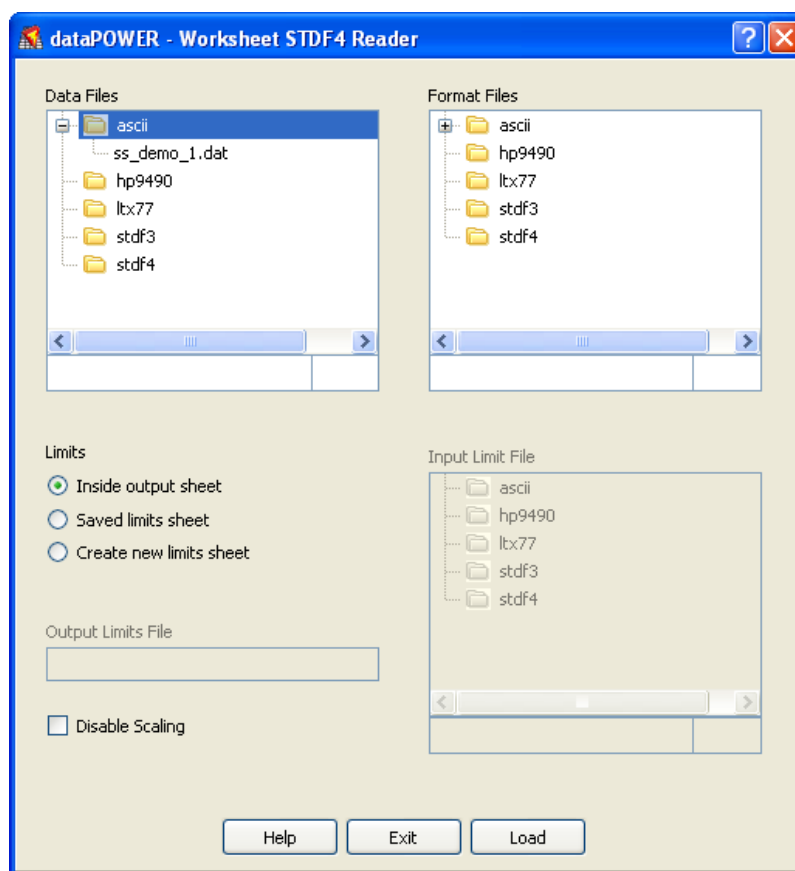


<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

STDF3 WORKSHEET READER INTERFACE

From Exensio –Yield, STDF3 files that are to be imported directly to the worksheet without using the Exensio –Yield database system will use the Data Import dialog. From this dialog, you will configure the data so it will be properly formatted for the Exensio –Yield environment. If your data import sessions will always utilize a database, you will not be using this function.

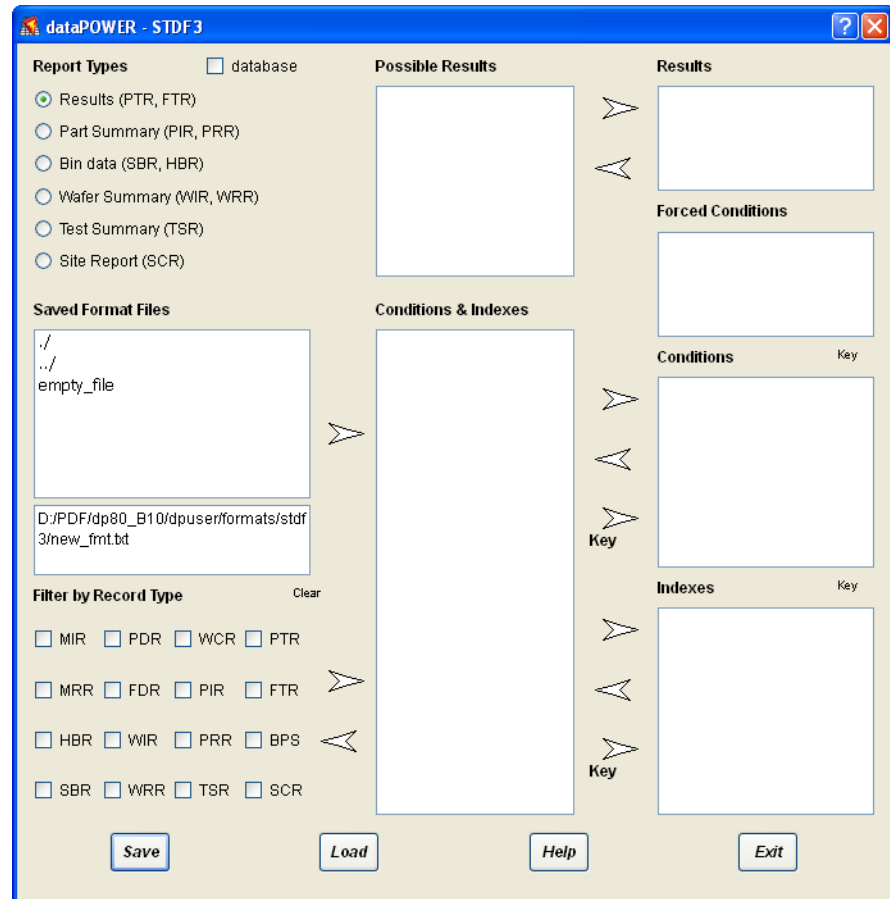
To invoke the dialog select **Data > Readers > STDF3**. (The button and menu option will be available if your system configuration includes the STDF3 worksheet reader.)



There are several primary components that must be specified from the data import dialog for a reader session to be successful, including the **Data Files**, **Format File**, — and, depending on your limits configuration, **Input Limits Table**, and **Output Limits File**. Each of these fields is populated using the file navigator in the dialog.

STDF3 FORMAT FILE — OVERVIEW

The STDF3 data reader requires a format file that defines what data will be treated as results, conditions, and indexes. The format file is easily generated using the window shown below, which allows you to choose from a library of STDF3 results, conditions and indexes. The window is accessed by selecting **Data > Create Format File > STDF3**.



I – REPORT TYPES

The data reader supports multiple results. The first step in creating a format file is deciding what results you will be using. Selecting an item from the **Report Type** field fills the **Possible Results** list box with all record fields that can be chosen as results for that particular report. For example, if you are interested in a results report then select the **Results** option, which will fill the **Possible Results** list box with record fields. To choose from the possible results, select any combination of items in the **Possible Results** list and click on the ➤ control pointing to the **Results** list box.

To remove an item from the **Results** list box, select the result(s) to be removed and click on the  control pointing back to the **Possible Results** list box.

With the **database** option selected, the supported Report Types are:

- Results: Results from PTR.
- Summary: Results from HBR, SBR and TSR

II – CONDITIONS AND INDEXES

The second step in creating a format file is deciding on the conditions (Column headings in the raw data table) and indexes (Row headings in the raw data table).

Definitions

Key Conditions The group of conditions that uniquely identify the data column. The combination of the values of all key conditions for any particular data column will be unique to that column.

Key Indexes The group of indexes that uniquely identify the data row. The combination of the values of all key indexes for any particular data row will be unique to that row.

For each Report Type the **Conditions & Indexes** list box is filled with all the record fields that can be chosen as Conditions or Indexes, including the datalog file name. For all Report Types the first three conditions are always forced to be the units, test name and test number.

A condition is included in the format file by highlighting it in the **Conditions & Indexes** list box and clicking the  control pointing to the **Conditions** list box. There are two controls, one is used to define the selected items as conditions and the other as key conditions. After a condition is already in the list it can be changed from key to non-key or vice-versa by clicking the **Key** button.

The  control pointing back to the **Conditions & Indexes** list box removes the selected condition(s) from the format file.

The **Indexes** list box operates exactly the same way as the **Conditions** list box.

Filter by record type

Since the list of possible conditions and indexes can be relatively long, you may need to look at a subset of the list. This is achieved by filtering out all undesired records from the list.

As an example if you are only interested in conditions from PRR records:

1. Click on the **Clear** button to clear all check-boxes.
2. Choose PRR by clicking on the PRR check-box.
3. Click on the arrow pointing to the **Conditions & Indexes** list box.

This updates the list to include items from PRR records only.

III – SAVING FORMAT FILES

After choosing the results, conditions and indexes the format file can be saved by clicking on the **Save** button. The name of the saved file is entered in the box below the **Saved Format Files** list box.

IV – LOADING FORMAT FILES

Previously created format files can be loaded by selecting the desired file in the **Saved Format Files** list box and clicking on the **Load** button or clicking on the arrow pointing to the **Conditions & Indexes** list box. This is useful for reviewing or editing existing files, or for creating a format file based on an existing one.

DATABASE

Fields that are declared as indexes can affect the binning and wafer database tables.

The **HIST_BIN** and **BIN_LOG** database tables are updated by the STDF3 reader only if an index or a parameter exists that contains bin information. (PRR.sbin_num or PRR.hbin_num)

The Database Summary report includes updating the **BIN_LOG** database table. This is an automatic operation as long as either the hard bin or soft bin is included as one of the results. If both hard bin and soft bin are included in this type of report, by default, the hard bin information is inserted in **BIN_LOG**. To override this default, use the **-softbin** command line argument as described in the [Command-line Arguments](#) section.

The **LOT...** and **LOT** database tables are updated by the STDF3 reader only if the MIR.lot_id exists and is valid.

The **WAF...** and **WAFER** database tables are updated by the STDF3 reader only if an index exists that is declared as containing wafer information. (WIR.wafer_id or WRR.wafer_id).

All files must contain a Test Program, and the Test Program must belong to a Program Class. This is done using the **-class** option described below.

Command-line Arguments

If the STDF3 reader is being run manually with the database option knowing the different command-line arguments becomes necessary. The STDF3 reader accepts the following options: (The first argument should always be the data file to be processed.)

-db	[database] Where database is the name of the database to write to.
-fmt	[format file] Where format file is the format to be used.
-db_accept	If used the processing of the files does not wait for any user input, this is useful if non of the filtering or sampling options are needed.
-semi_dynamic	A semi dynamic “Program” assumes a fixed number of Parameters, like the default, but in addition fills up all tables with keys in the OP_LOG table such as Product and Equipment.
-dynamic	Like semi-dynamic, but without a fixed number of Parameters.

-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.
-resex	[extent] Sets database table extents for the RES table. If not used, defaults to 5012 Kbytes. Acceptable range is 64 Kb to 128000 Kb.
-lotext	[extent] Sets database table extents for the LOT table. If not used, defaults to 64 Kbytes. Acceptable range is 16 Kb to 4000 Kb.
-defext	[extent] Sets database table extents for the DEF table. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 5012 Kb.
-wafext	[extent] Sets database table extents for the WAF table. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 20000 Kb.
-class	[program class] Sets Program Class, (New Programs).
-indexes	This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created. Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-lowercase	Forces Lot ID and Wafer ID to Lower Case.
-uppercase	Forces Lot ID and Wafer ID to Upper Case.
-outliers	[ni] Filters Outliers from the calculation of Wafer and Lot summaries using Box-Plot criteria.
-dboutliers	Filters Outliers from the calculation of Wafer and Lot summaries using database limits.

-pgext	[extension] Appends extension to program name.
-softbin	Forces the use of soft bins (default is hard bins) in BIN_LOG for Summary Program class when both soft bin and hard bin information is part of the Summary report.
-rework_action	[action] Sets the rework_action field in the PROGRAM table to “action”. Valid values for rework action are 1, 2, 3, 4.
-srclot	Gets Source Lot from file name (SrcLot_name)



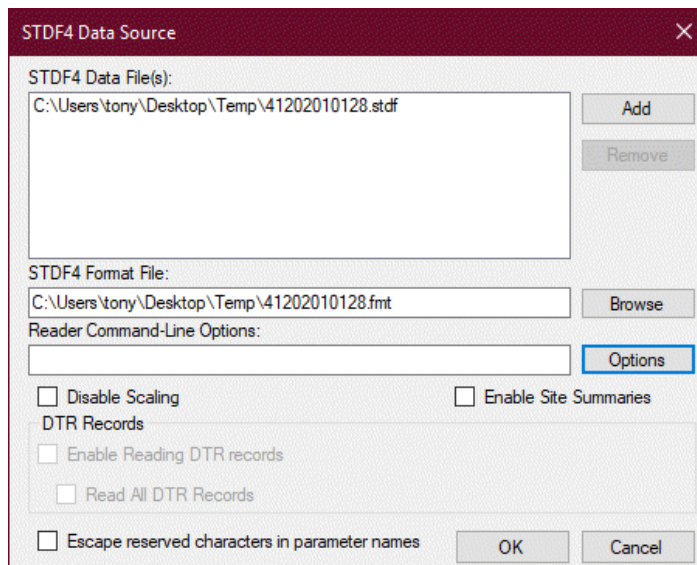
CHAPTER 7

STDF4 WORKSHEET READER AND ASCII UTILITY

STDF4 WORKSHEET READER INTERFACE

From Exensio –Yield, STDF4 files that are to be imported directly to the worksheet without using the database retrieval system. Instead, the Data Import dialog will be used. From this dialog, you will configure the data so it will be properly formatted for the Exensio –Yield environment. If your data import sessions will always utilize database the retrieval window, you will not be using this functionality.

To invoke the dialog, select Tools > Import > From STDF4 File(s).



There are several primary components that must be specified from the data import dialog for a reader session to be successful, including the **Data Files**, **Format File**, — and, depending on your requirements, additional configuration options.

From this dialog, you will configure the data STDF4 data file so it will be properly formatted for the Exensio –Yield environment.



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

Data File

1. In the **STDF4 Data File(s)** area, click on the **Add** button and when the file browser opens, navigate to the data file you want to import.
2. Select your STDF4 data file.

Format File

3. In the **STDF4 Format File** field, click on the **Browse** button to navigate to the format file for your data.
4. Select the format file.

This assumes that at least one specific format file has already been created for the proper formatting of your data files. Original creation of format files is covered in [“Exensio –YieldSTDF4 Format File — Overview,”](#) pg. 288.

Reader Command-Line Options

5. If you want to apply STDF4 command-line options during import, enter them in the **Reader Command-Line Options** field.

This functionality is used for STDF4 options that are not supported in this STDF4 Data Source dialog.

- a. Click on the **Options** button for a list of options that can be used with this function. The options’ descriptions are provided.
- b. Manually enter each command-line argument that you want applied; space-separated. Be sure to enter the “-” character in front of the option name, as displayed in the Options table.

Scaling

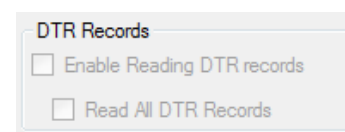
6. Select the **Disable Scaling** option if you want to disable default scaling of imported values.

Site Summaries

7. Select the **Enable Site Summaries** option if you want to load summary data at the site level. If your data does not include multiple sites, the option will not have any effect.

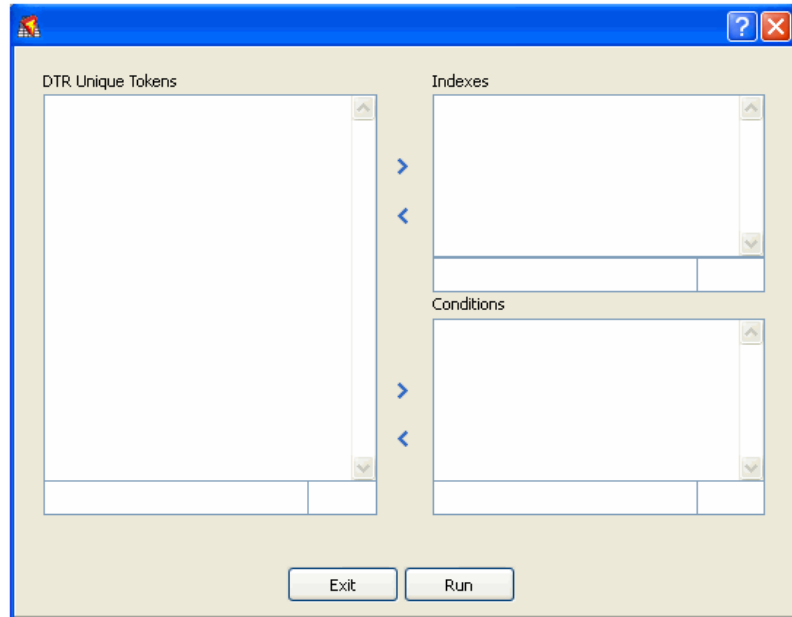
DTR Records

When selecting a format file, its content will be checked for DTR record (Datalog Text Records) definition (the last line). If a DTR record definition exists in the format file, the **Enable Reading DTR Records** option will be enabled, otherwise it will be disabled.



If the **Enable Reading DTR Records** option is selected, the **Read All DTR Records** option will also be enabled.

8. Select the **Enable Reading DTR Records** option to read DTR records in the selected data files.
 - When the **Enable Reading DTR Records** option is selected and if the **Read All DTR Records** option not selected, then upon pressing the **Load** button a second dialog will be shown in which you can select what DTR tokens will be conditions and what tokens will be indexes or both, according to where the **text_dat** token is placed in the main format file.



- If the **Read All DTR Records** option is also selected, this dialog will not be shown and all DTR tokens will be either indexes or conditions based on the selection of **DTR.text_dat** in the format file.

If DTR records exist in the selected file but the **Enable Reading DTR Records** option is not selected, the reader will ignore DTR options in the format file and ignores DTR records in the data file.

Reserved Characters

Parameter names that contain square bracket special characters (“[” and “]”) can create problems for Spotfire. This is because Spotfire’s accessing of columns is done by enclosing the column name in square brackets. Therefore, if a column (parameter) name contains any square brackets, it must be “escaped.”

9. Select the **Escape reserved characters in parameter names** option if you want, in accordance with SF functionality, one bracket to be added in front of the parameter name string and two brackets to be added at the end of the parameter name string.

For example, the parameter **IVF[5V]** would be changed to **[IVF[5V]]**.

Executing the WS Reader

- Click on the **OK** button to execute the reader with the selected data, format and/limits files. Your new analysis worksheet will be generated and populated with the data corresponding to your configuration.

Handling of Repeating Serial Number/ Part IDs

The **PRR.part_id** field sometimes contains the test counter for a specific part. The test count could potentially be repeated. When repeats occur, the **PRR.part_id** is left as it is, and the repeat counter is output in a new index called **PRR.part_id.count**, which will be automatically added after **PRR.part_id** whenever it is used as an index.

Worksheet Reader Command-Line Arguments

The **stdf4_x64.exe** STDF4 reader executable supports 64-bit operating systems. The **stdf4.exe** STDF4 executable supports systems running on a 32-bit OS.

-comma_replacement <rep> — Replacement for “,” used in sheet output files.

-Dfmt — points to the created DTR format file:

```
stdf4.exe <data file> -fmt <main format file> -Dfmt
<DTR format file>
```

OR

```
stdf4_x64.exe <data file> -fmt <main format file> -Dfmt
<DTR format file>
```

-DTR_scan — causes the reader to scan the data files for DTR records:

```
stdf4.exe <data file> -fmt <main format file> -DTR_scan
```

OR

```
stdf4_x64.exe <data file> -fmt <main format file> -DTR_scan
```

-export — Generates a *dpEXPORT* file (in *dpEXPORT*) and writes to the database. **-export** will be followed by the program class (mandatory), a comma and the extension of output file (optional - default is “res”).

STDF4 reader syntax (*use stdf4.exe or stdf4_x64.exe*):

```
stdf4.exe <data file> -fmt <format file> -export <prog. Class>,<extension>
```

or

```
stdf4.exe <data file> -fmt <format file> -export <prog. Class>
```

- If the argument is not passed, the reader behaves as usual.
- This argument cannot be used with **-char** argument. If it is, an error will be generated, and the reader won’t produce output files, only an error file (err.jnk).
- This argument can only be used with results report (specified in the format file), otherwise, an error is generated and the reader won’t produce output files, only an error file (err.jnk).

dpEXPORT is fully described in the “*dpEXPORT* and *djEXPORT*” chapter of the Exensio –Yield Administrator Tools manual.

-limitsinclude — (to be used with the **-export** option.) **-limitsinclude** causes the limits section to be included in the .res output file. It outputs the limits section (<BOL>...<EOL>) to the resulting *dpEXPORT* file. Scaling information is included in the results section (<BOR>...<EOR>). The resulting section contains the scaling factor too.

-map_head() — Match with SDR.head_num = 0 when there is only 1 SDR record.

-max_cols — specifies the total maximum number of output columns in all files. If not specified, the default value is 32,000.

-max_file_cols — specifies the maximum number of output columns per file. If not specified, the default value is 150 in *ascii_spreadsheet* and 75 in *stdf4_spreadsheet*.

-maxtime — The timeout of the reader in seconds. If not used, the reader won't timeout.

-no_filter — Do not filter empty results.

-noptr — when used, the reader will not error out if there are no PTRs and/or MPRs in data file.

-noscale — when used, disables auto-scaling of the test results and limits in PTR and MPR.

-retest — creates multiple export files when retest is detected and **-export** is used. This feature is only activated when the **-export** option is also used.

The default behavior of the reader without this option is to overwrite repeated data, as with typical retest data. In this context, repeated data means receiving data for the same key indexes and key conditions more than once.

When **-retest** (and **-export**) are implemented, repeated data gets split into more than one file on export. The file names in this case start with a retest number (0_, 1_, ...). The number of files depends on number of retests in the file.

-sheet — causes the output of all **res#.jnk** and **dev#.jnk** files to be output in one file named **out.jnk**, which is formatted as a ready spreadsheet. This also causes the output of **lim.jnk** to be in a raw sheet-like format, with new ordering of columns. (There is no maximum number of columns limitation when this option is used).

-skip_invalid_records — handles invalid records and duplicated FAR records.

Behavior when using the option:

- **Multiple FAR records:**
 - Accept first FAR record and ignore the rest of FAR records
- **Invalid record types:**
 - When used with **-sheet**: read the file until the first invalid record found

- When used with **-export**: stdf4.exe generates a warning (invalid record type)

Default behavior (without using the option):

- **Multiple FAR records:**
 - stdf4.exe errors out with “Error, datafile has multiple FAR records, must have only one.”
- **Invalid record types:**
 - When used with **-sheet**: read the file until the first invalid record found
 - When used with **-export**: stdf4.exe errors out with “Error reading file, invalid record type.”

-testtxt — replaces test name from TSR by first occurrence of test txt from PTR.

-testtxtkey — replaces the test name from TSR by each unique combination of test txt and test number for PTR and MPR.



The default values are used whenever these options are not used.

-tz <N> — shifts time-zone of date-time fields by valid range: 0.5 steps from [-23.5, 23.5].

userdef.cfg — Tools for Importing Flat Files Into Worksheets

Several API functions have been created to allow you to build a scripted tool that will facilitate the importing of flat files into worksheets using the worksheet readers. This functionality is related to the Import ASCII files utility and to the worksheet readers (ASCII, STDF4 and STDF3).

The idea is to create a script defining a set of rules which will be stored in userdef.cfg. These rules will tell the worksheet reader how to import the specified files. When *dataPOWER* is run and the user runs a worksheet reader, userdef.cfg automatically applies the rules as needed.

Any rule has the following structure:

Reader Key Word, File Extension, directory name, format file or separator

Acceptable values of:

- Reader Keywords: ASCII, STDF3, STDF4 and DELIMITED.
Any rule that does not have one of these keywords will be ignored.
- Separator: TAB, COMMA, WHITESPACE.
Any other value will be used as a separator. For example, if you set a rule with "coma" as separator, then "coma" is not recognized as keyword and the text in the imported files will be separated using the string, "coma".

Examples:

```

ASCII      asc */dpuser/data/ascii ascii_format.fmt
STDF4      stdf4 */dpuser/data/stdf4 stdf4_format.fmt
DELIMITED  csv */dpuser/tables comma
DELIMITED  txt */dpuser/tables tab
STDF3      std */dpuser/stdf3 stdf3_format.fmt

```

- Row 1 specifies that if a file with the “asc” extension is found under the ***/dpuser/data/ascii** directory then this file needs the ASCII worksheet reader to import it. The reader should use the `ascii_format.fmt` format file. The directory under which the tool will look for the format file is stored in `usedef.cfg`. The user can change it using the following API function:
`CALL imp.dps:SetDefaultFormatDir(directory,"ASCII")`
 Where
 "directory" is the directory where the format file will be found
 "ASCII" is the keyword to indicate that this directory is related to ASCII format files
- Row 2 specifies that if a file with the “stdf4” extension is found under the ***/dpuser/data/stdf4** directory then this file needs the STDF4 worksheet reader to import it. The reader should use `stdf4_format.fmt` format file. The directory under which the tool will look for the format file is stored in `usedef.cfg`. The user can change it using the following API function:
`CALL imp.dps:SetDefaultFormatDir(directory,"STDF")`
 Where
 "directory" is the directory where the format file will be found
 "STDF" keyword to indicate that this directory is related to STDF format files
- Row 3 specifies that if a file with the “csv” extension is found under the ***/dpuser/tables** directory then this is a flat COMMA file, and you need to use simple import with comma as separator.
- Row 4 specifies that if a file with the “txt” extension is found under the ***/dpuser/tables** directory then this is a flat TAB file, and you need to use simple import with tab as separator.

As an example, if the following files are selected to import:

```

dp7.0/dpuser/data/ascii/file1.asc
dp7.0/dpuser/tables/file2.csv
dp7.0/dpuser/tables/file3.csv
dp7.0/dpuser/tables/file4.txt
dp7.0/file5.ddd

```

Then the tool will recognize that:

- **dp7.0/dpuser/data/ascii/file1.asc** should be imported using the ASCII worksheet reader (rule 1) using `ascii_format.fmt`
- **dp7.0/dpuser/tables/file2.csv** and **dp7.0/dpuser/tables/file3.csv** should be imported using the simple import tool using COMMA as separator. The result of the import of these two files is one single worksheet (rule 3).

In this case the tool invokes a dialog that asks for the number of indexes and conditions of the data to import. Also it sets the separator value to what is defined by the rules (but the user can change it). Once all the files are imported, the tool merges the results of all these files into one Exensio – Yield worksheet.

- **dp7.0/dpuser/tables/file4.txt** should be imported using the simple import tool, using TAB as separator (rule 4)
- Since there is no match for **dp7.0/file5.ddd** in the rules then the tool uses the simple import tool using COMMA separator (but the user can change the separator).

List of API Functions

`imp.dps:ClearConfig()` Clear the current configuration.

`imp.dps:SetDefaultDir(directory)` Set the default directory in `userdef.cfg`, where the "Import" GUI should point to when opened the first time.

`imp.dps:GetDefaultDir()` Get the default data directory from `userdef.cfg`.

`imp.dps:SetDefaultFormatDir(directory,code)` Set the default directory in `userdef.cfg` where the format files are located.

`imp.dps:GetDefaultFormatDir(code)` Get the default directory from `userdef.cfg` where the format files are located.

`imp.dps:AppendToConfig(configArr)` Add the rules defined in `configArr` to the list of rules defined in `userdef.cfg`.

`imp.dps:GetConfig()` Get the current configuration from `userdef.cfg`.

Example of Usage

```

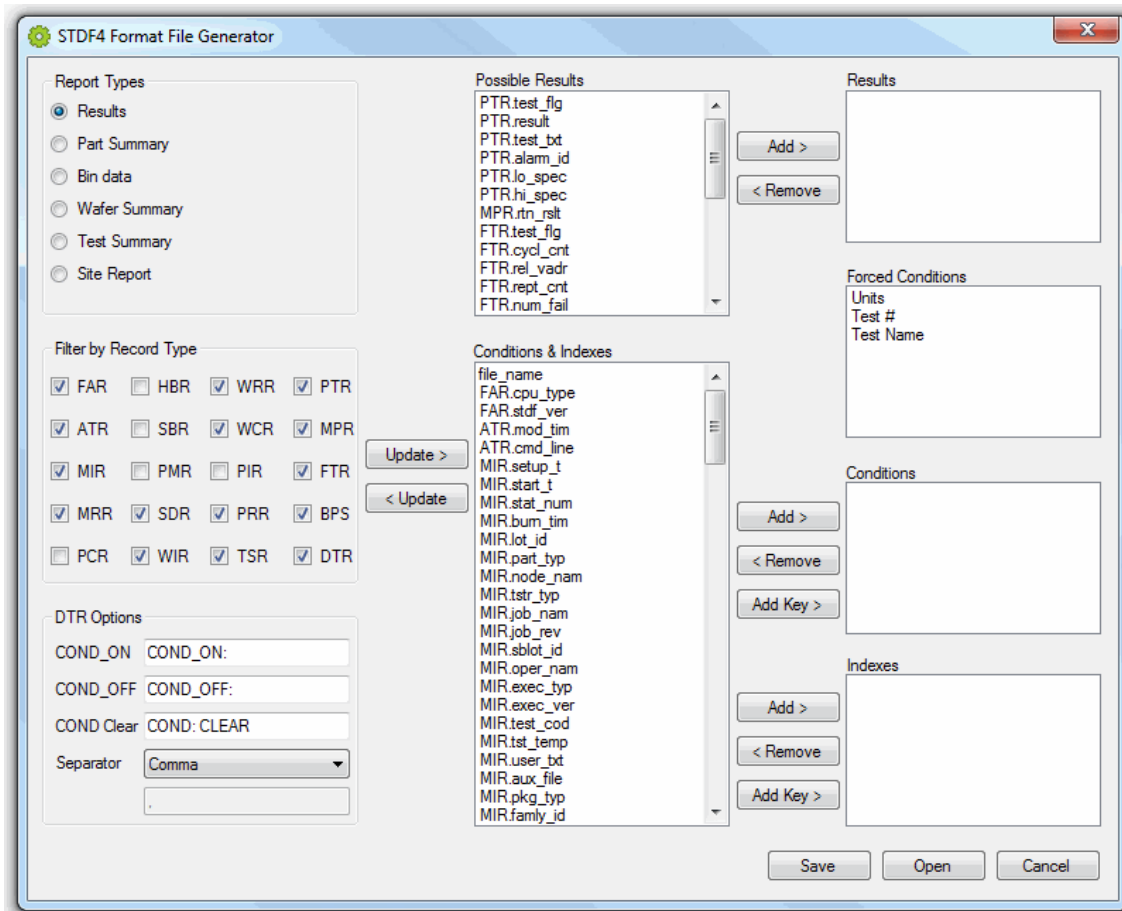
FUNCTION Test()
  DEFINE arr[6,4]

  GET arr[*],1] FROM "ASCII","STDF4","DELIMITED","DELIMITED",
  "DELIMITED","DELIMITED"
  GET arr[*],2] FROM "txt","std","txt","spc","csv","com"
  GET arr[*],3] FROM
    "*"dpuser\data\ascii",
    "*"dpuser\data\stdf4",
    "*"dpuser\data\flat",
    "*"dpuser\data\flat",
    "*"dpuser\data\flat",
    "*"dpuser\data\flat"
  GET arr[*],4] FROM
    "ss_demo_1.fmt",
    "stdf4.fmt",
    "Tab", { "Tab","WhiteSpace","Comma",Other user defined}
    "WhiteSpace",
    "Comma",
    " "
  CALL imp.dps:ClearConfig()
  CALL imp.dps:SetDefaultDir(user.dps:userPath &"\data\")
  CALL imp.dps:SetDefaultFormatDir(user.dps:userPath
    &"\formats\ascii\","ASCII")
  CALL imp.dps:SetDefaultFormatDir(user.dps:userPath
    &"\formats\stdf4\","STDF4")
  CALL imp.dps:AppendToConfig(arr)
END FUNCTION

```

EXENSIO –YIELDSTDF4 FORMAT FILE — OVERVIEW

The STDF4 data reader requires a format file that defines what data will be treated as results, conditions, and indexes. The format file can be generated using the window shown below, which allows you to choose from a library of STDF4 results, conditions and indexes. The window is accessed by selecting Tools > Import > STDF4 Format File Generator.



String length in the data file cannot exceed 255 characters. Any test name that is longer will be truncated.

I – REPORT TYPES

The data reader supports multiple results. The first step in creating a format file is deciding what results you will be using. Selecting an item from the **Report Type** field fills the **Possible Results** list box with all record fields that can be chosen as results for that particular report. For example, if you are interested in a results report then select the **Results** option, which will fill the **Possible Results** list box with record fields. To choose from the possible results, select any combination of items in the **Possible Results** list and click on the $\gg$ control pointing to the **Results** list box.

To remove an item from the **Results** list box, select the result(s) to be removed and click on the $\ll$ control pointing back to the **Possible Results** list box.

With the **database** option selected, the supported Report Types are:

- Results: Results from PTR.
- Summary: Results from HBR, SBR and TSR

II – CONDITIONS AND INDEXES

The second step in creating a format file is deciding on the conditions (Column headings in the raw data table) and indexes (Row headings in the raw data table).

Definitions

Key Conditions The group of conditions that uniquely identify the data column. The combination of the values of all key conditions for any particular data column will be unique to that column.

Key Indexes The group of indexes that uniquely identify the data row. The combination of the values of all key indexes for any particular data row will be unique to that row.

For each Report Type the **Conditions & Indexes** list box is filled with all the record fields that can be chosen as Conditions or Indexes, including the datalog file name. For all Report Types the first three conditions are always forced to be the units, test name and test number.

A condition is included in the format file by highlighting it in the **Conditions & Indexes** list box and clicking the $\gg$ control pointing to the **Conditions** list box. There are two controls, one is used to define the selected items as conditions and the other as key conditions. After a condition is already in the list it can be changed from key to non-key or vice-versa by clicking the **Key** button.

The red  control pointing back to the **Conditions & Indexes** list box removes the selected condition(s) from the format file.

The **Indexes** list box operates exactly the same way as the **Conditions** list box.

Filter by record type

Since the list of possible conditions and indexes can be relatively long, you may need to look at a subset of the list. This is achieved by filtering out all undesired records from the list.

As an example if you are only interested in conditions from PRR records:

1. Click on the **Clear** button to clear all check-boxes.
2. Choose PRR by clicking on the **PRR** check-box.
3. Click on the arrow pointing to the **Conditions & Indexes** list box.

This updates the list to include items from PRR records only.

III – DTR OPTIONS

DTR (Datalog Text Records) contain text information that is to be included in the datalog printout. DTRs may be written under the control of a job plan. For example, to highlight unexpected test results. They may also be generated by the tester executive software: for example, to indicate that the datalog sampling rate has changed. DTRs are placed as comments in the datalog listing.

DTR records contain only one sub-record, **text_dat**, it can be used to define test conditions' start and stop, and their values. Using the following syntax:

```
COND_ON: cond1=value1, cond2=value2
COND_OFF: cond1, cond2, cond3
COND: CLEAR
```

"COND_ON:" default token for turning conditions on

"COND_OFF:" default token for turning conditions off

"COND: CLEAR" default token for turning all conditions of

cond1, cond2, cond3: condition names

value1,value2: condition value

comma is used as a separator.

All white space at the end and beginning of condition/index names will be stripped.

The tokens and the separator can be user-defined in the GUI, as explained next.

When the **Report Types > Results** option is selected, the **DTR Options** area is enabled. Otherwise, this area will not be visible in the window.

When **Filter by Record Type > DTR** is selected (exclusively), the updated **Conditions & Indexes** list will have just two items for DTR record type: **file_name** and **DTR.text_dat**.

In the format file, DTR options are used to define the condition starting point (condition on) and stopping point (condition off). The DTR contains conditions that will be applied to all of the tests that follow this record, until they are cleared (condition clear).

In the **DTR Options** area, **COND_ON:**, **COND_OFF:**, and **COND: CLEAR** are configurable default names for the condition-on, condition-off and condition-clear operations, respectively. The separator is also configurable, with comma being the default.

Example:

```
COND_ON: Temp=25C,Vcc=4.5V
```

Condition definition ends with

```
COND_OFF: Temp,Vcc
```

Condition values are allowed to change without first turning the condition off.

The following DTR options are configurable:

All entries should be alphanumeric characters, underscore, colon or a space. The entries are case insensitive.

Custom entries cannot contain a single quote and are case insensitive.

- **Turn on conditions token name** — The default is “COND_ON”. You can enter a new term in the **COND_ON** text field.
- **Turn off conditions token name** — The default is “COND_OFF”. You can enter a new term in the **COND_OFF** text field.
- **Clear all conditions token name** — The default is “COND: CLEAR”. You can enter a new term in the **COND Clear** text field.

- **Conditions Separator** — All entries for the **Separator** field should be either an alphanumeric character, underscore, colon, comma or tab. A space a single quote is not a valid entry. The default is Comma.

When a another separator is needed, select the **Other** option and enter a single character separator in the text field that is enabled (using the above rules).

DTR Options

COND_ON COND_ON:

COND_OFF COND_OFF:

COND Clear COND: CLEAR

Separator Comma
Comma
Tab
Other

New Line and Separating Characters in DTR Records

The system supports more than one separating character at the same time, delimited by an apostrophe. This means that in the same run, whenever any of the characters is faced it will be treated as a separator. These separators will be provided in the format file as usual. The format file line for the DTR options will be like this:

```
DTR_OPTIONS COND: DUMMY 'DUMMY' ' ' ' ' , ' _ ' - '
```

The comma, dash and underscore are the separators in this case.

When a newline character is needed, '**NL**' shall be provided, and where a tab is needed, '**LT**' shall be provided.

IV – EXPORT BIN/TEST SUMMARIES

The STDF4 data reader command-line argument, **-export_summaries**, generates Test Summary and Bin Summary sections in the *dpEXPORT* export file. This argument is optional.

Bin Summary

When the STDF4 **-export_summaries** command-line argument is used, the Bin Summary section is generated by *dpEXPORT* in the export file. It summarizes the number of fails per each site and for total sites for all bins.

<BOBS>/<EOBS> — This section is for soft bins. It is exported in stdf4.exe only, in export mode. It is pulled exclusively from the SBR records.

```
<BOBS>
bin1|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
bin2|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
.
.
.
binO|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
<EOBS>
```

<BOHBS>/<EOHBS> — This section is for hard bins. It is exported in stdf4.exe only, in export mode. It is pulled exclusively from the HBR records.

```
<BOHBS>
bin1|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
bin2|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
.
.
.
binO|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
<EOHBS>
```

Test Summary

When the STDF4 **-export_summaries** command-line argument is used, the Test Summary section is generated by *dpEXPORT* in the export file. It summarizes the number of failed tests per each site and for total sites for all test numbers. It is pulled exclusively from the TSR records.

Following is the format of the bin summary section:

```
<BOTS>
bin1|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
bin2|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
.
.
.
binO|num_of_fails_in_site1|num_of_fails_in_site2|....|num_of_fails_in_site255
<EOTS>
```

V – EXPORT HARD/SOFT BINS

<BOHB>/<EOHB>: this section is for hard bins, it is parsed in *dpIMPORT* and it gets exported when using *dbascii*, *dpEXPORT* and *stdf4.exe*.

stdf4.exe Format:

bin number | bin name | P (Pass) or F (Fail) | -I | -I | -I

dbascii/dpexport Format:

bin number | bin name | P (Pass) or F (Fail) | bin order | red | green | blue

<BOSB>/<EOSB>: this section is for soft bins, it is exported in *stdf4.exe* only. It gets exported without using any special options.

format: bin number | bin name | P or F | -I | -I | -I

VI – SAVING FORMAT FILES

After choosing the results, conditions and indexes the format file can be saved by clicking on the **Save** button. The name of the saved file is entered in the box below the **Saved Format Files** list box.

VII – LOADING FORMAT FILES

Previously created format files can be loaded by selecting the desired file in the **Saved Format Files** list box and clicking on the **Load** button or clicking on the arrow pointing to the **Conditions & Indexes** list box. This is useful for reviewing or editing existing files, or for creating a format file based on an existing one.

IMPORTING STDF4 DATA FROM A DATABASE — stdf4ascii

This program was created to make it easy and efficient for the user to extract and translate certain sets of records from one or more stdf4 binary file into ASCII files. An stdf4 binary file can have 300MB of data while the user might be interested in a small portion of that data. This program will enable you to specify which sets of data to extract, and then do the translation from binary to ascii format.

The ascii file can then be uploaded into the database using the ASCII reader (dbascii) — *“ASCII Data Reader,”* pg. 55.

The program supports the six most commonly used sets or types of data (see *“Additional Notes,”* pg. 298). In addition, you can customize your requirement by adding more record types and you can extract multiple sets during the same execution. You can also use the **+ALL** command-line option, which will extract all the records from the data files.

By default, the program will do scaling on the PTR and MPR records, according to the stdf4 file specification. Scaling will also translate the unit name according to the scale of the result.

- If you want to turn off scaling, supply the **-noscale** command-line option.
- If there are no PTR or MPR records, the reader will return an error by default. If you want to bypass the error mode when there are no PTR/MPR records, supply the **-noptr** command-line option.

Format Files

PDF Solutions provides a set of sample format files for the dbascii reader to load the generated ascii files into the database.

You can modify these files to meet your requirements. The format files are in the **dbformats/stdf4ascii** directory in the server-side of the Exensio –Yield installation package.

USAGE

Usage Statement

stdf4ascii [<options>] <data file(s)>

- u Display this help message.
- type <type> Output records according to the value in <type>.
The supported output types are:
 - 1 --> Binmap
 - 2 --> Bin Summaries
 - 3 --> Bin Summaries/Parametric
 - 4 --> Parametric
 - 5 --> Functional
 - 6 --> Multi-Parametric
- +<record> Add <record> to the output.
<record> must all be in upper-case (e.g. +PTR).
To include all records, use +ALL.
- cfg <config> Use <config> as the config file to determine which fields to print in each record. DEFAULT: stdf4ascii.cfg
- gen_config Generate default config file which includes all fields in all records.
- out <out-dir> Specify a directory where to create the output file(s).
DEFAULT: .
- ext <ext> Use the specified <ext> as an extension for the output filenames.
DEFAULT: txt
- mode <mode> Choose output mode (1 or 2).
DEFAULT: 1
- delim <delim> Use <delim> as a field delimiter when in mode 2.
DEFAULT: |
- null <str> Use <str> as a null value when field values are missing in mode 2.
DEFAULT: (blank)
- compact Produce a compact version of the output in order to reduce its size.
- noscale Disable auto-scaling of the test results and limits in PTR and MPR.

-mode

- **Mode 1** — a full record, description, field names and values.

Partial Example:

```
-mode: mode 1:
MIR - Master Information Record
setup_t = 2004-10-09 14:33:32
start_t = 2004-10-09 19:15:59
stat_num = 0
mode_cod = P
rtst_cod =
prot_cod =
burn_tim = 0
cmod_cod =
lot_id = DN120588
part_typ = DN120588_04.8
...
```

- **Mode 2** — record name and field values only.

Partial Example:

mode: mode 2:

MIR

2004-10-09 14:33:32|2004-10-09

19:15:59|0|P|rn|l|rn|l|0|rn|l|DN120588|DN120588_04.8 ...

-compact

- When -mode = 1, removes spaces to produce a compact version of the output. No effect when -mode = 2.

Additional Notes

- If no data files are supplied, nothing will be done.
- An output file will be generated for each data file supplied. The output file name(s) will be generated by suffixing the data file name(s) with the “.txt” extension. You can choose a different extension using the -ext command-line option.
- The FAR, MIR and MRR records are always added to the output by default.
- Type 1 (Binmap) includes PIR, PRR, HBR, SBR, WIR, WRR and WCR.
Type 2 (Bin Summaries) includes HBR, SBR and WCR.
Type 3 (Bin Summaries/Parametric) includes HBR, SBR, WCR and TSR.
Type 4 (Parametric) includes HBR, SBR, WIR, WRR, WCR, PIR, PRR, TSR and PTR.
Type 5 (Functional) includes HBR, SBR, WIR, WRR, WCR, PIR, PRR, TSR and FTR.
Type 6 (Multi-Parametric) includes HBR, SBR, WIR, WRR, WCR, PIR, PRR, TSR, PTR and MPR.
- To include all records in the data files, use: **+ALL**.

USAGE EXAMPLES

```
stdf4ascii -type 3 +PRR +PTR data1.std data2.std data3.std
```

- » Will output FAR, MIR, MRR, Bin Summaries and Parametric records (type 3) plus PRR and PTR for data1.std, data2.std and data3.std.

```
stdf4ascii +PRR +PTR data1.std -noscale
```

- » Will output FAR, MIR, MRR, PRR and PTR for data1.std. No scaling is done.

```
stdf4ascii +ALL data1.std data2.std
```

- » Will output all records for data1.std and data2.std.



FRONT



CONTENTS



INDEX

exensio

CHAPTER 8

DEFECT READER

IMPORTING DATA

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file.

The Exensio –Yield Defect reader imports defect data directly into the Exensio –Yield database system.



The Exensio –Yield Defect Reader is a database reader and has no worksheet reader functionality. It therefore runs only on user configurations which include the Exensio –Yield dataBASE system.



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

DefectMap Data Retrieval

Defect data retrieval Exensio –Yield accomplished with the **Defect** level of the Retrieval interface.

The primary function of the window is to supply customized retrieval options so the end-user can easily populate a worksheet with defect data, and then use the tool for analysis.

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	Char	If	Target
KeyCond	False	AND	Integer	Else	LOL
LimCond	EQ	Const	Real	Exit	HOL
Index	NE	Var	String	LSL	LWL
KeyIndex	NOT	Begin	Boolean	HSL	HWL
Result	LE	End	While	LPL	FailBin
FileName	GE	Step	For	HPL	mod
Script	LT	GT	To	Tune	

All built-in functions described in *“Built-in Functions,”* pg. 308, also constitute part of the list of key words.

Comments The characters `/*` introduce a comment that terminates with the characters `*/`, such comments do not nest and they can not occur within strings. The characters `//` introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
    job_nam      =      "MyJob"      //String Constant
    Product      =      "MyProduct"  //String Constant
    NumOfBins    =      32            //Integer Constant
```

Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 255 characters)
- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                   condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                   condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The allowed types for conditions and indexes are Real, Integer and String. Also, note that the order that these conditions appear in is important and is different for dB and WS readers. The first three conditions should always be:

dB Reader —

Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)

WS Reader —

TestNum	Integer	(For Test Number)
Tname	String	(For Test Name)
Unit	String	(For Units)



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results

The Defect reader supports choosing more than one result each of a possibly different type. Results are declared in the VAR block and the following syntax is used:

```
Variable1,Variable2,...TypeResult// To declare a result
```

Every format file must have at least one result declared.

When using the Defect reader with the database option only one result is allowed of type real.

The Main Block

The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the Defect reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword "FileName" which is of type string. For example to assign the Index lot_id the value of "FileName" one would

write:

```
lot_id = FileName//lot_id must be of type string
```

In this example the keyword "lot_id" will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

```
AddSep(':',')
```

OPERATORS

<i>Arithmetic:</i>	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Assignment	=
	Modulus	mod

Logical: Used only with variables of type Boolean.

Logical Or	OR
Logical And	AND
Negation	NOT

<i>Relational:</i>	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression
statements
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression
statements
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression
if-statements
Else
else-statements
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

This section contains the functions that have been added to the generic Defect reader in order to retrieve defect related information from ASCII defect files such as KLARF.

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of Defect Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 309
 - **Go To** — pg. 309
 - **Separators** — pg. 309
 - **Skip Forward/Backward** — pg. 310
 - **Miscellaneous** — pg. 310
- **Data Retrieval** — pg. 310
 - **Data** — pg. 310
 - **Strings** — pg. 310
 - **Sub-Strings** — pg. 311
 - **Integers** — pg. 313
 - **Real** — pg. 313
- **String Manipulation** — pg. 313
- **Database** — pg. 315
 - **DefectMap** — pg. 315
 - **Fab** — pg. 328
 - **Technology** — pg. 329
 - **Family** — pg. 329
 - **Process** — pg. 329
 - **Product** — pg. 329
 - **Program** — pg. 330
 - **Lot** — pg. 331
 - **Wafer** — pg. 332
 - **Step** — pg. 333

- [Stage](#) — pg. 333
- [Equipment](#) — pg. 334
- [Operator](#) — pg. 335
- [Customer](#) — pg. 335
- [Wafer Configuration](#) — pg. 336
- [Rework](#) — pg. 336
- [Tagging](#) — pg. 336
- [Miscellaneous](#) — pg. 337
- [Mathematical](#) — pg. 337
- [Debugging](#) — pg. 338
- [System](#) — pg. 339
- [Zones](#) — pg. 339

File Navigation

<i>Go To</i>	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the error code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.

*Skip Forward/
Backward*

SkipLines (integer) —

Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.

SkipWords (integer) —

Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.

SkipChars (integer) —

Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.

Miscellaneous

NotEndOfFile —

Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.

GetLineLen —

Accepts no arguments and returns an integer. Returns the number of characters from the File Pointer to the end of line.

Data Retrieval

Data **OpenFile (string) —**

Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the **-file_path** command line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

Strings **GetWord —**

Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.

GetPrevWord —

Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

	GetQuotedWord (char) —	Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error code is set if no quoted word is found.)
	GetLine —	Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.
<i>Sub-Strings</i>	GetChar —	Accepts no arguments and returns the first character in the current word, leaving the File Pointer one character beyond the retrieved word.
	GetChars (integer) —	Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.
	GetCharsTrim (integer, char) —	Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)
	ExtractString —	Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.
	GetLeftChars (integer) —	Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetRightChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetMidChars (integer, integer) — Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordAfter (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.

If there are multiple occurrences of the passed character, the function returns all characters after the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) — Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

<i>Integers</i>	GetInt —	Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.
	GetPrevInt —	Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.
	<i>Real</i> GetReal —	Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.
	GetPrevReal —	Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.
	String Manipulation	ToLower (string) —
		Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.
		ToUpper (string) —
		Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.
		IsNumber (string) —
		Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.
		IsString (string) —
		Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.
		Right (string,integer) —
		Accepts two arguments of type string and integer and returns a string. Same as GetRightChars(), but operates on the passed string instead of the current word.
		Left (string,integer) —
		Accepts two arguments of type string and integer and returns a string. Same as GetLeftChars(), but operates on the passed string instead of the current word.
		Mid (string, integer, integer) —
		Accepts three arguments of type string, integer and integer and returns a string. Same as GetMidChars(), but operates on the passed string instead of the current word.

After (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordAfter()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

Before (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `'StrToInt ("16A3")'`, the output would be the integer 16; for `'StrToReal ("16.1A3")'`, the output would be 16.1.

IntToStr (integer) — Accepts one argument of type integer and returns a string. Translates the passed integer to a string.

StrTrim (string, char) — Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.

StrLen (string) — Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.

Argument —

Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument **-arg**.

Database

DefectMap **IsProgramNew —**

Returns a boolean variable indicating whether the given program name had been stored in the Program table in the given database.



Important: This function has to be called after **vDbProgram** or **DbProgram**.

dBDumpLayer —

Dump the defect data to the database.



Important: Only call after all the relevant information of a record* is collected.

An error is generated if the die dimensions are not provided.

An error is generated if the distance between the wafer center and die(0,0) is not provided.

DbCvInLine —

Accepts no arguments. If called, the defect program is marked as a CV inline program. This affects the way the program is treated in terms of overlay.

vDbAddToDieMapTable (diex –integer-, diey –integer, die_type –char-) —

Add an entry into the die map table. This table contains the indices of the inspected.

Input variables:

diex: The x index of the die.

diey: The y index of the die.

die_type: valid values are 'V' and 'R'.

'R' indicates a removed (not inspected die).

'V' indicates an inspected (valid) die.

*A record is defined as a single wafer inspection at a given layer.

vDbDefectSummary (test_num –integer-, def_cnt –integer-, insp_area –real-, def_dens –real-, insp_die –integer-, def_die –integer-) —
Enter a summary of the defect record.

Input variables:

test_num: The test number.

def_cnt: The number of defects in the record.

insp_area: The area inspected by the inspection tool (in μm^2).

def_dens: The defect density.

insp_die: The number of inspected die.

def_die: The number of defective die.



This function can be called up to twenty times within a single record.

vDbSlotNum (slot_num –integer-) —
Enter the slot number of the wafer.

Input variables:

slot_num: The slot number of the wafer.

vDbToleranceDimensions (tol_maj –real-, tol_min –real-, tol_or –real-) —

Enter the tolerance diameter for the layer. This is used to set the values for all defect types (adders, commons, repeaters, etc...). There is no default for circle or ellipse; if $x = y$ (**tol_maj** = **tol_min**), then a circle is defined.

Input variables:

tol_maj: The major axis of the tolerance ellipse.

tol_min: The minor axis of the tolerance ellipse.

tol_or: The orientation of the tolerance ellipse.



These values must be given in order to be able to compute defect type. If they are not provided, they will be forced to 1 micron. The orientation of the ellipse will not be supported at this stage. It is currently hard coded to zero.



Warning: If a value less than zero is passed to the function, a warning message is generated and the value is set to 1 micron.

vDbClusterToleranceDimensions (tol_maj real-, tol_min real-, min_cluster integer-) —

Enter the elliptical cluster tolerances.

Input variables:

tol_maj: The major axis of the tolerance ellipse.

tol_min: The minor axis of the tolerance ellipse.

min_cluster: The minimum number of defects needed to consider a group of defects a single cluster.

These values must be provided in order to be able to cluster defects together according to the distance between them.



Warning: If a value less than zero is passed to the function, a warning message is generated and the value is set to 1 micron.

vPrepareSizeSummary (lower_bound –float-, upper_bound –float-, size_type –integer-) —

When a defect program is first created, you can use the defect format to create size bins for which you would like to see precalculated summaries. This is accomplished using this function. By controlling the call to this built-in function in the format file, you can create as many or as few size bins as you wish (or create none at all).

Defines the size summaries generated by UpStat.

Input variables:

lower_bound: The lower bound of the size bin.

upper_bound: The upper bound of the size bin.

size_type: Only one of the following values are allowed:

size_type	Meaning
1	Use the 'x_size' field of the defect as size indicator
2	Use the 'y_size' field of the defect as size indicator
3	Use the 'df_area' field of the defect as size indicator
4	Use the 'dsize' field of the defect as size indicator



Warning: Provides an error if **size_type** is outside the (1-4) range.

vDbIsWaferPatterned (is_patterned –integer-) — Determine if the wafer is patterned.

Input variables:

is_patterned: Only one of the following two values are allowed:

is_patterned	meaning
0	No dies (patterns) on wafer.
1	Patterned wafer



Default value is 1.

vPosType (position_type – integer-) —

Determines if the coordinates are absolute or relative within the wafer.

Input variables:

position_type: Only one of the following three values are allowed:

Position_type	Definition
0	The xPos, yPos are relative positions from the lower left corner of the die the defect lies in
1	The xPos, yPos are absolute positions from the lower left corner of the wafer
2	The xPos, yPos are measured from wafer center



Through the use of the format file, the user can determine which position type is used in the data file.



Default value is 0.

vDbDefect

(df_index –integer-, xInd –integer-, yInd –integer-, xPos –real-, yPos –real-, x_size –real-, y_size –real-, df_area –real-, df_size –real-, cluster –integer-, intensity –integer-, test_num –integer-) —

Enter a defect and related information into the database.

Input variables:

df_index: The unique index given to the defect.

xInd: The x-axis index of the die the defect was found in.

yInd: The y-axis index of the die the defect was found in.

xPos: The x-position of the defect within the die or wafer (in microns).

yPos: The y-position of the defect within the die or wafer (in microns).

x_size: The size of the defect in the x direction (in microns).

y_size: The size of the defect in the y direction (in microns).

df_area: The area of the defect (in microns²).

df_size: An indication of the size of the defect (in microns).

cluster: Indicates if a defect is part of a cluster.

intensity: Found in Tencor binary file formats. An indication of defect size.

test_num: The index of the test that was run on the wafer.

vDbAddDefectImage (df_index –integer-, image_path –string-, image_file –string-, image_index –integer-, image_type –string-, img_src_code –integer-) —

Used to add defect images into the database.

Input variables:

df_Index: The unique index given to the defect.

image_path: The directory that contains the image files.

image_file: The image file name.

image_index: The index of the image within the image file.

image_type: Defines the image file format. Valid values are: “JPEG”, “TIFF”, “GIF”.

img_src_code: A code that identifies the source of the image.

The defaults are:

Unknown	ims_src_code = 1
SEM Internal	ims_src_code = 2
SEM External 1	ims_src_cod = 3
EDX	ims_src_code = 4
Optical	ims_src_code = 5



Warnings: Only TIFF files with multiple images are allowed. A maximum of five images per defect is allowed. If the image type is TIFF, then the **image_index** always starts from one (greater than or equal to one). The maximum number of TIFF files that can be loaded is 3,000 per wafer/layer combination.



Warnings: If the image type is JPEG or GIF then the **image_index** should be set to greater than or equal to 1.



You can set the second argument (*image_path*) to "NA" and use the **-image_path** command line argument to determine the image file path.

NoImageLoad —

Takes no arguments. Instructs the reader not to load the defect images into the database.

If you wish to store image pointers (as opposed to the images themselves) in the database, you would have to use the "**NoImageLoad**" built-in function in the defect format files. Then you would need to follow the procedures described in *"Using Defect Image Pointers,"* pg. 364.

vAddToClassLUT (index –integer–, class_name –string–) —

Add an entry into the class LUT. There are no bounds on how many entries are in the class LUT.

Input variables:

Index: The unique index given to the class.

Class_name: The name given to the class corresponding to the index.

Recommended use:

Use before individual defect information is read.



A class is not entered into the database unless one of the defects in the file is classified with that class.

Same Classification Code Across Multiple Classification Methods

- a. The Defect reader assumes that class codes are mutually exclusive across classification methods (types)

- b. If you have class codes that exists across many methods, you need to use the **vAddToClassLUT** built-in function for manual classification method (**df_key** = 1)

vDbAddDefectClass (df_index –integer-, class_num –integer-, classification_type –integer-) —

Ability to give a defect a classification using the defect class name as identifier. Can be called multiple times per defect, in which case a single defect will have more than one classification.

Input variables:

Df_index: The unique index given to the defect.

Class_num: The number of the class. The name of the class is extracted from the class LUT.

Classification_type: These are predefined types. Only the following values are allowed:

Classification_type	Classification Method
1	manual
2	roughbin
3	finebin
4	mansemclass
5	autosemclass
6	microsigclass
7	macrosigclass
8	adl



Warnings: A warning message will be generated if any of the following cases occurs:

- For “manual” classification, the class number is not in the class LUT (this includes the case that the table was not read in through the format file).
- The defect with index (**df_index**) was not read in through the format file.
- The **classification_type** is not one of the valid values given in the table shown above.
- For all classifications, the class number is not in the class LUT (this includes the case where the table is not read in through the format file).

Recommended uses:

- Use after the class LUT has been read through the function **vAddToClassLUT**.
- In order to save database space, do not call this function if the defect is classified as “Unclassified”.
- Use after the function **vDbDefect** has been called for the given **df_index**.

vDbAddDefectClassByCode (**df_index** –integer-, **class_num** –integer-, **classification_type** –integer-) —

Ability to give a defect a classification using the code as identifier. Can be called multiple times per defect, in which case a single defect will have more than one classification.

Input variables:

Df_index: The unique index given to the defect.

Class_num: The number of the class. The name of the class is extracted from the class LUT.

Classification_type: These are predefined types. Only the following values are allowed:

Classification_type	Classification Method
1	manual
2	roughbin
3	finebin
4	mansemclass
5	autosemclass
6	microsigclass
7	macrosigclass
8	adl



Warnings: A warning message will be generated if any of the following cases occurs:

For all classifications, the class number is not in the class LUT (this includes the case where the table is not read in through the format file).

- The defect with index (**df_index**) was not read in through the format file.
- The **classification_type** is not one of the valid values given in the table shown above.

Recommended uses:

- Use after the class LUT has been read through the function **vAddToClassLUT**.
- In order to save database space, do not call this function if the defect is classified as “Unclassified”.
- Use after the function **vDbDefect** has been called for the given df_index.

vDbSetCriticalDfClass (class_num -integer-, classification_type -integer-) —

Sets the flag for the given class as “Critical”. This flag indicates to the defect UpStat computations that summaries for this specific class are required. If not called, the class is set as “Not Critical”. If the given class already exists in the database, its critical flag is not updated.

Defect classes are defined as critical or non-critical the first time they are created in a database. This is accomplished using this function. A class is defined at the database level. Once a class is defined as critical it cannot be reset to non-critical status. By controlling the call to this built-in function, you can define as many or as few defect classes as you want.

Input arguments are:

Class_num: The number of the class.

Classification_type: These are predefined types. Only the following values are allowed:

Classification_type	Classification Method
1	manual
2	roughbin
3	finebin
4	mansemclass
5	autosemclass
6	microsigclass
7	macrosigclass
8	adl

vDbResultTime, DbResultTime (string, string) —

Accepts two arguments and returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the end time field in OP-LOG.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
%B Full month name.
%d Day of month as a decimal[00,...,31]
%H 24 hour clock.
%I 12 hour clock.
%M Minute as a decimal[00,...,59]
%m Month as a decimal[01,...,12]
%p a.m. or p.m.
%S Second as a decimal[00,...,59]
%y Year as a decimal[00,...,99]
%Y Year as a 4-digit decimal.
%% Allows for percent in the string.

As an example the format for the following string:

“jul 1 96 05:10:46”

would be:

“%b %d %y %H:%M:%S”

vDbSetupTime (string, string) —

Accepts two arguments of type string and returns a string. Returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file, the date is used as the start time field in OP-LOG.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
%B Full month name.
%d Day of month as a decimal[00,...,31]
%H 24 hour clock.
%I 12 hour clock.
%M Minute as a decimal[00,...,59]
%m Month as a decimal[01,...,12]
%p a.m. or p.m.

%S	Second as a decimal[00,...,59]
%y	Year as a decimal[00,...,99]
%Y	Year as a 4-digit decimal.
%%	Allows for percent in the string.

As an example the format for the following string:

“jul 1 96 05:10:46”

would be:

“%b %d %y %H:%M:%S”

DbTesterType —

Accepts no arguments and returns a string. Calls GetWord, returning the current word. If the current word does not exist as a tester in the EQUIPMENT database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbTesterType (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTesterType but uses the passed argument instead of reading from a file.

DbHandlerType —

Accepts no arguments and returns a string. Calls GetWord, returning the current word. If the current word does not exist as a handler in the EQUIPMENT database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbHandlerType (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbHandlerType but uses the passed argument instead of reading from a file.

DbDfTagAction

(tag_action -int-, dd_thresh -real-, dc_thresh -int-) —

Sets the thresholds for automatic defect tagging.

Input variables:

tag_action: A value of 1 means tagging is disabled. A value of 2 means tagging is enabled.

dd_thresh: A wafer is tagged with “High Defect Density” (flag = 5) if the defect density is higher than the value provided by this field.

dc_thresh: A wafer is tagged with “High Defect Count” (flag = 6) if the defect count is higher than the value provided by this field.

If the user does not want tagging to occur, two options are available:

1. Turn off automatic tagging completely by using **DbDfTagAction (1, DD_Threshold, DC_Threshold)**.
2. Do not call the **DbDfTagAction** function at all, which would set automatic tagging but with an extremely high threshold values (1e+19).



The user should be cautious using this function because wafers tagged by this function will be excluded from lot-level yield calculation by UpStat, unless the -tag option is used, which tells UpStat to included all tagged wafers.



Automatic tagging can be overridden by manual tagging using DbWfTag defectMAP function.



If the user wishes to perform only defect density tagging, dc_thresh should be given as a very high value (e.g. 100,000). Similarly, if the user wishes to perform only defect count tagging, dd_thresh should be set to a high value (e.g. 100,000).

DbUpdateDefect —

Accepts no arguments. If called, the data file being read is considered to have update information of a wafer-layer already in the database. If this function is called, the defect reader is used to add to the database defect classification information and/or defect image information (using the **vDbAddDefectClass** and **vDbAddDefectImage** built-in functions respectively).



*When using this function, The **vDbDefect** and **vDbWmapCfg** functions cannot be used.*



The wafer-layer being updated should already exist in the database.



*The defect index used in **vDbAddDefectClass** and **vDbAddDefectImage** should already exist in the database.*



When the **DbUpdateDefect** function is invoked, the reader adds defect classes to an existing wafer layer.

For certain defect/class methods, the command-line option, **-classoverwrite** (pg. 342) can be used to remove an old class and replace it with a new one.

When used, if a defect has a new defect class for a specific classification method, any old class associated with that defect for the given method is overwritten with the new one.

DbElectricUpdate —

Accepts no arguments. The data file read by this function should contain update information for a wafer, and CV electrical layer, that already exist in the database. If this function is called, the defect reader will add defect classification information and/or defect image information into the database (using the **vDbAddDefectClass** and **vDbAddDefectImage** built-in functions respectively).



The wafer-layer being updated should already exist in the database.



The defect index used in **vDbAddDefectClass** and **vDbAddDefectImage** should already exist in the database.

RemoveLayer —

Used to exclude a certain wafer-layer from defect type calculations and defect summary statistics computations.

Fab **DbFab (string)** —

Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a fab in the FAB database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbFab (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbFab**, but uses the passed argument instead of reading from the file.

<i>Technology</i>	DbTechnology (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the TECHNOLOGY database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbTechnology (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.
<i>Family</i>	DbFamily (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a family in the FAMILY database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbFamily (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.
<i>Process</i>	DbProcess (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the PROCESS database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbProcess (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but uses the passed argument instead of reading from the file.
<i>Product</i>	DbProduct (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the PRODUCT database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbProduct (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file.

Program DbProgram —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program name to the current word. This function should always be called when dumping to database.



Data readers and database retrieval supports program names up to a limit of 255 characters.

vDbProgram (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the database test program name to the passed argument. This function should always be called when dumping to database.



Data readers and database retrieval supports program names up to a limit of 255 characters.

DbProgRel (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. Sets the database test program release to the current word. The passed argument specifies the format of the date string being read.

vDbProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as DbProgRel but uses the first passed argument instead of GetWord as the date string.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.
ddd — Day of the week as a 3-letter abbreviation.
mm — Month as a 2-digit number.
mmm — Month as a 3-letter abbreviation.
yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.
yyyy — Year as a 4-digit number.



With the Oracle database, the ddd option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in date string.

DbProgRev —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.

vDbProgRev (string) — Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.

vDbProgGroup (string) — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

Lot DbLot —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Lot in the database (LOT Table), The Lot is added. If it does exist the function establishes the appropriate relations with other tables. Necessary if Lot summaries are to be calculated.

vDbLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbLot, but uses the passed argument instead of reading from the file

DbSrcLot — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Source Lot in the database (LOT Table), the Source Lot is added. If it does exist, the function establishes the appropriate relations with other tables. Works in conjunction with DbLot, sets the src_lot column in the LOT table and OP_LOG table.

vDbSrcLot (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbSrcLot, but uses the passed argument instead of reading from the file

vDbLotClass (string, string, string, string) — Accepts four arguments of type string: (**Lot_Name**, **Method_Name**, **Method_Type**, **Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

Wafer **vDbWaferClass (string, string, string, string)** — Accepts four arguments of type string and has no returns. The passed arguments are of the following types:

wafer_name - string
method_name - string
method_type - string
class_name - string

Only those wafers that are used in the data file will have their classes and methods logged. Wafers that are only passed to **vDbWaferClass()** but not used elsewhere in the data file will be ignored.

For those wafers logged, the following tables will be populated: CLS_METHOD, WAFER_CLASS, and WFCLS2WF. CLS_METHOD is populated with the **method_name** and **method_type**. WAFER_CLASS is populated with the **class_name**. WFCLS2WF is populated with the corresponding **wafer_names** and **class_names**.

DbWfNum (string, int) — Accepts two arguments of type string and integer and has no return. The first argument should be the wafer ID, while the second argument should be the wafer number associated with that wafer ID. (This function allows populating the **wf_num** column in the **WAFER** table. It should be called for every wafer/wafer number combination in the data file.)

DbWfDesc (string, string) — Accepts two arguments of type string and has no return. The first argument is the **wf_id**; the second argument is the wafer description (maximum 64 characters). This function allows a wafer description to be added to the **WAFER** table (in the **wf_desc** column), but only when a new wafer is encountered.

Step **DbStep (string) —** Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a process step in the **PROC_STEP** database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbStep (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStep**, but uses the passed argument instead of reading from the file.

Input variables: None

Stage **DbStage (string) —** Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a stage in the **TECH_STAGE** database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbStage (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStage**, but uses the passed argument instead of reading from the file.

Equipment **DbEquip3** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey3 in the OP_LOG table.)

vDbEquip3 (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip3**, but uses the passed argument instead of reading from the file.

DbEquip4 —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey4 in the OP_LOG table.)

vDbEquip4 (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip4**, but uses the passed argument instead of reading from the file.

DbEquip5 —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey5 in the OP_LOG table.)

vDbEquip5 (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip5**, but uses the passed argument instead of reading from the file.

DbEquip6 —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey6 in the OP_LOG table.)

	vDbEquip6 (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbEquip6, but uses the passed argument instead of reading from the file.
	DbTester (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a tester in the EQUIPMENT database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbTester (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTester, but uses the passed argument instead of reading from the file.
	DbHandler (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a handler in the EQUIPMENT database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbHandler (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbHandler, but uses the passed argument instead of reading from the file.
<i>Operator</i>	DbOperator (string) —	Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an operator name in the PEOPLE database table, it is added. If it does exist, the function only establishes the appropriate relations with other tables.
	vDbOperator (string) —	Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbOperator, but uses the passed argument instead of reading from the file.
<i>Customer</i>	vDbCustomer(string, ...) —	Accepts 12 arguments of type string, and has no return. Populates the CUSTOMER database table and establishes a relationship between the customer entry and the current lot. This function may be called several times in the same format file, in this way relating one lot to many

customers. The arguments passed to the function match exactly those of the CUSTOMER table. These fields (in their respective order) include: **customer name, address 1, address 2, address 3, postal code, city, state, country, contact, email, fax, phone.**

Wafer Configuration

vDbWmapCfg (...) —

Accepts fifteen arguments and has no return. The passed arguments fill the WMAP_CONFIG database table and are of the following types:

...wmap_name - String, wf_size - Real, wf_units - String, flat - Char, flat_type Char, die_wd - Real, die_ht - Real, offset_center_x - Real, offset_center_y - Real, pos_x - Char and pos_y - Char, fld_rows Integer, fld_cols Integer, row_offset integer, col_offset integer.



The defect reader ignores the wmap_name argument passed by the user and hard codes it to the product name.

Rework

DbReworkAction (int) —

Accepts one argument of type integer and has no return. Sets rework_action in the PROGRAM table. This function overwrites the command line argument **-rework_action**.

Tagging

DbLotTag (int) —

Accepts one argument of type integer and has no return. Used for manual lot tagging. The passed argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)
- High Defect Density (5)
- High Defect Count (6)
- High Defect Density and Count (7)

The above list is dependent on the contents of the **TAGS** table in the database.

DbWfTag (string, int) — Accepts two arguments of type string and integer and has no return. Used for manual wafer tagging. The first argument should be the wafer ID. The second argument should be one of the following values:

No Action (1)
Bad (2)
Scrap (3)
Experiment (4)
High Defect Density(5)
High Defect Count(6)
High Defect Density and Count(7)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (int) — Accepts one argument of type integer and has no return. Used for manual source lot tagging. The passed argument should be one of the following values:

No Action (1)
Bad (2)
Scrap (3)
Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

Miscellaneous **DbSampleType (Char) —** Accepts no arguments and has no return. Calls GetWord and assigns the first character of the current word to **test_mode** in the database OP_LOG table.

vDbSampleType (Char) — Accepts one argument of type Char and has no return. Assigns the passed character to test_mode in the database OP_LOG table.

DbPartNumber (int, string) — Accepts two arguments of type integer and string. This function is used to load a datafile in pieces. It takes two arguments:

- 1.**FileNum**: an integer > 1
- 2.**MapID**: The MapID of the file segment.

Mathematical **ScaleFactor (int) —** Accepts one argument of type integer and has no return. Sets the value of the current scaling factor.

Abs (real) — Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.

Sqr (real) — Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.

POW (real, int) — Accepts two arguments — first of type real, second of type integer; and returns a real. The function returns a real, which is the result of the 1st argument (type real), raised to the power of the 2nd argument (type integer).

Debugging

Print (...) — Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.

PrintToFile (...) — Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.

ErrorCode — Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the error code are **GoTo**, **GoBackTo**, and **OpenFile**.

ExitScript (string) — Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

exitcode (int) — Accepts one argument of type integer and has no return. This function sets the exit code with which the ExitScript function would exit. If not used, the **exitscript** function exits with code = 1.

System System (string) — Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function. Accepts up to 254 characters.

Zones The defect reader automatically define zones for each program. The reader uses dieX and dieY coordinates to create default zones for certain programs, when the following built-in functions are called:

- [DbCreateDefaultzones\(\)](#)
- [DbCircularZones\(string, int\)](#)
- [DbRadiusZones\(int\)](#)

DbCreateDefaultzones() — The **DbCreateDefaultzones** built-in function accepts seven arguments of type string that represent the zones for the system to create. The function has no returns.

- **RW** — Row
- **CL** — Column
- **CR** — Circle
- **RD** — Radial
- **QD** — Quadrants
- **SP** — Step
- **Z9** — 9 Zone

DbCreateDefaultzones works with the following zone types:

By Row: Zonal analysis by row displays bin or parametric results based on die row zones.

By Column: Zonal analysis by column displays bin or parametric results based on die column zones.

Circular (where *n* is the circular area or the circular radius): Circular zonal analysis displays bin or parametric results based on circular zones.

Radials (where *n* is the number of radials): Radial zonal analysis displays bin or parametric results based on radial (pie slice) zones.

Quadrant: Quadrant zonal analysis displays bin or parametric results based on quadrant zones.

Stepper Field: Stepper Field zonal analysis displays bin or parametric results by stepper field zones.

9 Zone: 9 Zone analysis displays three circles of equal radius, with the outer two circles divided into four quadrants.

DbCircularZones(string, int) —

Accepts two arguments of type string and integer. The system does not return any code or message for this built-in function.

The first argument is type string and it represents the type of circular zone: equal area or equal distance (between zone lines). Acceptable arguments are:

- area** — equal area
- distance** — equal distance between zones

The second argument is type integer and represents the number of circles.



If this built-in function is not used, the reader creates a default 10 circle chart with equal distance between the zones.

DbRadiusZones(int) —

Accepts one argument of type integer and has no returns. The argument represents the number of radials.



If this built-in function is not used, the reader creates a default 12 radial chart.

TAGGING WAFERS

Wafers can be tagged manually, from the Defect reader.

This tagging state can then be used as data retrieval criteria.

The predefined tag states are:

- Normal (0)
- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)
- High Defect Density (5)
- High Defect Count (6)
- High Defect Density and Count (7)

To set the tagging flag from the Defect reader, two functions are used:

- **DbDfTagAction (tag_action -int-, dd_thresh -real-, dc_thresh -int-)** — used for automatic wafer tagging
- **DbWfTag (string, int)** — used for manual wafer tagging

RUNNING THE DEFECT READER

The Defect reader can be run in batch mode in the same fashion used to run the ASCII reader (using DpLoad.pl and a configuration file).

To run the reader in command line mode, once your format file is created and saved the Defect Reader can be run using the newly created format. If there are any errors in your format file the reader will generate an error file (err.jnk) describing the nature of the error.

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the Defect reader, which is the format file preceded by the -fmt switch.

Example:

```
defect -fmt [format file]
```

Executing the Defect reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Command-line Arguments

If the Defect reader is being run manually with the database option, knowing the different command-line arguments becomes necessary. The Defect reader accepts the following options: (The first argument should always be the data file to be processed.)

Option	Definition	Default
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).	If not used, this action is not taken.
-classoverwrite	When the DbUpdateDefect (pg. 327) function is invoked, the reader adds defect classes to an existing wafer layer. For certain defect/class methods, -classoverwrite removes an old class and replaces it with a new one. When used, if a defect has a new defect class for a specific classification method, any old class associated with that defect for the given method is overwritten with the new one.	If not used, this action is not taken.
-db	[database]	This option is required with a valid value.
-db_accept	To automatically set accept_data flag in database	NA
-defectsize	Determines the pixel size of a single defect in the gallery. Valid values are: small, medium, large. Default is medium.	Default is medium.
-defext	[Kbytes] Sets DEF... Table extent. If not used, defaults to 32 Kbytes. Acceptable range is 16 Kb to 1024 Kb.	If not used, defaults to 32 Kbytes.

Option	Definition	Default
-file_path	[path to data file]	The current working directory.
-image_path	Path to defect image files. If not set, image files are assumed to be in path determined by the -file_path option. If the -file_path option is also not set, the image file is assumed to be in the current directory. However, the second argument in the format file function vDbAddDefectImage overrides the command line options if it is not set to "NA"	If not set, image files are assumed to be in path determined by the -file_path option. If the -file_path option is also not set, the image file is assumed to be in the current directory.
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created. Optional.	Default dbspace for the (Database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-lotext	[Kbytes] Sets LOT... Table extent. If not used, defaults to 64 Kbytes. Acceptable range is 16 Kb to 2048 Kb.	If not used, defaults to 64 Kbytes.
-lowercase	Forces Lot ID and Wafer ID to Lower Case	If not used, this action is not taken.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion. If used, this option should be followed by the maximum number of seconds to run the executable. If it is exceeded, the executable will terminate with an error.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.
-no_normalize	Do not normalize defect data.	If not used, data is normalized, as usual.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results never age.
-review_time	(Oracle only) Used for file loading review to check, by time, which wafer layer to update. When used, the ResultTimeStamp of the review file can be greater than or equal to the ResultTimeStamp of inspection layer being updated.	If not used, the ResultTimeStamp must be exactly equal to the ResultTimeStamp of the inspection layer being updated.

Option	Definition	Default
-rework_action	(Oracle only) Rework Action sets the Test Program Rework Action (integer). Valid values are 2 (append and mark) and 3 (overwrite).	Default is 3 (overwrite)
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.	If not used, summaries never age.
-u	usage	NA
-uppercase	Forces Lot ID and Wafer ID to Upper Case	If not used, this action is not taken.
-upstat	Runs the UpStat executable with the -defectonly option only for the entries loaded in the file. It is assumed that the upstat executable resides in the same path as the defect reader.	If not used, UpStat is not run.
-v	version	NA
-wafback-ground	Determines the shade of gray for the wafer in the gallery. Valid values are 0-255. Default is 240.	Default is 240.
-wafext	[Kbytes] Sets WAF... Table extent. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 5012 Kb.	If not used, defaults to 256 Kbytes.

Data and Wafer Map Normalization

Any data that includes X-Y coordinates, such as bin map data and defect data may be inspected in different wafer orientations. Different die indexing is also typically used. Thus, in order to align these two types of data, the data readers read and store the data in a “normalized” fashion that guarantees that they are aligned in the database.

Normalization takes the wafer configuration for each data type and shifts, rotates and/or mirrors raw data and the wafer configuration such that in the normalized state:

1. All inspection data is oriented according to a database global orientation specified when the schema was created using DpCreateDb.pl.
2. The die indexes (0,0) include the physical center of the wafer.
3. Positive x-direction is to the right. Positive y-direction is up.

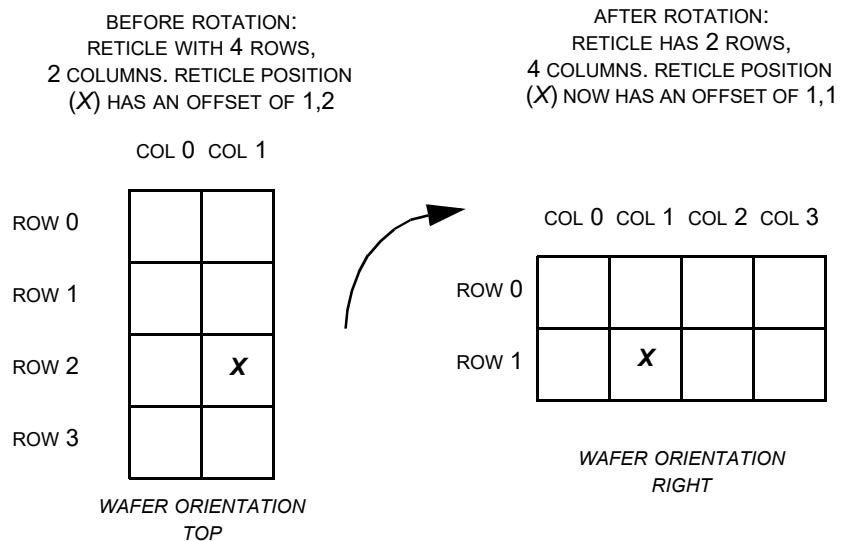


Once the data is normalized in a consistent manner, both back-end and front-end tools can make use of this fact to create summaries (Kill Ratio, Yield Impact, etc.) and map quickly for visualization.

The Defect reader always normalizes data.

Wafer configuration normalization takes into account the row offset (**row_offset**) and column offset (**col_offset**) arguments. Normalization is activated by default in the defect reader.

Reticle row and column offset normalization example:



REGARDLESS OF WAFER ORIENTATION AND NORMALIZATION,
COLUMN/ROW NUMBERING SYSTEM IS CONSTANT, WITH
COLUMN NUMBERS INCREASING TO THE RIGHT AND ROW
NUMBERS INCREASES GOING DOWN

Tencor™ Binary to ASCII Conversion Script

A Perl script, **tencor2ascii**, is provided to transform defect files in the Tencor SFS-7x00 file format into a generic ASCII file. A format file that can read the resulting ASCII file, **tencor.fmt**, is also provided.

To run the script, run the following on the command line:

```
tencor2ascii <data directory path> <file extension>
```

Example:

```
tencor2ascii /export/home/user/tencordata tff
```

This script will create a directory under the <data directory path> called **BinaryFiles**. It will transform all the binary files with the specified file extension to ASCII files with the “.ten” extension. And it will move all the binary files to the **BinaryFiles** subdirectory.

Now the ASCII files can be read using the procedure outlined in [“Running the Defect Reader,”](#) page 342.

defectMAP FORMAT FILE — OVERVIEW

This section includes two examples of format - data file combinations. The first set relates to KLARF data files, while the second set demonstrates the loading of defect images into the database.

Reference: [“How the Defect Reader Determines Center Die,”](#) pg. 49.

EXAMPLE ONE

klarf.fmt

The following is the complete text of the KLARF example format file provided with the DefectMap installation.

//Purpose: Format file for KLARF defect files

SCRIPT defect_klarf
VAR

```
//Conditions
    TestName      string      Cond
    Unit          string      Cond
    TestNum       integer     KeyCond

//Indexes
    Wafer         string      KeyIndex
    Layer         string      Index

//Results
    res           real        Result

    wfid          integer
    str           string
    HowManyRecords integer
    record        integer
    counter       integer
    LotID_Str     string
    WaferID_Str   string
    Setup_Time_Str string
    Results_Time_Str string
    ProductID_Str string
    StepID_Str    string
    wmap_name_str string
    wf_size       real
    flat_type     char
    wf_units      string
    Flat_Type_Str string
    die_wd        real
    die_ht        real
    center_x      real
    center_y      real
    wf_xcenter    real
    wf_ycenter    real
    line_str      string
    def_cnt       integer
```

Exensio Data Readers

test_num	integer
dd	real
insp_die	integer
def_die	integer
major_axis	real
minor_axis	real
el_orient	real
num	integer
Class_Str	string
df_index	integer
xInd	integer
yInd	integer
xPos	real
yPos	real
x_size	real
y_size	real
df_area	real
df_size	real
manualclass	integer
finebin	integer
roughbin	integer
cluster	integer
bin	integer
pos_type	integer
wafer_type	integer
intensity	integer
tmpvar	integer
slot_num	integer
Flat_Location_Str	string
flat_location	char
test_area	real
SrcLotID_Str	string
IsProgNew	boolean
do_imagelist	boolean
do_cluster	boolean
do_roughbin	boolean
do_finebin	boolean
do_review	boolean
LowerLimit	real
UpperLimit	real
LowerLimit_Str	string
UpperLimit_Str	string
fld_rows	integer
fld_cols	integer
row_offset	integer
col_offset	integer
diex[10000]	integer
diey[10000]	integer
dieType[10000]	char
Tester_Str	string
TesterType_Str	string
tech_Str	string
proc_Str	string
Prog_Str	string
size_type	integer
DD_Threshold	real
DC_Threshold	integer
img_list	integer
img_indx	integer
img_file	string
img_path	string
review	integer
shift	integer
dieCnt	integer
j	integer

BEGIN

```
//define seperators
AddSep("")
AddSep(' ')
AddSep(';')
```

```
major_axis = 150
minor_axis = 150
```



FRONT



CONTENTS



INDEX

```
el_orient = 0.0 //always for this version of defectMap
vDbToleranceDimensions(major_axis, minor_axis, el_orient)

GoToBOF
GoTo("SetupID")
ProductID_Str = GetLeftChars(5)
Prog_Str = vDbProgram(StrCat("DF_", ProductID_Str))

////////////////////////////////////
//Is program new?
//If so, read limits of size bins used for summaries and.

If (IsProgramNew = FALSE)

    //////////////////////////////////////
    //get the sample type
    GoToBOF
    GoTo("SampleType")

    DbSampleType

    //////////////////////////////////////
    //get the product name
    ProductID_Str = vDbProduct(ProductID_Str)
    Str = vDbTechnology("MyTech")
    Str = vDbFab("MyFab")
    Str = vDbStage("MyStage")
    Str = vDbProcess("MyProcess")

    //////////////////////////////////////
    //Get test equipment name
    GoToBOF
    GoTo("InspectionStationID")
    If (ErrorCode=0)
        str=GetQuotedWord("")
        TesterType_Str=GetQuotedWord("")
        TesterType_Str=vDbTesterType(TesterType_Str)

        Tester_Str=GetQuotedWord("")
        Tester_Str=vDbTester(Tester_Str)
    Else
        Print("Warning:No tester name provided", 'NL')
    End If

    //////////////////////////////////////
    //Get die pitch
    GoToBOF
    GoTo("DiePitch")
    If (ErrorCode = 0)
        die_wd=GetReal
        die_ht=GetReal
    Else
        ExitScript("ERROR: No die dimensions provided.")
    End If

    //////////////////////////////////////
    //Get wafer diameter
    GoToBOF
    GoTo("SampleSize")
    If (ErrorCode=0)
        SkipWords(1)
        wf_size = GetReal
        wf_size = wf_size*1000
        //multiplication important since measurements are expected in micron
    Else
        ExitScript("ERROR: No wafer diameter provided.")
    End If

    //Get lotID
    GoToBOF
    GoTo("LotID")
    If (ErrorCode=0)
        str = GetWord
```

```

        LotID_Str = vDbLot(StrCat(str, ".1"))
        SkipWords(-1)
        SrcLotID_Str = vDbSrcLot(StrCat(str, ".S"))
    Else
        ExitScript("ERROR: No LotID provided.")
    End If

//Get ResultTimeStamp
GoToBOF
GoTo("ResultTimestamp")
If (ErrorCode=0)
    Results_Time_Str = GetWord //two digit year?
    str = Mid(Results_Time_Str, 7, 2)
    Results_Time_Str = Mid(Results_Time_Str, 1, 6)

    If (StrToInt(str) > 95) // year 95 (1995)
        str = StrCat("19", str)
        str = StrCat(str, "--")
    Else
        str = StrCat("20", str)
        str = StrCat(str, "--")
    End If

    Results_Time_Str = StrCat(Results_Time_Str, str)
    Results_Time_Str = StrCat(Results_Time_Str, GetWord)

    str = vDbResultTime(Results_Time_Str, "%m-%d-%Y--%H:%M:%S")
Else
    ExitScript("ERROR: No result time provided.")
End If

//Get SetupTimeStamp
GoToBOF
GoTo("SetupID")
If (ErrorCode=0)
    Setup_Time_Str = GetQuotedWord("")
    Setup_Time_Str = GetWord
    str = Mid(Setup_Time_Str, 7, 2)
    Setup_Time_Str = Mid(Setup_Time_Str, 1, 6)

    If (StrToInt(str) > 95)
        str = StrCat("19", str)
        str = StrCat(str, "--")
    Else
        str = StrCat("20", str)
        str = StrCat(str, "--")
    End if

    Setup_Time_Str = StrCat(Setup_Time_Str, str)
    Setup_Time_Str = StrCat(Setup_Time_Str, GetWord)
    str = vDbSetupTime(Setup_Time_Str, "%m-%d-%Y--%H:%M:%S")
Else
    Print("Warning: No Setup time provided", 'NL')
End If

//Get StepID
GoToBOF
GoTo("StepID")
If (ErrorCode=0)
    StepID_Str = vDbStep(GetQuotedWord(""))
Else
    ExitScript("ERROR: No step ID provided.")
End If

//////////
//Get Orientations
GoToBOF
GoTo("SampleOrientationMarkType")
If (ErrorCode = 0)
    Flat_Type_Str = GetWord
    If (Flat_Type_Str = "NOTCH")
        flat_type='N'
    Else If (Flat_Type_Str = "FLAT")

```

```

                                flat_type='F'
                                Else ExitScript("ERROR: INVALID SampleOrientationMarkType provided.")
                                End If
                                Else
                                Print("Warning: SampleOrientationMarkType not provided.", 'NL')
                                End If

                                GoToBOF
                                GoTo("OrientationMarkLocation")
                                If (ErrorCode = 0)
                                Flat_Location_Str=GetWord
                                If (Flat_Location_Str = "DOWN")
                                flat_location='B'
                                Else If (Flat_Location_Str = "UP")
                                flat_location='T'
                                Else If (Flat_Location_Str = "LEFT")
                                flat_location='L'
                                Else If (Flat_Location_Str = "RIGHT")
                                flat_location='R'
                                Else
                                ExitScript("ERROR: INVALID OrientationMarkLocation provided.")
                                End If
                                End If

                                //////////////////////////////////
                                //When wafer is not patterned, the die_wd=wafer diameter
                                If(wf_size = die_wd)
                                pos_type=1
                                wafer_type=0
                                Else
                                pos_type=0
                                wafer_type=1
                                End If
                                vPosType(pos_type)
                                vDbIsWaferPatterned(wafer_type)

                                //////////////////////////////////
                                //Get Class LUT
                                GoToBOF
                                GoTo("ClassLookup")
                                If (ErrorCode=0)
                                SkipWords(-1)
                                SkipLines(1)

                                While (IsNumber(GetWord))
                                SkipWords(-1)
                                num=GetInt
                                Class_Str=GetQuotedWord("")
                                vAddToClassLUT(num, Class_Str)
                                End While
                                Else
                                Print("Warning:No class LUT provided", 'NL')
                                End If

                                //////////////////////////////////
                                //Get Die Map
                                dieCnt = 0
                                GoToBOF
                                GoTo("RemovedDieList")
                                If (ErrorCode=0)
                                SkipWords(-1)
                                SkipLines(1)

                                While (IsNumber(GetWord))
                                dieCnt = dieCnt + 1
                                SkipWords(-1)
                                diex[dieCnt]=GetInt
                                diey[dieCnt]=GetInt
                                dieType[dieCnt]= 'R'
                                //vDbAddToDieMapTable(diex, diey, 'R')
                                End While
                                Else
                                Print("Warning: No RemovedDieList provided", 'NL')
                                End If

```

```

GoToBOF
GoTo("SampleTestPlan")
If (ErrorCode=0)
    SkipWords(-1)
    SkipLines(1)

    While (IsNumber(GetWord))
        dieCnt = dieCnt + 1
        SkipWords(-1)
        diex[dieCnt]=GetInt
        diey[dieCnt]=GetInt
        dieType[dieCnt]= 'V'
        //vDbAddToDieMapTable(diex, diey, 'V')
    End While
Else
    Print("Warning: No SampleTestPlan provided", 'NL')
End If

////////////////////////////////////
// Get the TIFF file name if present. We assume 1 TIFF file/ KLARF file
//GoToBOF
//img_file = "NA"
//GoTo("TiffFileName")
//IF (ErrorCode=0)
//    img_file=GetWord
//Else
//    Print("WARNING:No TIFF file", 'NL')
//End If

////////////////////////////////////
//Count how many records (Layers) are in the ASCII (KLA) file
GoToBOF
HowManyRecords=0
GoTo("WaferID")
If (ErrorCode=0)
    While(ErrorCode=0)
        HowManyRecords = HowManyRecords + 1
        GoTo("WaferID")
    End While
    Print('NL',"Number of records ", HowManyRecords, 'NL')
Else
    ExitScript("ERROR: Could not count how many records in file.")
End If

////////////////////////////////////
// Get wafer-layer info
GoToBOF
For record=1 To HowManyRecords

//Get WaferID
GoTo("SummaryList")
GoBackTo("WaferID")
If (ErrorCode=0)
    WaferID_Str = GetWord
    str = WaferID_Str

    If(IsNumber(WaferID_Str))
        wfid = StrToInt(WaferID_Str)
        str = IntToStr(wfid)
    Else
        wfid = -1
    End If

    WaferID_Str=StrCat(LotID_Str," ")
    WaferID_Str=StrCat(WaferID_Str, str)
    //Print("-->",WaferID_Str,'NL')
    DbWfNum(WaferID_Str, wfid)
Else
    ExitScript("ERROR: No wafer ID provided.")
End If

//Get slot number
GoTo("SummaryList")

```



```
GoBackTo("Slot")
If (ErrorCode=0)
    vDbSlotNum(GetInt)
Else
    Print("Warning: No slot number provided", 'NL')
End If

//Get test_area
GoTo("SummaryList")
GoBackTo("AreaPerTest")
If (ErrorCode=0)
    test_area=GetReal
Else
    Print("Warning: No test area provided", 'NL')
End If

//Get Wafer center in coordinate system
GoTo("SummaryList")
GoBackTo("SampleCenterLocation")
If (ErrorCode=0)
    wf_xcenter = GetReal
    wf_ycenter = GetReal
Else
    ExitScript("ERROR: No wafer center location provided.")
End If

// Wafer configuration
wmap_name_str = ProductID_Str//always force this
wf_units = "microns"

GoTo("SummaryList")
GoBackTo("DieOrigin")
If (ErrorCode=0)
    center_x = GetReal
    center_y = GetReal
    center_x = center_x - wf_xcenter
    center_y = center_y - wf_ycenter
Else
    ExitScript("ERROR: No die origin provided.")
End If

//reticle info should be obtained from other sources
fld_rows = 1
fld_cols = 1
row_offset = 0
col_offset = 0

vDbWmapCfg(wmap_name_str, wf_size, wf_units,
    flat_location, flat_type, die_wd, die_ht, center_x,
    center_y, 'R','U', fld_rows, fld_cols, row_offset, col_offset)

For j=1 TO dieCnt
    vDbAddToDieMapTable(diex[j], diey[j], dieType[j])
End For

//Get Defect Info
Goto("SummaryList")

GoBackTo("WaferID")

GoTo("DefectRecordSpec")
If (ErrorCode=0)
    //we will assume the following fields are always there:
    //DEFECTID XREL YREL XINDEX YINDEX XSIZE YSIZE DEFECTAREA
    //DSIZE CLASSNUMBER TEST

    //Lets check for the rest,
    // but we will assume they will come in the order shown

    do_cluster = FALSE
    do_roughbin = FALSE
    do_finebin = FALSE
    do_review = FALSE
    //do_imagelist = FALSE
```

```

GoTo("TEST")
str=GetWord
while (str <> "DefectList")

    If (str = "CLUSTERNUMBER")
        do_cluster = TRUE
    else if (str = "ROUGHBINNUMBER")
        do_roughbin = TRUE
    else if (str = "FINEBINNUMBER")
        do_finebin = TRUE
    else if (str = "REVIEWSAMPLE")
        do_review = TRUE
    //else if (str = "IMAGELIST")
        //do_imagelist = TRUE
    end if

    str = GetWord

end while

SkipWords(-1)
SkipLines(1)

//this is a klarf not tencor
intensity=-1 //since we are reading a klarf file

str = GetWord
While (IsNumber(str)) OR (str = "TiffFileName")
    if (str = "TiffFileName")
        //SkipWords(-1)
        //SkipLines(1)
        //str=GetWord
        //while(IsString(str))
            // SkipWords(-1)
            // SkipLines(1)
            // str=GetWord
        //end while
    end if

    SkipWords(-1)
    df_index = GetInt
    xPos = GetReal
    yPos = GetReal
    xInd = GetInt
    yInd = GetInt
    x_size = GetReal
    y_size = GetReal
    df_area = GetReal
    df_size = GetReal
    manualclass = GetInt
    test_num = GetInt

    if (do_cluster = TRUE)
        cluster= GetInt
    end if

    if (do_roughbin = TRUE)
        roughbin=GetInt
    end if

    if (do_finebin = TRUE)
        finebin=GetInt
    end if

    if (do_review = TRUE)
        review = GetInt
    end if

    //if (do_imagelist = TRUE)
    //img_list=GetInt

    //if (img_list <> 0)
    //img_indx=GetInt
    //end if

```



```

//end if
SkipWords(-1)//to make sure we are not at end of line
SkipLines(1)

vDbDefect(df_index, xInd, yInd, xPos, yPos,
x_size, y_size, df_area, df_size, cluster, intensity,
test_num)

if(manualclass > 0) //other than "unclassified"
    vDbAddDefectClass(df_index, manualclass, 1)
end if

if(roughbin > 0) AND (do_roughbin = TRUE)//other than "unclassified"
    vDbAddDefectClass(df_index, roughbin, 2)
end if

if(finebin > 0) AND (do_finebin = TRUE)
//other than "unclassified", vDbAddDefectClass(df_index, finebin, 3)
    vDbSetCriticalDfClass(finebin, 3)
end if

str=GetWord
End While //ends defect loop

////////////////////////////////////
//get file summaries
GoTo("SummaryList")//we can have multiple summaries for the record
If (ErrorCode=0)
    SkipWords(-1)
    SkipLines(1)
    While (IsNumber(GetWord) AND NotEndOfFile)
        SkipWords(-1)
        test_num=GetInt
        def_cnt=GetInt
        dd=GetReal
        insp_die=GetInt
        def_die=GetInt

        //manual tagging
        // If(dd > 100)
        //   DbWfTag(WaferID_Str, 5)
        // Else if (def_cnt > 1)
        //   DbWfTag(WaferID_Str, 6)
        // End If
        vDbDefectSummary(test_num, def_cnt, test_area, dd,
            insp_die, def_die)
        SkipWords(-1)//to make sure we are not at end of line
        SkipLines(1)
    End While //ends summary loop
Else
    Print("Warning:Could not find summaries in KLARF file", 'NL')
End If
dBDumpLayer() //stores layer info in DB

Else
    ExitScript("ERROR: Could not find defectlist")
End If

End For //ends record loop

Else //the program is new
    Print("The program is New!", 'NL')
    TestNum = 0

    if (1 = 1)

        OpenFile("size_limits.txt")
        GoToBOF
        GoTo("SizeType")
        If (ErrorCode=0)
            size_type=GetInt
        Else
            ExitScript("ERROR: Could not find SizeType")
        End If
    End If
End If

```

```

                                End If

                                GoToBOF
                                GoTo("SizeUnit")
                                IF (ErrorCode=0)
                                    Unit = GetWord
                                Else
                                    ExitScript("ERROR: Could not find Unit")
                                End If

                                GoToBOF
                                counter=0
                                str=GetWord
                                While (NotEndOfFile)
                                    SkipWords(-1)
                                    TestName=StrCat("SizeBin",IntToStr(counter))
                                    LowerLimit_Str=StrCat("LowerSizeBin",IntToStr(counter))
                                    UpperLimit_Str=StrCat("UpperSizeBin",IntToStr(counter))
                                    Goto(LowerLimit_Str)
                                    IF (ErrorCode=0)
                                        LowerLimit=GetReal
                                    Else
                                        ExitScript("ERROR: Could not find LowerLimit_Str")
                                    End If

                                    Goto(UpperLimit_Str)
                                    IF (ErrorCode=0)
                                        UpperLimit=GetReal
                                    Else
                                        ExitScript("ERROR: Could not find UpperLimit_Str")
                                    End If

                                    counter = counter+1
                                    TestNum = TestNum+1
                                    vPrepareSizeSummary(LowerLimit, UpperLimit, size_type)
                                    str=GetWord
                                End While

                                End If

                                DD_Threshold = 2.3
                                DC_Threshold = 1000
                                DbDfTagAction(2, DD_Threshold, DC_Threshold)
                                DbDumpLayer()//store required summaries in DB
                                End If
End

```

demo.kla **demo.kla** is the example KLARF data file that can be read using the provided example format file.

```

FileVersion 1 1;
FileTimestamp 03-21-99 23:48:18;
InspectionStationID "" "TTYPE" "TSTR";
SampleType WAFER;
ResultTimestamp 04-21-99 23:30:20;
LotID "LOT1";
SampleSize 1 200;
SetupID "PROG1LYR1" 04-21-99 23:28:37;
StepID "LYR1";
SampleOrientationMarkType NOTCH;
OrientationMarkLocation DOWN;
DiePitch 11410.1220 11409.7725;
DieOrigin 0 0;
WaferID "21";
Slot 11;
SampleCenterLocation 11453.3863 8608.0686;
ClassLookup 0;
DefectClusterSpec 3 THRESHOLD MINSIZE MERGETOL;
DefectClusterSetup 1000 10 3;
RemovedDieList 8
    6 6
    7 5
    8 3
    4 -7
    -3 -7
    -7 3
    -6 5
    -5 6;
InspectionTest 1;
SampleTestPlan 198
    8 -2
    8 -1
    8 0
    8 1
    8 2
    7 -4
    7 -3
    7 -2
    7 -1
    7 0
    7 1
    7 2
    7 3
    7 4
    6 -5
    6 -4
    6 -3
    6 -2
    6 -1
    6 0
    6 1
    6 2
    6 3
    6 4
    6 5
    5 -6
    5 -5
    5 -4
    5 -3
    5 -2
    5 -1
    5 0
    5 1
    5 2
    5 3
    5 4
    5 5
    5 6
  
```

4-6
4-5
4-4
4-3
4-2
4-1
40
41
42
43
44
45
46
47
3-7
3-6
3-5
3-4
3-3
3-2
3-1
30
31
32
33
34
35
36
37
2-7
2-6
2-5
2-4
2-3
2-2
2-1
20
21
22
23
24
25
26
27
28
1-7
1-6
1-5
1-4
1-3
1-2
1-1
10
11
12
13
14
15
16
17
18
0-7
0-6
0-5
0-4
0-3
0-2
0-1
00
01
02
03
04
05



0 6
0 7
0 8
-1 -7
-1 -6
-1 -5
-1 -4
-1 -3
-1 -2
-1 -1
-1 0
-1 1
-1 2
-1 3
-1 4
-1 5
-1 6
-1 7
-1 8
-2 -7
-2 -6
-2 -5
-2 -4
-2 -3
-2 -2
-2 -1
-2 0
-2 1
-2 2
-2 3
-2 4
-2 5
-2 6
-2 7
-3 -6
-3 -5
-3 -4
-3 -3
-3 -2
-3 -1
-3 0
-3 1
-3 2
-3 3
-3 4
-3 5
-3 6
-3 7
-4 -6
-4 -5
-4 -4
-4 -3
-4 -2
-4 -1
-4 0
-4 1
-4 2
-4 3
-4 4
-4 5
-4 6
-5 -5
-5 -4
-5 -3
-5 -2
-5 -1
-5 0
-5 1
-5 2
-5 3
-5 4
-5 5
-6 -4
-6 -3

-6 -2
 -6 -1
 -6 0
 -6 1
 -6 2
 -6 3
 -6 4
 -7 -2
 -7 -1
 -7 0
 -7 1
 -7 2;

AreaPerTest 2.06294e+010;

DefectRecordSpec 17 DEFECTID XREL YREL XINDEX YINDEX XSIZE YSIZE
 DEFECTAREA DSIZE CLASSNUMBER TEST CLUSTERNUMBER ROUGHBINNUMBER
 FINEBINNUMBER REVIEWSAMPLE IMAGECOUNT IMAGELIST;

DefectList

1 5425.4215 4277.5917 1 8 0.6242 0.6242 0.9204 0.6242 0 1 0 0 333 1 0 0
 2 1295.3345 3357.9511 0 8 0.6242 0.6242 0.5861 0.6242 0 1 0 0 0 0 0 0
 3 3775.9884 2521.9511 0 8 1.8725 1.8725 3.9884 1.8725 0 1 0 0 333 1 0 0
 4 4159.8541 1784.1777 0 8 0.6242 0.6242 0.6342 0.6242 0 1 0 0 0 0 0 0
 5 407.3306 255.7402 0 8 0.6242 0.6242 0.9204 0.6242 0 1 0 0 0 0 0 0
 6 1348.9528 57.8730 1 8 0.6242 0.6242 0.6136 0.6242 0 1 0 0 333 1 0 0
 7 5769.5932 11077.2548 -2 7 0.6242 0.6242 0.5861 0.6242 0 1 0 0 0 0 0 0
 8 4482.5483 10788.2627 -2 7 0.6242 0.6242 0.6136 0.6242 0 1 0 0 0 0 0 0
 9 1192.2837 11169.0048 -1 7 0.6242 0.6242 0.6342 0.6242 0 1 1 0 0 0 0 0
 10 10712.7827 11029.1923 -1 7 1.8725 1.8725 3.6816 1.8725 0 1 0 0 333 1 0 0

SummarySpec

5 TESTNO NDEFECT DEF DENSITY NDIE NDEFDIE;

SummaryList

1 10 0.048474 198 4;

EndOfFile;

size_limits.txt

This file contains the size bin information that the UpStat process will use to prepare summaries for.

SizeType	2
SizeUnit	"micron"
LowerSizeBin0	0.0
UpperSizeBin0	0.01
LowerSizeBin1	0.01
UpperSizeBin1	0.1
LowerSizeBin2	0.1
UpperSizeBin2	1.00
LowerSizeBin3	1.00
UpperSizeBin3	2.00
LowerSizeBin4	2.00
UpperSizeBin4	3.00
LowerSizeBin5	3.00
UpperSizeBin5	5.00

EXAMPLE TWO

MINI-KLARF.fmt //This is a format file for MINI-KLARF defect files

```

SCRIPT mini_klarf
VAR
    //Conditions
    TestName      string      Cond
    Unit          string      Cond
    TestNum       integer     KeyCond

    //Indexes
    Wafer         string      KeyIndex
    Layer         string      Index

    //Results
    res           real        Result

    str
    LotID_Str     string
    Results_Time_Str string
    StepID_Str    string
    df_index      integer
    finebin       integer
    slot_num      integer
    img_list      integer
    img_indx      integer
    img_file      string
    img_path      string
    SrcLotID_Str  string
    img_count     integer

BEGIN
    //define seperators
    AddSep("")
    AddSep(' ')
    AddSep(';')

    DbUpdateDefect

    //////////////////////////////////////
    //get the sample type
    GoToBOF
    GoTo("SampleType")
    DbSampleType

    //Get lotID
    GoToBOF
    GoTo("LotID")
    IF (ErrorCode=0)
        str = GetWord
        LotID_Str = vDbLot(StrCat(str, ".1"))
        SkipWords(-1)
        SrcLotID_Str = vDbSrcLot(StrCat(str, ".S"))
    Else
        ExitScript("ERROR: No LotID provided.")
    End If

    //////////////////////////////////////
    //Get ResultTimeStamp
    GoToBOF
    GoTo("ResultTimestamp")
    IF (ErrorCode=0)
        Results_Time_Str = GetWord //two digit year?
        str = Mid(Results_Time_Str, 7, 2)
        Results_Time_Str = Mid(Results_Time_Str, 1, 6)
        if (StrToInt(str) > 95) // year 95 (1995)
            str = StrCat("19", str)

```

```

                                str = StrCat(str, "--")
else
                                str = StrCat("20", str)
                                str = StrCat(str, "--")
end if

Results_Time_Str = StrCat(Results_Time_Str, str)
Results_Time_Str = StrCat(Results_Time_Str, GetWord)
Print("Results_Time_Str=", Results_Time_Str, "<--", 'NL')
str = vDbResultTime(Results_Time_Str, "%m-%d-%Y--%H:%M:%S")
Else
    ExitScript("ERROR: No result time provided.")
End If

////////////////////////////////////
//Get StepID
GoToBOF
GoTo("StepID")
IF (ErrorCode=0)
    StepID_Str=vDbStep(GetWord)
Else
    ExitScript("ERROR: No step ID provided.")
End If

////////////////////////////////////
// Get the TIFF file name if present.
GoToBOF
img_file = "NA"
GoTo("TiffFilename")
IF (ErrorCode=0)
    img_file=GetWord
Else
    Print("WARNING:No TIFF file", 'NL')
End If

////////////////////////////////////
// Get defect info

GoToBOF
//Get slot number
GoTo("Slot")
IF (ErrorCode=0)
    vDbSlotNum(GetInt)
Else
    Print("Warning: No slot number provided", 'NL')
End If

GoTo("DefectList")
IF (ErrorCode=0)
    // we will assume the following fields are always there:
    // DEFECTID CLASSNUMBER IMAGECOUNT IMAGELIST
    SkipWords(-1)
    SkipLines(1)

    str = GetWord
    While (IsNumber(str))
        SkipWords(-1)

        df_index = GetInt
        finebin = GetInt
        img_count = GetInt
        if (img_count > 0)
            img_list = GetInt
            if (img_list > 0)
                img_indx = GetInt
            end if
        end if

        SkipWords(-1)//to make sure we are not at end of line
        SkipLines(1)

        if (img_list <> 0) AND (img_file <> "NA")

```

```

img_path="/data/images/"
vDbAddDefectImage(df_index, img_path,
img_file, img_idx, "TIFF",5)
end if
if (finebin <> 0)
vAddToLUT(finebin, 3)
vDbAddDefectClass(df_index, finebin, 3)
end If
str = GetWord
End While //ends defect loop
dBDumpLayer() //stores layer info in DB
Else
ExitScript("ERROR: Could not find defectlist")
End If
End

```

MINI_KLARF.txt

```

FileVersion 1 1;
FileTimestamp 03-20-99 20:15:38;
TiffFilenameMyImage.tif;
StepID LR24;
InspectionStationID "" "TYP1" "EQ44";
SampleType WAFER;
ResultTimestamp 03-20-00 19:36:18;
LotID "LOT11009";
Slot 13;
DefectRecordSpec 4 DEFECTID CLASSNUMBER IMAGECOUNT IMAGELIST;
DefectList
19 0 1 1 1 0
21 0 1 1 2 0
21 0 1 1 3 0
33 0 1 1 4 0
33 0 1 1 5 0
33 0 1 1 6 0
37 0 1 1 7 0
37 0 1 1 8 0
37 0 1 1 9 0
37 0 1 1 10 0
37 0 1 1 11 0;
EndOfFile;

```

USING DEFECT IMAGE POINTERS

If you wish to store image pointers in the database instead of storing the actual images themselves, the following requirements and steps are the recommended solution. You will also have to use the **NoImageLoad** built-in function in the defect format files. (*"NoImageLoad,"* pg. 321)

RECOMMENDED INSTALLATION AND SETUP FOR INFORMIX

I - Environment Variables and Database Engine Configuration

1. The **IDSHOME** environment variable should be set in both the "Admin" and "Informix" accounts.
2. The **IDSHOME** variable has two directories: **bin** and **lib**. The **bin** directory holds the **ImageLoad** executable and the **CallLoad** script. The **lib** directory holds the **defect_lib.so** library.
3. It is recommended that the Informix account also define the **IDSTMP** variable. This variable should point to a directory in which temporary files should be created. If the variable is not defined, the temporary files will be created in a **/tmp** directory on the machine.
4. To define these variables, (**IDSHOME** and **IDSTMP**) the Informix **.cshrc** file should be edited and sourced. Then the engine should be taken offline and then put back online in order for the engine to recognize these new variables.
5. Both the "Admin" and "Informix" accounts should have read and execute privileges on these directories and the files inside them.
6. The Informix account should have read privilege on the directories holding the image files.
7. The Informix **ONCONFIG** file should be edited (and the engine restarted) to add a Virtual Processor for the external function to run on.

To do that:

- a. Comment out the **SINGLE_CPU_VP** variable.
- b. Add the following line:

```
VPCLASS newvp, num=3
```

II - DpCreateDb

1. A option called **-dfpoint** adds the **FillImage** SPL function, but only if run with the **-defect** option for new databases or with the **-add** option for old (existing) databases.
2. DpCreateDb detects the **IDSHOME** environment variable and returns an error if it is not found.
3. It grants execute on the function to public (optional).

III - LoadImage Executable

1. Placed in **IDSHOME/bin** directory.



2. The Informix account runs the executable when the **FillImage** function is called.
3. **LoadImage** looks for the **IDSTMP** environment variable in the temp directory. If it is not found, it uses the **/tmp** directory to create temporary files.
4. This executable also clears images loaded more than two days prior, after it finishes loading images for the current lg_key.

IV - defect_lib.so

1. Placed in **IDSHOME/lib** directory.
2. This file should be owned by Informix and have the file privileges “**744**”. No other user should have the right to write to this file.

V - CallLoad

1. Placed in **IDSHOME/bin** directory.
2. This is a shell script that sets the correct environment for ImageLoad.

RECOMMENDED INSTALLATION AND SETUP FOR ORACLE

I - Environment Variables

1. The **IDSHOME** environment variable should be set in both the “Admin” and “Oracle” accounts.
2. The **IDSHOME** variable has two directories: **bin** and **lib**. The **bin** directory holds the **ImageLoad** executable and the **CallLoad** script. The **lib** directory holds the **defect_lib.so** library.
3. Both the Admin and Oracle accounts should have read and execute privileges on these directories and the files inside them.
4. It is recommended that the Oracle account also define the **IDSTMP** variable. This variable should point to a directory in which temporary files should be created. If the variable is not defined, the temporary files will be created in a **/tmp** directory on the machine.
5. Another environment variable, **DPDFUSER**, can be defined in the format: “**username/password@servicename**”. If not defined, the **ImageLoad** executable assumes a hard coded value:
dp_imgload/dp_imgload/@servicename.
The service name is extracted from the database name sent from the front-end in the format: **dbname@servicename** (second argument to the **FillImage** function).

6. The **listener.ora** file has to be modified by defining the **\$IDSHOME** variable (and any other needed variables) using the **ENVS** keyword. Below is an example:

```
SID_LIST_LISTENER =

(SID_LIST =

  (SID_DESC =

    (SID_NAME = PLSExtProc)

    (ORACLE_HOME = /db1/app/oracle/product/10.2)

    *      (ENVS = IDSHOME= should put the needed directory path here) *

    (PROGRAM = extproc)

  )

  (SID_DESC =

    (GLOBAL_DBNAME = odpserver)

    (ORACLE_HOME = /db1/app/oracle/product/10.2)

    (SID_NAME = odpserver)

  )

)
```

7. To define these variables, the Oracle account **.cshrc** file should be edited and sourced. The Listener should be restarted to recognize these new variables. This is done with the following two steps:
 - a. **lsnrctl stop**
 - b. **lsnrctl start**
8. Both "Admin" and "Oracle" accounts should have read and execute privileges on these directories and the files inside them.

II - DpCreateDb

1. The option **-dfpoint** adds the **FillImage PL/SQL** function, but only if run with the **-defect** option for new databases or with the **-add** option for old (existing) databases. This option has to be run on the server side.
2. DpCreateDb detects the **IDSHOME** environment variable and returns an error if it is not found.

III - LoadImage Executable

1. Placed in **IDSHOME/bin** directory.
2. The Oracle account runs the executable when the **FillImage** function is called.
3. **LoadImage** would look for the environment variable **IDSTMP** for the temp directory, and if not found would use the **/tmp** directory for creating temporary files.

4. Once the executable is called, it connects from the Oracle Unix account as the **DPDFUSER** environment variable (if defined in the format "**username/password@servicename**"). If this variable not defined, the **ImageLoad** executable assumes a hard coded value **dp_imgload/dp_imgload/@servicename**. The service name is extracted from the database name sent from the front-end in the form.
5. This executable also clears images loaded more than two days prior, after it finishes loading images for the current lg_key.

IV - defect_lib.so

Must be placed in **IDSHOME/lib**

V - CallLoad

1. Placed in IDSHOME/bin directory.
2. This is a shell script that sets the correct environment for ImageLoad.

EXAMPLE FORMAT FILE - DEFECT IMAGE POINTERS

```
// Filename: klarf.fmt

//Purpose: Format file for KLARF defect files with images

SCRIPT defect_demo
CONST
    D = FALSE
    E = FALSE
VAR
    //Conditions
    TestName string Cond
    Unit      string Cond
    TestNum   string KeyCond

    //Indexes
    Wafer      string KeyIndex
    layer      string Index

    //Results
    res        real      Result
    wfid       integer
    str        string
    str1       string
        HowManyRecords   integer
        SampleNum         integer
        record            integer
        counter           integer
        SourceLot         string
    LotID_Str    string
        Plot             string
        Plot2            string
    Plot_len integer
    Plot_1st_letter string
        DeviceID_Str     string
        WaferID_Str       string
        Setup_Time_Str    string
        Results_Time_Str  string
        ProductID_Str     string
        StepID_Str        string
        wmap_name_str     string
        wf_size           real
        flat_type         char
        wf_units          string
        Flat_Type_Str     string
        die_wd            real
        die_ht            real
        center_x          real
        center_y          real
        wf_xcenter        real
        wf_ycenter        real
        line_str          string
        def_cnt           integer
```


test_num	integer
dd	real
insp_die	integer
def_die	integer
el_orient	real
num	integer
Class_Str	string
df_index	integer
xInd	integer
yInd	integer
xPos	real
yPos	real
x_size	real
y_size	real
df_area	real
df_size	real
manualclass	integer
finebin	integer
roughbin	integer
samplebin	integer
reviewsample	integer
cluster	integer
bin	integer
pos_type	integer
wafer_type	integer
intensity	integer
tmpvar	integer
slot_num	integer
Flat_Location_Str	string
flat_location	char
test_area	real
IsProgNew	boolean
LowerLimit	real
UpperLimit	real
LowerLimit_Str	string
UpperLimit_Str	string
fld_rows	integer
fld_cols	integer
row_offset	integer
col_offset	integer
diex[10000]	integer
diey[10000]	integer
dieType[10000]	char
Tester_Str	string
TesterType_Str	string
tech_Str	string
Prog_Str	string
size_type	integer
DD_Threshold	real
DC_Threshold	integer
img_list	integer
img_indx	integer
img_file	string
img_path	string
img_type	string

```

review                integer
shIft                integer
dieCnt                integer
j                    integer
    sizeFile          string
    i                  integer
    DRSArray[20]       string
    DRSLen             integer
    image_count        integer
    image_index[100]   string
MyDir                 string
    Family             string
    totalRecords       integer
    prodArray[10,2]    string
    NumberOfProd       integer
    DataFileName       string
    image_load         integer
    image_id           integer
    spec_flag          integer
BEGIN

    NoImageLoad // To use image pointers

    AddSep('')
    AddSep(' ')
    AddSep(';')

    GoToBOF
    GoTo("LotID")
    If (ErrorCode=0)
        LotID_Str = GetWord
    Else
        ExitScript("ERROR: No LotID provided.")
    End If
    Plot = before(LotID_Str, '.')

    if strlen(Plot) < 1
        Plot = before(LotID_Str, '-')
        if strlen(Plot) < 1
            Plot = before(LotID_Str, '_')
            if strlen(Plot) < 1
                Plot = LotID_Str
            end if
        end if
    end if

    ProductID_Str = "MY_PROD"

    If ProductID_Str <> ""
        prog_Str = vDbProgram(StrCat("DF_", ProductID_Str))
        LotID_Str = vDbLot(LotID_Str)
        ProductID_Str = vDbProduct(ProductID_Str)
    Else

```

```
        ExitScript(Str)
    End If

    If (IsProgramNew = FALSE)

        GoToBOF
        GoTo("SampleType")
        DbSampleType

        GoToBOF
        GoTo("InspectionStationID")
        If (ErrorCode=0)
            str=GetQuotedWord('')
            TesterType_Str=GetQuotedWord('')
            TesterType_Str=vDbTesterType(TesterType_Str)

            Tester_Str=GetQuotedWord('')
            Tester_Str=vDbTester(Tester_Str)
        Else
            Print("Warning:No tester name provided", 'NL')
        End If

        GoToBOF
        GoTo("DiePitch")
        If (ErrorCode = 0)
            die_wd=GetReal
            die_ht=GetReal
        Else
            ExitScript("ERROR: No die dimensions provided.")
        End If

        GoToBOF
        GoTo("SampleSize")
        If (ErrorCode=0)
            SkipWords(1)
            wf_size = GetReal
            wf_size = wf_size*1000
        Else
            ExitScript("ERROR: No wafer diameter provided.")
        End If

        GoToBOF
        GoTo("ResultTimestamp")
        If (ErrorCode=0)
            DelSep(' ')
            SkipChars(1)
            Results_Time_Str = GetWord
            AddSep(' ')
            Results_Time_Str = StrTrim(Results_Time_Str, ' ')

            Results_Time_Str = vDbResultTime(Results_Time_Str,
            "%Y-%m-%d %H:%M:%S")
        End If
    End If
```

```

Else
    ExitScript("ERROR: No result time provided.")
End If

GoToBOF
GoTo("SetupID")
If (ErrorCode=0)
    DelSep(' ')
    SkipChars(1)
    StepID_Str = GetQuotedWord('')
    SkipChars(1)
    Setup_Time_Str = GetWord
    AddSep(' ')
    Setup_Time_Str = StrTrim(Setup_Time_Str, ' ')

    Setup_Time_Str = vDbSetupTime(Setup_Time_Str, "%Y-
    %m-%d %H:%M:%S")
Else
    Print("Warning: No Setup time provided", 'NL')
End If

GoToBOF
GoTo("StepID")
If (ErrorCode=0)
    str = GetQuotedWord('')
    If (StrLen(Str) < 2)
        StepID_Str = vDbStep(StepID_Str)
    Else
        StepID_Str = vDbStep(Str)
    End If
    Print("Step = ", StepID_Str, 'NL')
Else
    ExitScript("ERROR: No step ID provided.")
End If

GoToBOF
GoTo("SampleOrientationMarkType")
If (ErrorCode = 0)
    Flat_Type_Str = GetWord
    If (Flat_Type_Str = "NOTCH")
        flat_type='N'
    Else If (Flat_Type_Str = "FLAT")
        flat_type='F'
    Else ExitScript("ERROR: INVALID
    SampleOrientationMarkType provided.")
    End If
Else
    Print("Warning: SampleOrientationMarkType not
    provided.", 'NL')
End If

```

```
GoToBOF
GoTo("OrientationMarkLocation")
If (ErrorCode = 0)
    Flat_Location_Str=GetWord
    If (Flat_Location_Str = "DOWN")
        flat_location='B'
    Else If (Flat_Location_Str = "UP")
        flat_location='T'
    Else If (Flat_Location_Str = "LEFT")
        flat_location='L'
    Else If (Flat_Location_Str = "RIGHT")
        flat_location='R'
    Else
        ExitScript("ERROR: INVALID
        OrientationMarkLocation provided.")
    End If
End If

If(wf_size = die_wd)
    pos_type=1
    wafer_type=0
Else
    pos_type=0
    wafer_type=1
End If
vPosType(pos_type)
vDbIsWaferPatterned(wafer_type)

GoToBOF
GoTo("ClassLookup")
If (ErrorCode=0)
    SkipWords(-1)
    SkipLines(1)

    While (IsNumber(GetWord))
        SkipWords(-1)
        num=GetInt
        Class_Str=GetQuotedWord('')
        vAddToClassLUT(num, Class_Str)
    End While
Else
    Print("Warning:No class LUT provided", 'NL')
End If

dieCnt = 0
GoToBOF
GoTo("RemovedDieList")
If (ErrorCode=0)
    SkipWords(-1)
    SkipLines(1)
```

```

        While (IsNumber(GetWord))
            dieCnt = dieCnt + 1
            SkipWords(-1)
            diex[dieCnt]=GetInt
            diey[dieCnt]=GetInt
            dieType[dieCnt]='R'
        End While
    Else
        Print("Warning: No RemovedDieList provided", 'NL')
    End If

    GoToBOF
    SampleNum=0
    GoTo("SampleTestPlan")
    If (ErrorCode=0)
        While(ErrorCode=0)
            SampleNum = SampleNum + 1
            GoTo("SampleTestPlan")
        End While
    Else
        ExitScript("ERROR: Could not count how many
            SampleTestPlans in file.")
    End If

    GoToBOF
    For record=1 To SampleNum

        GoTo("SampleTestPlan")
        If (ErrorCode=0)
            SkipWords(-1)
            SkipLines(1)

            While (IsNumber(GetWord))
                dieCnt = dieCnt + 1
                SkipWords(-1)
                diex[dieCnt]=GetInt
                diey[dieCnt]=GetInt
                dieType[dieCnt]='V'
            End While
        Else
            Print("Warning: No SampleTestPlan provided", 'NL')
        End If
    End For

    GoToBOF
    GoTo("WaferID")
    If (ErrorCode=0)
        WaferID_Str = StrTrim(StrTrim(GetWord,'"'),'@')
    Else
        ExitScript("ERROR: Could not find WaferID in file.")
    End If

    str = WaferID_Str
    If(IsNumber(WaferID_Str))
        wfid = StrToInt(WaferID_Str)
    
```

```
str = IntToStr(wfid)
if(wfid) < 10 str = StrCat("0",str) end if
Else
wfid = -1
End If

GoToBOF
GoTo("WaferScribe")
If (ErrorCode=0)
    WaferID_Str=StrTrim(GetWord,'')
Else
    ExitScript("ERROR: Could not find WaferScribe in
        file.")
End If

DbWfNum(WaferID_Str, wfid)

GoToBOF
GoTo("AreaPerTest")
If (ErrorCode=0)
    test_area=GetReal
Else
    Print("Warning: No test area provided", 'NL')
End If

GoToBOF
GoTo("SampleCenterLocation")
If (ErrorCode=0)
    wf_xcenter = GetReal
    wf_ycenter = GetReal
Else
    ExitScript("ERROR: No wafer center location
        provided.")
End If

wmap_name_str = ProductID_Str//always force this
wf_units = "microns"

GoToBOF
GoTo("DieOrigin")
If (ErrorCode=0)
    center_x = GetReal
    center_y = GetReal
    center_x = center_x - wf_xcenter
    center_y = center_y - wf_ycenter
Else
    ExitScript("ERROR: No die origin provided.")
End If

fld_rows = 1
fld_cols = 1
row_offset = 0
col_offset = 0
```

```

//Record the wafermap
vDbWmapCfg(wmap_name_str, wf_size,
           wf_units, flat_location,
           flat_type, die_wd,
           die_ht, center_x,
           center_y, 'R',
           'U', fld_rows,
           fld_cols, row_offset,
           col_offset)
For j =1 TO dieCnt
    vDbAddToDieMapTable(dieX[j], dieY[j], dieType[j])

End For

//Get Defect Info
GoToBOF
GoTo("SummarySpec")
If (ErrorCode=0)
    spec_flag=1
Else
    ExitScript("ERROR: No SummarySpec provided.")
End If

GoToBOF
GoTo("DefectRecordSpec")
If (ErrorCode=0) //If DefectRecordSpec
    DRSLen = GetInt
    For i = 1 to DRSLen
        DRSArray[i] = GetWord
    End For

    intensity = -1

    SkipWords(-2)
    SkipLines(1)
    str = GetWord
    While (str <> "SummarySpec")
        SkipWords(-1)
        df_index = -1
        xPos = -999.999
        yPos = -999.999
        xInd = -999
        yInd = -999
        x_size = -1.0
        y_size = -1.0
        df_area = -1.0
        df_size = -1.0
        manualclass = -1
        finebin = -1
        roughbin = -1
        samplebin = -1
        reviewsample = -1
        cluster = -1
        test_num = -1
    End While
End If

```



```

For i = 1 to DRSLen
  If DRSArray[i] = "DEFECTID"
    df_index = GetInt
  Else If DRSARRAY[i] = "XREL"
    xPos = GetReal
  Else If DRSARRAY[i] = "YREL"
    yPos = GetReal
  Else If DRSARRAY[i] = "XINDEX"
    xInd = GetInt
  Else If DRSARRAY[i] = "YINDEX"
    yInd = GetInt
  Else If DRSARRAY[i] = "XSIZE"
    x_size = GetReal
  Else If DRSARRAY[i] = "YSIZE"
    y_size = GetReal
  Else If DRSARRAY[i] = "DEFECTAREA"
    df_area = GetReal
  Else If DRSARRAY[i] = "DSIZE"
    df_size = GetReal
  Else If DRSARRAY[i] = "CLASSNUMBER"
    manualclass = GetInt
  Else If DRSARRAY[i] =
    "FINEBINNUMBER"
    finebin = GetInt
  Else If DRSARRAY[i] = "TEST"
    test_num = GetInt
  Else If DRSARRAY[i] =
    "CLUSTERNUMBER"
    cluster = GetInt
  Else If DRSARRAY[i] =
    "ROUGHBINNUMBER"
    roughbin = GetInt
  Else If DRSARRAY[i] = "REVIEWSAMPLE"
    reviewsample = GetInt
  Else If DRSARRAY[i] = "IMAGECOUNT"
    image_count = GetInt
  Else If DRSARRAY[i] = "IMAGELIST"
    Str = GetWord
  Else
    Str = GetWord
  End If
End For

```

```

If E
    Print("Defect Index: ", df_index,
        " X Pos: ", xPos,
        " Y Pos: ", yPos,
        " X Indx: ", xInd,
        " Y Indx: ", yInd,
        " X Size: ", x_size,
        " Y Size: ", y_size,
        " DF Area: ", df_area,
        " DF Size: ", df_size,
        " Man Class: ", manualclass,
        " Fine Bin: ", finebin,
        " Test Num: ", test_num,
        " ImageCount: ", image_count,
        v'NL')
End If

SkipWords(-1)
SkipLines(1)
vDbDefect(df_index, xInd, yInd,
    xPos, yPos, x_size,
    y_size, df_area, df_size,
    cluster, intensity, test_num)

If manualclass >= 0
    vDbAddDefectClass(df_index, manualclass,
        1)
    vDbSetCriticalDfClass(manualclass, 1)
End If

If roughbin > 0
    vAddToClassLUT(roughbin,
        StrCat("roughbin_", IntToStr(roughbin)))
    vDbAddDefectClass(df_index, roughbin, 2)
    vDbSetCriticalDfClass(roughbin, 2)
End If

If finebin > 0
    vAddToClassLUT(finebin,
        StrCat("finebin_", IntToStr(finebin)))
    vDbAddDefectClass(df_index, finebin, 3)
    vDbSetCriticalDfClass(finebin, 3)
End If

```

```
If (image_count > 0)
    image_id = 0
    img_path = "NA"
    While(image_id < image_count)
        Str = Getword
        If (Str = "TiffFileName")
            img_file = GetWord
            Str = after(img_file, '.')
            If (Str = "jpg")
                img_type = "JPEG"
            Else
                img_type = "TIFF"
            End If
            SkipWords(-1)
            SkipLines(1)
        Else
            img_indx = StrToInt(Str)
            SkipWords(-1)
            SkipLines(1)
            image_id = image_id + 1
            vDbAddDefectImage(df_index,
                img_path,
                img_file, img_indx, img_type, 1)
        End If
    End While
End If
str = GetWord
End While
Else
    ExitScript("ERROR: No DefectRecordSpec provided.")
End If

GoTo("SummaryList")
If ErrorCode = 0
    SkipWords(-1)
    SkipLines(1)
    While IsNumber(GetWord) AND NotEndOfFile
        SkipWords(-1)
        test_num = GetInt
        def_cnt = GetInt
        dd = GetReal
        insp_die = GetInt
        def_die = GetInt
        Print("Test_Num: ", test_num, 'NL')
        vDbDefectSummary(test_num, def_cnt,
            test_area, dd, insp_die, def_die)
        SkipWords(-1)
        SkipLines(1)
    End While
Else
    Print("Warning: Could not find summaries in KLARF
        file", 'NL')
End If

Print("Start dBDumpLayer ", 'NL')
```

```
        dBDumpLayer()  
        Print("End dBDumpLayer ", 'NL')  
  
Else //the program is new  
    Print("The program is New!", 'NL')  
    TestNum = IntToStr(0)  
  
    DD_Threshold = 2.3  
    DC_Threshold = 1000  
    DbDfTagAction(2, DD_Threshold, DC_Threshold)  
    DbDumpLayer()//store required summaries in DB  
End If  
  
End
```

EXAMPLE DATA FILE - DEFECT IMAGE POINTERS

```

FileTimestamp 2011-01-01 11:11:11;
InspectionStationID "TESTER_TYPE""TESTER1""TESTER2";
SampleType WAFER;
ResultTimestamp 2011-01-01 11:11:12;
LotID "MY_LOT";
SampleSize 1 300;
SetupID "ID1" 2011-01-11 11:01:11;
StepID "MY_STEP";
DeviceID "MY_DEVICE";
SampleOrientationMarkType NOTCH;
OrientationMarkLocation DOWN;
DiePitch 20000.0 30000.0;
ClassLookup 3
0 "Not Classified"
1 "NonVisible"
10 "Particle";
InspectionTest 1;
SampleTestPlan 1
-5 0
-5 1;
AreaPerTest 1000000.00;
WaferID "03";
WaferScribe "Scribe1";
DieOrigin 0.000000 0.000000;
Slot 1;
SampleCenterLocation 12000.1 15000.1;
DefectRecordSpec 21 DEFECTID XREL YREL XINDEX YINDEX XSIZE
YSIZE DSIZE DEFECTAREA CLASSNUMBER OTYPE VOLUME GRADE TEST
ROUGHBINNUMBER FINEBINNUMBER REVIEWSAMPLE REGIONID
CLUSTERNUMBER IMAGECOUNT IMAGELIST;
1 7.6934787256e+002 3.0707644388e+004 -5 1 9.036854 0.577817
1.467832 5.221648 0 16 140 219 1 0 0 0 2 0 4 4
TiffFileName "example.tif";
1 30
2 32
3 30
4 32
2 7.8514590564e+002 2.8568470145e+004 -5 0 0.518913 0.538045
0.125326 0.279199 0 10 21 206 1 0 0 0 2 0 4 4
TiffFileName "example.tif";
5 30
6 32
7 30
8 32
SummarySpec 5 TESTNO NDEFECT DEF DENSITY NDIE NDEFDIE;
SummaryList
1 2 2.0e-08 69 2;
EndOfFile;

```


CHAPTER 9

LTX77 DATA READER

IMPORTING DATA / FORMAT FILE CONFIGURATION

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file. Exensio –Yield readers can access data directly from files or through a database using the Exensio –Yield database system. Whether the data is imported through a database or directly from a data file, the objective is the same: to access your data and format it properly for comprehensive analysis with Exensio –Yield.

To accommodate these two primary means of importing data, there are two types of data readers: database (*dB*) readers, which import the formatted data into the Informix database for access through the Exensio –Yield database system; and worksheet (*WS*) readers, which import the formatted data directly into the Exensio –Yield environment. dB readers operate in the background, as part of the overall process of configuring and importing data from the database. WS reader sessions are configured and implemented from a reader-specific interface that is called by the user when it is time to load a new raw data file.

For a general overview of how Exensio –Yield readers function, refer to “[Exensio Data Readers — Overview](#),” pg. 9.

In this document, the LTX WS reader functionality is described in the following section. Throughout the rest of the document, beginning with “[LTX77 Format — Overview](#),” pg. 385, sections dealing exclusively with dB readers will be marked with the  icon.

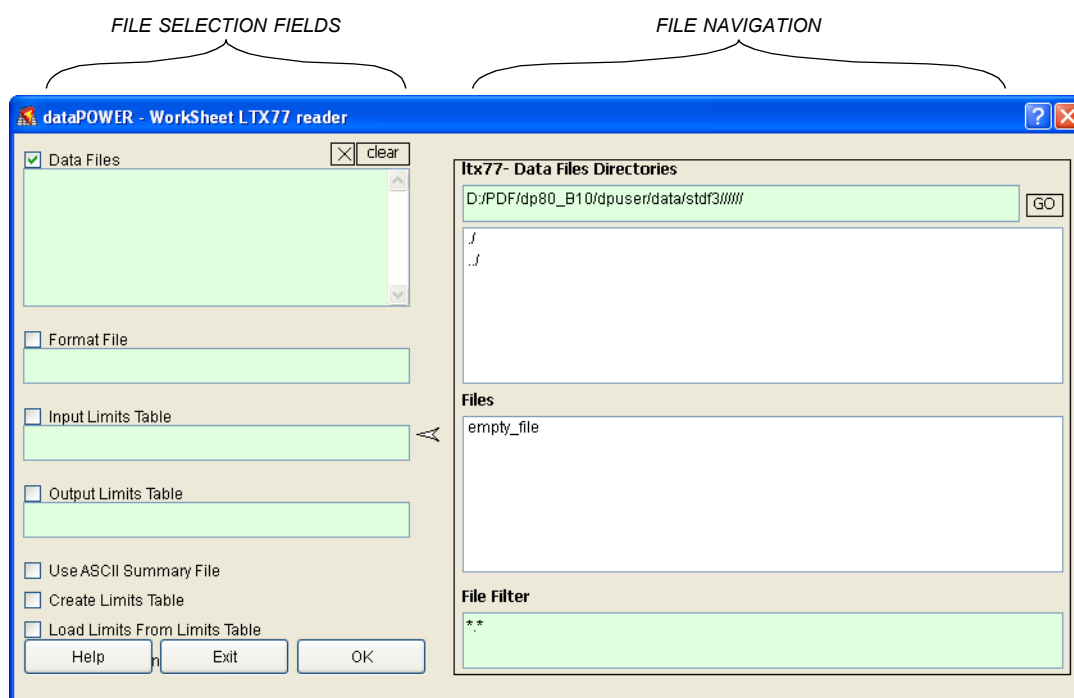


<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

LTX77 WORKSHEET READER INTERFACE

From Exensio –Yield, LTX77 files that are to be imported directly to the worksheet without using the Exensio –Yield database system will use the Data Import dialog. From this dialog, you will configure the data so it will be properly formatted for the Exensio –Yield environment. If your data import sessions will always utilize the database, you will not be using this function.

To invoke the dialog, click on the  button or select **Data > Readers > LTX77**. (The button and menu option will be available if your system configuration includes the LTX77 worksheet reader.)



There are several primary components that must be specified from the data import dialog for a reader session to be successful, including the **Data Files**, **Format File**, **Input Limits Table**, and **Output Limits Table**. Each of these fields is populated using the file navigator. When you click to select any of the file selection fields, the file navigator will automatically point to the appropriate directory for that type of file and list all of the files it contains. Then you can use the file navigator's functionality to find the correct file to populate the field. Any file you load will be moved into the selected field.

LTX77 FORMAT — OVERVIEW

Command-line Arguments

The LTX77 data reader accepts the following command-line arguments:

-fmt	[format file] Where format file is the format to be used.
-db	[database] Where database is the name of the database to write to.
-db_accept	If used the processing of the files does not wait for any user input, this is useful if non of the filtering or sampling options are needed.
-semi_dynamic	A semi dynamic “Program” assumes a fixed number of Parameters, like the default, but in addition fills up all tables with keys in the OP_LOG table such as Product and Equipment.
-dynamic	Like semi-dynamic, but without a fixed number of Parameters.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.
-resex	[extent] Sets database table extents for the RES table. If not used, defaults to 5012 Kbytes. Acceptable range is 64 Kb to 10024 Kb.
-lotext	[extent] Sets database table extents for the LOT table. If not used, defaults to 64 Kbytes. Acceptable range is 16 Kb to 1024 Kb.
-defext	[extent] Sets database table extents for the DEF table. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 5012 Kb.
-wafext	[extent] Sets database table extents for the WAF table. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 5012 Kb.
-class	[program class] Sets Program Class, (New Programs).

-indexes	This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbspace in the database, then [dbspace] is where the indexes will be created. Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-lowercase	Forces Lot ID and Wafer ID to Lower Case.
-uppercase	Forces Lot ID and Wafer ID to Upper Case.
-outliers	[ni] Filters Outliers from the calculation of Wafer and Lot summaries using Box-Plot criteria.
-dboutliers	Filters Outliers from the calculation of Wafer and Lot summaries using database limits.
-rework_action	[action] Sets the rework_action field in the PROGRAM table to “action”. Valid values for rework action are 1, 2, 3, 4.
-srclot	Gets Source Lot from file name (SrcLot_name)
-wfnum	Gets wafer number from file name (SrcLot_wfnum_name)
-lsum	Reads ASCII Lot summary file

FORMAT FILE

The format file specifies the conditions, key conditions, indexes and key indexes. Every test program in the database can not use more than one format file. The test name, unit, and test number are forced conditions and should not be in the format file.

In addition the format file may contain conditions and indexes from the following list:

DevNum	(Device number)
BinNum	(Bin number)
Cond	(LTX77 condition)
FileName	(The data file name without the path)



FRONT



CONTENTS



INDEX

exensio

TesterNum	(Test number 16003.0)
HeadNum	(Test number 16004.0)
Lot	(Lot from File Name)
Wafer	(Wafer from File Name)
die_X	(die_X Test)
die_Y	(die_Y Test)

A condition is indicated with the letter “C,” a key condition with the letters “KC,” an index with the letter “K” and a key index with the letters “KI” and as a special case an “R” can be placed after BinNum to make it a result (one of the parameters).

Every Format file should have at least one key index. All format files should list all conditions before the indexes.

Example:

Cond	C
DevNum	KI
Lot	KI
BinNum	I



DATABASE

Fields that are declared as indexes can affect the binning and wafer database tables.

The **HIST_BIN** and **BIN_LOG** database tables are updated by the LTX77 reader only if an index or a Parameter exists that contains bin information. (BinNum)

The **LOT...** and **LOT** database tables are updated by the LTX77 reader only if the Lot_id exists and is valid.

The **WAF...** and **WAFER** database tables are updated by the LTX77 reader only if an index exists that is declared as containing wafer information. (Wafer).

All files must contain a Test Program, and the Test Program must belong to a Program Class. This is done using the -class option as described earlier.



CHAPTER 10

BITMAP READER

IMPORTING DATA

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file.

The Exensio –Yield BitMap reader imports logical bit fault data directly into the Exensio –Yield database system.



The Exensio –Yield BitMap Reader is a database reader and has no worksheet reader functionality. It therefore runs only on user configurations which include the Exensio –Yield dataBASE system.



All data points between $-1e-38$ and $1e-38$ are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

BitMap Data Retrieval

Bit fault data retrieval into the BitMap tool is accomplished with the **BitMap** level of the Mapping Retrieval interface.

The functionality of BitMap retrieval is fully described in the “*Data Retrieval*,” chapter of the Exensio End-Users Manual. The primary function of the window is to supply customized retrieval options so the end-user can easily populate a worksheet with bit fault data, and then use the tool for analysis.

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	Char	If	Target
KeyCond	False	AND	Integer	Else	LOL
LimCond	EQ	Const	Real	Exit	HOL
Index	NE	Var	String	LSL	LWL
KeyIndex	NOT	Begin	Boolean	HSL	HWL
Result	LE	End	While	LPL	FailBin
FileName	GE	Step	For	HPL	mod
Script	LT	GT	To	Tune	

All built-in functions described in *“Built-in Functions,”* pg. 399, also constitute part of the list of key words.

Comments The characters `/*` introduce a comment that terminates with the characters `*/`, such comments do not nest and they can not occur within strings. The characters `//` introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file contains constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example looks like this:

```
CONST
    job_nam      =      "MyJob"      //String Constant
    Product      =      "MyProduct"  //String Constant
    NumOfBins    =      32            //Integer Constant
```

Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 254 characters)
- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key
                                   condition
Variable1,Variable2,...TypeLimCond// To declare a limit
                                   condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The allowed types for conditions and indexes are Real, Integer and String. Also, note that the order that these conditions appear in is important and is different for dB and WS readers. The first three conditions should always be:

dB Reader —

Tname	String	(For Test Name)
Unit	String	(For Units)
TestNum	Integer	(For Test Number)



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results

The bitmap data reader supports choosing more than one result each of a possibly different type. Results are declared in the VAR block and the following syntax is used:

```
Variable1,Variable2,...Type Result// To declare a result
```

Every format file must have at least one result declared.

The Main Block

The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the ASCII file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the bitmap data reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write the following in the format file:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index **lot_id** the value of “FileName” one would

write:

lot_id = FileName //lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

```
AddSep(':')
```

OPERATORS

<i>Arithmetic:</i>	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Assignment	=
	Modulus	mod

Logical: Used only with variables of type Boolean.

Logical Or	OR
Logical And	AND
Negation	NOT

<i>Relational:</i>	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression  
    statements  
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression  
    statements  
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression  
    if-statements  
Else  
    else-statements  
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of BitMap Reader built-in functions, which generally fall into the following categories and sub-categories:

- **File Navigation** — pg. 400
 - **Go To** — pg. 400
 - **Separators** — pg. 400
 - **Skip Forward/Backward** — pg. 401
 - **Miscellaneous** — pg. 401
- **Data Retrieval** — pg. 401
 - **Data** — pg. 401
 - **Strings** — pg. 401
 - **Sub-Strings** — pg. 402
 - **Integers** — pg. 403
 - **Real** — pg. 404
- **String Manipulation** — pg. 404
- **Database** — pg. 406
 - **BitMap** — pg. 406
 - **Logging Pattern Instances** — pg. 412
 - **Composite Class Instances** — pg. 422
 - **Fab** — pg. 422
 - **Technology** — pg. 423
 - **Family** — pg. 423
 - **Process** — pg. 424
 - **Product** — pg. 424
 - **Lot** — pg. 425
 - **Program** — pg. 425
 - **Step** — pg. 426
 - **Stage** — pg. 427
 - **Equipment** — pg. 427

- **Operator** — pg. 428
- **Customer** — pg. 429
- **Wafer Configuration** — pg. 429
- **Tagging** — pg. 430
- **Storing Raw BitMap File Information** — pg. 431
- **Miscellaneous** — pg. 432
- **Mathematical** — pg. 433
- **Debugging** — pg. 433
- **System** — pg. 434
- **Zones** — pg. 434

File Navigation

<i>Go To</i>	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful, the File Pointer is moved one character beyond the passed string. Otherwise, the File Pointer is not changed and the error code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.

<i>Skip Forward/ Backward</i>	SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive, N lines are skipped in the forward direction. If N is negative, the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
	SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.
	SkipChars (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.

<i>Miscellaneous</i>	NotEndOfFile —	Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.
	GetLineLen —	Accepts no arguments and returns an integer. Returns the number of characters from the File Pointer to the end of line.

Data Retrieval

<i>Data</i>	OpenFile (string) —	Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the -file_path command line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.
<i>Strings</i>	GetWord —	Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.
	GetPrevWord —	Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

GetQuotedWord (char) — Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error code is set if no quoted word is found.)

GetLine — Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.

Sub-Strings **GetChar** — Accepts no arguments and returns the first character in the current word, leaving the File Pointer one character beyond the retrieved word.

GetChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.

GetCharsTrim (integer, char) — Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)

GetLeftChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetRightChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetMidChars (integer, integer) —

Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordAfter (char) —

Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.

If there are multiple occurrences of the passed character, the function returns all characters after the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) —

Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

ExtractString —

Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.

Integers **GetInt —**

Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.

String Manipulation

GetPrevInt —	Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.
<i>Real</i> GetReal —	Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.
GetPrevReal —	Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.
ToLower (string) —	Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.
ToUpper (string) —	Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.
IsNumber (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.
IsString (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.
Right (string,integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetRightChars() , but operates on the passed string instead of the current word.
Left (string,integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetLeftChars() , but operates on the passed string instead of the current word.
Mid (string, integer, integer) —	Accepts three arguments of type string, integer and integer and returns a string. Same as GetMidChars() , but operates on the passed string instead of the current word.

After (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordAfter()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

Before (string, char) — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.

If there are multiple occurrences of the passed character, the function returns all characters before the last instance.

StrCat (string, string) — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.

StrToReal (string) — Accepts one argument of type string and returns a real. Translates the passed string to a real.

StrToInt (string) — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and StrToReal will convert the first numerical part of the provided string.

For example, for `StrToInt ("16A3")`, the output would be the integer 16; for `StrToReal ("16.1A3")`, the output would be 16.1.

IntToStr (integer) — Accepts one argument of type integer and returns a string. Translates the passed integer to a string.

RealToInt (real_num -real-) — Accepts one real argument and returns its integer value.

Input variables:

real_num: A real valued number.

IntToReal (int_num -integer-) — Accepts one integer argument and returns its real value.

Input variables:

int_num: An integer valued number.

StrTrim (string, char) — Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.

StrLen (string) — Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.

Argument — Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument **-arg**.

Database

BitMap **vDBAddMemCellClass —**
(inDesignName -string-, inDesignType -string-, inDescription -string-,
inRowCount -integer-, inColumnCount -integer-,
inBusWidth -integer-, inBlockCount -integer-,
inTwistScheme -integer-, width -real- height - real -) —

Creates a description of a memory cell macro unique inDesignName.

This memory cell can be used in any number of products.

If cell with inDesignName already exists and at least one of the remaining attributes is different, no action is taken and error is returned.

Input variables:

inDesignName: Unique name of this memory cell design.

inDesignType: Identifies the design type: DRAM, SRAM, FLASH.

inDescription: Optional description of the cell design details and purpose.

inRowCount: Number of rows in this memory cell.

inColumnCount: Number of columns in this memory cell.

inBusWidth: Number of I/O channels of this memory.

inBlockCount: Number of blocks in this memory cell

inTwistScheme: Number used as an identification of the column twisting scheme in DRAM. 0 Means no twisting. Reserved for future use.

width: width along the of the cell in microns (along OX axis)

height: height of the cell in microns (along the OY axis)

vDbMemCellBlockConfig
(**inBlockNumber** -integer-, **inBlockName** -string-, **inBlockType** -char-, **inBlockCount** -integer-, **inColumnCount** -integer-, **inX1** -integer-, **inY1** -integer-, **inX2** -integer-, **inY2** -integer-, **inFirstColumn** -integer-, **inFirstRow** -integer-) —

Creates a definition of the memory block for a currently selected memory design (by `vDBAddMemCellClass`). The memory design must already exist.

Input variables:

inMemCellName: Unique name of a memory instance this block belongs to.

inBlockNumber: Number given to this block — unique for a given memory instance.

inBlockName: Optional name for the block.

inBlockType: Defines a block type.
Predefined values are:

- M** — the usual memory block
- P** — periphery

inBlockCount: Number of rows in this block.

inColumnCount: Number of columns in this block.

inX1: Position of the left side of the block, relative to the left edge of the memory instance.

inY1: Position of the bottom side of the block relative, to the bottom edge of the memory.

inX2: Position of the right side of the block, relative to the right edge of the memory.

inY2: Position of the top side of the block, relative to the top edge of the memory.

inFirstColumn: topological bit number of the first column.

inFirstRow: topological bit number of the first row.

vDbMemConfig —

(inDesignName -string-, inMemInstanceName -string-, inMemoryNumber -integer-, inDescription -string-, inX1 -integer-, inY1 -integer-, inX2 -integer-, inY2 -integer-, inColumnDirection -char-, inRowDirection -char-) —

Creates a description of a memory instance within the current chip. This is done for the current product. The instance is defined by reference to **mem_dsgn_name** value, which must already exist in the MEM_CELL_CLASS table. If a memory instance with name **inMemInstanceName** does not yet exist in the MEM_CONFIG table for a given product, it is added. Otherwise, the attributes provided are checked against the ones already present in the database for that memory instance and that product. If at least one of the remaining attributes is different, no action is taken and error is returned.

Input variables:

inDesignName: Reference to unique name of the macro used — must be already defined (see [vDBAddMemCellClass](#)).

inMemInstanceName: Name assigned to this instance — must be unique.

inMemoryNumber: Number assigned to the memory instance. Unique for a given product.

inDescription: Optional description of the purpose for this memory instance.

inX1: Left coordinate of the bounding box, in chip coordinates.

inY1: Bottom coordinate of the bounding box, in chip coordinates.

inX2: Right coordinate of the bounding box, in chip coordinates.

inY2: Top coordinate of the bounding box, in chip coordinates.

inColumnDirection: Direction of column, counting:

R - columns are vertical, numbered left to right,
L - columns are vertical, numbered right to left,
D - columns are horizontal, numbered top down,
U - columns are horizontal, numbered bottom up.

inRowDirection: Direction of row, counting:

R - rows are vertical, numbered left to right,
L - rows are vertical, numbered right to left,
D - rows are horizontal, numbered top down,
U - rows are horizontal, numbered bottom up.

vDbMemBlockConfig

(**inBlockNumber** -integer-, **inBlockName** -string-, **inBlockType** -char-,
inBlockCount -integer-, **inColumnCount** -integer-, **inX1** -integer-, **inY1** -integer-,
inX2 -integer-, **inY2** -integer-, **inFirstColumn** -integer-,
inFirstRow -integer-) —

Creates a definition of the memory block for a given memory instance (by vDbMemConfig). The memory instance must already exist.

Input variables:

inMemInstanceName: Unique name of a memory instance this block belongs to.

inBlockNumber: number given to this block — unique for a given memory instance.

inBlockName: Optional name for the block.

inBlockType: Defines a block type.

Predefined values are:

M — the usual memory block

P — periphery

inBlockCount: Number of rows in this block.

inColumnCount: Number of columns in this block.

inX1: Position of the left side of the block, relative to the left edge of the memory instance.

inY1: Position of the bottom side of the block relative, to the bottom edge of the memory.

inX2: Position of the right side of the block, relative to the right edge of the memory.

inY2: Position of the top side of the block, relative to the top edge of the memory.

inFirstColumn: topological bit number of the first column.

inFirstRow: topological bit number of the first row.

vdbPatternSet (patternSetName -string-, memoryCellName -string-, isLibDefault -string-) —

Specifies pattern set to be used for a given memory design during loading of the current data file. If this call is not present for a given memory design, then an implicit pattern setup mechanism is in effect.

If the Pattern Set name was not provided to the reader and the **-debug** option is not given on input and the MEM_LIB2PROD table for this product and this memory design already contains an entry that is different from <product name>_<memory design name>_set, then the reader aborts and exists with an error. Otherwise, if the pattern setup name was not provided, the reader assumes implicitly that the name of the setup is <product name>_<memory design name>_set. If such a name is not present in the MEM_MCLIB table yet, it is inserted there and MEM_LIB2PROD is updated accordingly. In this implicit setup mechanism, a pattern that is present in MEM_CLASS is allowed to appear in the input file and will be added to the MEM_MCL2MC table as needed with **rec_order** field set to 0. Implicit pattern set can be used only when no other one exists for a given product.

If you created a pattern set through this mechanism but want to continue using it with other patterns sets created later, you must rename this pattern set.

patternSetName — name of the pattern set that is already defined in the database

memoryCellName — name of the memory design that is already defined an associated with the patternSetName

isLibDefault — if ‘y’ or ‘Y’ or ‘yes’ use that pattern set a default one from now on for this memoryCellName in current product (stored in the MEM_LIB2PROD table). Otherwise, use this library during the current run but do not change the default pattern set.

vdbSetDefaultTest (instanceName -string-) - —

Makes a current test for a given memory instance to be a default test (stored in the MEM_DEFAULT_TEST table). If this call is not executed, a default test will be the one that was set earlier. Moreover, the first test for which results are loaded for a given memory instance becomes the default test.

vDbDieMemory — (inDieX -integer-, inDieY -integer-, inDevNum -integer-, inMemInstanceName -string-, inMemInstanceNumber -integer-)

Creates an entry in the MEM_LOG table and, if needed, an entry in the MEM_MEMORY table for the currently tested die.

Input variables:

inDieX: The x index of the die within a wafer.

inDieY: The y index of the die within a wafer.

inDevNum: The device number on the wafer.

inMemInstanceName: Unique name of the memory instance (see [vDbMemConfig](#)).

inMemInstanceNumber: Instance number of the memory for which the results follow.

The bitmap reader requires the **vDbDieMemory** built-in function to be called before some of the database built-in functions.

The functions that are affected are:

- Time functions (start and end time)
- File and path functions (**vDbRawFile**, **vDbRawFilePath**...)
- Fail pattern functions (**vDbLogBox**, **vDbLogRow**...)

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE ANY OF THESE BUILT-IN FUNCTIONS. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THESE FUNCTIONS WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

This is the complete list of functions that are affected:

- DbStartTime()
- VDbStartTime()
- DbEndTime()
- VDbEndTime()
- VDbLogBox()
- VDbLogCustom()
- VDbLogMultiRow()
- VDbLogMultiColumn()
- VDbLogFog()
- VDbLogCross()
- VDbLogMiss()
- VDbLogColumn()
- VDbLogRow()
- VDbLogSpot()
- VDBFileSetup()
- VDBRawFile()
- VDBRawFilePath()
- VDBRootFilePath()

*Logging Pattern
Instances*

vDBLogSpot
(inClassName -string-, inMinCol -integer-, inMinRow -integer-,
inMaxCol -integer-, inMaxRow -integer-) —

Creates a row in the MEM_SPOT table to represent a spot-like pattern instance for a currently selected die and memory (see **vDbDieMemory**). There is no requirement for the newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type spot.

inMinCol: smallest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width depending upon pattern definition)

inMinRow: smallest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width depending upon pattern definition).

inMaxCol: biggest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogRow —

(inClassName - char-, inMinCol -integer-, inMinRow -integer-, inMaxCol -integer-, inMaxRow -integer-, inFailCount -integer-)

Creates a row in the MEM_ROWS table to represent a row-like pattern instance for a currently selected die and memory (see **vDbDieMemory**). There is no requirement for the newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type row.

inMinCol: smallest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMinRow: smallest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogColumn
(inClassName -string-, inMinCol -integer-, inMinRow -integer-, inMaxCol -integer-, inMaxRow -integer-, inFailCount) —

Creates a row in the MEM_COLS table to represent a column-like pattern instance for a currently selected die and memory (see **vDbDieMemory**). There is no requirement for that newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type column.

inMinCol: smallest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMinRow: smallest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogBox

(inClassName -string-, invertFailBits -integer-, inMinCol -integer-, inMaxCol -integer-, inMinRow -integer-, inMaxRow -integer-, inFailCount -integer-) —

Creates a row in the MEM_BOX table to represent a box-like pattern instance for a currently selected die and memory (see **vDbDieMemory**). There is no requirement for that newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type box.

invertFailBits: reserved for future use.

inMinCol: smallest column at which the pattern instance occurs.

inMinRow: smallest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogOther
(inClassName -string-, invertFailBits -integer-, inMinCol -integer-, inMaxCol -integer-, inMinRow -integer-, inMaxRow -integer-, inFailCount -integer-) —

Creates a row in the MEM_MISS table to represent a pattern describing a set of failed bits that were not classified for a currently selected die and memory (see **vDbDieMemory**). There is no requirement for that newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type box.

inMinCol: smallest column at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMinRow: smallest row at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance.

vDBLogCustom

(inClassName -string-, invertFailBits -integer-, inMinCol -integer-, inMaxCol -integer-, inMinRow -integer-, inMaxRow -integer-, inFailCount -integer-) —

For a currently selected die and memory (see [vDbDieMemory](#)), creates a row in the MEM_CUSTOM table to represent a pattern instance for a class that does not fit one of several predefined types. There is no requirement for that newly created table entry to be unique.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type custom.

invertFailBits: reserved for future use.

inMinCol: smallest column at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMinRow: smallest row at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the bits occur, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogCross (inClassName -string-, inFailCount -integer-) —

For a currently selected die and memory (see [vDbDieMemory](#)), creates a row in the MEM_CROSS table to represent a pattern instance for a cross-like class representing and intersection of an instance of column class and instance of row class. There is no requirement for that newly created table entry to be unique. The column pattern instance and row pattern instance must be provided immediately after this call by calling [vDBLogRow\(\)](#) and [vDBLogColumn\(\)](#), and [vDBEndComposite\(\)](#).

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type cross.

inFailCount: number of bits that failed in this pattern instance. If it is ≤ 0 the bitmap reader will calculate fail count by summing up fail count for column and row pattern instances that comprise this cross pattern instance.

IMPORTANT NOTE: The bitmap reader requires the [vDbDieMemory](#) built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogFog (inClassName -string-, inMinCol -integer-, inMinRow -integer-, inMaxCol -integer-, inMaxRow -integer-, inFailCount) —

Creates a row in the MEM_FOG table to represent a box-like pattern instance for a currently selected die and memory (see [vDbDieMemory](#)). There is no requirement for that newly created table entry to be unique. The pattern represents a cloud of spot pattern instances. These component instances must be

provided immediately after this call by calling `vDBLogSpot()` for each of the relevant spot pattern instance and after that calling `vDBEndComposite()`. The references to those component pattern instances are put into `MEM_FOG_BITS` table.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type fog.

inMinCol: smallest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMinRow: smallest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxCol: biggest column at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inMaxRow: biggest row at which the pattern instance occurs, excluding pattern neighborhood (which may be a non-zero width, depending upon pattern definition).

inFailCount: number of bits that failed in this pattern instance. If it is ≤ 0 the bitmap reader will calculate fail count by summing up fail bit count for spot pattern instances that comprise this fog pattern instance.

IMPORTANT NOTE: The bitmap reader requires the `vDbDieMemory` built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogMultiColumn (inClassName -string-, inFailCount -integer-) —

Creates a row in the MEM_MULTI_COL table to represent a bar code-like pattern instance for a currently selected die and memory (see [vDbDieMemory](#)). There is no requirement for that newly created table entry to be unique. The pattern represents a group of regularly spaced column pattern instances. These component instances must be provided immediately after this call by calling [vDBLogColumn\(\)](#) for each of the relevant spot pattern instance and after that calling [vDBEndComposite\(\)](#). The references to those component pattern instances are put into MEM_MULTI_COL_BITS table.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type multirow.

inFailCount: number of bits that failed in this pattern instance. If it is ≤ 0 the bitmap reader will calculate fail count by summing up fail bit count for column pattern instances that comprise this multicolumn pattern instance.

IMPORTANT NOTE: The bitmap reader requires the [vDbDieMemory](#) built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vDBLogMultiRow (inClassName -string-, inFailCount -integer-) —

Creates a row in the MEM_MULTI_ROW table to represent a bar code-like pattern instance for a currently selected die and memory (see [vDbDieMemory](#)). There is no requirement for that newly created table entry to be unique. The pattern represents a group of regularly spaced rows pattern instances. These component instances must be provided immediately after this call by calling [vDBLogRow\(\)](#) for each of the relevant spot pattern instance and after that calling [vDBEndComposite\(\)](#). The references to those component pattern instances are put into MEM_MULTI_ROW_BITS table.

inClassName: Name that uniquely identifies a pattern class. The class with that name must be already defined in the database and be of type multirow.

inFailCount: number of bits that failed in this pattern instance. If it is ≤ 0 the bitmap reader will calculate fail count by summing up fail bit count for column pattern instances that comprise this multicolumn pattern instance.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

Composite Class Instances

vDBEndComposite() —

Use for all the composite classes in order to end recording of component instances. This call is used whenever one of the following calls occurs: **vDBLogCross()**, **vDBLogFog()**, **vDBLogMultiRow()**, **vDBLogColumn()**. As described above, the component pattern instances for each of these composite patterns must be recorded. **vDBEndComposite()** marks the end of the list of the component patterns.

Fab DbFab --- default --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbFab --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbFab (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbFab**, but uses the passed argument instead of reading from the file.

Technology **DbTechnology --- default --- —** Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbTechnology --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbTechnology (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbTechnology**, but uses the passed argument instead of reading from the file.

Family **DbFamily —** Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the program type is fixed and the current word does not exist as a family in the database (FAMILY Table), the function does nothing. If the program type is not fixed (semi-dynamic, dynamic) and the current word does not exist as a family in the database (FAMILY table), the family is added to the database and the function establishes the appropriate relations with other tables.

vDbFamily (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbFamily**, but uses the passed argument instead of reading from the file.

Process DbProcess --- default --- —

Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a process in the database (PROCESS Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbProcess --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbProcess**, but uses the passed argument instead of reading from the file.

Product DbProduct --- default --- —

Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

When the product is created by the reader, the program is automatically created. The program name is created by prefixing the product name with "bit". For example, if the product is DVD07 the program name is bitDVD07.



If the product name is longer than 57 characters, only the first 57 characters are used for creation of the program name.

DbProduct --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbProduct (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file.

Program vDbProgGroup (string) — Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

Lot DbLot —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Lot in the database (LOT Table), The Lot is added. If it does exist, the function establishes the appropriate relations with other tables. Necessary if Lot summaries are to be calculated.

vDbLot (string) —

Accepts four arguments of type string: (**Lot_Name, Method_Name, Method_Type, Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

DbSrcLot — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Source Lot in the database (LOT Table), the Source Lot is added. If it does exist, the function establishes the appropriate relations with other tables. Works in conjunction with DbLot, sets the **src_lot** column in the LOT table and OP_LOG table.

vDbSrcLot (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbSrcLot, but uses the passed argument instead of reading from the file

vDbLotClass (string) — string, string, string, string) — Accepts four arguments of type string: (**Lot_Name, Method_Name, Method_Type, Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

Step **DbStep --- default ---** — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStep --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStep (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbStep**, but uses the passed argument instead of reading from the file.

Stage **DbStage --- default ---** — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStage --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStage (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbStage, but uses the passed argument instead of reading from the file.

Equipment **DbEquip3 —** Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as an equipment in the database (EQUIPMENT Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables. (Sets eqkey3 in the OP_LOG table.)

vDbEquip3 (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbEquip3**, but uses the passed argument instead of reading from the file.

DbTester --- default --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a tester in the database (EQUIPMENT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbTester --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a tester in the database (EQUIPMENT Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbTester (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbTester**, but uses the passed argument instead of reading from the file.

DbHandler --- default --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a handler in the database (EQUIPMENT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbHandler--- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a handler in the database (EQUIPMENT Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbHandler (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbHandler**, but uses the passed argument instead of reading from the file.

Operator **DbOperator --- default --- —** Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a name in the PEOPLE Table, The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbOperator --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls **GetWord** returning the current word. If the current word does not exist as a name in the PEOPLE Table, it is added with the “role” field set to Operator. If it does exist the function only establishes the appropriate relations with other tables.

vDbOperator (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as **DbOperator**, but uses the passed argument instead of reading from the file.

Customer **vDbCustomer(string, ...)** — Accepts 12 arguments of type string, and has no return. Populates the CUSTOMER database table and establishes a relationship between the customer entry and the current lot. This function may be called several times in the same format file, in this way relating one lot to many customers. The arguments passed to the function match exactly those of the CUSTOMER table. These fields (in their respective order) include: **customer name, address 1, address 2, address 3, postal code, city, state, country, contact, email, fax, phone.**

Wafer Configuration **vDbWmapCfg (...)** — Accepts 16 arguments and has no return. The passed arguments fill the WMAP_CONFIG database table and are of the following types:

...
wf_size - Real
wf_units - String
flat - Char
flat_type Char
die_wd - Real
die_ht - Real
center_x - Real
center_y - Real
pos_x - Char
pos_y - Char
fld_rows Integer
fld_cols Integer
row_offset integer
col_offset integer
diex_offset integer
diey_offset integer

Acceptable values for:

- **Flat** —
 - **R** - right
 - **L** - left
 - **T** - top
 - **B** - bottom
- **flat_type** —
 - **F**- flat
 - **N** - notch
- **pos_x** —
 - **R** - right

- **L** - left
- **pos_y** —
- **U** - up
- **D** - down

DbWfDesc (string, string) —

Accepts two arguments of type string and has no return. The first argument is the `wf_id`; the second argument is the wafer description (maximum 64 characters). This function allows a wafer description to be added to the WAFER table (in the **wf_desc** column), but only when a new wafer is encountered.

Tagging **DbLotTag (int) —**

Accepts one argument of type integer and has no return. Used for manual lot tagging. The passed argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)
- High Defect Density(5)
- High Defect Count(6)
- High Defect Density and Count(7)

The above list is dependent on the contents of the **TAGS** table in the database.

DbWfTag (string, int) —

Accepts two arguments of type string and integer and has no return. Used for manual wafer tagging. The first argument should be the wafer ID. The second argument should be one of the following values:

- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)
- High Defect Density(5)
- High Defect Count(6)
- High Defect Density and Count(7)

The above list is dependent on the contents of the **TAGS** table in the database.

DbSrcLotTag (int) — Accepts one argument of type integer and has no return. Used for manual source lot tagging. The passed argument should be one of the following values:
No Action (1)
Bad (2)
Scrap (3)
Experiment (4)

The above list is dependent on the contents of the **TAGS** table in the database.

*Storing Raw BitMap
File Information*

vdbRawFile (string, string, string, string) — Accepts four arguments of type string and has no returns — **name1, name2, name3, name4**.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vdbRawFilePath (string, string, string, string) — Accepts four arguments of type string and has no returns — **path1, path2, path3, path4**.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vdbRootFilePath (string, string, string, string) — Accepts four arguments of type string and has no returns — **rpath1, rpath2, rpath3, rpath4**.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

vdbFileSetup (int itemIndex, int start_row, int start_col, string transf, string setup_name) —

Accepts five arguments of type integer and string and has no returns —

itemIndex — the ordinal number (counted from 1) for a die within a given raw bitmap file. This is useful for file formats, such as BIM, that can store bitmap data for the entire wafer. In such a case, the index identifies the die counting from the beginning of the file (starting from 1).

start_row — corresponding to input **InputBitmapSplitStartRow** from the MemBrain **fileSetupFile** used for that memory.

start_col — corresponding to input **InputBitmapSplitStartColumn** from the MemBrain **fileSetupFile** used for that memory.

transf — corresponds to **InputBitmapOrientation** from the MemBrain **fileSetupFile** used for that memory.

setup_name — Corresponds to **InputBitmapSetupName** from the MemBrain **fileSetupFile** used for that memory. Represents the file configuration setup name.

This function should be called after calling **vDbDieMemory**.

IMPORTANT NOTE: The bitmap reader requires the **vDbDieMemory** built-in function to be called before this built-in function.

THIS FUNCTIONALITY WILL IMPACT ANY FORMAT FILES THAT USE THIS BUILT-IN FUNCTION. ALL PRE-EXISTING BitMap READER FORMAT FILES USING THIS FUNCTION WILL REQUIRE EDITING SO THAT THEY WILL CONTINUE TO RUN CORRECTLY.

Miscellaneous **DbTestMode —**

Accepts no arguments and has no return. Calls **GetWord** and assigns the first character of the current word to **test_mode** in the database OP_LOG table.

- vDbTestMode (Char)** — Accepts one argument of type Char and has no return. Assigns the passed character to **test_mode** in the database OP_LOG table.
- DbWfNum (string, int)** — Accepts two arguments of type string and integer and has no return. The first argument should be the wafer ID, while the second argument should be the wafer number associated with that wafer ID. (This function allows populating the **wf_num** column in the **WAFER** table. It should be called for every wafer/wafer number combination in the data file.)

Mathematical

- ScaleFactor (int)** — Accepts one argument of type integer and has no return. Sets the value of the current scaling factor.
- Abs (real)** — Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.
- Sqr (real)** — Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.
- POW (real, int)** — Accepts two arguments — first of type real, second of type integer; and returns a real. The function returns a real, which is the result of the 1st argument (type real), raised to the power of the 2nd argument (type integer).

Debugging

- Print (...)** — Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.
- PrintToFile (...)** — Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.
- ErrorCode** — Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the error code are **GoTo**, **GoBackTo**, and **OpenFile**.
- ExitScript (string)** — Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the `DpLoad.pl` script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

System System (string) —

Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function.

Zones

The BitMap reader automatically define zones for each program. The reader uses dieX and dieY coordinates to create default zones for certain programs, when the following built-in functions are called:

- [DbCreateDefaultzones\(\)](#)
- [DbCircularZones\(string, int\)](#)
- [DbRadiusZones\(int\)](#)

DbCreateDefaultzones() — The **DbCreateDefaultzones** built-in function accepts seven arguments of type string that represent the zones for the system to create. The function has no returns.

- **RW** — Row
- **CL** — Column
- **CR** — Circle
- **RD** — Radial
- **QD** — Quadrants
- **SP** — Step
- **Z9** — 9 Zone

DbCreateDefaultzones works with the following zone types:

By Row: Zonal analysis by row displays bin or parametric results based on die row zones.

By Column: Zonal analysis by column displays bin or parametric results based on die column zones.

Circular (where n is the circular area or the circular radius): Circular zonal analysis displays bin or parametric results based on circular zones.

Radials (where n is the number of radials): Radial zonal analysis displays bin or parametric results based on radial (pie slice) zones.

Quadrant: Quadrant zonal analysis displays bin or parametric results based on quadrant zones.

Stepper Field: Stepper Field zonal analysis displays bin or parametric results by stepper field zones.

9 Zone: 9 Zone analysis displays three circles of equal radius, with the outer two circles divided into four quadrants.

DbCircularZones(string, int) —

Accepts two arguments of type string and integer. The system does not return any code or message for this built-in function.

The first argument is type string and it represents the type of circular zone: equal area or equal distance (between zone lines). Acceptable arguments are:

- area** — equal area
- distance** — equal distance between zones

The second argument is type integer and represents the number of circles.



If this built-in function is not used, the reader creates a default 10 circle chart with equal distance between the zones.

DbRadiusZones(int) —

Accepts one argument of type integer and has no returns. The argument represents the number of radials.



If this built-in function is not used, the reader creates a default 12 radial chart.

RUNNING THE *bit*MAP READER

The BitMap reader can be run in batch mode in the same fashion used to run the ASCII reader (using DpLoad.pl and a configuration file).

To run the reader in command line mode, once your format file is created and saved the BitMap reader can be run using the newly created format. If there are any errors in your format file the reader will generate an error file (err.jnk) describing the nature of the error.

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the BitMap reader, which is the format file preceded by the -fmt switch.

Example:

```
bitmap -fmt format file
```

Executing the BitMap reader in this way generates an error file (err.jnk) containing any compilation errors. (0 is an indication of no errors.)

Command-Line Arguments

If the BitMap reader is being run manually, knowing the different command-line arguments becomes necessary. The *BitMap* reader accepts the following options: (The first argument should always be the data file to be processed.)

-v	print version string
-u	usage
-db <i>database</i>	(Where <i>database</i> is the name of the database to write to).
-fmt <i>format file</i>	(Where <i>format file</i> is the format to be used).
-db_accept	To automatically set accept_data flag in database.
-file_path <i>path</i>	Sets the path to data files that may be opened using the OpenFile built-in function.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.
-stats_aging	[days] Sets test program statistics aging in days for summaries. No default value. If not provided, then it is set to NULL in the database.
-class	[program class] sets test program class (integer).

-resext <i>Kbytes</i>	Set db extents for MEM_RES tables. Acceptable range is 64 Kb-1,000,000 Kb. Default=128,000 Kb.
-wafext <i>Kbytes</i>	Set db extents for MEM_WAF tables. Acceptable range is 32 Kb-60,000 Kb. Default = 16,000 Kb.
-lotext <i>Kbytes</i>	Set db extents for MEM_LOT tables. Acceptable range is 16 Kb-10,000 Kb. Default = 1,000 Kb.
-b2dext <i>Kbytes</i>	Set db extents for MEM_B2D and MEM_P2D tables. Acceptable range is 32 Kb-100,000 Kb. MEM_B2D default = 5,000 Kb; MEM_P2D default = 1,000Kb.
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid Dbospace in the database, then [dbspace] is where the indexes will be created. Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-img	Generate thumbnail images for bitmaps.
-lowercase	Forces lot ID and wafer ID to lower case.
-uppercase	Forces lot ID and wafer ID to upper case.
-start-time	Includes start-time as part of the detection of rework.
-end-time	Includes end-time as part of the detection of rework.
-addpatts	Add an existing pattern to a pattern set, if needed.
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).

-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion. If used, this option should be followed by the maximum number of seconds to run the executable. If it is exceeded, the executable will terminate with an error.
-multimemconfig	To allow multiple memory configurations per run.

bitMAP FORMAT FILE - OVERVIEW

Proposed Data File Format

Although the BitMap reader can read from any ASCII file, the following section describes a format suggested by Exensio –Yield to use in conjunction with this reader.

- The capitalized words will be key words to look for.
- The ':' symbol will be used as a separator.
- A single program, test, product, lot, wafer and die per file is allowed.

Example Data File

This section gives an example of a data file.

example input file	comment (not to be included in actual file)
<BOH>	begin general production information
PRODUCT:ASIC_1	
TECHNOLGY:DEEP_SUBMICRON	
STEP:NA	
STAGE:NA	
EQUIPMENT:NA	
OPERATOR:Fred	
LOTID:12101	
SRCLLOT:12101.s	optional
WAFERID:12101_07	
WAFERNUM:07	
<EOH>	end general production information
<BWC>	begin wafer map configuration
WAFER_ORIENTATION:B	
WF_POS_X:R	
WF_POS_Y:U	
WAFER_DIAMETER:200000	
FLAT_TYPE:Notch	
DIE_WIDTH:7079.951171875	
DIE_HEIGHT:12899.6015625	
CENTER_DIEX:18	
CENTER_DIEY:11	
RETICLE_ROW:3	
RETICLE_COL:2	
UNIT:micron	
RET_ROW_OFFSET:0	
RET_COL_OFFSET:0	
DIEX_OFFSET:-12899.6016	
DIEY_OFFSET:-215.516006	

example input file	comment (not to be included in actual file)
<EWC>	end wafer map configuration
<BMC>	begin cell macro definition
MEMORYNAME:DP_a	
MEMORYTYPE:Dual_Port	
MEMORY_DESIGN_DESC:No Desc	
MEMORYROWS:256	
MEMORYCOLUMNS:512	
BUS_WIDTH:8	
BLOCK_COUNT:8	
TWIST_SCHEME:0	
GOM_WIDTH:1076	if unknown put 0 - will be calculated from instance
GOM_HEIGHT:540	if unknown put 0 - will be calculated from instance
<BCB>	begin block definition per cell macro. If not present, must be provided in <BOM><EOM> elements for all the instances of that macro
BLOCK_NAME:blk0	
BLOCK_NUM:0	
BLOCK_TYPE:M	
BLOCK_COLS:64	
BLOCK_ROWS:256	
X1:0	uses the cell coordinate system
Y1:0	uses the cell coordinate system
X2:104	uses the cell coordinate system
Y2:540	uses the cell coordinate system
FIRST_LOGICAL_COL:0	
FIRST_LOGICAL_ROW:0	
<ECB>	end of cell level block definition
<BCB>	begin repeats for each block within the cell macro
....	
<ECB>	
<EMC>	end of cell macro definition
<BMC>	begin next cell macro definition
.....	
<BCB>	
....	
<ECB>	
<EMC>	end of cell macro definition
<BOM>	begin memory instance for the macro given by MEMORYNAME
MEMORYNUM:0	
MEMORYNAME:DP_a	
INSTANCENAME:DP_a_1	
MEMORYDESC:No Desc	

example input file	comment (not to be included in actual file)
ROW_ORIENTATION:D COL_ORIENTATION:R X1:4046 Y1:4054.6015625 X2:5122 Y2:4594.6015625	uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION
<BOB>	begin block definition for this instance. Insert if and only if blocks are not defined for the corresponding memory macro
BLOCK_NAME:blk0 BLOCK_NUM:0 BLOCK_TYPE:M BLOCK_COLS:64 BLOCK_ROWS:256 X1:4046 Y1:4054.6015625 X2:4150 Y2:4594.6015625 FIRST_LOGICAL_COL:0 FIRST_LOGICAL_ROW:0	uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION uses the chip coordinate system according to WAFER_ORIENTATION
<EOB>	end of per instance block definition
<BOB>	next per instance block
....	
<EOB>	end of next per instance block definition
<EOM>	end of memory instance definition
<BOM>	begin definition memory instance
....	
<BOB>	begin first block definition for this next instance
...	
<EOB>	end of first block definition for this next instance
<BOB>	begin next block definition for this next instance
....	
<EOB>	end next block definition for this next instance
<EOM>	end of next memory instance definition
## End Config ##	
<BDEFL> DMEMORYCELLNAME:DP_a DLIBRARYNAME:lib1 DISDEFAULT:yes <EDEFL>	begin select pattern set for memory design pattern set name yes or no for making it a default one end select pattern set
<BDEFL> ...	begin select next pattern set

example input file	comment (not to be included in actual file)
<EDEF>	end select next pattern set
SETDEFAULTTEST: DP_a_1	for memory instance DP_a_1, the test is to be the default one from now on
<BOT>	begin of test results
TESTTIME:NA	
TESTNUM:0	
TESTNAME:Chkbrd_Vmin	
RAW_-	
FILE_NAME:"test.wsf":""":"""	
RAW_FILE_PATH:"product3/	
":"SRAM":"VDDL":"""	
ROOT_FILE_PATH:"/homes/	
":"users/test/":""""	
RAW_FILE_-	
SETUP:1:1024:2048:"UR"	
DIEX:5	
DIEY:10	
DEVICE_NUM:-1	
INSTANCENAME:DP_a_1	
MEMORYNUM:0	
<EOT>	end of test results attributes
<BFL>	begin of fail pattern instances recognized for the test results
SPOT:L3A:0:25:20:25:20:3	format: <meta_- class_tag>:<classname>:0:<min_row>:<min_col>:<max_row>:<max_col>:<fail_count>
ROW:Row1:0:5:1:5:63:60	instance of class Row1 of type ROW in row 5 from column 1 to 63 with 60 failing bits
COL:Col1:0:0:446:255:446:222	instance of class Col1 of type COL in column 446 from row 0 to row 255 with 222 failing bits
BOX:Array:0:11:0:22:13:101	instance of class Array from row 11 to 22 and column 0 to 13, with 101 failing bits
SPOT:SB:0:44:20:44:20:1	instance of SB of type SPOT in row 44 and col20
OTHER:Missed:0:24:0:44:13:20	instance of class Missed (bits not classified into any other class) of type OTHER in rows from 24 to 44 and rows from 0 to 13 with 20 failing bits
CROSS:Cross1:0:0:0:0:0:-1	Cross1 instance; it is a composite class of type CROSS; since fail count was given as -1 it will be calculated from component classes
ROW:Row1:0:3:1:3:63:60	First component is Row1 class instance
COL:Col1:0:0:55:255:55:222	Second component is Col1 class instance
ENDC	end of enumeration of components for the composite class
FOG:FogSB:0:0:0:0:0:3	Fog instance; Fog is a composite class; since fail count was given as > 0 it will be used instead of calculating from the component classes
SPOT:SB:0:22:20:22:20:1	first component is SB instance
SPOT:SB:0:28:20:28:20:1	second component is SB instance
SPOT:SB:0:16:20:16:20:1	third component is SB instance



example input file	comment (<i>not to be included in actual file</i>)
ENDC	end of enumeration of components for the composite class
MUROW:Multi- Row1:0:7:1:13:63:180 ROW:Row1:0:7:1:7:63:60 ROW:Row1:0:10:1:10:63:60 ROW:Row1:0:13:1:13:63:60 ENDC	end of enumeration of components for the composite class
MUCOL:MultiCol- umn1:0:0:11:13:22:663 COL:Col1:0:0:446:255:446:221 COL:Col1:0:0:446:255:446:221 COL:Col1:0:0:446:255:446:221 ENDC <EFL>	end of enumeration of components for the composite class
<BOT> <EOT>	
<BFL> <EFL>	

Example Format File

This section provides the format file for the previous data file.

```
//-----
// FILE : bit3.fmt
// AUTHORS : Omar Kamal
// REVISION : 1.1
//          DATE: 19-02-2001
// SCCS ID: %W %G %U
//-----
SCRIPT bit_data

VAR
  //Conditions
  TestName      string  keyCond
  TestUnit      string  Cond
  TestNumber    integer Cond
  Conditions3    integer Cond
  Conditions4    integer Cond

  //Indexes
  Wafer         string  KeyIndex

  //Results
  res           real    Result

  //Variables
  Str           string
  WaferNumber   integer
  die_X         integer
  die_Y         integer
  dev_num       integer
  wf_size       real
  flat_type     char
  flat_location char
  die_wd        real
  die_ht        real
  center_x      real
  center_y      real
  ret_row       integer
  ret_col       integer
  mem_name      string
  instance_name string

  bus_width     integer
  block_count   integer
  twist_scheme  integer
  meta_class_name string
  raw_file_name1 string
  raw_file_name2 string
  raw_file_name3 string
  raw_file_name4 string
  raw_file_path1 string
  raw_file_path2 string
  raw_file_path3 string
  raw_file_path4 string
  root_file_path1 string
```



root_file_path2	string
root_file_path3	string
root_file_path4	string
file_setup_ind	integer
file_start_row	integer
file_start_col	integer
file_transf	string

mem_num	integer
mem_desc	string
mem_type	string
mem_rows	integer
mem_cols	integer
block_rows	integer
block_cols	integer
x1	real
y1	real
x2	real
y2	real
col_orient	char
row_orient	char
fail_type	integer
fail_type_str	string
col_row	integer

fail_class_name	string
fg_invert	integer
min_col	integer
min_row	integer
max_col	integer
max_row	integer
fail_count	integer

pos_x	char
pos_y	char
test_name	string
test_num	integer

j	integer
column	integer
row	integer
fail_cnt	integer
col_unit	real
first	integer
block_name	string
block_num	integer
block_type	char
first_col	integer
first_row	integer
row_offset	integer
col_offset	integer
unit	string
ret_row_offset	integer
ret_col_offset	integer

diex_offset	real
diey_offset	real
memorycellname	string
libraryName	string
isLibDefault	string
geom_width	real
geom_height	real

BEGIN

AddSep("")
AddSep(':')
AddSep(' ')

TestName = "NA"
TestNumber = -1
TestUnit = "NA"

GoTo("<BOH>")
If (ErrorCode <> 0)
ExitScript("Missing <BOH> Token")
End If
SkipWords(-1)
Str = GetWord

GoTo("PRODUCT")
If(ErrorCode <> 0)
ExitScript("No Product - Missing PRODUCT Token")
End If
Str = DbProduct

GoTo("TECHNOLOGY")
If (ErrorCode <> 0)
Print("Warning: Missing TECHNOLOGY Token", 'NL')
ELSE
Str = DbTechnology
End If

Str = vDbProcess("Process1")

GoTo("STEP")
If (ErrorCode <> 0)
ExitScript("No Tester - Missing STEP Token")
End If
Str = DbStep

GoTo("STAGE")
If (ErrorCode <> 0)
ExitScript("No Tester - Missing STAGE Token")
End If
Str = DbStage

GoTo("EQUIPMENT")

```
If (ErrorCode <> 0)
    ExitScript("No Tester - Missing EQUIPMENT Token")
End If
Str = DbTester

GoTo("OPERATOR")
If (ErrorCode <> 0)
    Print("Warning: No Operator - Missing OPERATOR Token", 'NL')
ELSE
    Str = DbOperator
End If

GoTo("LOTID")
If (ErrorCode <> 0)
    ExitScript("Missing LOTID Token")
End If
Str = GetWord
Str = vDbLot(Str)

GoTo("SRCLOT")
If (ErrorCode <> 0)
    Print("Warning: Missing SRCLOT Token")
ELSE
    Str = GetWord
    Str = vDbSrcLot(Str)
End If

GoTo("WAFERID")
If (ErrorCode <> 0)
    ExitScript("Missing WAFERID Token")
End If
Wafer = GetWord

GoTo("WAFERNUM")
If (ErrorCode <> 0)
    ExitScript("Missing WAFERNUM Token")
End If
WaferNumber = GetInt

DbWfNum(Wafer, WaferNumber)

//Get Wafer configuration
GoTo("WAFER_ORIENTATION")
If (ErrorCode <> 0)
    ExitScript("Missing WAFER_ORIENTATION Token")
End If
flat_location = GetChar

GoTo("WF_POS_X")
If (ErrorCode <> 0)
    ExitScript("Missing WAFER_POS_X Token")
End If
pos_x = GetChar

GoTo("WF_POS_Y")
```

```
If (ErrorCode <> 0)
    ExitScript("Missing WAFER_POS_Y Token")
End If
pos_y = GetChar

GoTo("WAFER_DIAMETER")
If (ErrorCode <> 0)
    ExitScript("Missing WAFER_DIAMETER Token")
End If
wf_size = GetReal

GoTo("FLAT_TYPE")
If (ErrorCode <> 0)
    ExitScript("Missing FLAT_TYPE Token")
End If
flat_type = GetChar

GoTo("DIE_WIDTH")
If (ErrorCode <> 0)
    ExitScript("Missing DIE_WIDTH Token")
End If
die_wd = GetReal

GoTo("DIE_HEIGHT")
If (ErrorCode <> 0)
    ExitScript("Missing DIE_WIDTH Token")
End If
die_ht = GetReal

GoTo("CENTER_DIEX")
If (ErrorCode <> 0)
    ExitScript("Missing CENTER_DIEX Token")
End If
center_x = GetReal

GoTo("CENTER_DIEY")
If (ErrorCode <> 0)
    ExitScript("Missing CENTER_DIEY Token")
End If
center_y = GetReal

GoTo("RETICLE_ROW")
If (ErrorCode <> 0)
    ExitScript("Missing RETICLE_ROW Token")
End If
ret_row = GetInt

GoTo("RETICLE_COL")
If (ErrorCode <> 0)
    ExitScript("Missing RETICLE_COL Token")
End If
ret_col = GetInt

GoTo("UNIT")
If (ErrorCode <> 0)
```



```

        ExitScript("Missing UNIT Token")
    End If
    unit = GetWord

    GoTo("RET_ROW_OFFSET")
    If (ErrorCode <> 0)
        ExitScript("Missing RET_ROW_OFFSET Token")
    End If
    ret_row_offset = GetInt

    GoTo("RET_COL_OFFSET")
    If (ErrorCode <> 0)
        ExitScript("Missing RET_COL_OFFSET Token")
    End If
    ret_col_offset = GetInt

    GoTo("DIEX_OFFSET")
    If (ErrorCode <> 0)
        //Print("Warning:Missing RET_ROW_OFFSET Token")
        diex_offset = 0.0
    Else
        diex_offset = GetReal
    End If

    GoTo("DIEY_OFFSET")
    If (ErrorCode <> 0)
        //Print("Warning:Missing RET_COL_OFFSET Token")
        diey_offset = 0.0
    Else
        diey_offset = GetReal
    End If

    vDbWmapCfg(wf_size, unit,
        flat_location, flat_type, die_wd, die_ht,
        center_x, center_y, pos_x,
        pos_y, ret_row, ret_col,
        ret_row_offset, ret_col_offset, diex_offset, diey_offset)

    GoTo("<BMC>")
    If (ErrorCode <> 0)
        Print("getting memory definitions from DB")
    Else
        SkipWords(-1)
        Str = GetWord

        While (Str = "<BMC>")

            GoTo("MEMORYNAME")
            If (ErrorCode <> 0)
                ExitScript("Missing MEMORYNAME Token")
            End If

```

```
mem_name = GetWord

GoTo("MEMORYTYPE")
If (ErrorCode <> 0)
    ExitScript("Missing MEMORYTYPE Token")
End If
mem_type = GetWord

GoTo("MEMORY_DESIGN_DESC")
If (ErrorCode <> 0)
    ExitScript("Missing MEMORY_DESIGN_DESC Token")
End If
mem_desc = GetWord

GoTo("MEMORYROWS")
If (ErrorCode <> 0)
    ExitScript("Missing MEMORYROWS Token")
End If
mem_rows = GetInt

GoTo("MEMORYCOLUMNS")
If (ErrorCode <> 0)
    ExitScript("Missing MEMORYCOLUMNS Token")
End If
mem_cols = GetInt

GoTo("BUS_WIDTH")
If (ErrorCode <> 0)
    ExitScript("Missing BUS_WIDTH Token")
End If
bus_width = GetInt

GoTo("BLOCK_COUNT")
If (ErrorCode <> 0)
    ExitScript("Missing BLOCK_COUNT Token")
End If
block_count = GetInt

GoTo("TWIST_SCHEME")
If (ErrorCode <> 0)
    ExitScript("Missing TWIST_SCHEME Token")
End If
twist_scheme = GetInt

GoTo("GEOM_WIDTH")
If (ErrorCode <> 0)
    geom_width = 0.0
Else
    geom_width = GetReal
End If

GoTo("GEOM_HEIGHT")
If (ErrorCode <> 0)
    geom_height = 0.0
Else
```

```

        geom_height = GetReal
    End If

    vDBAddMemCellClass(mem_name, mem_type, mem_desc,
        mem_rows, mem_cols,
        bus_width, block_count, twist_scheme,
        geom_width, geom_height)

    SkipWords(-1)

    GoTo("<BCB>")
    If (ErrorCode = 0)

        SkipWords(-1)
        Str = "<BCB>"

        While (Str = "<BCB>")
            GoTo("BLOCK_NAME")
            If (ErrorCode <> 0)
                ExitScript("Missing BLOCK_NAME Token")
            End If
            block_name = GetWord

            GoBackTo("<BCB>")

            GoTo("BLOCK_NUM")
            If (ErrorCode <> 0)
                ExitScript("Missing BLOCK_NUM Token")
            End If
            block_num = GetInt

            GoBackTo("<BCB>")

            GoTo("BLOCK_TYPE")
            If (ErrorCode <> 0)
                ExitScript("Missing BLOCK_TYPE Token")
            End If
            block_type = GetChar

            GoBackTo("<BCB>")

            GoTo("BLOCK_COLS")
            If (ErrorCode <> 0)
                ExitScript("Missing BLOCK_COLS Token")
            End If
            block_cols = GetInt

            GoBackTo("<BCB>")

            GoTo("BLOCK_ROWS")
            If (ErrorCode <> 0)
                ExitScript("Missing BLOCK_ROWS Token")
            End If
            block_rows = GetInt

```

```
GoBackTo("<BCB>")
```

```
GoTo("X1")  
If (ErrorCode <> 0)  
    ExitScript("Missing block X1 Token")  
End If  
x1 = GetReal
```

```
GoBackTo("<BCB>")
```

```
GoTo("Y1")  
If (ErrorCode <> 0)  
    ExitScript("Missing block Y1 Token")  
End If  
y1 = GetReal
```

```
GoBackTo("<BCB>")
```

```
GoTo("X2")  
If (ErrorCode <> 0)  
    ExitScript("Missing block X2 Token")  
End If  
x2 = GetReal
```

```
GoBackTo("<BCB>")
```

```
GoTo("Y2")  
If (ErrorCode <> 0)  
    ExitScript("Missing block Y2 Token")  
End If  
y2 = GetReal
```

```
GoBackTo("<BCB>")
```

```
GoTo("FIRST_LOGICAL_COL")  
If (ErrorCode <> 0)  
    ExitScript("Missing FIRST_LOGICAL_COL Token")  
End If  
first_col = GetInt
```

```
GoBackTo("<BCB>")
```

```
GoTo("FIRST_LOGICAL_ROW")  
If (ErrorCode <> 0)  
    ExitScript("Missing FIRST_LOGICAL_ROW Token")  
End If  
Str = GetWord  
  
first_row = StrToInt(Str)
```

```

vdbMemCellBlockConfig(block_name,
                        block_num,    block_type,
                        block_cols,    block_rows,
                        x1,            y1,
                        x2,            y2,
                        first_col,     first_row)

GoTo("<ECB>")
If (ErrorCode <> 0)
    ExitScript("Missing <ECB> Token")
End If

Str = GetWord
End While // block definition loop

//end blocks
SkipWords(-1)
End If //<BCB>

GoTo("<EMC>")
If (ErrorCode <> 0)
    ExitScript("Missing <EMC> Token")
End If

Str = GetWord

End While // <BMC>

SkipWords(-1)
GoTo("<BOM>")
If (ErrorCode <> 0)
    ExitScript("Missing <BOM> Token")
End If
SkipWords(-1)
Str = GetWord

While (Str = "<BOM>")
    GoTo("MEMORYNUM")
    If (ErrorCode <> 0)
        ExitScript("Missing MEMORYNUM
                    Token")
    End If
    mem_num = GetInt

    GoTo("MEMORYNAME")
    If (ErrorCode <> 0)
        ExitScript("Missing MEMORYNAME Token")
    End If
    mem_name = GetWord

    GoTo("INSTANCENAME")
    If (ErrorCode <> 0)
        Print(GetWord)
    End If
End While

```

```
ExitScript("Missing INSTANCENAME Token ")
End If
instance_name = GetWord

GoTo("MEMORYDESC")
If (ErrorCode <> 0)
ExitScript("Missing MEMORYDESC Token")
End If
mem_desc = GetWord

GoTo("ROW_ORIENTATION")
If (ErrorCode <> 0)
ExitScript("Missing ROW_ORIENTATION Token")
End If
row_orient = GetChar

GoTo("COL_ORIENTATION")
If (ErrorCode <> 0)
ExitScript("Missing COL_ORIENTATION Token")
End If
col_orient = GetChar

GoTo("X1")
If (ErrorCode <> 0)
ExitScript("Missing X1 Token")
End If
x1 = GetReal

GoTo("Y1")
If (ErrorCode <> 0)
ExitScript("Missing Y1 Token")
End If
y1 = GetReal

GoTo("X2")
If (ErrorCode <> 0)
ExitScript("Missing X2 Token")
End If
x2 = GetReal

GoTo("Y2")
If (ErrorCode <> 0)
ExitScript("Missing Y2 Token")
End If
y2 = GetReal

vDbMemConfig(mem_name, instance_name,
             mem_num, mem_desc,
             x1, y, x2, y2,
             col_orient, row_orient)

GoTo("<BOB>")
If (ErrorCode = 0)
```

```

SkipWords(-1)
Str = GetWord

While (Str = "<BOB>")
  GoTo("BLOCK_NAME")
  If (ErrorCode <> 0)
    ExitScript("Missing BLOCK_NAME Token")
  End If
  block_name = GetWord
  //Print( "___2___ ", block_name, "___2___ ")

GoBackTo("<BOB>")

  GoTo("BLOCK_NUM")
  If (ErrorCode <> 0)
    ExitScript("Missing
                BLOCK_NUM Token")
  End If
  block_num = GetInt

  GoBackTo("<BOB>")

  GoTo("BLOCK_TYPE")
  If (ErrorCode <> 0)
    ExitScript("Missing
                BLOCK_TYPE Token")
  End If
  block_type = GetChar

GoBackTo("<BOB>")

  GoTo("BLOCK_COLS")
  If (ErrorCode <> 0)
    ExitScript("Missing BLOCK_COLS Token")
  End If
  block_cols = GetInt

GoBackTo("<BOB>")

  GoTo("BLOCK_ROWS")
  If (ErrorCode <> 0)
    ExitScript("Missing
                BLOCK_ROWS Token")
  End If
  block_rows = GetInt

  GoBackTo("<BOB>")

  GoTo("X1")
  If (ErrorCode <> 0)
    ExitScript("Missing block X1
                Token")
  End If
  x1 = GetReal

```

```
GoBackTo("<BOB>")

GoTo("Y1")
If (ErrorCode <> 0)
    ExitScript("Missing block Y1
                Token")
End If
y1 = GetReal

GoBackTo("<BOB>")

GoTo("X2")
If (ErrorCode <> 0)
    ExitScript("Missing block X2
                Token")
End If
x2 = GetReal

GoBackTo("<BOB>")

GoTo("Y2")
If (ErrorCode <> 0)
    ExitScript("Missing block Y2
                Token")
End If
y2 = GetReal

GoBackTo("<BOB>")

GoTo("FIRST_LOGICAL_COL")
If (ErrorCode <> 0)
    ExitScript("Missing
                FIRST_LOGICAL_COL Token")
End If
first_col = GetInt

GoBackTo("<BOB>")

GoTo("FIRST_LOGICAL_ROW")
If (ErrorCode <> 0)
    ExitScript("Missing
                FIRST_LOGICAL_ROW Token")
End If
Str = GetWord

first_row = StrToInt(Str)

vdbMemBlockConfig(block_name,
                  block_num, block_type,
                  block_cols, block_rows,
                  x1, y1,
                  x2, y2,
                  first_col, first_row)
```



```

GoTo("<EOB>")
If (ErrorCode <> 0)
    ExitScript("Missing <EOB>
                Token")
End If

Str = GetWord
End While // block definition loop
End If

//end blocks
SkipLines(-1)

GoTo("<EOM>")
If (ErrorCode <> 0)
    Str = GetWord
    Print("current word is: ")
    Print(Str, 'NL')
    ExitScript("Missing <EOM> Token")
End If

Str = GetWord

End While // <EOM>
End If //getting memory form file if <BOM> tag is provided
SkipLines(-1)
GoTo("<BDEFL>")
If (ErrorCode <> 0)
    Print("Warning: Missing pattern setup implicit mechanism assumed")
Else
    Str = "<BDEFL>"
    While (Str = "<BDEFL>")
        GoTo("DMEMORYCELLNAME")
        If (ErrorCode <> 0)
            ExitScript("Missing DMEMORY
                        CELLNAME Token")
        End If

        memoryCellName = GetWord

        GoTo("DLIBRARYNAME")
        If (ErrorCode <> 0)
            ExitScript("Missing DLIBRARYNAME Token")
        End If

        libraryName = GetWord

        GoTo("DISDEFAULT")
        If (ErrorCode <> 0)
            ExitScript("Missing DISDEFAULT Token")
        End If

        isLibDefault = GetWord

        vdbPatternSet(libraryName, memoryCellName, isLibDefault)

```

```

        Str = GetWord

    End While //<BDEFL>

End If

    SkipLines(-1)
    // <BOT> word or SETDEFAULTTEST follows directly <EOM> word or the
    optional <BDEFL>

    GoTo("SETDEFAULTTEST")
    If (ErrorCode = 0)
        Str = "SETDEFAULTTEST"
        While (Str = "SETDEFAULTTEST")
            Str = GetWord
            vdbSetDefaultTest(Str)
            Str = GetWord
        End While
    End If

    SkipLines(-1)
    GoTo("<BOT>")
    If (ErrorCode <> 0)
        ExitScript("Missing <BOT> Token")
    Else
        Str = "<BOT>"
    End If

    While (Str = "<BOT>")
        raw_file_name1 = ""
        raw_file_name2 = ""
        raw_file_name3 = ""
        raw_file_name4 = ""
        raw_file_path1 = ""
        raw_file_path2 = ""
        raw_file_path3 = ""
        raw_file_path4 = ""
        root_file_path1 = ""
        root_file_path2 = ""
        root_file_path3 = ""
        root_file_path4 = ""
        file_setup_ind = 0
        file_start_row = 0
        file_start_col = 0
        file_transf = "UR"

        GoTo("TESTNAME")
        If (ErrorCode <> 0)
            ExitScript("Missing TESTNAME
                        Token")
        End If
        TestName = GetWord
        GoBackTo("<BOT>")
    End While

```

```

GoTo("TESTNUM")
If (ErrorCode <> 0)
    ExitScript("Missing TESTNUM
                Token")
End If
TestNumber = GetInt
GoBackTo("<BOT>")

GoTo("TESTTIME")
If (ErrorCode <> 0)
    ExitScript("No Time - Missing
                TESTTIME Token")
End If
Str = DbEndTime
    ("%Y-%m-%d %H-%M-%S")
GoBackTo("<BOT>")

    GoTo("DIEX")
    If (ErrorCode <> 0)
        ExitScript("Missing DIEX Token")
    End If
    die_X = GetInt
GoBackTo("<BOT>")

    GoTo("DIEY")
    If (ErrorCode <> 0)
        ExitScript("Missing DIEY Token")
    End If
    die_Y = GetInt
GoBackTo("<BOT>")

    GoTo("DEVICE_NUM")
    If (ErrorCode <> 0)
ExitScript("Warning: Missing DEVICE_NUM
            Token")
    End If
    dev_num = GetInt
GoBackTo("<BOT>")

    GoTo("INSTANCENAME")
    If (ErrorCode <> 0)
        Print(GetWord)
        ExitScript("2Missing INSTANCENAME Token ")
    End If
    instance_name = GetWord
        GoBackTo("<BOT>")

    GoTo("MEMORYNUM")
    If (ErrorCode <> 0)
        ExitScript("Missing MEMORYNUM Token")
    End If
    mem_num = GetInt

GoBackTo("<BOT>")

```

```

GoTo("RAW_FILE_NAME")
If (ErrorCode = 0)
    SkipChars(-1)
raw_file_name1 = GetQuotedWord(':')
    SkipChars(-1)
raw_file_name2 = GetQuotedWord(':')
    SkipChars(-1)
raw_file_name3 = GetQuotedWord(':')
    SkipChars(-1)
If GetLineLen > 1
    raw_file_name4 = GetWord
    Else
        raw_file_name4 = ""
    End If

    GoBackTo("<BOT>")

GoTo("RAW_FILE_PATH")
If (ErrorCode <> 0)
    ExitScript("Missing RAW_FILE_PATH Token ")
End If
    SkipChars(-1)
raw_file_path1 = GetQuotedWord(':')
    SkipChars(-1)
raw_file_path2 = GetQuotedWord(':')
    SkipChars(-1)
raw_file_path3 = GetQuotedWord(':')
    SkipChars(-1)
If GetLineLen > 1
    raw_file_path4 = GetWord
    Else
        raw_file_path4 = ""
    End If

    GoBackTo("<BOT>")

GoTo("ROOT_FILE_PATH")
If (ErrorCode <> 0)
    ExitScript("Missing ROOT_FILE_PATH Token ")
End If
    SkipChars(-1)
root_file_path1 = GetQuotedWord(':')
    SkipChars(-1)
root_file_path2 = GetQuotedWord(':')
    SkipChars(-1)
root_file_path3 = GetQuotedWord(':')
    SkipChars(-1)
If GetLineLen > 1
root_file_path4 = GetWord
    Else
        root_file_path4 = ""
    End If
    GoBackTo("<BOT>")

```

```

      GoTo("RAW_FILE_SETUP")
      If (ErrorCode <> 0)
ExitScript("Missing RAW_FILE_SETUP Token ")
End If

      Str = GetWord
      If(IsNumber(Str))
        file_setup_ind = StrToInt(Str)
      Else
ExitScript("file_setup_ind required in
          RAW_FILE_SETUP")
      End If

      Str = GetWord
      If(IsNumber(Str))
        file_start_row = StrToInt(Str)
      Else
ExitScript("file_start_row required in
          RAW_FILE_SETUP")
      End If

      Str = GetWord
      If(IsNumber(Str))
        file_start_col = StrToInt(Str)
      Else
ExitScript("file_start_col required in
          RAW_FILE_SETUP")
      End If

file_transf = GetWord

      vdbRawFile(raw_file_name1, raw_file_name2, raw_file_name3,
        raw_file_name4)
      vdbRawFilePath(raw_file_path1, raw_file_path2, raw_file_path3,
        raw_file_path4)
      vdbRootFilePath(root_file_path1, root_file_path2, root_file_path3,
        root_file_path4)
      vdbFileSetup(file_setup_ind, file_start_row, file_start_col,
        file_transf)
    Else
      Print("Warning: Missing RAW_FILE_NAME Token ")
    End If

//      Print('NL',"raw_file_name:")
//      Print('NL',raw_file_name)
//      Print('NL',raw_file_path1)
//      Print('NL',raw_file_path2)
//      Print('NL',raw_file_path3)
//      Print('NL',raw_file_path4,'NL')
//      Print('NL', "kuku")
//      Print('NL',raw_file_name1)
//      Print('NL',raw_file_name2)
//      Print('NL',raw_file_name3)
//      Print('NL',raw_file_name4)
//      Print('NL',raw_file_path1)

```

```

//          Print('NL',raw_file_path2)
//          Print('NL',raw_file_path3)
//          Print('NL',raw_file_path4)
//          Print('NL',root_file_path1)
//          Print('NL',root_file_path2)
//          Print('NL',root_file_path3)
//          Print('NL',root_file_path4)
//          Print('NL',file_setup_ind)
//          Print('NL',file_start_row)
//          Print('NL',file_start_col)
//          Print('NL',file_transf)

          GoBackTo("<BOT>")
          GoTo("<EOT>")
          If (ErrorCode <> 0)
ExitScript("Missing <EOT> Token")
          End If

          vDbDieMemory(die_X, die_Y, dev_num, instance_name,
mem_num)

          fail_type = 2

          GoTo("<BFL>")
          If (ErrorCode <> 0)
              ExitScript("Missing <BFL> Token")
          End If

          Str = GetWord
          While (Str <> "<EFL>")

              //here dispatch for fail bit reading
              meta_class_name = Str
              //Print("__Pattern Instance__")
              //Print(meta_class_name, 'NL')
              //Print(" ")
              If ((meta_class_name = "SPOT") OR (meta_class_name = "ROW")
                  OR (meta_class_name = "COL") OR (meta_class_name = "BOX")
                  OR (meta_class_name = "CUST") OR (meta_class_name =
                      "OTHER")
                  OR (meta_class_name = "FOG") OR (meta_class_name = "CROSS")
                  OR (meta_class_name = "MUROW") OR (meta_class_name =
                      "MUCOL")
                  OR (meta_class_name = "BLI"))

                  fail_class_name = GetWord

                  Str = GetWord
                  If(IsNumber(Str))
                      fg_invert = StrToInt(Str)

                  Else
                      ExitScript("fg_invert required in class instance")
                  End If

```

```

        Str = GetWord
        If(IsNumber(Str))
            min_row = StrToInt(Str)
    Else
        ExitScript("min_row required in class instance")
    End If

        Str = GetWord
        If(IsNumber(Str))
            min_col = StrToInt(Str)
    Else
        ExitScript("min_col required in class instance")
    End If

        Str = GetWord
        If(IsNumber(Str))
            max_row = StrToInt(Str)
    Else
        ExitScript("max_row required in class instance")
    End If

        Str = GetWord
        If(IsNumber(Str))
            max_col = StrToInt(Str)
    Else
        ExitScript("max_col required in class instance")
    End If

        Str = GetWord
        If(IsNumber(Str))
            fail_count = StrToInt(Str)
    Else
        ExitScript("fail_count required in class instance")
    End If

    If (meta_class_name = "SPOT")
        vDbLogSpot(fail_class_name, min_col, min_row, max_col,
            max_row)
    Else
        If (meta_class_name = "ROW")
            vDBLogRow(fail_class_name, min_col, min_row, max_col,
                max_row, fail_count)
        Else
            If (meta_class_name = "COL")
                vDBLogColumn(fail_class_name, min_col, min_row, max_col,
                    max_row, fail_count)
            Else
                If (meta_class_name = "BOX")
                    vDBLogBox(fail_class_name, fg_invert, min_col, min_row,
                        max_col, max_row, fail_count)
                Else
                    If (meta_class_name = "CUST")
                        vDBLogCustom(fail_class_name, fg_invert, min_col, min_row,
                            max_col, max_row, fail_count)
                    Else

```

```

    If (meta_class_name = "OTHER")
        vDBLogMiss(fail_class_name, fg_invert, min_col, min_row,
            max_col, max_row, fail_count)
    Else
    If (meta_class_name = "CROSS")
        vDBLogCross(fail_class_name, fail_count)
    Else
    If (meta_class_name = "FOG")
        vDBLogFog(fail_class_name, min_col, min_row, max_col,
            max_row, fail_count)
    Else
    If (meta_class_name = "MUROW")
        vDBLogMultiRow(fail_class_name, fail_count)
    Else
    If (meta_class_name = "MUCOL")
        vDBLogMultiColumn(fail_class_name, fail_count)
    //Else
        //Print('NL', "--- ignoring metaclass token", meta_class_name, " for
            now", 'NL')
        End If

    Else If (meta_class_name = "ENDC")
        vDBEndComposite
    Else If meta_class_name = "BLI"
        print("Skip BLI.", 'NL')
    Else
        Print("Bad Meta Class Token name: ", meta_class_name,
            'NL')
        ExitScript("Bad Meta Class Token")
        End If

    Str = GetWord

    End While //fail list while loop

    SkipLines(-1)
    GoTo("<BOT>")
    If (ErrorCode <> 0)
        Str = "END"
    Else
        Str = "<BOT>"
    End If

    End While //test while loop
END

```

Storing Raw BitMap File Information

The .bit file within the <BOT> section contains the following attributes:

RAW_FILE_NAME:<part1>:<part2>:<part3>:<part4>

RAW_FILE_PATH:<part1>:<part2>:<part3>:<part4>



ROOT_FILE_PATH:<part1>:<part2>:<part3>:<part4>

RAW_FILE_SETUP:<index>:<start_col>:<start_row>:<transf>

There are four separate fields for the file and path variables because, in the format file, the size of string is currently limited to 63 characters.

RAW_FILE_SETUP is used to pass the FileSetup information for a given memory as provided in **fileSetupFile** in the MemBrain input. Additionally, <index>, provides the ordinal number (counted from 1) for a die within a given raw bitmap file. This is useful for file formats, such as BIM, that can store bitmap data for the entire wafer. In such a case, the <index> identifies the die counting from the beginning of the file (starting from 1). The following four attributes — <start_col>:<start_row>:<transf>:<setup_name> — provide for the current memory instance of the FileSetup attributes **InputBitmapSplitStartRow**, **InputBitmapSplitStartColumn**, **InputBitmapOrientation**, **InputBitmapSetupName**, respectively.

The above attributes are optional. In the reference format file implementation, either none of the attributes exists or all of them exist.

For example, the file:"/homes/users/test/product3/SRAM/VDDL/test.wsf"

Could be stored as follows:

RAW_FILE_NAME:"test.wsf":"":"":""

RAW_FILE_PATH:"product3/":"SRAM":"VDDL/":""

ROOT_FILE_PATH:"/homes/":"users/test/":"":""

RAW_FILE_SETUP:1:1024:2048:"UR":"SNAME"



FRONT



CONTENTS



INDEX

CHAPTER 11

EVENTS READER

IMPORTING DATA

All Exensio –Yield readers function as the device for importing raw datalog files into the analysis environment. The objective of the reader is to organize data into a standard form that can be handled in the Exensio –Yield environment, regardless of how the data is formatted in the original file.

The Exensio –Yield Events reader imports data directly into the database.



All data points between -1e-38 and 1e-38 are considered by the system to be tester error codes, and therefore invalid data. Data points in these ranges will be loaded as NULL in the Exensio –Yield readers.



<TAB> is a reserved character in Exensio and should not be part of the parameter names of data files.

Events Data Retrieval

Data retrieval into the Event reader tool is accomplished with the Bring-in interface in Exensio –Yield. The functionality of the Events Bring-In window is fully described in “*Bring-In Data Tools*” section of the Exensio –Yield End-User Manual, Retrieval chapter. The primary function of the window is to supply customized retrieval options so the end-user can easily populate a worksheet with events data, and then use the appropriate tool for analysis.

EVENTS FORMAT FILE — OVERVIEW

The Exensio –Yield format file is a way to specify the data format in which the data will be expected from an incoming source (usually a disk file). The format file is written in an easy to use script language specifically designed to navigate through an data file and extract all relevant information, storing it in a pre-defined database. The language has a set of keywords ([“Keywords,”](#) pg. 469) and built-in functions ([“Built-in Functions,”](#) pg. 479) and it allows for user-defined variables, constants and separators. Blanks, horizontal and vertical tabs, new-lines, form-feeds and comments as described below (collectively referred to as “white space”) serve only to separate tokens.

OBJECTIVE

The objective of the Events reader is to import data into a Exensio –Yield table, organizing it into a standard form regardless of how the data is organized in the original data file. First the data is stored in the database and later retrieved into a Exensio –Yield table.

Conditions and Indexes — Definitions

conditions	— the column headings in the raw data table. These conditions are the identifiers for parameters that uniquely identify each column.
key conditions	— the group of conditions that uniquely identify the data column. The combination of all key conditions for any particular data column will be unique to that column.
indexes	— the row headings in the raw data table. Each row in the Data Table represents an event that uses a “tag” or index. The index of a data row contains information about the event.
key indexes	— the group of indexes that uniquely identify the data row. The combination of all key indexes for any particular data row will be unique to that row.

LEXICAL CONVENTIONS

Tokens There are six classes of tokens. Identifiers, keywords, constants, string literals, operators and separators. “White Space,” as described previously, is used to separate tokens.

Identifiers An identifier is a sequence of letters and digits including the under-score character. The first character of an identifier must be a letter. The language does not differentiate between upper and lower case letters.

Keywords The following identifiers are reserved for use as keywords and may not be used otherwise.

Cond	True	OR	CharIf	Target
KeyCond	False	AND	IntegerElse	LOL
LimCond	EQ	ConstReal	Exit	HOL
Index	NE	Var	String	LSL
KeyIndex	NOT	Begin	Boolean	HSLHWL
Result	LE	End	While	LPL
FileName	GE	Step	For	HPL
Script	LT	GT	To	Tune
				mod

All built-in functions described in “*Built-in Functions*,” pg. 479, also constitute part of the list of key words.

Comments The characters /* introduce a comment that terminates with the characters */, such comments do not nest and they can not occur within strings. The characters // introduce a comment that terminates with the end-of-line character.

FORMAT FILE BLOCKS

The format file is made up of several declaration blocks and one main processing block. The blocks provide a logical grouping of the keywords and commands. Keywords and commands dictate how the block will function and what information is to be retrieved within that particular block.

Each format file should begin with the keyword “Script”, followed by an identifier which serves as the format file name.

Constants

The value of a keyword may be known, however not noted in the input stream. Under these circumstances you may assign the keyword a “constant” value. A common example is if the programmer knows the name of the test program that processed the data but the input stream does not reflect this value anywhere. If a file is to contain constants, these are included in the beginning of the file as the first block starting with the keyword CONST.

There are several kinds of constants, each having a data type. (Data types are discussed in the following section). The basic types are:

Character Constants — A Character Constant consists of one character enclosed in single quotes. Some special characters, like new-lines can not be represented in this form. The following can be used instead:

New Line	‘NL’
Horizontal Tab	‘HT’
Vertical Tab	‘VT’
Form Feed	‘FF’
Carriage Return	‘CR’
Single Quote	‘SQ’

Integer Constants — An integer constant consists of a sequence of digits. Negative Integer constants are preceded by the “-” sign.

Real Constants — A Real Constant consists of an integer part, a decimal point and a fraction part. both the integer part and the fraction part consist of a sequence of digits. Negative Real Constants are preceded by the “-” sign.

String Constants — A string constant (or lateral) is a sequence of characters enclosed in double quotes.

An identifier is assigned a constant value using the “=” operator. An example would look like this:

```
CONST
  job_nam      =      "MyJob"      //String Constant
  Product      =      "MyProduct" //String Constant
  NumOfBins    =      32             //Integer Constant
```

Variables

The following types of variables are supported:

- Integer (Four Bytes)
- Real (Four Bytes)
- Char (One Byte)
- String (Maximum length of 255 characters)



The Events reader is the only reader with maximum string length 254. All other readers have a maximum string length of 64.

- Boolean (Can only take on the values True or False)

The variables block starts with the keyword “VAR” following the “CONST” block. A variable is declared using the following syntax:

```
Variable1, Variable2, ...Type
```

For example, to declare “TestNum” as an integer:

```
TestNum Integer
```

To declare “TestName” as a string:

```
TestName String
```

Arrays

One-Dimensional Arrays

One-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N the variable name is followed by brackets enclosing the size N.

For example, to declare “VCC” as an array of 6 integers:

```
VCC[6] integer
```

To declare “Pin” as an array of 8 strings:

```
Pin[8] string
```

The indexing for arrays starts at one. The Nth member of an array is “array[N]”.

For example to assign to an integer “Cond1” the fifth member of the array “Temp”:

```
Cond1 = Temp[5]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Two-Dimensional Arrays

Two-dimensional arrays of all variable types (excluding conditions, indexes and results) are supported. To declare an array of size N, M the variable name is followed by brackets enclosing the sizes N, M.

For example, to declare “VCC” as an array of 2, 3 integers:

```
VCC[2,3] integer
```

To declare “Pin” as an array of 2, 4 strings:

```
Pin[2,4] string
```

The indexing for arrays starts at one.

For example to assign to an integer “Cond1” to elements 2, 3 of the array “VCC”:

```
Cond1 = VCC[2,3]
```

If the declared bounds of an array are exceeded, the array is automatically reallocated to double the original or current length.

Conditions and Indexes

Declaring conditions and indexes is done in the VAR block and is very similar to declaring other variables, the following syntax is used:

```
Variable1,Variable2,...TypeCond// To declare a condition
Variable1,Variable2,...TypeKeyCond// To declare a key condition
Variable1,Variable2,...TypeLimCond// To declare a limit condition
Variable1,Variable2,...TypeIndex// To declare an index
Variable1,Variable2,...TypeKeyIndex// To declare a key index
```

A minimum of one key condition and one key index must be included in the VAR block. The allowed types for conditions and indexes are Real, Integer and String. Also, note that the order that these conditions appear in is important. The first three conditions should always be:

```
dB Reader —
Tname      String      (For Test Name)
Unit       String      (For Units)
TestNum    Integer     (For Test Number)
```



*The second condition — **Unit** — should not be set as **KeyCond** or **LimCond**. It is hard-coded to non-key condition in the reader.*

Results

The Events data reader supports choosing more than one result each of a possibly different type. Results are declared in the VAR block and the following syntax is used:

```
Variable1,Variable2,...TypeResult// To declare a result
```

Every format file must have at least one result declared.

The Main Block The main processing block starts with the keyword BEGIN and ends with the keyword END. Every program must have these two keywords which enclose everything not in the declaration blocks discussed above.

ASSIGNMENT OF VARIABLES

The format file usually contains a number of variables including the conditions, indexes and results. The user may specify that the value of a certain variable be assigned by the input stream or by other means which will be covered later. Formats are built assuming that the programmer knows the Events file format that the data will be coming as. With this format in mind, the programmer will construct a format file that guides the Events data reader through the data file.

ASSIGNMENT USING “=”

Any variable can be assigned a value which is the result of an expression using the “=” operator. An expression is any combination of operators, variables and possibly function calls. For example if the input stream looked like this:

6 23

and we want to assign to the condition “cond1” the sum of these two numbers multiplied by a factor of 3, we would write in the format file the following:

cond1 = 3*(GetInt + GetInt)

where GetInt is one of the built-in functions, to be described later, that retrieves an integer from the input stream.

ASSIGNMENT FROM THE INPUT STREAM FILE NAME

A variable may also be assigned a value from the data file's actual file name. This is useful if multiple data files are being processed for one data set and each file name, or portion of the file name, contains data that is to be assigned to a variable.

To assign the file name to a variable use the pre-defined keyword “FileName” which is of type string. For example to assign the Index lot_id the value of “FileName” one would

write:

lot_id = FileName//lot_id must be of type string

In this example the keyword “lot_id” will be assigned the value of the current data file's name. Since FileName is of type string it can be used in conjunction with all pre-defined string functions and operators.

ORACLE ROLLBACK TABLE

With an Oracle database, when a reader error is detected and the reader exits unexpectedly, for reasons such as:

- Connection to the Oracle engine abruptly lost.
- The user disrupting the reader by pressing **Ctrl+C** multiple times.

...the reader may exit before the buffer has fully executed.

When this happens, a rollback sequence takes place to prevent the Oracle database from becoming corrupted. The rollback statements are stored in a dynamic temporary table, `DP_ROLLBACK_STMTS_...`, per program.

If the reader exits without fully executing the rollback statements, the table will continue to hold the remaining rollback statements, for that `pg_key`. This allows the reader to recover the rollback statements and execute them, should the above exit cases occur.

When a premature exit occurs, this rollback recovery operation is triggered automatically; the administrator does not have to perform any function to make this operation occur.

INVALID DATA

Results, conditions and indexes can sometimes assume values in the data file that are invalid, and should not be logged to the data table. To achieve this a list of invalid data values is maintained that the user can add to or delete from using the built-in functions described in detail in the built-in functions section.

As an example if the result 999.99 is used to designate invalid data in the data file it can be added to the list of invalid data by calling the function **AddInvReal(999.99)**.

SEPARATORS

A list of separators is needed to indicate to the data reader where one string ends and another begins in the data file. The new-line character, carriage return and tab are the default separators. The user can add to the list of separators, but can not remove the defaults. Three functions are available for adding a separator, deleting a separator and clearing all user defined separators. these functions are described in detail in the built-in functions section. A separator is only one character and a maximum of five separators can be added by the user.

For example to add the colon to the list of separators:

AddSep(',')

OPERATORS

<i>Arithmetic:</i>	Addition	+
	Subtraction	-
	Multiplication	*
	Division	/
	Assignment	=
	Modulus	mod

Logical: Used only with variables of type Boolean.

Logical Or	OR
Logical And	AND
Negation	NOT

<i>Relational:</i>	Equal	EQ	or	=
	Not equal	NE	or	<>
	Greater than or equal	GE	or	>=
	Greater than	GT	or	>
	Less than or equal	LE	or	<=
	Less than	LT	or	<

LOOP CONTROL

Iteration statements let you loop through a set of statements. The language supports two forms of iteration: While and For loops.

For: The general format for this statement is:

```
For initialization-expression To conditional-expression  
    statements  
End For
```

The **initialization-expression** initializes a loop counter. The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of For statements may be nested.

While: The general format for this statement is:

```
While conditional-expression  
    statements  
End While
```

The loop statements are executed repeatedly until the **conditional-expression** compares equal to FALSE. Any number of While statements may be nested.

CONDITIONAL CONTROL

Conditional control refers to selecting from alternative courses of action by testing certain values. There is one type of selection statement, the If ... Else.

If ... Else: The general format for this statement is:

```
If conditional-expression  
    if-statements  
Else  
    else-statements  
End If
```

The **if-statements** are executed if the **conditional-expression** evaluates to True otherwise the **else-statements** are executed. Any number of If ... Else statements may be nested. A nested “Else If” should not end with an “End if”.

BUILT-IN FUNCTIONS

All functions that do not accept any arguments do not end with “()”.

<i>Definitions</i>	File Pointer —	The current location in the data file.
	Word —	Any sequence of characters not including any of the defined separators.
	Error Code —	The return value of the built-in function, ErrorCode .

There are a number of Events Reader built-in functions, which generally fall into the following categories and sub-categories:

- [File Navigation](#) — pg. 480
 - [Go To](#) — pg. 480
 - [Separators](#) — pg. 480
 - [Skip Forward/Backward](#) — pg. 481
 - [Miscellaneous](#) — pg. 481
- [Data Retrieval](#) — pg. 481
 - [Data](#) — pg. 481
 - [Strings](#) — pg. 482
 - [Sub-Strings](#) — pg. 483
 - [Integers](#) — pg. 484
 - [Real](#) — pg. 485
- [String Manipulation](#) — pg. 485
- [Database](#) — pg. 487
 - [Fab](#) — pg. 487
 - [Technology](#) — pg. 487
 - [Family](#) — pg. 488
 - [Process](#) — pg. 488
 - [Product](#) — pg. 488
 - [Program](#) — pg. 489
 - [Lot](#) — pg. 491
 - [Wafer](#) — pg. 492
 - [Step](#) — pg. 492
 - [Stage](#) — pg. 493
 - [Equipment](#) — pg. 493
 - [Recipe](#) — pg. 494
 - [Operator](#) — pg. 494

- [Customer](#) — pg. 495
- [Events Indexes](#) — pg. 495
- [Date-Time](#) — pg. 497
- [Tagging](#) — pg. 498
- [Miscellaneous](#) — pg. 499
- [Mathematical](#) — pg. 499
- [Debugging](#) — pg. 499
- [System](#) — pg. 500

File Navigation

<i>Go To</i>	Goto (string) —	Accepts one argument of type string and has no return. Searches the file in the forward direction for the passed argument (a Word). If the search is successful the File Pointer is moved one character beyond the passed string, otherwise the File Pointer is not changed and the error code is set to 1.
	GoBackTo (string) —	Similar to GoTo, but searches in the backward direction.
	GotoEOF —	Accepts no arguments and has no return. Moves the File Pointer to the end of the file.
	GotoBOF —	Accepts no arguments and has no return. Moves the File Pointer to the beginning of the file.
<i>Separators</i>	AddSep (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of separators.
	DelSep (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of separators.
	ClearSep —	Accepts no arguments and has no return. Clears all separators from the list of separators except the defaults.

<i>Skip Forward/ Backward</i>	SkipLines (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N lines are skipped in the forward direction. If N is negative the File Pointer is moved backwards skipping N lines. Skipping a line amounts to skipping one end of line character and moving the File Pointer to the beginning of the line following that character.
	SkipWords (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N words are skipped in the forward direction. If N is negative the File Pointer is moved backwards N words.
	SkipChars (integer) —	Accepts one argument of type integer and has no return. If the passed argument (N) is positive N characters are skipped in the forward direction. If N is negative the File Pointer is moved backwards N Characters.
<i>Miscellaneous</i>	NotEndOfFile —	Accepts no arguments and returns a Boolean. The returned value is False when the File Pointer is at the end of the data file, otherwise it is True.
	GetLineLen —	Accepts no arguments and returns an integer. Returns the number of characters from the FilePointer to the end of line.

Data Retrieval

<i>Data</i>	LogResult (result) —	Accepts one argument of type result and has no return. Reads in the current word as a result and logs it to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values.
	vLogResult (result, real) —	Accepts two argument of type result and real and has no return. Logs the second argument as the result to the data table. This function should only be called after all key conditions and key indexes have already been set to the current values. The first argument passed to this function specifies the name and type of the result, while the second argument is the result itself.

AddInvString (string) —	Accepts one argument of type string and has no return. Adds the passed argument to the list of invalid data.
AddInvReal (real) —	Accepts one argument of type real and has no return. Adds the passed argument to the list of invalid data.
AddInvInteger (integer) —	Accepts one argument of type integer and has no return. Adds the passed argument to the list of invalid data.
AddInvChar (char) —	Accepts one argument of type char and has no return. Adds the passed argument to the list of invalid data.
DelInvString (string) —	Accepts one argument of type string and has no return. Deletes the passed argument from the list of invalid data.
DelInvReal (real) —	Accepts one argument of type real and has no return. Deletes the passed argument from the list of invalid data.
DelInvInteger (integer) —	Accepts one argument of type integer and has no return. Deletes the passed argument from the list of invalid data.
DelInvChar (char) —	Accepts one argument of type char and has no return. Deletes the passed argument from the list of invalid data.
OpenFile (string) —	Accepts one argument of type string and has no return. Replaces the currently open file with the file whose name is the passed argument. The passed argument may be the full path name or may be used with the -file_path command line argument. If the new file does not exist, the error code is set to 1 and the open file remains the old file.

<i>Strings</i>	GetWord —	Accepts no arguments and returns a string. Returns the current word leaving the File Pointer one character beyond the retrieved word.
	GetPrevWord —	Accepts no arguments and returns a string. Returns the previous word leaving the File Pointer one character beyond the retrieved word.

GetQuotedWord (char) — Accepts one argument of type char and returns the current word. In this instance, a word is defined as all the characters in the quoted string. The character used to identify a quote is the passed argument. (Error code is set if no quoted word is found.)

GetLine — Accepts no arguments and returns a string. Returns all characters from the File Pointer to the first end-of-line character. The File Pointer is moved to the end of the line being read.

Sub-Strings **GetChars (integer)** — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the N characters starting from the first non-separator after the File Pointer.

GetCharsTrim (integer, char) — Accepts two arguments of type integer and char and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains a maximum of N characters starting from the File Pointer. Leading and trailing characters are trimmed. (The second argument decides which character to trim.)

ExtractString — Accepts no arguments and returns a string. Returns the current word minus anything in the beginning of the word that is a number leaving the File Pointer one character beyond the retrieved word.

GetLeftChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the first N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetRightChars (integer) — Accepts one argument of type integer and returns a string. Returns a string of length N, where N is the passed argument. The returned string contains the last N characters of the current word. If the length of the current word is less than N, the whole word is returned. The File Pointer is moved to one character beyond the retrieved word.

GetMidChars (integer, integer) —

Accepts two arguments of type integer and returns a string. Returns a string of length N, where N is the second argument. The returned string contains the middle N characters of the current word starting at the Mth character. If the length of the current word - M is less than N, all the characters after the Mth character are returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordAfter (char) —

Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring after the passed character.

If there are multiple occurrences of the passed character, the function returns all characters after the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

GetWordBefore (char) —

Accepts one argument of type char and returns a string. The returned string contains all the characters of the current word occurring before the passed character.

If there are multiple occurrences of the passed character, the function returns all characters before the first instance.

If the passed character is not found in the current word an empty string is returned. The File Pointer is moved to one character beyond the retrieved word.

Integers **GetInt —**

Accepts no arguments and returns an integer. Returns the current word as an integer leaving the File Pointer one character beyond the retrieved word.

GetPrevInt —

Accepts no arguments and returns an integer. Returns the previous word as an integer leaving the File Pointer one character beyond the retrieved word.

String Manipulation	Real	GetReal —	Accepts no arguments and returns a real. Returns the current word as a real leaving the File Pointer one character beyond the retrieved word.
		GetPrevReal —	Accepts no arguments and returns a real. Returns the previous word as a real leaving the File Pointer one character beyond the retrieved word.
		ToLower (string) —	Accepts one argument of type string and returns a string. The returned string is the lower-case equivalent of the passed argument.
		ToUpper (string) —	Accepts one argument of type string and returns a string. The returned string is the upper-case equivalent of the passed argument.
		IsNumber (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is a number, False otherwise.
		IsString (string) —	Accepts one argument of type string and returns a Boolean. Returns True if the passed string is not a number, False otherwise.
		Right (string,integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetRightChars(), but operates on the passed string instead of the current word.
		Left (string,integer) —	Accepts two arguments of type string and integer and returns a string. Same as GetLeftChars(), but operates on the passed string instead of the current word.
		Mid (string, integer, integer) —	Accepts three arguments of type string, integer and integer and returns a string. Same as GetMidChars(), but operates on the passed string instead of the current word.
		After (string, char) —	Accepts two arguments of type string and character and returns a string. Same as GetWordAfter(), but operates on the passed string instead of the current word. If there are multiple occurrences of the passed character, the function returns all characters after the last instance.

- Before (string, char)** — Accepts two arguments of type string and character and returns a string. Same as `GetWordBefore()`, but operates on the passed string instead of the current word.
- If there are multiple occurrences of the passed character, the function returns all characters before the last instance.
- StrCat (string, string)** — Accepts two arguments of type string and returns a string. Concatenates the two strings and returns the result.
- StrToReal (string)** — Accepts one argument of type string and returns a real. Translates the passed string to a real.
- StrToInt (string)** — Accepts one argument of type string and returns an integer. Translates the passed string to an integer.



StrToInt and **StrToReal** will convert the first numerical part of the provided string.

For example, for `'StrToInt ("16A3")`, the output would be the integer 16; for `'StrToReal ("16.1A3")`, the output would be 16.1.

- IntToStr (integer)** — Accepts one argument of type integer and returns a string. Translates the passed integer to a string.
- StrTrim (string, char)** — Accepts two arguments of type string and character; and returns a string. Trims leading and trailing characters from the first argument using the second argument as the character to be trimmed.
- StrLen (string)** — Accepts one argument of type string and returns an integer. The value returned is the length of the string parameter in characters.
- Argument** — Accepts no arguments and returns a string. The returned string is what is passed to the reader using the command-line argument **-arg**.

Database

Fab **DbFab --- default ---** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbFab --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a fab in the database (FAB Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbFab (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFab, but uses the passed argument instead of reading from the file.

Technology **DbTechnology --- default ---** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbTechnology --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a technology in the database (TECHNOLOGY Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbTechnology (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbTechnology, but uses the passed argument instead of reading from the file.

Family **DbFamily** —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the program type is fixed and the current word does not exist as a family in the database (FAMILY Table), the function does nothing. If the program type is not fixed (semi-dynamic, dynamic) and the current word does not exist as a family in the database (FAMILY Table), the family is added to the database and the function establishes the appropriate relations with other tables.

vDbFamily (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbFamily, but uses the passed argument instead of reading from the file.

Process **DbProcess** --- default --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), the function does nothing. If it does exist, the function establishes the appropriate relations with other tables.

DbProcess --- with -semi_dynamic option --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process in the database (PROCESS Table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbProcess (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProcess, but uses the passed argument instead of reading from the file.

Product **DbProduct** --- default --- —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbProduct --- with -semi_dynamic option ---

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a product in the database (PRODUCT Table), It is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbProduct (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbProduct, but uses the passed argument instead of reading from the file

Program **DbProgram —**

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program name to the current word. This function should always be called when dumping to database.



Data readers and database retrieval supports program names up to a limit of 255 characters.

vDbProgram (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Sets the database test program name to the passed argument. This function should always be called when dumping to database.



Data readers and database retrieval supports program names up to a limit of 255 characters.

DbProgRel (string) —

Accepts one argument of type string and returns a string. Calls GetWord returning the current word. Sets the database test program release to the current word. The passed argument specifies the format of the date string being read.

vDbProgRel (string, string) —

Accepts two arguments of type string and returns a string. Same as DbProgRel but uses the first passed argument instead of GetWord as the date string.

The following describes the formatting of the DATE string:

dd — Day of the month as a 2-digit number.

ddd — Day of the week as a 3-letter abbreviation.

mm — Month as a 2-digit number.

mmm — Month as a 3-letter abbreviation.

yy — Year as a 2-digit number in the 2000s. The two-digit input will be appended onto “20__”.

yyyy — Year as a 4-digit number.



With the Oracle database, the ddd option cannot be used. Additionally, in Oracle, the format has to consist of only the date format, with no additional text added in date string.

DbProgRev —

Accepts no arguments and returns a string. Calls GetWord returning the current word. Sets the database test program revision to the current word.

vDbProgRev (string) —

Accepts one argument of type string and returns a string. Same as DbProgRev but uses the passed argument instead of GetWord.

DbProgClass (int) —

Accepts one argument of type integer and has no return. Sets pgc_key in the PROGRAM table. This function overwrites the command line argument **-class**.

vDbProgGroup (string) —

Accepts one argument of type string and returns a string. The accepted argument is the group name to be associated with the program.

The program group should already exist. If it does not exist, an error will be generated (similar to the case where a program class does not exist). The PROGRAM.pg_grp_key table will only be filled when the program is new.



A new program group can be created by the following SQL statement:

For Oracle:

```
Insert into program_group (PG_GRP_KEY, PG_GRP_NAME,
PGC_KEY, Em_key, INSERT_TIME, PG_GRP_DESC)
Values (PROGRAM_GROUP_SEQ.NEXTVAL, 'NewProgGroup',
Pgc_Key, 'Who Created It',SYSDATE, 'A new group');
Commit;
```

For Informix:

```
Insert into program_group (PG_GRP_NAME, PGC_KEY, Em_key,
INSERT_TIME, PG_GRP_DESC)
Values ("NewProgGroup", Pgc_Key, 'Who Created It', current,
"A new group");
Commit;
```

Lot DbSrcLot —

Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a Source Lot in the database (LOT Table), the Source Lot is added. If it does exist, the function establishes the appropriate relations with other tables. Works in conjunction with DbLot, sets the src_lot column in the LOT table and OP_LOG table. This function can only be used with program class 15, LotEvent.

vDbSrcLot (string) —

Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbSrcLot, but uses the passed argument instead of reading from the file. This function can only be used with program class 15, LotEvent.

vDbLotClass (string, string, string, string) —

Accepts four arguments of type string: (**Lot_Name, Method_Name, Method_Type, Class_Name**) and has no return. Works in conjunction with **DbLot**. Sets the lot class from the passed argument for the lot specified using **DbLot**. The function supports multiple classes, per lot.

DbUpdateLot (int) —

Accepts one argument of type integer, and has no return. The first argument is a fab flag that allows for updating the Fab relationship if argument >0.

The function allows for the update of the LOT table's fab relationships.

This is useful when a particular lot already exists in the database and there is a need to update the fab relationships to that lot.

The fab is updated to the fab identified by either of the functions, **DbFab** or **vDbFab**.

Wafer DbWfDesc (string, string, string) — Accepts three arguments of type string and has no return. The first argument should be the lot ID, the second argument should be the wafer ID, the third argument is the wafer description (maximum 255 characters). The description goes into the **wf_desc** column of the WAFER table.

DbWfNum (string, string, int) — Accepts three arguments of type string, string and integer and has no return. The first argument should be the lot ID, the second argument should be the wafer ID, the third argument should be the wafer number associated with that wafer ID. This function allows populating the **wf_num** column in the WAFER table.

Step DbStep --- default --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStep --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a process step in the database (PROC_STEP Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStep (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbStep, but uses the passed argument instead of reading from the file.

Stage DbStage --- default --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), the function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbStage --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a stage in the database (TECH_STAGE Table), it is added. If it does exist the function only establishes the appropriate relations with other tables.

vDbStage (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbStage, but uses the passed argument instead of reading from the file.

Equipment vDbEquipment (string, string) — Accepts two arguments and has no return. The 1st argument is the equipment type (string). The 2nd argument is the equipment class (string).

This function can only be used if equipment is declared as index using the DbEventEquipIndex function.

DbEventEquipNumIndex (int, string) — Accepts two arguments and has no return. The 1st argument is the equipment number (int). The 2nd argument is the equipment name (string).

The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the equipment2 index used for updating the EQUIPMENT database table. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index.

vDbEquipName (int, string) — Accepts two arguments and has no return. The 1st argument is the equipment number (int). The 2nd argument is the equipment name (string).

If the passed value for second argument (equipment name) does not exist as an equipment in the database (EQUIPMENT table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

vDbEquipDet (int, string, string) — Accepts three arguments and has no return. The 1st argument is the equipment number (int). The 2nd argument is the equipment type (string). The 3rd argument is the equipment class (string).

This function can only be used if equipment name is declared as index using the DbEventEquipNumIndex function or by vDbEquipName function (above).

Recipe **DbEventRecipeIndex (string)** — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the Recipe index used for updating the RECIPE database table. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index.

vDbRecipe (String) — Accepts one argument of type string and returns a string. The returned string is the passed argument. The passed argument should be the recipe name. If it does not exist as a recipe in the database (RECIPE table), it is added. If it does exist, the function only establishes the appropriate relations with other tables.

Operator **DbOperator --- default ---** — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a name in the PEOPLE Table, The function does nothing. If it does exist the function establishes the appropriate relations with other tables.

DbOperator --- with -semi_dynamic option --- — Accepts no arguments and returns a string. Calls GetWord returning the current word. If the current word does not exist as a name in the PEOPLE Table, it is added with the “role” field set to Operator. If it does exist the function only establishes the appropriate relations with other tables.

vDbOperator (string) — Accepts one argument of type string and returns a string. Returns the passed argument. Same as DbOperator, but uses the passed argument instead of reading from the file.

Customer **vDbCustomer(string, ...)** — Accepts 12 arguments of type string, and has no return. Populates the CUSTOMER database table and establishes a relationship between the customer entry and the current lot. This function may be called several times in the same format file, in this way relating one lot to many customers. The arguments passed to the function match exactly those of the CUSTOMER table. These fields (in their respective order) include: **customer name, address 1, address 2, address 3, postal code, city, state, country, contact, email, fax, phone.**

Events Indexes **DbMetaIndex (string, string)** — Accepts two arguments of type string and has no return value. This function is used to store meta data in **meta_value** column for each event. One meta name can be chosen per program. The first passed argument must be the name of meta (in double quotes) and the second passed argument must be the name of one of the declared indexes (in double quotes). That index becomes the meta index used for updating the CASS_EVENT_LOG and EVENT_DATE_BUCKET database tables. This function can be called only once in the format file.

Restrictions / Requirements: This function is used with Cassandra databases and will not work with Oracle databases.

DbEventLotIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the lot index used for updating the LOT database table. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index. This function must be used with program class 15 (LotEvent) and program class 22 (WaferEvent). It also can be used with program class 16 (EquipEvent).

DbEventEquipIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the equipment index used for updating the EQUIPMENT database table. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index. This function must be used with program class 16 (EquipEvent), but can also be used with program class 15 (LotEvent) and program class 22 (WaferEvent).

DbEventWafIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the wafer index used for updating the WAFER database table. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index. This function must be used with program class 22 (WaferEvent). It can be used with program class 16 (EquipEvent), but it can't be used with program class 15 (LotEvent).

DbEventIndex (string, string, string, string, string, string) —

Accepts six arguments of type string and has no return. The passed arguments must be the names (in double quotes) of some declared indexes. If some of those arguments are not needed, place "NA" in their place. Those indexes become the: source lot, people, process step, product, process, and/or stage indexes (in that specific sequence) used for updating their respective tables in the database. Those indexes are consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming hidden indexes.

DbChamberIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. That index becomes the chamber index used for updating the EQUIPMENT database table. The **eq_level** field in EQUIPMENT table is set to 200 to distinguish chamber from other equipment values. This index is consequently removed from the RES... table and placed in the EVENT_LOG table, thus becoming a hidden index.

NormalizeIndex (string) — Accepts one argument of type string and has no return. The passed argument must be the name (in double quotes) of one of the declared indexes. A table is created for that index and named EV_... (where ... is the index name). This function can't be used with one of the hidden indexes.



- *The normalized index name should not exceed 14 characters, otherwise it is truncated.*
- *Normalize index cannot be used when loading to Cassandra, If used, then it will be skipped.*

Date-Time DbStartTime (string) — Accepts one argument of type string and returns a string. Calls GetWord returning the current word. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file.

vDbStartTime (string, string) — Accepts two arguments of type string and returns a string. Returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file.

The following describes the formatting of the DATETIME string:

%b Abbreviated month name.
%B Full month name.
%d Day of the month as a decimal [01,...,31].
%H 24 hour clock.
%I 12 hour clock
%M Minute as a decimal [00,...,59].
%m Month as a decimal [01,...,12].
%p a.m. or p.m.
%S Second as a decimal [00,...,59].
%y Year as a decimal [00,...,99].
%Y Year as a 4-digit decimal.
%% Allows for percent in the string.

As an example the format for the following string:

“jul 1 96 05:10:46”

would be:

“%b %d %y %H:%M:%S”

DbEndTime (string) — Accepts one argument of type string and returns a string. Calls GetWord returning the current word. The passed string is the format (SQL DATETIME) that describes the date as it appears in the file.

vDbEndTime (string, string) — Accepts two arguments of type string and returns a string. Returns the first argument. The first argument is the date-time as a string and the second is the format (SQL DATETIME) that describes the date as it appears in the file.

Tagging **DbLotTag (int) —** Accepts one argument of type integer and has no return. Used for manual lot tagging. The passed argument should be one of the following values:
Normal (0)
No Action (1)
Bad (2)
Scrap (3)
Experiment (4)
High defect density (5)
High defect count (6)
High defect density and high defect count(7)

The above list is dependent on the contents of the **TAGS** table in the database. This function can only be used with program class 15, LotEvent.

DbSrcLotTag (int) — Accepts one argument of type integer and has no return. Used for manual source lot tagging. The passed argument should be one of the following values:
Normal (0)
No Action (1)
Bad (2)
Scrap (3)
Experiment (4)
High defect density (5)
High defect count (6)
High defect density and high defect count (7)

The above list is dependent on the contents of the **TAGS** table in the database. This function can only be used with program class 15, LotEvent.

Miscellaneous **DbParGrp (string, int, string) —**

Accepts three arguments of type string, integer, string, and has no return. Allows for the creation of parameter groups from within the format file. This function may be called several times, thus creating multiple parameter groups related to the current program. The arguments passed to the function include the following fields (in their respective order): **parameter group name**, **condition number**, and **wild-card string**.

The function creates the parameter groups based on the value of one of the conditions (argument 2) and based on the value of that condition for the different parameters (argument 3, which could be an exact match or a wild-card match).

Mathematical **Abs (real) —**

Accepts one argument of type real and returns a real. The returned value is the absolute value of the passed argument.

Sqr (real) —

Accepts one argument of type real and returns a real. The returned value is the square root of the passed argument.

POW (real, int) —

Accepts two arguments — first of type real, second of type integer; and returns a real. The function returns a real, which is the result of the 1st argument (type real), raised to the power of the 2nd argument (type integer).

Debugging **Print (...) —**

Accepts a variable number of arguments and has no return. Prints to screen the passed arguments.

PrintToFile (...) —

Accepts a variable number of arguments and has no return. The first argument is the file name to print to. Appends to the file the remaining arguments.

ErrorCode —

Accepts no arguments and returns an integer. The returned value is zero if the previously called function was successful; otherwise the return value is one. Currently the only functions that set the error code are **GoTo**, **GoBackTo**, and **OpenFile**.

ExitScript (string) —

Accepts one argument of type string and has no return. Causes the reader to exit, reporting the passed string as an error message.

If the passed string is a null string “” no error file is generated and the execution is aborted. This is useful in conjunction with the DpLoad.pl script, where there is a need to move the file to the **Processed** directory without actually processing the file — as opposed to generating an error file causing the file to be moved to the **NotProcessed** directory.

System

System (string) —

Accepts one argument of type string and returns an integer. This function allows access to the “C” System function, where the passed argument is the UNIX command to be executed and the returned value is the return value of the “C” System function. Accepts up to 254 characters.

DATABASE

In addition to the database functions described in the previous section, variables that are declared as indexes can affect the database tables.

Some indexes can be declared as hidden indexes. Hidden indexes help determine if a new row is inserted into the database (RES...), just like any other index. The only difference is that they are not stored in the RES... table, but stored in EVENT_LOG as normalized keys. These indexes are associated with any of the following list of EVENT_LOG columns: Lot, Source Lot, Equipment, Employee, Step, Product, Process, and/or Stage. The Events reader can support serial keys beyond 2^{31} .

Any other index can be normalized using the built-in function **NormalizeIndex**. This would create extra tables dynamically for those indexes.

LOT database table is updated from the Events reader only if the functions are used to read in the Lot ID.

All files must contain a Test Program, and the Test Program must belong to a Program Class. This is done using the -class option described below. Only three valid values for the program class are accepted. These are **15, EquipEvent; 16, LotEvent; and 22, WaferEvent**.

TAGGING LOTS

Lots are tagged manually.

This tagging state can then be used as data retrieval criteria.

The predefined tag states are:

- Normal (0)
- No Action (1)
- Bad (2)
- Scrap (3)
- Experiment (4)
- High defect density (5)
- High defect count (6)
- High defect density and high defect count (7)

To set the tagging flag from the Events reader, two functions are used:

- **DbLotTag (int)** — used for manual lot tagging
- **DbSrcLotTag (int)** — used for manual source lot tagging

The previous functions only operate if tagging is enabled at the time of the creation of the program. Enabling of tagging is done using the `-tag_action` command line argument.

PUTTING IT ALL TOGETHER

The combination of keywords, commands and processing blocks make up the format file. A thorough concept of the input stream and a few lines in a format file will allow the user to process any defined data file. In writing the format file, it is important to know the inherent nesting order of the input stream. Once this is known, it is only a matter of matching the keywords with the values being expected. Every format file can be started from the following skeleton:

```
Script      default
Const
Var
           // At least the default conditions (Test name, test number and units)
           // At least one key index
           // At least one result

Begin
           // Add separators and invalid data
           // For Database at least DbProgram has to be called
           // Set condition and index values
           // Log results
           // Log Limits

End
```

There are three types of format files that the Events reader can handle. The selection of the type of which format file to use depends on the program class that the data file falls in. It can be a **LotEvent** (pgc_key = 15), an **EquipEvent** (pgc_key = 16) or a **WaferEvent** (pgc_key = 22).

Below are three data files each followed by a script, the first data and script file are an example of an EquipEvent; the second is an example of a LotEvent; the third is an example of WaferEvent.

Data File Example 1: EquipEvent

```
<HEADER>
VERSION = 1
CREATION_DATE = NA
PROGRAM_CLASS = 16
PROGRAM = EVNT_EQUIP_14
RELEASE = NA
REVISION = NA
</HEADER>

<DATA>

Fab_15, molding, STEP_2, STAGE_11, tester_33, tester, eq_class_5, John, jan-01-2001-10:55:16, jan-02-2001-10:58:37, "Equipment broken", "Equipment needs fixing"

Fab_16, molding, STEP_1, NA, tester_76, tester, eq_class_5, Eric, jan-02-2001-16:33:19, jan-03-2001-16:36:44, "Equipment low on oil", "Need to change oil"

</DATA>
```

Script Example: EquipEvent

```

SCRIPT equipEvents

CONST

VAR

//Conditions

TestName          string    Cond
TestUnit          string    Cond
TestNumber        integer   KeyCond

//Indexes

EQUIP             string    KeyIndex

OPER             string    Index
STP              string    KeyIndex

PROC             string    KeyIndex

//Results

res              string    Result

//Variables

Str string

BEGIN

AddSep(' ')
AddSep(',')
AddSep('=')

TestUnit = "NA"
TestName = "Comment"

//Header

Goto("PROGRAM_CLASS")
If (ErrorCode <> 0)
    ExitScript("Program Class Not Found")
End If
DbProgClass(GetInt)

Goto("PROGRAM")
If (ErrorCode <> 0)
    ExitScript("Program Not Found")
End If

Str = DbProgram

// Index declarations

DbEventEquipIndex("EQUIP")

DbEventIndex("NA", "OPER", "STP", "NA", "NA", "NA")

// Normalized indexes

NormalizeIndex("PROC")

// Raw Data

Goto("<DATA>")

```



```

If (ErrorCode <> 0)
  ExitScript("<DATA> Not Found")
End If

Str = GetWord

While (NotEndOfFile AND (Str <> "</DATA>"))

  SkipWords(-1)

  Str = DbFab

  PROC = GetWord

  STP = GetWord

  Str = DbStage

  EQUIP = GetWord

  vDbEquipment(GetWord, GetWord)

  OPER = GetWord

  Str = DbStartTime("%b-%d-%Y-%H:%M:%S")
  Str = DbEndTime("%b-%d-%Y-%H:%M:%S")

  TestNumber = 1
  Str = GetQuotedWord("")
  vLogResult(res, Str)

  TestNumber = 2
  Str = GetQuotedWord("")
  vLogResult(res, Str)

  Str = GetWord

End While

END

```

Data File Example 2: LotEvent

```

<HEADER>
VERSION = 1
CREATION_DATE =
PROGRAM_CLASS = 15
PROGRAM = EVNT_LOT_22
RELEASE = NA
REVISION = NA
</HEADER>

<DATA>

Fab_56, Tech_67, Fam_5, molding, CMOS_6, STEP_2, STAGE_11, lot_453, 2,
slot_333, 2, lrg_lot, tester_3, tester, eq_cls_5, John, jan-01-2001-10:55:16, jan-
02-2001-10:58:37, "Lot incomplete", "Lot needs rework"

Fab_16, NA, Fam_3, molding, BCMOS, STEP_1, STAGE_12, lot_453, 0,
slot_333, 0, small_lot, prober_76, prober, eq_class_8, Eric, jan-02-2001-
16:33:19, jan-03-2001-16:36:44, "Defective lot", "Lot need fixing"

Fab_17, Tech_6, Fam_7, burning, VRAM, STEP_18, STAGE_13, lot_453, 2,
slot_333, 3, small_lot, handler_76, handler, eq_class_11, Eric, jan-05-2001-
11:43:09, jan-05-2001-11:44:58, "Junk lot", "Lot to be junked"

</DATA>

```



Script Example: LotEvent

```

SCRIPT lotEvents

CONST

VAR

//Conditions

TestName          string    Cond
TestUnit          string    Cond
TestNumber        integer   KeyCond

//Indexes

LOT               string    KeyIndex
EQUIP             string    Index

PROD              string    Index
PROC              string    Index
STAG              string    KeyIndex
SLOT              string    Index
OPER              string    Index
STP               string    KeyIndex

FAB               string    Index
TECH              string    KeyIndex

//Results

res               string    Result

//Variables

Str               string

BEGIN

AddSep(' ')
AddSep(',')
AddSep('=')

TestUnit = "NA"
TestName = "Comment"

//Header

Goto("PROGRAM_CLASS")
If (ErrorCode <> 0)
    ExitScript("Program Class Not Found")
End If
DbProgClass(GetInt)

Goto("PROGRAM")
If (ErrorCode <> 0)
    ExitScript("Program Not Found")
End If
Str = DbProgram

// Index declarations

DbEventLotIndex("LOT")
DbEventEquipIndex("EQUIP")

DbEventIndex("SLOT", "OPER", "STP", "PROD", "PROC", "STAG")

// Normalized indexes

```

```
NormalizeIndex("FAB")
NormalizeIndex("TECH")

// Raw Data

Goto("<DATA>")
If (ErrorCode <> 0)
  ExitScript("<DATA> Not Found")
End If

Str = GetWord

While(NotEndOfFile AND (Str <> "</DATA>"))

  SkipWords(-1)

  FAB = DbFab

  TECH = DbTechnology

  Str = DbFamily

  PROC = GetWord

  PROD = GetWord

  STP = GetWord

  STAG = GetWord

  LOT = GetWord

  DbLotTag( GetInt)

  SLOT = GetWord

  DbSrcLotTag( GetInt)

  vDbLotClass( GetWord)

  EQUIP = GetWord

  vDbEquipment( GetWord, GetWord)

  OPER = GetWord

  Str = DbStartTime("%b-%d-%Y-%H:%M:%S")
  Str = DbEndTime("%b-%d-%Y-%H:%M:%S")

  TestNumber = 1
  Str = GetQuotedWord("")
  vLogResult( res, Str)

  TestNumber = 2
  Str = GetQuotedWord("")
  vLogResult( res, Str)

  Str = GetWord

End While

END
```

Data File Example 3: WaferEvent

```
<HEADER>
VERSION = 1
CREATION_DATE =
PROGRAM_CLASS = 22
PROGRAM = EVNT_WAFER_22
RELEASE= NA
REVISION= NA
</HEADER>
<DATA>
Fab_56, Tech_67, Fam_5, molding, CMOS_6, STEP_2, STAGE_11, lot_453, 2,
slot_333, 2, lrg_lot, waf_1, waf_desc1, 1, tester_3, tester, eq_cls_5, John, jan-
01-2001-10:55:16, jan-02-2001-10:58:37,
"Lot incomplete", "Lot needs rework"
Fab_16, Tech_23, Fam_3, molding, BCMOS, STEP_1, STAGE_12, lot_453, 0,
slot_333, 0, small_lot, waf_2, waf_desc2, 2, prober_76, prober, eq_class_8, Er-
ic, jan-02-2001-16:33:19, jan-03-2001-16:36:44,
"Defective lot", "Lot need fixing"
Fab_17, Tech_6, Fam_7, burning, VRAM, STEP_18, STAGE_13, lot_453, 2,
slot_333, 3, small_lot, waf_3, waf_desc3, 3, handler_76, handler, eq_class_11,
Eric, jan-05-2001-11:43:09, jan-05-2001-11:44:58,
"Junk lot", "Lot to be junked"
</DATA>
```



Script Example: WaferEvent

```
SCRIPT lotEvents
CONST
VAR
//Conditions
TestName string Cond
TestUnit string Cond
TestNumber integer KeyCond
//Indexes
WAF string KeyIndex
LOT string KeyIndex
EQUIP string Index
PROD string Index
PROC string Index
STAG string KeyIndex
SLOT string KeyIndex
OPER string Index
STP string KeyIndex
FAB string Index
TECH string KeyIndex
//Results
res string Result
//Variables
Str string
BEGIN
AddSep(' ')
AddSep(',')
AddSep('=')
TestUnit = "NA"
TestName = "Comment"
//Header
Goto("PROGRAM_CLASS")
If (ErrorCode <> 0)
ExitScript("Program Class Not Found")
End If
DbProgClass(GetInt)
Goto("PROGRAM")
If (ErrorCode <> 0)
ExitScript("Program Not Found")
End If
Str = DbProgram
// Index declarations
```

```
DbEventLotIndex("LOT")
DbEventWafIndex("WAF")
DbEventEquipIndex("EQUIP")
DbEventIndex("SLOT", "OPER", "STP", "PROD", "PROC", "STAG")
// Normalized indexes
NormalizeIndex("FAB")
NormalizeIndex("TECH")
// Raw Data
Goto("<DATA>")
If (ErrorCode <> 0)
ExitScript("<DATA> Not Found")
End If
Str = GetWord
While(NotEndOfFile AND (Str <> "</DATA>"))
SkipWords(-1)
FAB = DbFab
TECH = DbTechnology
Str = DbFamily
PROC = GetWord
PROD = GetWord
STP = GetWord
STAG = GetWord
LOT = GetWord
DbLotTag( GetInt)
SLOT = GetWord
DbSrcLotTag( GetInt)
vDbLotClass( GetWord)
WAF = GetWord
DbWfDesc(LOT, WAF, GetWord)
DbWfNum(LOT, WAF, GetInt)
EQUIP = GetWord
vDbEquipment( GetWord, GetWord)
OPER = GetWord
Str = DbStartTime("%b-%d-%Y-%H:%M:%S")
Str = DbEndTime("%b-%d-%Y-%H:%M:%S")
TestNumber = 1
Str = GetQuotedWord("")
vLogResult( res, Str)
TestNumber = 2
Str = GetQuotedWord("")
vLogResult( res, Str)
Str = GetWord
End While
END
```

EVENTS FILE FORMAT SPECIFICATION (ORACLE ONLY)

```

<File generated by dpexport utility version number>
<DatabaseVersion = dbinfo.version, dbinfo.cdate, dbinfo.waf_orient>
FormatType = Events
FormatVersion = 3
Separator = '|'

<BOH>
ProgRel|program.release_date
ProgRev|program.revision
ProgClassName|prog_class.pgc_name // Where prog_class.pgc_key = 15 or 16
or 22
ProgClassKey|program.pgc_key //where program.pgckey is 15 or 16 or 22
ReworkAction|program.rework_action
Program|program.ppid
ProgGroup|program_group.pg_grp_name
NumOfConds|program.conditions
NumOfTests|program.tests
NumOfIndexes|program.indexes
NumOfHiddenIndexes|count event_config //where pg_key = program.pg_key
TimeFormat| date_time format
<EOH>

<BOC> // Where program.pg_key =condition.pg_key //
and "type" is database column type of the condition. condition.cond_index|con-
dition.cond_name|type|condition.key_type...
...
<EOC>

<BOI> //First contains the hidden indexes Where event_config.pg_key = pro-
gram.pg_key
//All hidden indexes names starts with _EX_ followed by 2 characters represents
the hidden
index as follows:
//Waf as WF to be the name _EX_WF
//Lot as LT to be the name _EX_LT
//Source Lot as SL to be the name _EX_SL
//Equipment as PE to be the name _EX_PE
//Operator as OP to be the name _EX_OP
//Step as ST to be the name _EX_ST
//Product as PD to be the name _EX_PD
//Process as PR to be the name _EX_PR
//Stage as SG to be the name _EX_SG
//Chamber as CH to be the name _EX_CH
// Equipment2 as E2 to be the name _EX_E2
// Equipment3 as E2 to be the name _EX_E3
// Recipe as RC to be the name _EX_RC
// Followed by the rest of the not hidden indexes
// Where def<pg_key>.test_index <= program.indexes and "type" is database
column type of the index.

def.test_index|type|def.dist_type|def.cond0|def.filter_flag|def.c1
...
<EOI>

<BOT>
def.scale_factor|def.test_type|def.c20|def.cond0|def.cond1|.....
...
...
<EOT>
<BOCST>
customer.cust_name|customer.address1|customer.address2|customer.ad-
dress3|customer.postal_code|customer.city|customer.state|customer.country|

```

```
customer.contact|customer.email|customer.fax|customer.phone
...
<EOCST>

<BOD>
<BOE>
Lot.Lot_ID|Lot.src_lot_id|Fab.Fab_name|Equipment.Eq_name|Equip-
ment.Eq_name|Equip_Type.Eqt_name|Equip_Class.Ec_name|Equip-
ment.Eq_name2|Equip_Type.Eqt_name2|Equip_Class.Ec_name2|
Equipment.Eq_name3|Equip_Type.Eqt_name3|Equip_Class.Ec_name3|Reci-
pe.Rcp_name|People.Name|Product.Pd_name|Family.Fm_name|Pro-
cess.Pr_name|
Technology.tech_name|Proc -
step.Step_name|Tech_stage.Stage_name|Event_log.Start_time|Event_log.End -
time|res.i0|res.i1...res.iN|test_index1|res.t1...test_indexY|res.tY

OR if program class is 22 (WaferEvent)

Lot.Lot_ID|Lot.src_lot_id|Fab.Fab_name|Wafer.Wf_ID|Wafer.wf_desc|Wa-
fer.wf_num|Wafer.wf_type|Equipment.Eq_name|Equipment.Eq_name|Equip -
Type.Eqt_name|Eq
uip_Class.Ec_name|Equipment.Eq_name2|Equip_Type.Eqt_name2|Equip_Clas
s.Ec_name2|Equipment.Eq_name3|Equip_Type.Eqt_name3|Equip -
Class.Ec_name3|Recipe.Rcp_name|People.Name|Product.Pd_name|Fami-
ly.Fm_name|Process.Pr_name|
Technology.tech_name|Proc -
step.Step_name|Tech_stage.Stage_name|Event_log.Start_time|Event_log.End -
time|res.i0|res.i1...res.iN|test_index1|res.t1...test_indexY|res.tY
<EOE>

...
//First Equipment.Eq_name is for the chamber
//Second Equipment.Eq_name is for the equipment
//Any Normalized index it's res.i will be replaced with the name of the index table
that looks like this EV_<index_name>.<index_name>_VAL
<EOD>
```

RUNNING THE EVENTS READER

The Events reader can be run in batch mode in the same fashion used to run the ASCII reader (using DpLoad.pl and a configuration file).

To run the reader in command line mode, once your format file is created and saved, the Events reader can be run using the newly created format. If there are any errors in your format file the reader will generate an error file (err.jnk) describing the nature of the error.

Compiling Format Files Without Executing

Format files may be compiled without executing by passing one argument to the Events reader, which is the format file preceded by the -fmt switch.

Example:

```
events -fmt [format file]
```

Executing the Events reader in this way generates an error file (err.jnk) containing any compilation errors. (0 0 are an indication of no errors.)

Command-Line Arguments

If the Events reader is being run manually with the database option, knowing the different command-line arguments becomes necessary. The Events reader accepts the following options: (The first argument should always be the data file to be processed.)

Command-Line Arguments

Option	Definition	Default
-arg	[string] Passes the string to the format file (returned by the Argument built-in function).	If not used, this action is not taken.
-class	[class] Sets Program Class to class.	If not used, program class must be set within format file.
-cassandra	Stores EVENT_LOG table in Cassandra.	If not used, this action is not taken.
-cass_threads	Number of threads to create when storing the data to Cassandra (Allowed range [1-50]; default = 5).	If not used, defaults to 5.

Command-Line Arguments

Option	Definition	Default
-datadbs	<dbspace> Places reader created tables in specified dbspace.	Default dbspace for the (database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.
-db	[database] (Where database is the name of the database to write to)	This option is required with a valid value.
-db_accept	If used, automatically sets accept_data flag in database.	If not used, this action is not taken.
-defext	[extent] Sets the extent for the database table DEF... to extent. If not used, defaults to 256 Kbytes. Acceptable range is 32 Kb to 20,000 Kb.	If not used, defaults to 256 Kbytes.
-dynamic	<max added tests> Same as semi_dynamic, but with a possibly growing number of parameters. The max added tests is optional, and sets the maximum number of tests to be added per run [percent].	NOTE: If not used, no new parameters or tests can be added.
-end_time	Includes end_time as part of the detection of reload.	If not used, this action is not taken.
-file_path	[path] Sets the path to data files that may be opened using the built-in function OpenFile.	The current working directory.
-fix_cond_values	In all string condition values, replace the '[' with '(' and ']' with ')'. If dbspace is specified and exists as a valid dbspace in the database, then [dbspace] is where the indexes will be created.	If not used, this action is not taken.
-fmt	[format file] (Where format file is the format to be used)	This option is required with a valid value.
-indexes	[dbspace] This argument is only needed to identify the dbspace where indexes should be created for the dynamic tables. If not used, indexes are still created, but in the default dbspace. If dbspace is specified and exists as a valid dbspace in the database, then [dbspace] is where the indexes will be created.	Default dbspace for the (database/schema). Note: Default dbspace is the tablespace in which the database/schema is created. If the database was created in datadbs, for example, the indexes will be created in datadbs. Ask your database administrator for default dbspace details.

Command-Line Arguments

Option	Definition	Default
-lowercase	Forces Lot ID and Wafer ID to Lower Case	If not used, this action is not taken.
-maxtime	[seconds] Reader is terminated if maxtime is exceeded before completion. If used, this option should be followed by the maximum number of seconds to run the executable. If it is exceeded, the executable will terminate with an error.	WARNING: If not used, there is no maxtime limit on the reader, and the reader could run indefinitely.
-normext	[extent] Sets the extent for the database tables EV... to extent.	If not used, defaults to 64 Kbytes. Acceptable range is 64 Kb to 512 Kb.
-reload	Used to handle cases of reloading of the same events. For the Events reader, the detection of reload depends on start-time or end-time. Two command-line arguments, -start_time and -end_time , allow the user to specify the reload dependency on time. (Either -start_time or -end_time flag should be used with reload option.) The reload for the event will be if the start-time/end-time in the data file equals to start-time/end-time in the database (in addition to all other hidden key indexes defined using hidden indexes functions*). In this case, the reader considers the new data file as a reload of the same data. In which case, the old data is deleted and is replaced by the new data file. *Hidden indexes functions: DbEventIndex(), DbEventLotIndex(), DbEventEquipIndex(), DbEventWafIndex(), DbChamberIndex(), DbEventRecipeIndex() and DbEventEquipNumIndex().	If not used, all passed events in the data file will be inserted as new events.
-res_aging	[days] Sets aging in days for results. No default value. If not provided, then it is set to NULL in the database.	If not used, results never age.
-resext	[extent] Sets the extent for the database table RES... to extent. If not used, defaults to 5012 Kbytes. Acceptable range is 64 Kb-128,000 Kb.	If not used, defaults to 5012 KBytes.
-semi_dynamic	A semi dynamic "Program" assumes a fixed number of parameters, like the default, but in addition fills up all tables with keys in the EVENT_LOG table such as Product and Equipment.	NOTE: If not used, and -dynamic is not used, readers will not fill in fields in tables pointed to by EVENT_LOG.
-start_time	Includes start_time as part of the detection of reload.	If not used, this action is not taken.
-tag_action	[Tag Action] Sets Test Program Tag Action (integer).	2 WARNING: tagging is enabled.
-u	usage	NA

Command-Line Arguments

Option	Definition	Default
-updatelot	[flag] Same as the built-in function DbUpdateLot (<i>“DbUpdateLot (int),”</i> pg. 491). Used when DbUpdate lot is required, but is not included in the format file. The flag following -updatelot should have only one character, where the character is the fab flag. For example, -updatelot 1 would indicate that the fab relationship should be updated.	If not used, this action is not taken.
-uppercase	Forces Lot ID and Wafer ID to Upper Case	If not used, this action is not taken.
-v	version	NA

EVENT READER WINDOWS SERVICE – INSTALLATION AND CONFIGURATION

The Event reader can be run as a Windows Service. When an installation site utilizes this capability, a user will be able to start/stop/monitor the Event Reader from the Windows Services panel (services.msc).

This capability is typically used in conjunction with the Exensio EPT (Equipment Performance Tracking) feature, although it functions independently from EPT.

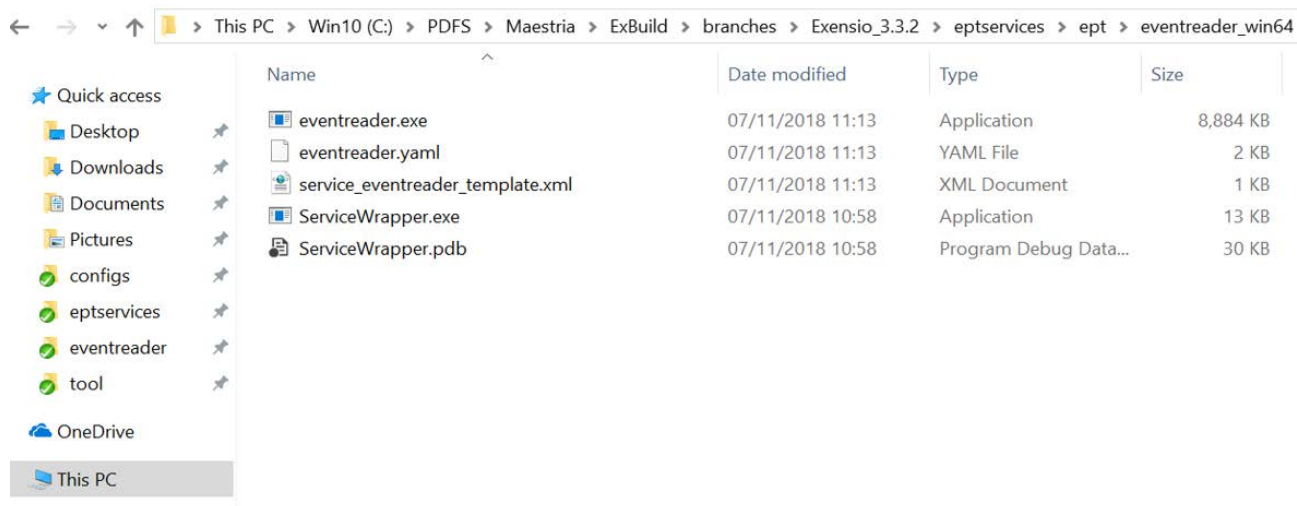
The primary reasons for running the Event reader as a service are:

- The ability to automatically start the Event reader when Windows starts/reboots.
- The ability to automatically restart the Event reader in the event of a crash or unexpected termination.

For more information on EPT, refer to the “*Equipment Performance Tracking*” section in the Exensio End-User Manual.

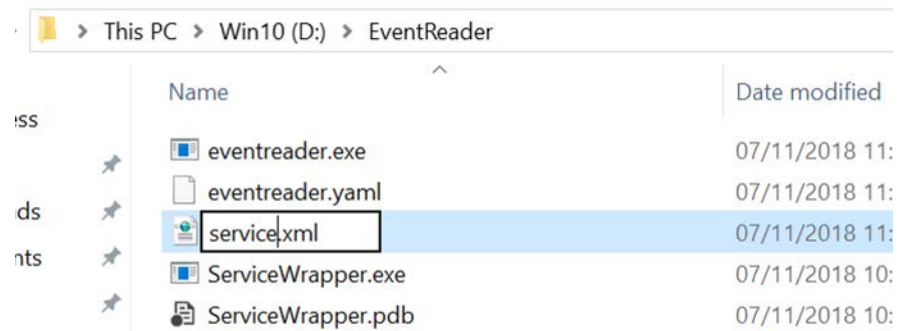
INSTALL EPT_READER WINDOWS SERVICE

1. Unzip the **eptservices.tar.gz** file.
2. Navigate to the folder: **leptleventreader_win64**.
3. Copy **ServiceWrapper.exe**, **ServiceWrapper.pdb**, and **service_eventreader_template.xml**.

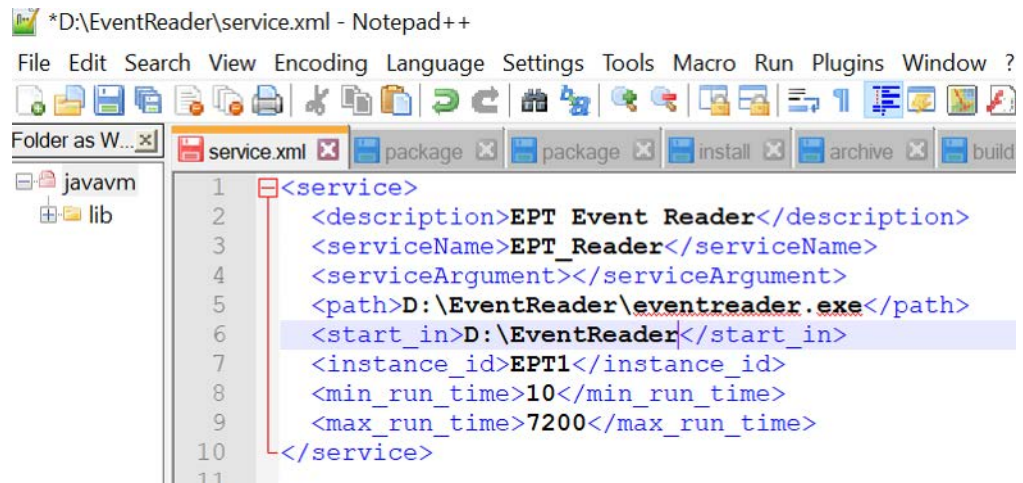


4. Paste the files you copied to the Event Reader location (e.g. **D:\EventReader**)

5. Rename “**service_eventreader_template.xml**” to “**service.xml**”.



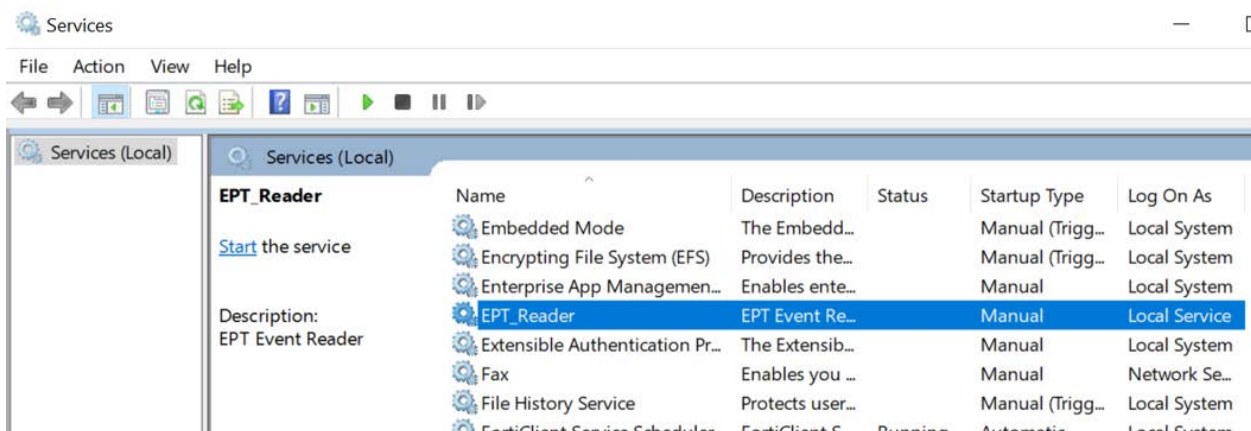
6. Edit **service.xml**, using the values below:



If you are using this functionality independent of the EPT feature, you can apply any appropriate serviceName, e.g. “<serviceName>Event_Reader</serviceName>”.

7. Save the file.
8. Open “**cmd**” as Administrator.
9. Navigate to the EventReader location, e.g.:
D:\>cd EventReader
10. Run the following command:
D:\EventReader>ServiceWrapper.exe -install
11. In Windows, navigate to Administrative Tools > Services.

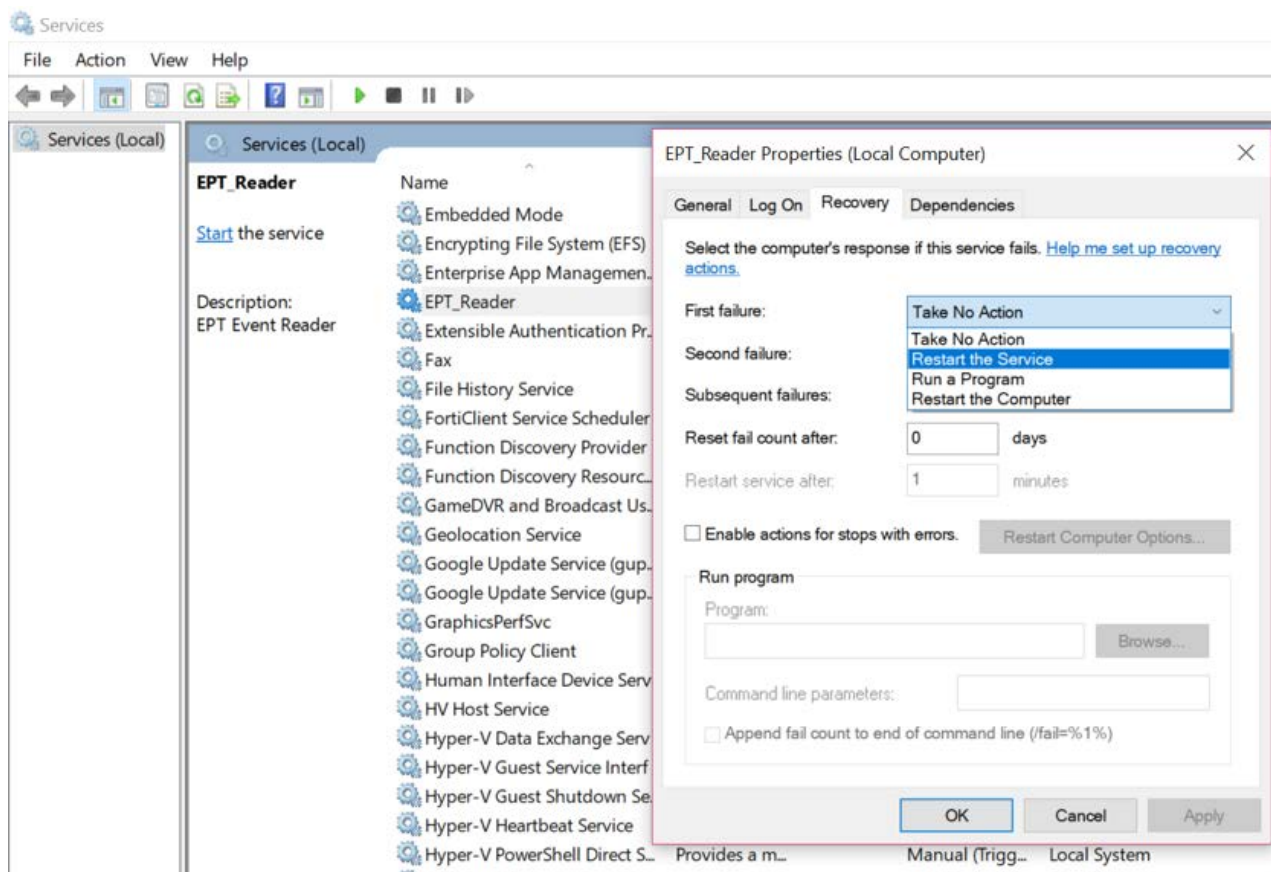
You should see the **EPT_Reader** service installed:



EPT_READER SERVICE RECOVERY OPTIONS

To set recovery options in case of failure:

1. Right-click on the **EPT_Reader** service and select **Properties** from the context menu.
2. Click on the **Recovery** tab.



START/STOP EPT_READER SERVICE

- To start EPT_Reader service:
Right-click on the **EPT_Reader** service and select **Start** from the context menu.
- To stop EPT_Reader service:
Right-click on the **EPT_Reader** service and select **Stop** from the context menu.

UNINSTALL EVENT READER WINDOWS SERVICE

To uninstall Event Reader Windows Service:

1. Open “**cmd**” as Administrator.
2. Navigate to **EventReader** location, e.g.:
D:\>cd EventReader
3. Run the command:
D:\EventReader>ServiceWrapper.exe -uninstall

Index

Symbols

=
 assigning variables 474

A

Abs
 ASCII 117
 bitMAP 433
 defectMAP 338
 Events 499
 Fab 255
 LEH 173
 WEH 208
Accept_Data flag 16
AddInvChar
 ASCII 80
 Events 482
 Fab 236
AddInvInteger
 ASCII 80
 Events 482
 Fab 236
AddInvReal 71
 ASCII 79
 Events 476, 482
 Fab 236
AddInvString
 ASCII 79
 Events 482
 Fab 236
-addpatts
 bitMAP 437
AddSep 72, 306
 ASCII 78
 bitMAP 396, 400
 defectMAP 309
 Events 476, 480
 Fab 234
 LEH 161
 WEH 195

AemReport
 ASCII 139
 Fab 261
After
 ASCII 84
 bitMAP 405
 defectMAP 314
 Events 485
 Fab 240
 LEH 166
 WEH 200
Alarm and Events Rules
 Manager
 ASCII 139
 Fab 260
-arg
 ASCII 127
 bitMAP 437
 defectMAP 342
 Events 512
 Fab 258
 LEH 176
 WEH 211
Argument
 ASCII 85
 bitMAP 406
 defectMAP 315
 Events 486
 Fab 242
 LEH 167
 WEH 201
Arithmetic 73, 306
 bitMAP 397
 Events 477
Arrays
 ASCII 63
 bitMAP 392
 defectMAP 302
 Events 471
 Fab 221
 LEH 153
 WEH 187
ASCII
 Command Line Arguments
 127

ASCII Data Reader 55
ASCII Format File 56
avg 35

B

-b2dext
 bitMAP 437
Before
 ASCII 84
 bitMAP 405
 defectMAP 314
 Events 486
 Fab 240
 LEH 166
 WEH 200
-bigcond
 Fab 258
-bigstring
 ASCII 127
 Fab 258
Bin Map Data 39
Bin Summaries 33
BIN_LOG 50
BIN_LOG table 30
-bincons
 ASCII 127
-bingal
 ASCII 127
Binning
 built-in functions, ASCII
 111
bitMAP
 Command Line Arguments
 436
Bricking 102
Built-in Functions
 ASCII 76
 bitMAP 406
 defectMAP 308
 Events 479
 Fab 233
 LEH 160
 WEH 194

Built-in Functions Definitions

ASCII 76
 bitMAP 399
 defectMAP 308
 Events 479
 Fab 233
 LEH 160
 WEH 194

C

Carriage Return

ASCII 62
 bitMAP 391
 defectMAP 301
 Events 470
 Fab 220
 LEH 152
 WEH 186

-cass_threads 512

LEH 176, 211

-cassandra

ASCII 127
 Events 512
 LEH 176
 WEH 211

-cassandra_level

ASCII 127

Character Constants 62, 301

bitMAP 391
 Events 470
 Fab 220
 LEH 152
 WEH 186

character limit, string

ascii 63
 bitMAP 392
 Defect Reader 302
 Events Reader 471
 Fab 221
 LEH 153
 WEH 187

-class

ASCII 127
 bitMAP 436
 Events 512
 LTX77 385
 STDF3 277

-classoverwrite

defectMAP 342

ClearLimits

ASCII 80
 Fab 237

ClearSep

ASCII 78
 bitMAP 400
 defectMAP 309
 Events 480

Fab 234

LEH 161

WEH 195

Cluster Factor

calculation 51

cnt 35

Command Line Arguments 15

ASCII 127
 bitMAP 436
 defectMAP 342
 Events 512
 Fab 258
 LEH 176
 LTX77 385
 STDF3 276
 WEH 211

Comments

ASCII 61
 bitMAP 390
 defectMAP 300
 Events 469

-complete_stats

ASCII 128

Composite Class Instances

built-in functions, bitMAP
 422

CONDITION table 30

Conditional Control

ASCII 74
 bitMAP 398
 defectMAP 307
 Events 478
 Fab 232
 LEH 159
 WEH 193

Conditions 34, 57, 468

ASCII 64
 bitMAP 393
 built-in functions, Fab 251
 defectMAP 303
 Events 472
 Fab 222
 LEH 154
 WEH 188

-conditions

ASCII 128
 Fab 258

Conditions and Indexes

STDF3 274
 STDF4 289

Configuration File 13

-cons

ASCII 128

-cons_latest

ASCII 128

Constants

ASCII 62
 bitMAP 391
 Character 62, 301, 391, 470
 Character, Fab 220
 Character, LEH 152
 Character, WEH 186
 defectMAP 301
 Events 470
 Fab 220
 Integer 62, 301, 391, 470
 Integer, Fab 220
 Integer, LEH 152
 Integer, WEH 186
 LEH 152
 Real 62, 301, 391, 470
 Real, Fab 220
 Real, LEH 152
 Real, WEH 186
 String 62, 301, 391, 470
 String, Fab 220
 String, LEH 152
 String, WEH 186
 WEH 186

Contents 3



- Coordinate Normalization [138](#)
 - built-in functions, ASCII [109](#)
- CR
 - ASCII [62](#)
 - bitMAP [391](#)
 - defectMAP [301](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- Customer
 - built-in functions, ASCII [99](#)
 - built-in functions, bitMAP [429](#)
 - built-in functions, defectMAP [335](#)
 - built-in functions, Events [495](#)
- D**
- d0 [33](#), [51](#)
- d0 calculation [51](#)
- Data
 - built-in functions, ASCII [79](#)
 - built-in functions, bitMAP [401](#)
 - built-in functions, defectMAP [310](#)
 - built-in functions, Events [481](#)
 - built-in functions, Fab [235](#)
 - built-in functions, LEH [162](#)
 - built-in functions, WEH [196](#)
- Data File, example
 - bitMAP [439](#)
- Data Integration
 - Multibin [140](#)
- Data Partitioning [86](#)
- Data Retrieval
 - bitMAP [389](#)
- Data Retrieval, built-in functions
 - ASCII [79](#)
 - bitMAP [401](#)
 - defectMAP [310](#)
 - Events [481](#)
 - Fab [235](#)
 - LEH [162](#)
 - WEH [196](#)
- Database
 - ASCII [122](#)
 - Events [501](#)
 - LTX77 [388](#)
 - STDF3 [276](#)
- dataBASE Readers
 - Database Model [30](#)
- Database, built-in functions
 - ASCII [86](#)
 - bitMAP [406](#)
 - defectMAP [315](#)
 - Events [487](#)
 - Fab [242](#)
 - LEH [167](#)
 - WEH [201](#)
- datadbs
 - ASCII [129](#)
 - Events [513](#)
 - Fab [258](#)
 - LEH [176](#)
 - WEH [211](#)
- Datalog Files
 - importing data from [10](#)
- Date-Time
 - built-in functions, ASCII [103](#)
 - built-in functions, Events [497](#)
 - built-in functions, Fab [251](#)
 - built-in functions, LEH [172](#)
 - built-in functions, WEH [207](#)
- db
 - ASCII [129](#)
 - bitMAP [436](#)
 - defectMAP [342](#)
 - Events [513](#)
 - Fab [258](#)
 - LEH [176](#)
 - LTX77 [385](#)
 - STDF3 [276](#)
 - WEH [211](#)
- db_accept [16](#)
 - ASCII [129](#)
 - bitMAP [436](#)
 - Events [513](#)
 - LEH [176](#)
 - LTX77 [385](#)
 - STDF3 [276](#)
 - WEH [211](#)
- DbAlignDefect
 - ASCII [110](#), [138](#)
- DbAlignDefectOnly
 - ASCII [110](#)
- DbBinIndex
 - ASCII [102](#)
- DbBinName
 - ASCII [111](#), [112](#)
- DbBinNameColorOrder
 - ASCII [113](#)
- DbBinNameOnly
 - ASCII [111](#)
- DbBinNameOrder
 - ASCII [112](#)
- DbBinSum
 - ASCII [113](#)
- DbBinTest
 - ASCII [102](#)
- DbBinTypeIndex
 - ASCII [102](#)
 - Events [497](#)
- DbCalcQueueTime
 - LEH [173](#)
 - WEH [208](#)
- DbChamberIndex
 - Events [496](#)
- DbCircularZones
 - ASCII [119](#), [340](#)
 - bitMAP [435](#)

- DbCreateDefaultzones
 - ASCII [119](#)
 - bitMAP [434](#)
 - defectMAP [339](#)
- DbCvInLine
 - defectMAP [315](#)
- DbDataDesc
 - LEH [162](#)
 - WEH [196](#)
- DbDfTagAction
 - defectMAP [326](#)
- DbDieCnt
 - ASCII [89](#)
- DbDieId
 - ASCII [85](#)
- DbDieIdIndex
 - ASCII [85](#)
- DbDieMap
 - ASCII [110](#)
- DbDieXYIndexes
 - ASCII [103](#)
 - Fab [251](#)
- DbDtFormat
 - LEH [172](#)
 - WEH [207](#)
- dBDumpLayer
 - defectMAP [315](#)
- DbElectricUpdate
 - defectMAP [328](#)
- DbEndTime
 - ASCII [105](#)
 - Events [498](#)
- DbEquip3
 - ASCII [96](#)
 - bitMAP [427](#)
 - defectMAP [334](#)
 - Fab [249](#)
- DbEquip4
 - ASCII [97](#)
 - defectMAP [334](#)
 - Fab [249](#)
- DbEquip5
 - ASCII [97](#)
 - defectMAP [334](#)
 - Fab [249](#)
- DbEquip6
 - ASCII [97](#)
 - defectMAP [334](#)
 - Fab [250](#)
- DbEventEquipIndex
 - Events [496](#)
- DbEventEquipNumIndex
 - Events [493](#)
- DbEventIndex
 - Events [496](#)
- DbEventLotIndex
 - Events [495](#)
- DbEventRecipeIndex
 - Events [494](#)
- DbEventWafIndex
 - Events [496](#)
- DbFab
 - ASCII [86](#)
 - bitMAP [422](#)
 - defectMAP [328](#)
 - Events [487](#)
 - LEH [167](#)
 - WEH [201](#)
- DbFamily
 - ASCII [87](#)
 - bitMAP [423](#)
 - defectMAP [329](#)
 - Events [488](#)
 - Fab [245](#)
 - LEH [168](#)
 - WEH [202](#)
- DbHandler
 - ASCII [96](#)
 - bitMAP [428](#)
 - defectMAP [335](#)
- DbHandlerType
 - defectMAP [326](#)
- DbHistLimits [67](#), [68](#), [226](#)
 - ASCII [107](#)
 - Fab [253](#)
- DbIndexes
 - LEH [172](#)
 - WEH [206](#)
- DbInline
 - Fab [255](#)
- DbLimDieCnt
 - ASCII [89](#)
- DbLimitsSet
 - ASCII [107](#)
 - Fab [253](#)
- DbLot
 - ASCII [92](#)
 - bitMAP [425](#)
 - defectMAP [331](#)
 - Events [491](#)
- DbLotTag
 - ASCII [108](#)
 - bitMAP [430](#)
 - defectMAP [336](#)
 - Events [498](#)
 - Fab [254](#)
 - LEH [171](#)
 - WEH [205](#)
- DbMDate
 - Fab [252](#)
- DbMEquip
 - Fab [248](#)
- DbMetaData
 - ASCII [117](#)
- DbMetaIndex
 - Events [495](#)
- DbModuleIndex
 - ASCII [85](#)
- DbNoRes
 - LEH [171](#)
 - WEH [205](#)
- DbNormalizeMap
 - ASCII [109](#), [138](#)
- DbNoSumCalc
 - ASCII [117](#)
- DbNumberResRework
 - ASCII [106](#)
- DbOperator
 - ASCII [98](#), [99](#)
 - bitMAP [428](#)
 - defectMAP [335](#)
 - Events [494](#)
 - Fab [250](#)
- DbOrgDieXYIndexes
 - ASCII [109](#)

-dboutliers	DbProgRel	DbSrcLotTag
ASCII 129	ASCII 90	ASCII 109
Fab 258	defectMAP 330	bitMAP 431
LTX77 386	Events 489	defectMAP 337
STDF3 277	LEH 169	Events 498
DbPackage	WEH 203	Fab 254
ASCII 98	DbProgRev	LEH 172
DbPackageType	ASCII 91	WEH 206
ASCII 98	defectMAP 330	DbStage
DbParGrp	Events 490	ASCII 93
ASCII 115	Fab 243	bitMAP 427
Events 499	LEH 169	defectMAP 333
DbPartitionAction	WEH 203	Events 493
ASCII 86	DbRadiusZones	Fab 247
DbPDate	ASCII 120	DbStage(semi-dynamic)
Fab 253	bitMAP 435	ASCII 94
DbPEquip	defectMAP 340	bitMAP 427
Fab 248	DbRecipe	Events 493
DbProcess	ASCII 94	DbStageCond
ASCII 88	Events 488	LEH 172
bitMAP 424	Fab 248	WEH 206
defectMAP 329	DbResultTime	DbStartTime
Events 488	defectMAP 325	ASCII 103
Fab 245	DbRetstBin	Events 497
LEH 168	ASCII 114	DbStep
WEH 202	DbRework	ASCII 93
DbProduct	ASCII 105	bitMAP 426
ASCII 88	DbReworkAction	defectMAP 333
bitMAP 424	ASCII 105	Events 492
defectMAP 329	defectMAP 336	Fab 247
Events 488, 489	Events 490	DbStepCond
Fab 246	Fab 253	LEH 172
LEH 168	DbRowCnt	WEH 206
WEH 202	ASCII 114	DbSumLevel
DbProdWmapCfg	Events 489	ASCII 116
ASCII 89	DbSampleType	DbTagAction
DbProgClass	defectMAP 337	ASCII 108
ASCII 91	DbSiteIndex	DbTechnology 167, 201
Events 490	ASCII 103	ASCII 87
DbProgram	Fab 251	bitMAP 423
ASCII 89	DbSrcLot	defectMAP 329
defectMAP 330	ASCII 92	Events 487
Events 489	bitMAP 426	Fab 244
LEH 169	defectMAP 332	DbTester
WEH 203	Events 491	ASCII 96
	DbSrcLotIndex	bitMAP 427
	LEH 172	defectMAP 335

- DbTesterType
 - defectMAP 326
- DbTestMode
 - ASCII 115
 - bitMAP 432
- DbTimeResRework
 - ASCII 105
- DbUglyDie
 - ASCII 115
- DbUpdateBinColor
 - ASCII 113
- DbUpdateBinName
 - ASCII 113
- DbUpdateDefect
 - defectMAP 327
- DbUpdateLimits 68
 - ASCII 107
- DbUpdateLot
 - ASCII 93
 - Events 491
 - Fab 246
- DbUseSrcLot
 - WEH 206
- DbWaferAppend
 - ASCII 101
- DbWaferIndex
 - ASCII 103
 - Fab 251
- DbWfDesc
 - ASCII 102
 - bitMAP 430
 - defectMAP 333
 - Events 492
 - WEH 205
- DbWfNum
 - ASCII 115
 - bitMAP 433
 - defectMAP 333
 - Events 492
 - WEH 208
- DbWfTag
 - ASCII 109
 - bitMAP 430
 - defectMAP 337
 - Events 498
- DbWfTestMode
 - ASCII 101
- DbZonalBins
 - ASCII 100
- dc_thresh
 - defectMAP 327
- dd_thresh
 - defectMAP 326
- Debugging, built-in functions
 - ASCII 117
 - bitMAP 433
 - defectMAP 338
 - Events 499
 - Fab 255
 - LEH 173
 - WEH 209
- DEF table 30
- Defect Density
 - calculation 51
- Defect Image Pointers 364
- defectMAP
 - Command Line Arguments 342
 - defectMAP Format File 347
 - defectMAP-Specific Functions
 - built-in 315
 - defectsize
 - defectMAP 342
 - defext
 - ASCII 129
 - defectMAP 342
 - Events 513
 - Fab 259
 - LEH 176
 - LTX77 385
 - STDF3 277
 - WEH 211
- DelInvChar
 - ASCII 80
 - Events 482
 - Fab 236
- DelInvInteger
 - ASCII 80
 - Events 482
 - Fab 236
- DelInvReal
 - ASCII 80
 - Events 482
 - Fab 236
- DelInvString
 - ASCII 80
 - Events 482
 - Fab 236
- DelSep
 - ASCII 78
 - bitMAP 400
 - defectMAP 309
 - Events 480
 - Fab 234
 - LEH 161
 - WEH 195
- dfalign
 - ASCII 129
- Dfmt
 - STDF4 282
- Die -Level Traceability 85
- DpLoad.pl 12
 - How to Run 16
- DpLoadMgr.pl 29
- dtexit
 - ASCII 129
- DTR Records 280
- DTR_scan
 - STDF4 282
- dynamic
 - ASCII 129
 - Events 513
 - Fab 259
 - LTX77 385
 - STDF3 276
- dynamic 31
- Dynamic Programs 37
- E**
- end_time
 - ASCII 129
 - Events 513
 - LEH 176
 - WEH 211
- endpoint
 - ASCII 139
 - Fab 261
- end-time
 - bitMAP 437
 - rework 39
- Environment Variables 17

- Equipment
 - built-in functions, ASCII 94
 - built-in functions, bitMAP 427
 - built-in functions, defectMAP 334
 - built-in functions, Events 493
 - built-in functions, Fab 248
- Error Code
 - ASCII 76
 - bitMAP 399
 - defectMAP 308
 - Events 479
 - Fab 233
 - LEH 160
 - WEH 194
- ErrorCode
 - ASCII 118
 - bitMAP 433
 - defectMAP 338
 - Events 499
 - Fab 256
 - LEH 174
 - WEH 209
- Events
 - Command Line Arguments 512
- exitcode
 - ASCII 118, 339
- ExitScript
 - ASCII 118
 - bitMAP 433
 - defectMAP 338
 - Events 499
 - Fab 256
 - LEH 174
 - WEH 209
- exitscript 118
- export 123
 - ASCII 129
 - STDF4 282
- ExportExt
 - ASCII 121
- exportonly 123
 - ASCII 130
- extend_par
 - ASCII 130
- Extension 15, 16
- ExtractString
 - ASCII 81
 - bitMAP 403
 - defectMAP 311
 - Events 483
 - Fab 238
 - LEH 164
 - WEH 198
- F**
- Fab
 - built-in functions, ASCII 86
 - built-in functions, bitMAP 422
 - built-in functions, defectMAP 328
 - built-in functions, Events 487
 - built-in functions, LEH 167
 - built-in functions, WEH 201
- FAB Data Reader 215
- Fab Data Reader
 - dataPOWER Interface 257
- Family
 - built-in functions, ASCII 87
 - built-in functions, bitMAP 423
 - built-in functions, defectMAP 329
 - built-in functions, Events 488
 - built-in functions, Fab 245
 - built-in functions, LEH 168
 - built-in functions, WEH 202
- fcenterxy
 - ASCII 130
- FF
 - ASCII 62
 - bitMAP 391
 - defectMAP 301
 - Events 470
 - Fab 220
 - LEH 152
 - WEH 186
- File Navigation
 - WS readers 280
- File Navigation, built-in functions
 - ASCII 77
 - bitMAP 400
 - defectMAP 309
 - Events 480
 - Fab 234
 - LEH 161
 - WEH 195
- File Pointer
 - ASCII 76
 - bitMAP 399
 - defectMAP 308
 - Events 479
 - Fab 233
 - LEH 160
 - WEH 194
- file_path
 - ASCII 130
 - bitMAP 436, 437
 - defectMAP 343
 - Events 513
 - Fab 259
 - LEH 176
 - WEH 211
- filter
 - ASCII 130
 - Fab 259
- first_pass
 - ASCII 130
- fix_cond_values
 - ASCII 130
 - Events 513
 - Fab 259
 - LEH 176
 - WEH 211

-fmt

ASCII [130](#)
bitMAP [436](#)
Events [513](#)
Fab [259](#)
LEH [176](#)
LTX77 [385](#)
STDF3 [276](#)
WEH [211](#)

Form Feed

ASCII [62](#)
bitMAP [391](#)
defectMAP [301](#)
Events [470](#)
Fab [220](#)
LEH [152](#)
WEH [186](#)

Format File

ASCII [56](#)
defectMAP [347](#)
LEH [148](#)
LTX77 [386](#)
WEH [181](#)
WS readers [280](#)

Format File Blocks

ASCII [62](#)
bitMAP [391](#)
Events [470](#)
Fab [220](#)
LEH [152](#)
WEH [186](#)

Format File, example

bitMAP [444](#)

Format File, Overview

bitMAP [439](#)

Format Files [11](#)

STDF3 [273](#)
STDF4 [288](#)

Format Files and Options

LTX77 [385](#)

Format Files, Loading

STDF3 [275](#)
STDF4 [295](#)

Format Files, Saving

STDF3 [275](#)
STDF4 [295](#)

FullPath of Reader [15](#)**G****GetChar**

bitMAP [402](#)
defectMAP [311](#)

GetChars

ASCII [81](#)
bitMAP [402](#)
defectMAP [311](#)
Events [483](#)
Fab [237](#)
LEH [163](#)
WEH [197](#)

GetCharsTrim

ASCII [81](#)
bitMAP [402](#)
defectMAP [311](#)
Events [483](#)
Fab [238](#)
LEH [163](#)
WEH [197](#)

GetInt

ASCII [83](#)
bitMAP [403](#)
defectMAP [313](#)
Events [484](#)
Fab [239](#)
LEH [165](#)
WEH [199](#)

GetLeftChars

ASCII [81](#)
bitMAP [402](#)
defectMAP [311](#)
Events [483](#)
Fab [238](#)
LEH [164](#)
WEH [198](#)

GetLine

ASCII [81](#)
bitMAP [402](#)
defectMAP [311](#)
Events [483](#)
Fab [237](#)
LEH [163](#)
WEH [197](#)

GetLineLen

ASCII [79](#)
bitMAP [401](#)
defectMAP [310](#)
Events [481](#)
Fab [235](#)
LEH [162](#)
WEH [196](#)

GetMidChars

ASCII [82](#)
bitMAP [403](#)
defectMAP [312](#)
Events [484](#)
Fab [238](#)
LEH [164](#)
WEH [198](#)

GetPrevInt

ASCII [83](#)
bitMAP [404](#)
defectMAP [313](#)
Events [484](#)
Fab [239](#)
LEH [165](#)
WEH [199](#)

GetPrevReal

ASCII [83](#)
bitMAP [404](#)
defectMAP [313](#)
Events [485](#)
Fab [239](#)
LEH [165](#)
WEH [199](#)

GetPrevWord

ASCII [81](#)
bitMAP [401](#)
defectMAP [310](#)
Events [482](#)
Fab [237](#)
LEH [163](#)
WEH [197](#)

GetQuotedWord
 ASCII 81
 bitMAP 402
 defectMAP 311
 Events 483
 Fab 237
 LEH 163
 WEH 197
 GetReal
 ASCII 83
 bitMAP 404
 defectMAP 313
 Events 485
 Fab 239
 LEH 165
 WEH 199
 GetRightChars
 ASCII 82
 bitMAP 402
 defectMAP 312
 Events 483
 Fab 238
 LEH 164
 WEH 198
 GetWord
 ASCII 81
 bitMAP 401
 defectMAP 310
 Events 482
 Fab 237
 LEH 163
 WEH 197
 GetWordAfter
 ASCII 82
 bitMAP 403
 defectMAP 312
 Events 484
 Fab 239
 LEH 164
 WEH 198

GetWordBefore
 ASCII 82
 bitMAP 403
 defectMAP 312
 Events 484
 Fab 239
 LEH 165
 WEH 199
 Go To
 built-in functions, ASCII 77
 built-in functions, bitMAP 400
 built-in functions, defectMAP 309
 built-in functions, Events 480
 built-in functions, Fab 234
 built-in functions, LEH 161
 built-in functions, WEH 195
 GoBackTo
 ASCII 77
 bitMAP 400
 defectMAP 309
 Events 480
 Fab 234
 LEH 161
 WEH 195
 Goto
 ASCII 77
 bitMAP 400
 defectMAP 309
 Events 480
 Fab 234
 LEH 161
 WEH 195
 GotoBOF
 ASCII 77
 bitMAP 400
 defectMAP 309
 Events 480
 Fab 234
 LEH 161
 WEH 195

GotoEOF
 ASCII 77
 bitMAP 400
 defectMAP 309
 Events 480
 Fab 234
 LEH 161
 WEH 195
H
 Hardware
 WEH 208
 HIST_BIN table 30
 Historical Limits
 ASCII 66
 Fab 224
 HOL 36
 Horizontal Tab
 ASCII 62
 bitMAP 391
 defectMAP 301
 Events 470
 Fab 220
 LEH 152
 WEH 186
 -hosted_retest
 ASCII 130
 HPL 36
 HSL 36
 HT
 ASCII 62
 bitMAP 391
 defectMAP 301
 Events 470
 Fab 220
 LEH 152
 WEH 186
 HWL 36
I
 Identifiers
 ASCII 61
 bitMAP 390
 defectMAP 300
 Events 469
 IDS_CLEANUP_DAYS 16, 17
 IDS_FILES_PER_LOOP 17

- IDS_MOD_TIME [17](#)
- IDS_RM_UNUSED_DAYS [17](#)
- IDS_SLEEP_SECONDS [17](#)
- image_path
 - defectMAP [343](#)
- img
 - bitMAP [437](#)
- in_place_alter
 - ASCII [131](#)
- inBlockCount
 - bitMAP [407](#)
- inBlockName
 - bitMAP [407](#)
- inBlockNumber
 - bitMAP [407](#)
- inBlockType
 - bitMAP [407](#)
- inClassName
 - bitMAP [415](#), [416](#)
- inColumnCount
 - bitMAP [407](#)
- inColumnDirection
 - bitMAP [409](#)
- inDescription
 - bitMAP [408](#)
- inDesignName
 - bitMAP [408](#)
- inDevNum
 - bitMAP [411](#)
- Index
 - Data Table [58](#), [468](#)
 - Data Table, Fab [217](#)
- Indexes [34](#), [58](#), [468](#)
 - ASCII [64](#)
 - bitMAP [393](#)
 - built-in functions, ASCII [102](#)
 - built-in functions, Events [495](#)
 - built-in functions, Fab [251](#)
 - built-in functions, LEH [172](#)
 - built-in functions, WEH [206](#)
 - defectMAP [303](#)
 - Events [472](#)
 - Fab [222](#)
 - LEH [154](#)
 - WEH [188](#)
- indexes
 - ASCII [130](#)
 - bitMAP [437](#)
 - defectMAP [343](#)
 - Events [513](#)
 - Fab [259](#)
 - LEH [177](#)
 - LTX77 [386](#)
 - STDF3 [277](#)
 - WEH [212](#)
- inDieX
 - bitMAP [411](#)
- inDieY
 - bitMAP [411](#)
- inFailCount
 - bitMAP [415](#), [416](#)
- inFirstColumn
 - bitMAP [408](#)
- inFirstRow
 - bitMAP [408](#)
- inMaxCol
 - bitMAP [415](#), [416](#)
- inMaxRow
 - bitMAP [415](#), [416](#)
- inMemCellName
 - bitMAP [407](#)
- inMemInstanceName
 - bitMAP [408](#), [411](#)
- inMemInstanceNumber
 - bitMAP [411](#)
- inMemoryNumber
 - bitMAP [408](#)
- inMinCol
 - bitMAP [415](#), [416](#)
- inMinRow
 - bitMAP [415](#), [416](#)
- Input Stream [70](#), [156](#), [190](#)
 - bitMAP [394](#)
 - defectMAP [305](#)
 - Events [474](#)
 - Fab [228](#)
- inRowDirection
 - bitMAP [409](#)
- instanceName
 - bitMAP [411](#)
- Integer Constants [62](#), [301](#)
 - bitMAP [391](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- Integers
 - built-in functions, ASCII [83](#)
 - built-in functions, bitMAP [403](#)
 - built-in functions, defectMAP [313](#)
 - built-in functions, Events [484](#)
 - built-in functions, Fab [239](#)
 - built-in functions, LEH [165](#)
 - built-in functions, WEH [199](#)
- IntToReal
 - bitMAP [405](#)
- IntToStr
 - ASCII [84](#)
 - bitMAP [405](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [167](#)
 - WEH [201](#)

Invalid Data
 ASCII 71
 Events 476
 Fab 230
 invertFailBits
 bitMAP 415, 416
 inX1
 bitMAP 407, 409
 inX2
 bitMAP 408, 409
 inY1
 bitMAP 408
 inY2
 bitMAP 408, 409
 isLibDefault
 bitMAP 410, 411
 IsNumber
 ASCII 83
 bitMAP 404
 defectMAP 313
 Events 485
 Fab 240
 LEH 165
 WEH 199
 IsProgramNew
 defectMAP 315
 IsString
 ASCII 83
 bitMAP 404
 defectMAP 313
 Events 485
 Fab 240
 LEH 165
 WEH 199

K

Key Conditions 58, 468
 Key Indexes 58, 468
 -keyIndexes
 ASCII 131
 Keywords
 ASCII 61
 bitMAP 390
 defectMAP 300
 Events 469

L

Left
 ASCII 83
 bitMAP 404
 defectMAP 313
 Events 485
 Fab 240
 LEH 166
 WEH 200
 LEH Data Reader 147
 LEH Format File 148
 LEH Reader
 dataPOWER Interface 175
 Lexical Conventions
 ASCII 61
 bitMAP 390
 Events 469
 Fab 219
 LEH 151
 WEH 184
 LIM table 30, 31
 -lim_path
 ASCII 131
 -limext
 ASCII 131
 Fab 259
 LimFile
 ASCII 80
 Fab 237
 Limit Conditions 59
 Limits 36
 ASCII 65
 built-in functions, ASCII
 80, 107
 built-in functions, Fab 237,
 253
 Fab 224
 Limits and Scaling
 ASCII 65
 Limits Table 31
 -limitsinclude
 STDF4 283
 -limitonly 67
 ASCII 131
 Fab 259
 LimitsProgram
 Fab 243

Logging Pattern Instances
 built-in functions, bitMAP
 412
 Logical 73, 306
 bitMAP 397
 Events 477
 LogLimits
 ASCII 80
 Fab 237
 LogResult
 ASCII 79
 Events 481
 Fab 235
 LEH 162
 WEH 196
 LOL 36
 lol 36
 Loop Control
 ASCII 74
 bitMAP 398
 defectMAP 307
 Events 478
 Fab 232
 LEH 159
 WEH 193
 Lot
 built-in functions, ASCII
 92
 built-in functions, bitMAP
 425
 built-in functions,
 defectMAP 331
 built-in functions, Events
 491
 built-in functions, Fab 246
 built-in functions, LEH 171
 built-in functions, WEH
 205
 LOT table 30
 -lotext
 ASCII 131
 bitMAP 437
 defectMAP 343
 Fab 259
 LTX77 385
 STDF3 277

- lowercase
 - ASCII [131](#)
 - bitMAP [437](#)
 - defectMAP [343](#)
 - Events [514](#)
 - Fab [259](#)
 - LEH [177](#)
 - LTX77 [386](#)
 - STDF3 [277](#)
 - WEH [212](#)
- LPL [36](#)
- LSL [36](#)
- lsum
 - LTX77 [386](#)
- LTX77 Data Reader [383](#)
- LWL [36](#)
- M**
- m [33](#), [51](#)
 - calculation [51](#)
- Main Block
 - ASCII [65](#)
 - bitMAP [394](#)
 - defectMAP [304](#)
 - Events [473](#)
 - Fab [223](#)
 - LEH [155](#)
 - WEH [189](#)
- Mathematical, built-in
 - functions
 - ASCII [117](#)
 - bitMAP [433](#)
 - defectMAP [337](#)
 - Events [499](#)
 - Fab [255](#)
 - LEH [173](#)
 - WEH [208](#)
- max [36](#)
- max_cols
 - STDF4 [283](#)
- max_file_cols
 - STDF4 [283](#)
- Maximum Number of
 - Parameters
 - dbascii [57](#)
- Maximum Wafers Per Lot
 - dbascii [69](#)
- maxtests
 - LEH [177](#)
 - WEH [212](#)
- maxtime
 - ASCII [131](#)
 - bitMAP [438](#)
 - defectMAP [343](#)
 - Events [514](#)
 - Fab [259](#)
 - LEH [177](#)
 - WEH [212](#)
- md0
 - ASCII [131](#)
- mem_dsgn_name
 - bitMAP [408](#)
- memoryCellName
 - bitMAP [410](#), [411](#)
- Mid
 - ASCII [83](#)
 - bitMAP [404](#)
 - defectMAP [313](#)
 - Events [485](#)
 - Fab [240](#)
 - LEH [166](#)
 - WEH [200](#)
- min [36](#)
- Miscellaneous
 - built-in functions, ASCII [114](#)
 - built-in functions, bitMAP [432](#)
 - built-in functions, defectMAP [337](#)
 - built-in functions, Events [499](#)
 - built-in functions, Fab [255](#)
 - built-in functions, WEH [207](#)
- Multibin [140](#)
- multimemconfig [438](#)
- N**
- n
 - ASCII [131](#)
- New Line
 - ASCII [62](#)
 - bitMAP [391](#)
 - defectMAP [301](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- NL
 - ASCII [62](#)
 - bitMAP [391](#)
 - defectMAP [301](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- nlo [35](#)
- nls [36](#)
- no_normalize [343](#)
- nodata
 - ASCII [132](#)
- NoImageLoad
 - defectMAP [321](#)
- nolimprogram
 - ASCII [132](#)
 - Fab [260](#)
- noptr
 - STDF4 [283](#)
- nores
 - LEH [177](#)
 - WEH [212](#)
- normalize
 - ASCII [132](#)
- NormalizeIndex
 - Events [497](#)
- normext
 - Events [514](#)
- noscale
 - STDF4 [283](#)
- NOT_ACCEPTED [16](#)

NotEndOfFile
 ASCII 79
 bitMAP 401
 defectMAP 310
 Events 481
 Fab 235
 LEH 162
 WEH 196

NotProcessed 15, 16

-noxycheck
 ASCII 132

nuo 36

nus 36

O

One-Dimensional Arrays

ASCII 63
 bitMAP 392
 defectMAP 302
 Events 471
 Fab 221
 LEH 153
 WEH 187

Open File
 ASCII 78

OpenDirFile
 ASCII 79

OpenFile
 ASCII 78, 80
 bitMAP 401
 defectMAP 310
 Events 482
 Fab 236
 LEH 163
 WEH 197

Operator
 built-in functions, ASCII 98
 built-in functions, bitMAP 428
 built-in functions, defectMAP 335
 built-in functions, Events 494
 built-in functions, Fab 250

Operators
 ASCII 73
 bitMAP 397
 defectMAP 306
 Events 477
 Fab 231
 LEH 158
 WEH 192

Options 15, 16

Oracle Parameter Limit
 dbascii 75

Oracle Rollback Table
 dbascii 71
 Events 475
 Fab 229
 LEH 157
 WEH 191

-otca
 ASCII 132

-otgr
 ASCII 132

-otin
 ASCII 132

-otsc
 ASCII 132

-otu
 ASCII 132

-otun
 ASCII 133

Outlier Filtering 43

-outliers
 ASCII 133
 Fab 260
 LTX77 386
 STDF3 277

P

p1 35
 p10 35
 p5 35
 p90 35
 p95 35
 p99 35
 Package
 built-in functions, ASCII 98
 -parallel_stats
 ASCII 133

Parameter Limit, dbascii
 Oracle 75

-partition
 ASCII 133

-partition_action
 ASCII 133

paternSetName
 bitMAP 411

patternSetName
 bitMAP 410

-pct_increase
 ASCII 133

pg_key 30

-pgext
 STDF3 278

POW
 ASCII 117
 bitMAP 433
 defectMAP 338
 Events 499

Print
 ASCII 117
 bitMAP 433
 defectMAP 338
 Events 499
 Fab 255
 LEH 173
 WEH 209

PrintToFile
 ASCII 118
 bitMAP 433
 defectMAP 338
 Events 499
 Fab 255
 LEH 173
 WEH 209

Process

- built-in functions, ASCII 88
- built-in functions, bitMAP 424
- built-in functions, defectMAP 329
- built-in functions, Events 488
- built-in functions, Fab 245
- built-in functions, LEH 168
- built-in functions, WEH 202

Processed 15, 16**Product**

- built-in functions, ASCII 88
- built-in functions, bitMAP 424
- built-in functions, defectMAP 329
- built-in functions, Events 488
- built-in functions, Fab 246
- built-in functions, LEH 168
- built-in functions, WEH 202

-profile

- ASCII 134

Program

- built-in functions, ASCII 89
- built-in functions, bitMAP 425
- built-in functions, defectMAP 330
- built-in functions, Events 489
- built-in functions, Fab 242
- built-in functions, LEH 169
- built-in functions, WEH 203

Program Class 35**PROGRAM table** 30**Programs**

- definition 30

Q

- q1 35
- q2 35
- q3 35

R

- Raw bitMAP File Information
 - built-in functions, bitMAP 431, 464

reader configuration 10**Real**

- built-in functions, ASCII 83
- built-in functions, bitMAP 404
- built-in functions, defectMAP 313
- built-in functions, Events 485
- built-in functions, Fab 239
- built-in functions, LEH 165
- built-in functions, WEH 199

Real Constants 62, 301

- bitMAP 391
- Events 470
- Fab 220
- LEH 152
- WEH 186

RealToInt

- bitMAP 405

Recipe

- built-in functions, ASCII 94
- built-in functions, Events 494
- built-in functions, Fab 248

Regression Calculation 51**Relational** 73, 306

- bitMAP 397
- Events 477

-reload

- Events 514

RemoveLayer

- defectMAP 328

-replace_pipe

- ASCII 134

Report Types

- STDF3 273
- STDF4 289
- RES table 30
- res_aging
 - ASCII 134
 - bitMAP 436
 - defectMAP 343, 385
 - Events 514
 - Fab 260
 - LEH 177
 - STDF3 277
 - WEH 212

-resex

- ASCII 134
- bitMAP 437
- Events 514
- Fab 260
- LEH 177
- LTX77 385
- STDF3 277
- WEH 212

Results

- ASCII 65
- bitMAP 393
- defectMAP 303
- Events 472
- Fab 223
- LEH 155
- WEH 189

-retest

- ASCII 134
- STDF4 283

-review_time

- defectMAP 343

Rework 38

- built-in functions, ASCII 105
- built-in functions, defectMAP 336
- built-in functions, Fab 253

-rework

- LEH 177
- WEH 212

rework

- end-time 39
- start-time 39

-rework_action
 ASCII 134
 defectMAP 344
 Fab 260
 LTX77 386
 STDF3 278

rework_action 38

rework_flag 38

ReworkFiles 15, 16

Right

ASCII 83
 bitMAP 404
 defectMAP 313
 Events 485
 Fab 240
 LEH 166
 WEH 200

S

ScaleFactor 39, 69

ASCII 117
 bitMAP 433
 defectMAP 337
 Fab 255

Scaling 39

ASCII 65, 69
 Fab 224, 227

Script Example 124

Events, EquipEvent 504
 Events, LotEvent 506, 508

Search Dir 15

Search Directory 16

-semi_dynamic

ASCII 134
 Events 514
 LTX77 385
 STDF3 276

semi_dynamic 31

Separators

ASCII 72
 bitMAP 396
 built-in functions, ASCII 78
 built-in functions, bitMAP 400
 built-in functions, defectMAP 309
 built-in functions, Events 480
 built-in functions, Fab 234
 built-in functions, LEH 161
 built-in functions, WEH 195
 Events 476
 Fab 230
 LEH 158
 WEH 192

-sheet

STDF4 283

Single Quote

ASCII 62
 bitMAP 391
 defectMAP 301
 Events 470
 Fab 220
 LEH 152
 WEH 186

Site-Level Alignment 40

Skip

built-in functions, ASCII 78
 built-in functions, bitMAP 401
 built-in functions, defectMAP 310
 built-in functions, Events 481
 built-in functions, Fab 235
 built-in functions, LEH 161
 built-in functions, WEH 195

-skip_invalid_records

STDF4 283

SkipChars

ASCII 78
 bitMAP 401
 defectMAP 310
 Events 481
 Fab 235
 LEH 162
 WEH 196

SkipLines

ASCII 78
 bitMAP 401
 defectMAP 310
 Events 481
 Fab 235
 LEH 161
 WEH 195

SkipWords

ASCII 78
 bitMAP 401
 defectMAP 310
 Events 481
 Fab 235
 LEH 161
 WEH 195

-softbin

STDF3 278

-special_char

ASCII 135

SQ

ASCII 62
 bitMAP 391
 defectMAP 301
 Events 470
 Fab 220
 LEH 152
 WEH 186

Sqr

ASCII 117
 bitMAP 433
 defectMAP 338
 Events 499
 Fab 255
 LEH 173
 WEH 209

-srclot

LTX77 386
 STDF3 278

- ss [36](#)
- ssc
 - ASCII [135](#)
- Stage
 - built-in functions, ASCII [93](#)
 - built-in functions, bitMAP [427](#)
 - built-in functions, defectMAP [333](#)
 - built-in functions, Events [493](#)
 - built-in functions, Fab [247](#)
- start_time
 - ASCII [135](#)
 - Events [514](#)
 - LEH [177](#)
 - WEH [212](#)
- start-time
 - rework [39](#)
- static [31](#)
- Statistics
 - program class [35](#)
- Statistics Update Process [50](#)
- stats_aging
 - ASCII [135](#)
 - bitMAP [436](#)
 - defectMAP [344](#)
 - Fab [260](#)
 - LTX77 [385](#)
 - STDF3 [277](#)
- STATS_CFG [36](#)
- stdev [35](#)
- STDF3 Data Reader [271](#)
- STDF4 Data Reader [279](#)
- Step
 - built-in functions, ASCII [93](#)
 - built-in functions, bitMAP [426](#)
 - built-in functions, defectMAP [333](#)
 - built-in functions, Events [492](#)
 - built-in functions, Fab [247](#)
- Stepper Offset Normalization
 - ASCII [136](#)
 - defectMAP [344](#)
- StrCat
 - ASCII [84](#)
 - bitMAP [405](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [166](#)
 - WEH [200](#)
- String Constants [62](#), [301](#)
 - bitMAP [391](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- String Manipulation, built-in functions
 - ASCII [83](#)
 - bitMAP [404](#)
 - defectMAP [313](#)
 - Events [485](#)
 - Fab [240](#)
 - LEH [165](#)
 - WEH [199](#)
- string, character limit
 - ascii [63](#)
 - bitMAP [392](#)
 - Defect Reader [302](#)
 - Events Reader [471](#)
 - Fab [221](#)
 - LEH [153](#)
 - WEH [187](#)
- Strings
 - built-in functions, ASCII [81](#)
 - built-in functions, bitMAP [401](#)
 - built-in functions, defectMAP [310](#)
 - built-in functions, Events [482](#)
 - built-in functions, Fab [237](#)
 - built-in functions, LEH [163](#)
 - built-in functions, WEH [197](#)
- StrLen
 - ASCII [84](#)
 - bitMAP [406](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [167](#)
 - WEH [201](#)
- StrToInt
 - ASCII [84](#)
 - bitMAP [405](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [166](#)
 - WEH [200](#)
- StrToReal
 - ASCII [84](#)
 - bitMAP [405](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [166](#)
 - WEH [200](#)
- StrTrim
 - ASCII [84](#)
 - bitMAP [406](#)
 - defectMAP [314](#)
 - Events [486](#)
 - Fab [241](#)
 - LEH [167](#)
 - WEH [201](#)
- Sub-Strings
 - built-in functions, ASCII [81](#)
 - built-in functions, bitMAP [402](#)
 - built-in functions, defectMAP [311](#)
 - built-in functions, Events [483](#)
 - built-in functions, Fab [237](#)
 - built-in functions, LEH [163](#)
 - built-in functions, WEH [197](#)
- sum [36](#)

- System
 - ASCII 118
 - bitMAP 434
 - defectMAP 339
 - Events 500
 - Fab 256
 - LEH 174
 - WEH 209
- System, built-in functions
 - ASCII 118
 - defectMAP 339
 - Fab 256
 - LEH 174
 - WEH 209
- T**
 - tag_action
 - ASCII 135
 - defectMAP 326
 - Events 514
 - Tagging
 - built-in functions, ASCII 108
 - built-in functions, bitMAP 430
 - built-in functions, defectMAP 336
 - built-in functions, Events 498
 - built-in functions, Fab 254
 - built-in functions, LEH 171
 - built-in functions, WEH 205
 - Technology
 - built-in functions, ASCII 87
 - built-in functions, bitMAP 423
 - built-in functions, defectMAP 329
 - built-in functions, Events 487
 - built-in functions, Fab 244
 - built-in functions, LEH 167
 - built-in functions, WEH 201
 - Tester Summary 31
 - testtxt
 - STDF4 284
 - testtxtkey
 - STDF4 284
 - Tokens
 - ASCII 61
 - bitMAP 390
 - defectMAP 300
 - Events 469
 - ToLower
 - ASCII 83
 - bitMAP 404
 - defectMAP 313
 - Events 485
 - Fab 240
 - LEH 165
 - WEH 199
 - ToUpper
 - ASCII 83
 - bitMAP 404
 - defectMAP 313
 - Events 485
 - Fab 240
 - LEH 165
 - WEH 199
 - trigger_db
 - ASCII 136
 - Triggers 120
 - triggersonly
 - ASCII 135
 - Two-Dimensional Arrays
 - ASCII 64
 - bitMAP 392
 - defectMAP 302
 - Events 472
 - Fab 222
 - LEH 154
 - WEH 188
 - U**
 - u
 - ASCII 136
 - bitMAP 436
 - defectMAP 344
 - Events 514
 - Fab 260
 - LEH 177
 - WEH 212
 - Unit Scaling 39
 - UnUsedFiles 15
 - uol 36
 - update
 - Fab 260
 - update_all
 - Fab 260
 - UPDATE_STATS 50
 - updatebinname
 - ASCII 136
 - updatelot
 - ASCII 136
 - Events 515
 - uppercase
 - ASCII 136
 - bitMAP 437
 - defectMAP 344
 - Events 515
 - Fab 260
 - LEH 178
 - LTX77 386
 - STDF3 277
 - WEH 213
 - UpStat 13, 50
 - upstat
 - defectMAP 344
 - use_db_wmap
 - ASCII 136
 - V**
 - v
 - ASCII 136
 - defectMAP 344
 - Events 515
 - Fab 260
 - LEH 178
 - WEH 213
 - vAddToClassLUT
 - defectMAP 321
 - validate_wmcfg
 - ASCII 136

Variables

- ASCII [63](#)
- assignment [70](#), [156](#), [190](#), [474](#)
- assignment, Fab [228](#)
- bitMAP [392](#)
- defectMAP [302](#)
- Events [471](#)
- Fab [221](#)
- LEH [153](#)
- WEH [187](#)
- vDbAddDefectClass
 - defectMAP [322](#)
- vDbAddDefectClassByCode
 - defectMAP [323](#)
- vDBAddMemCellClass
 - bitMAP [406](#), [407](#)
- vDbAddToDieMapTable
 - defectMAP [315](#)
- vDbBrick
 - ASCII [102](#)
- vDbClusterToleranceDimensions
 - defectMAP [317](#)
- vDbConsEquipment
 - ASCII [94](#)
- vDbCustomer
 - ASCII [99](#), [429](#)
 - defectMAP [335](#)
 - Events [495](#)
- vDbDefect
 - defectMAP [319](#)
- vDbDefectSummary
 - defectMAP [316](#)
- vDbDieId
 - ASCII [85](#)
- vDbDieMemory
 - bitMAP [411](#)
- vDBEndComposite
 - bitMAP [422](#)
- vDbEndTime
 - ASCII [105](#)
 - Events [498](#)
- vDbEquip3
 - ASCII [97](#)
 - bitMAP [427](#)
 - defectMAP [334](#)
 - Fab [249](#)
- vDbEquip4
 - ASCII [97](#)
 - defectMAP [334](#)
 - Fab [249](#)
- vDbEquip5
 - ASCII [97](#)
 - defectMAP [334](#)
 - Fab [250](#)
- vDbEquip6
 - ASCII [97](#)
 - defectMAP [335](#)
 - Fab [250](#)
- vDbEquipDet
 - Events [494](#)
- vDbEquipment
 - ASCII [94](#)
 - Events [493](#)
- vDbEquipName
 - Events [493](#)
- vDbFab
 - ASCII [86](#)
 - bitMAP [423](#)
 - defectMAP [328](#)
 - Events [487](#)
 - LEH [167](#)
 - WEH [201](#)
- vDbFamily
 - ASCII [87](#)
 - bitMAP [423](#)
 - defectMAP [329](#)
 - Events [488](#)
 - Fab [245](#)
 - LEH [168](#)
 - WEH [202](#)
- vdbFileSetup
 - bitMAP [432](#)
- vDbGuid
 - ASCII [117](#)
- vDbHandler
 - ASCII [96](#)
 - bitMAP [428](#)
 - defectMAP [335](#)
- vDbHandlerType
 - defectMAP [326](#)
- vDbIsWaferPatterned
 - defectMAP [318](#)
- vDbLimEquipment
 - ASCII [95](#)
 - Fab [250](#)
- vDbLimFab
 - ASCII [86](#)
- vDbLimFamily
 - ASCII [87](#)
 - Fab [245](#)
- vDbLimLot
 - ASCII [92](#)
 - Fab [246](#)
- vDbLimLotClass
 - ASCII [93](#)
 - Fab [246](#)
- vDbLimPackage
 - ASCII [98](#)
- vDbLimPackageType
 - ASCII [98](#)
- vDbLimProcess
 - ASCII [88](#)
 - Fab [245](#)
- vDbLimProduct
 - ASCII [89](#)
 - Fab [246](#)
- vDbLimProgRel
 - ASCII [90](#)
- vDbLimSrcLot
 - ASCII [92](#)
 - Fab [246](#)
- vDbLimStartTime
 - ASCII [104](#)
 - Fab [251](#)
- vDbLimTechnology
 - ASCII [87](#)
 - Fab [245](#)
- vDBLogBox
 - bitMAP [415](#)
- vDBLogColumn
 - bitMAP [414](#)
- vDBLogCross
 - bitMAP [419](#)
- vDBLogCustom
 - bitMAP [417](#)



vDBLogFog	vDbPackage	vDbProgRel
bitMAP 419	ASCII 98	ASCII 90
vDBLogMultiColumn	vDbPackageType	defectMAP 330
bitMAP 421	ASCII 98	Events 489
vDBLogOther	vdbPatternSet	LEH 169
bitMAP 416	bitMAP 410	WEH 203
vDBLogRow	vDbPDate	vDbProgRev
bitMAP 413	Fab 253	ASCII 91
vDBLogSpot	vDbPEquip	defectMAP 331
bitMAP 412	Fab 249	Events 490
vDbLot	vDbProcess	Fab 243
ASCII 92	ASCII 88	LEH 170
bitMAP 425	bitMAP 424	WEH 204
defectMAP 331	defectMAP 329	vDbProgStep
vDbLotClass	Events 488	Fab 244
ASCII 92	Fab 245	vdbRawFile
bitMAP 426	LEH 168	bitMAP 431
defectMAP 332	WEH 202	vdbRawFilePath
Events 491	vDbProduct	bitMAP 431
Fab 246	ASCII 88	vDbRecipe
LEH 171	bitMAP 425	ASCII 94
WEH 205	defectMAP 329	Events 494
vDbMDate	Events 489	Fab 248
Fab 252	Fab 246	vDbResultTime
vDbMemBlockConfig	LEH 168	defectMAP 325
bitMAP 409	WEH 202	vdbRootFilePath
vDbMemCellBlockConfig	vDbProgGroup	bitMAP 431
bitMAP 407	ASCII 91	vDbSampleType
vDbMemConfig	bitMAP 425	defectMAP 337
bitMAP 408, 409	defectMAP 331	vDbSetCriticalDfClass
vDbMEquip	Events 490	defectMAP 324
Fab 248	Fab 244	vdbSetDefaultTest
vDbModuleCfg	LEH 170	bitMAP 411
ASCII 85	WEH 204	vDbSetupTime
vDbMStepPStep	vDbProgLotWaf	defectMAP 325
Fab 247	Fab 242	vDbSlotNum
vDbOperator	vDbProgProcess	defectMAP 316
ASCII 99	LEH 170	vDbSrcLot
bitMAP 428	WEH 204	ASCII 92
defectMAP 335	vDbProgProduct	bitMAP 426
Events 495	ASCII 91	defectMAP 332
Fab 250	vDbProgram	Events 491
vDbOplogIndex	ASCII 89	
Fab 242	defectMAP 330	
vDbOutliers	Events 489	
ASCII 115	LEH 169	
	WEH 203	

- vDbStage
 - ASCII [94](#)
 - bitMAP [427](#)
 - defectMAP [333](#)
 - Events [493](#)
 - Fab [248](#)
- vDbStartTime
 - ASCII [104](#)
 - Events [497](#)
- vDbStep
 - ASCII [93](#)
 - bitMAP [426](#)
 - defectMAP [333](#)
 - Events [492](#)
 - Fab [247](#)
- vDbTechnology [167](#)
 - ASCII [87](#)
 - bitMAP [423](#)
 - defectMAP [329](#)
 - Events [487](#)
 - Fab [244](#)
 - WEH [201](#)
- vDbTester
 - ASCII [96](#)
 - bitMAP [428](#)
 - defectMAP [335](#)
- vDbTesterType
 - defectMAP [326](#)
- vDbTestMode
 - ASCII [115](#)
 - bitMAP [433](#)
- vDbToleranceDimensions
 - defectMAP [316](#)
- vDbTrigger
 - ASCII [120](#)
- vDbTriggerInput
 - ASCII [121](#)
- vDbWaferClass
 - ASCII [100](#)
 - defectMAP [332](#)
 - Fab [255](#)
 - WEH [207](#)
- vDbWmapCfg
 - ASCII [99](#)
 - bitMAP [429](#)
 - defectMAP [336](#)
- vDbZWmapCfg
 - ASCII [100](#)
- Vertical Tab
 - ASCII [62](#)
 - bitMAP [391](#)
 - defectMAP [301](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- vLimitsProgram
 - Fab [243](#)
- vLogResult
 - ASCII [79](#)
 - Events [481](#)
 - Fab [235](#)
 - LEH [162](#)
 - WEH [196](#)
- vPosType
 - defectMAP [319](#)
- vPrepareSizeSummary
 - defectMAP [318](#)
- VT
 - ASCII [62](#)
 - bitMAP [391](#)
 - defectMAP [301](#)
 - Events [470](#)
 - Fab [220](#)
 - LEH [152](#)
 - WEH [186](#)
- W**
 - WAF table [30](#)
 - wafbackground
 - defectMAP [344](#)
 - Wafer
 - built-in functions,
 - defectMAP [332](#)
 - built-in functions, Events
 - [492](#)
 - built-in functions, WEH
 - [205](#)
 - Wafer Configuration
 - built-in functions, ASCII
 - [99](#)
 - built-in functions, bitMAP
 - [429](#)
 - built-in functions,
 - defectMAP [336](#)
 - wafext
 - ASCII [136](#)
 - bitMAP [437](#)
 - defectMAP [344](#)
 - Fab [260](#)
 - LTX77 [385](#)
 - STDF3 [277](#)
 - Warnings [15, 16](#)
 - Wd [15](#)
 - WEH Data Reader [179](#)
 - WEH Format File [181](#)
 - WEH Reader
 - dataPOWER Interface [210](#)
 - wfnun
 - LTX77 [386](#)
 - Word
 - ASCII [76](#)
 - bitMAP [399](#)
 - defectMAP [308](#)
 - Events [479](#)
 - Fab [233](#)
 - LEH [160](#)
 - WEH [194](#)
 - Worksheet Reader Interface
 - LTX77 [384](#)
 - STDF3 [272](#)
 - STDF4 [279](#)
 - Worksheet Readers [10](#)
 - Worksheet Storage [31](#)
 - wsreport
 - ASCII [139](#)
 - Fab [261](#)
- X**
 - X and Y coordinates [39](#)
- Y**
 - Y coordinates [39](#)

Z

Zones, built-in functions

ASCII [118](#)

bitMAP [434](#)

defectMAP [339](#)



FRONT



CONTENTS